

NEURONCITE

Neural-Powered Citation & Passage Retrieval

Software Architecture Document

Semantic Document Search Engine

Written in Rust

Version: 0.1.4
Date: March 17, 2026
Platform: Windows (primary), Linux (secondary)
Language: Rust 2024 Edition
License: AGPL-3.0-only

Contents

1	Project Overview	5
1.1	Purpose	5
1.2	Scope	5
1.3	Technology Stack	6
1.4	Design Principles	6
2	Workspace Structure	9
2.1	Crate Overview	9
2.2	Dependency Graph	13
2.3	Internal Module Structure	14
3	Core Abstractions	53
3.1	Domain Types	53
3.2	The Embedding Backend Trait	55
3.3	The Reranker Trait	55
3.4	The Chunk Strategy Trait	56
4	PDF Extraction Pipeline	57
4.1	File Discovery	57
4.2	Extraction Backends	57
4.3	Automatic OCR Fallback	58
4.4	Text Extraction Output	59
4.5	Error Handling	59
5	PDF Annotation Pipeline	60
5.1	Architecture Overview	60
5.2	5-Stage Text Location Pipeline	60
5.3	Annotation Creation	61
5.4	Verification	62
5.5	Job Integration	62
5.6	Interface Exposure	62
6	Citation Verification Pipeline	63
6.1	Architecture Overview	63
6.2	Database Schema	63
6.3	Cite Groups and Batching	65
6.4	Verdict Types	65
6.5	Interface Exposure	66
7	Chunking Strategies	67
7.1	Page-Based Chunking	67
7.2	Word-Window Chunking	67
7.3	Token-Window Chunking	67
7.4	Sentence-Based Chunking	68

7.5	Chunk Boundary Determinism	69
7.6	Context Window for Display	69
8	Embedding Backends	70
8.1	Backend: ONNX Runtime (<code>backend-ort</code>)	70
8.2	Backend Selection at Runtime	72
8.2.1	Global Model Selection	72
8.2.2	Model Manager Panel	72
9	Storage Layer	73
9.1	Storage Location	73
9.2	Connection Pooling	74
9.3	Table: <code>index_session</code>	74
9.4	Table: <code>indexed_file</code>	76
9.5	Table: <code>page</code>	78
9.6	Table: <code>chunk</code>	79
9.7	Table: <code>job</code>	82
9.8	Table: <code>idempotency</code>	83
9.9	Table: <code>annotation_quote_status</code>	83
9.10	Table: <code>web_source</code>	84
9.11	Full-Text Search Index (FTS5)	86
9.12	HNSW Index File	87
9.13	Embedding Blob Format	88
9.14	External Embedding Storage (Optional)	89
10	Search and Retrieval	90
10.1	Vector Search (HNSW)	90
10.2	Keyword Search (BM25)	90
10.3	Hybrid Search via Reciprocal Rank Fusion	90
10.4	Cross-Encoder Reranking	91
10.5	Cosine Similarity	92
10.6	Citation Generation	92
11	REST API Design	93
11.1	Shared Application State	93
11.2	Endpoint Overview	93
11.3	Request and Response Formats	97
11.4	Idempotency and Job Model	98
11.5	API Versioning Policy	98
11.6	Claim Verification Endpoint	99
11.7	Network Binding and Access Control	99
12	Data Flow	101
12.1	Indexing Pipeline	101
12.2	Search Pipeline	102
13	Concurrency Model	103

13.1 Thread Architecture	103
13.2 Shared State Synchronization	104
13.3 Graceful Shutdown	104
14 Cargo Feature Flags	106
14.1 Feature Definitions	106
14.2 Build Commands	106
14.3 Conditional Compilation	106
15 Error Handling	108
15.1 Strategy	108
15.2 Error Propagation	108
16 Model Management	109
16.1 Model Cache	109
16.2 Model Manifest (<code>models.lock</code>)	109
16.3 Download and Verification	109
16.4 Offline Operation	110
16.5 Local Model Registration	110
17 Build and Deployment	111
17.1 Build Output	111
17.2 Runtime Dependencies	111
17.3 Distribution Profiles	111
17.4 Dependency Doctor	112
17.5 Continuous Integration	113
18 Command-Line Interface	114
18.1 Subcommands	114
18.2 Exit Codes	114
18.3 JSON Output Contract	114
18.4 Model Management Subcommands	114
18.4.1 <code>models list</code>	116
18.4.2 <code>models info <model-id></code>	116
18.4.3 <code>models download <model-id></code>	116
18.4.4 <code>models verify <model-id></code>	116
18.4.5 <code>models system</code>	117
19 Operational Concerns	118
19.1 Configuration File	118
19.2 Logging	118
19.3 Storage Location for Operational Files	118
20 Agent Integration and Python Client	121
20.1 Integration Modes	121
20.2 Server Lifecycle for Agents	121
20.3 Python Client Library	121
20.4 OpenAPI Specification	123

20.5 LLM Agent Tool-Use Contract	123
21 Model Context Protocol Server	124
21.1 Protocol Architecture	124
21.2 Tool Catalog	124
21.3 Registration and Lifecycle	128
22 Testing Strategy	129
22.1 Testing Requirements and Toolchain	129
22.2 Test Infrastructure	130
22.2.1 Synthetic PDF Generation	130
22.2.2 OCR Fallback Test Fixtures	130
22.2.3 Test Database Lifecycle	131
22.2.4 Deterministic Embedding Stub	131
22.2.5 Temporary Directory Management	131
22.3 Unit Test Catalog	131
22.4 Integration Test Catalog	300
22.5 Property-Based and Regression Test Catalog	312
22.6 Test Summary	318
22.7 Continuous Integration Test Matrix	318
22.8 Architecture-Code Consistency Validator	320
22.8.1 Invocation and Arguments	320
22.8.2 LaTeX Parsing Approach	320
22.8.3 Rust Source Tree Scanning	321
22.8.4 Validation Checks	321
22.8.5 Output Format	322
22.8.6 Bidirectional Enforcement Rationale	323
A Glossary	324

1 Project Overview

1.1 Purpose

NeuronCite is a local, privacy-preserving semantic document search engine. It processes collections of PDF documents, extracts their textual content, computes dense vector embeddings using transformer-based language models running on a local GPU, stores the resulting vectors in an HNSW index and metadata in an embedded SQLite database, and provides ranked retrieval of the most semantically relevant passages for a given natural-language query.

The application ships as a single executable file that contains an embedded SolidJS frontend, an HTTP REST API server, and a command-line interface with subcommands. Double-clicking the executable starts the web server and opens a native OS window (WebView2 on Windows, WebKit on macOS/Linux). If the native window cannot be created (e.g. missing WebKitGTK on Linux), the application falls back to the default browser. Running the executable from a terminal with the `serve` subcommand starts the API server in headless mode without opening a browser, enabling automation by scripts and LLM agents. A CPU-only build using the default `pdf-extract` backend has zero external runtime dependencies. The optional pdfium extraction backend and OCR fallback require additional shared libraries at runtime. GPU-accelerated builds require the NVIDIA CUDA toolkit and, depending on the selected embedding backend, further shared libraries (see Section 17.2).

1.2 Scope

The system covers the complete pipeline from raw documents to ranked search results, citation verification, and PDF annotation:

1. Recursive discovery of PDF files in a user-selected directory tree
2. Text extraction from PDF documents using multiple backends with automatic OCR fallback for image-heavy pages
3. HTML web page fetching, caching, and crawling with readability-based boilerplate removal and sitemap-based URL discovery
4. Splitting of extracted text into configurable chunks (token, word, sentence, and page strategies)
5. Dense vector embedding of each chunk using a local transformer model (ONNX Runtime with CUDA acceleration)
6. Persistent storage of chunks in SQLite and their embeddings in an HNSW approximate nearest neighbor index
7. Hybrid retrieval combining keyword search (BM25 via FTS5) with vector similarity (HNSW), followed by optional cross-encoder reranking
8. Presentation of ranked results with source citations through a graphical interface and a REST API
9. LaTeX citation verification: parses `\cite` commands from `.tex` files, matches cited works to indexed PDFs, and manages a batch-based claim/submit workflow with

seven verdict types

10. PDF annotation: highlights quoted text passages and adds comment annotations via a 5-stage text matching pipeline (exact, normalized, fuzzy, fallback extraction, OCR)
11. LLM integration via Ollama for agent-driven citation verification
12. A Model Context Protocol (MCP) server that exposes all functionality as JSON-RPC 2.0 tools for integration with AI coding assistants
13. A command-line interface with subcommands for headless indexing, searching, and server operation, producing machine-readable JSON output
14. A versioned REST API with OpenAPI specification, job-based asynchronous operation tracking, and idempotent request handling for programmatic integration by scripts and LLM agents

1.3 Technology Stack

1.4 Design Principles

- **Single Binary:** The entire application compiles to one executable. A CPU-only build using the default `pdf-extract` backend requires no external runtime dependencies. The optional `pdfium` extraction backend and OCR fallback require external shared libraries regardless of CPU or GPU mode. GPU builds additionally require the NVIDIA CUDA toolkit and backend-specific shared libraries.
- **Local-First:** All processing happens on the user’s machine. No data leaves the local environment.
- **Modular Crates:** Each concern is isolated in its own Rust crate with well-defined trait boundaries.
- **Backend Agnostic:** The embedding subsystem abstracts over multiple ML frameworks via a common trait. Backends are selected at compile time through feature flags and at runtime through a user-facing dropdown.
- **Incremental Indexing:** The system tracks each file individually within a session. When a file is added, modified, or deleted in the indexed directory, only the affected files are re-processed. File-level change detection uses modification timestamps and file sizes as a fast pre-filter, falling back to SHA-256 content hashing when metadata changes are ambiguous. Deleted files are removed from the index, and modified files trigger a targeted re-extraction, re-chunking, and re-embedding of that file’s chunks only.
- **Evidence-Based Retrieval:** Every search result carries a structured citation (source file, page number, text excerpt) that enables the user to verify and reference the original passage.
- **Dual-Mode Operation:** The binary supports two execution modes from the same executable. Web mode (default, launched by double-click) starts the Axum server with the embedded SolidJS frontend and opens the default browser. Headless mode (`neuroncite serve`) starts the API server without opening a browser, suitable for server deployments, automated pipelines, and LLM agent integration.
- **Agent-Ready API:** The REST API uses a versioned URL prefix (`/api/v1/`), provides an OpenAPI specification at `/api/v1/openapi.json`, returns deterministic JSON responses, and tracks long-running operations as first-class job objects with

Component	Technology
Programming Language	Rust (2024 Edition)
Build System	Cargo workspace with feature flags
HTTP Server	axum (async, tower-based)
Async Runtime	tokio (multi-threaded)
PDF Extraction (primary)	pdf-extract (pure Rust, fast)
PDF Extraction (fallback)	pdfium-render (handles complex layouts, multi-column)
OCR	Tesseract CLI subprocess (installed via Doctor panel or PATH)
Embedding (ONNX)	ort (Rust bindings for ONNX Runtime with CUDA)
ANN Index	hnsw_rs (Hierarchical Navigable Small World graph)
Database	SQLite via rusqlite (embedded, zero-configuration)
Connection Pooling	r2d2 / r2d2_sqlite
Full-Text Search	SQLite FTS5 (BM25 keyword ranking)
Sentence Segmentation	unicode-segmentation
Stemming	rust-stemmers (Porter stemmer for keyword overlap in verify endpoint)
System PDF Viewer	opener (cross-platform “open file” command)
Serialization	serde / serde_json
Error Handling	thiserror (libraries), anyhow (application)
File Dialogs	rfd (Rusty File Dialog, OS-native)
CLI Framework	clap (derive-based argument parsing with subcommands)
OpenAPI Generation	utoipa (compile-time OpenAPI spec from axum handlers)
Logging	tracing / tracing-subscriber (structured, leveled logging)
Configuration	toml (human-readable config file format)
PDF Annotation	pdfium-render (highlight and text annotations)
BibTeX Parsing	nom-bibtex (citation verification input)
String Matching	strsim (Jaro-Winkler similarity for PDF-to-title matching)
HTML Fetching	reqwest (HTTP client with configurable User-Agent)
HTML Parsing	scraper / readability (boilerplate removal, content extraction)
LLM Integration	Ollama HTTP API via neuroncite-llm
MCP Protocol	JSON-RPC 2.0 over stdin/stdout (protocol version matches crate version)
Frontend	SolidJS (TypeScript SPA, embedded via rust-embed)

Table 1: Technology stack overview.

unique identifiers. This contract enables LLM agents and Python scripts to interact with the system programmatically without parsing human-readable output.

- **Deterministic CLI Output:** All CLI subcommands produce machine-readable JSON on stdout by default. Human-readable output is available via the `--format text` flag. Exit codes follow Unix conventions (0 for success, 1 for application error, 2 for usage error). This enables reliable scripting and pipeline integration.

2 Workspace Structure

The project is organized as a Cargo workspace with fifteen library crates, one binary crate, and one standalone test-fixture generator (`neuroncite-testgen`), for a total of sixteen workspace members. Each library crate encapsulates a single architectural concern.

2.1 Crate Overview

Crate	Type	Responsibility
<code>neuroncite-core</code>	Library	Shared domain types, trait definitions (Embedding-Backend, ChunkStrategy, Reranker), configuration structures, the document-offset-to-page resolution utility, and error types. Has no internal dependencies.
<code>neuroncite-pdf</code>	Library	Recursive PDF file discovery in a directory tree, page-by-page text extraction with multiple backends (pdf-extract, pdfium), and automatic OCR fallback for image-heavy pages.
<code>neuroncite-chunk</code>	Library	Text splitting strategies: page-based, word-window with configurable size and overlap, and sentence-based chunking using unicode-segmentation.
<code>neuroncite-embed</code>	Library	The embedding backend abstraction and its ONNX Runtime implementation (ort), gated behind a Cargo feature flag. Also implements the Reranker trait for cross-encoder models and manages the platform-specific model cache (download, checksum verification, offline resolution).
<code>neuroncite-store</code>	Library	SQLite schema management via r2d2 connection pool, organized into three internal layers: repository (CRUD for sessions, files, pages, chunks), index (HNSW file, FTS5, external embeddings), and workflow (job tracking, idempotency).
<code>neuroncite-search</code>	Library	HNSW approximate nearest neighbor queries, BM25 keyword search via FTS5, hybrid result fusion (Reciprocal Rank Fusion), optional cross-encoder reranking, and top-K ranking.

continued on next page

Crate	Type	Responsibility
neuroncite-api	Library	The axum-based REST API server with shared application state (<code>AppState</code> decomposed into <code>AuthState</code> , <code>IndexState</code> , and <code>SseChannels</code>), route definitions, request/response types, middleware, and the citation agent. Implements the <code>PipelineContext</code> trait from <code>neuroncite-pipeline</code> ; its <code>worker</code> , <code>indexer</code> , and <code>executor</code> modules are thin re-export shims over the pipeline crate.
neuroncite-pipeline	Library	Background job executor, GPU worker, and two-phase indexing pipeline, extracted from <code>neuroncite-api</code> . Defines the <code>PipelineContext</code> trait (12 methods for database pool, worker, HNSW map, configuration, and SSE channels) enabling independent testing via <code>StubPipelineContext</code> . Implements the GPU Worker (<code>WorkerHandle</code>) that serializes embedding and reranking via priority channels, the shared two-phase indexing pipeline (Phase 1: parallel rayon extraction; Phase 2: sequential GPU embedding and storage), and the background job executor.
neuroncite-web	Library	Browser-based SolidJS frontend served via Axum. Embeds the compiled SPA into the binary via rust-embed for single-file distribution. Provides SSE broadcast channels for real-time log streaming, indexing progress, and model operations. Adds web-specific REST handlers for file system browsing, model catalog management, dependency probes, MCP registration, and configuration reading. Opens the default browser on server startup.
neuroncite-html	Library	HTML web page fetching, caching, parsing, and crawling. Fetches pages via HTTP with configurable User-Agent and caches raw HTML to disk. Extracts visible text and metadata (title, Open Graph tags, canonical URL, author, language). Applies a readability algorithm for boilerplate removal (navigation, ads, sidebars). Splits content into heading-based sections (H1/H2) analogous to PDF pages. Supports BFS website crawling with depth/domain/pattern filtering and sitemap-based URL discovery.

continued on next page

Crate	Type	Responsibility
<code>neuroncite-mcp</code>	Library	Model Context Protocol server implementation. Reads JSON-RPC 2.0 requests from stdin and writes responses to stdout. Implements the MCP specification (protocol version derived from crate version) with tool definitions for search, indexing, session management, job monitoring, citation export, model listing, page retrieval, and system diagnostics. Includes registration logic for writing the server entry into Claude Code’s MCP configuration file.
<code>neuroncite-annotate</code>	Library	PDF annotation pipeline: highlights quoted text passages and adds comment annotations. Takes CSV or JSON input with title, author, and quote fields. Resolves PDFs via Jaro-Winkler filename matching, locates text through a 5-stage pipeline (exact, normalized, fuzzy, fallback extraction, OCR), and creates pdfium highlight and text annotations.
<code>neuroncite-citation</code>	Library	LaTeX citation verification pipeline. Parses <code>\cite</code> commands from <code>.tex</code> files and resolves cite-keys against BibTeX entries. Matches cited works to indexed PDFs via Jaro-Winkler similarity, groups nearby citations into batches for sub-agent claiming, and manages the claim/submit/status/export workflow with seven verdict types and LaTeX correction suggestions.
<code>neuroncite-llm</code>	Library	LLM abstraction layer with zero workspace-internal dependencies. Defines the <code>LlmBackend</code> trait for chat completion (blocking and streaming) and provides a concrete Ollama HTTP client implementation. The crate is a leaf dependency — swapping the LLM backend requires only adding a new struct that implements <code>LlmBackend</code> in this crate; no other crates change.
<code>neuroncite</code>	Binary	The main entry point. Parses CLI arguments via clap to determine the execution mode (web, serve, index, search, doctor, sessions, export, annotate, version, models, mcp). In web mode, initializes the tokio runtime, starts the Axum server with the embedded SolidJS frontend, and opens a native OS window (or falls back to the default browser). In headless mode, runs the API server, executes a single CLI command, or starts the MCP stdio server and exits.

continued on next page

Crate	Type	Responsibility
<code>neuroncite-testgen</code>	Binary	Standalone test-fixture generator (<code>publish = false</code>). Produces three synthetic PDFs in <code>tests/PDFs/ocr/</code> that trigger specific failure modes in the <code>pdf-extract</code> crate: cross-reference table corruption, malformed CID-range CMap encoding, and an unterminated Type1 FontFile stream. These fixtures replace copyrighted academic papers that cannot be distributed on GitHub. Only depends on <code>lopdf</code> ; has no workspace-internal dependencies. Run via <code>cargo run -p neuroncite-testgen</code> .

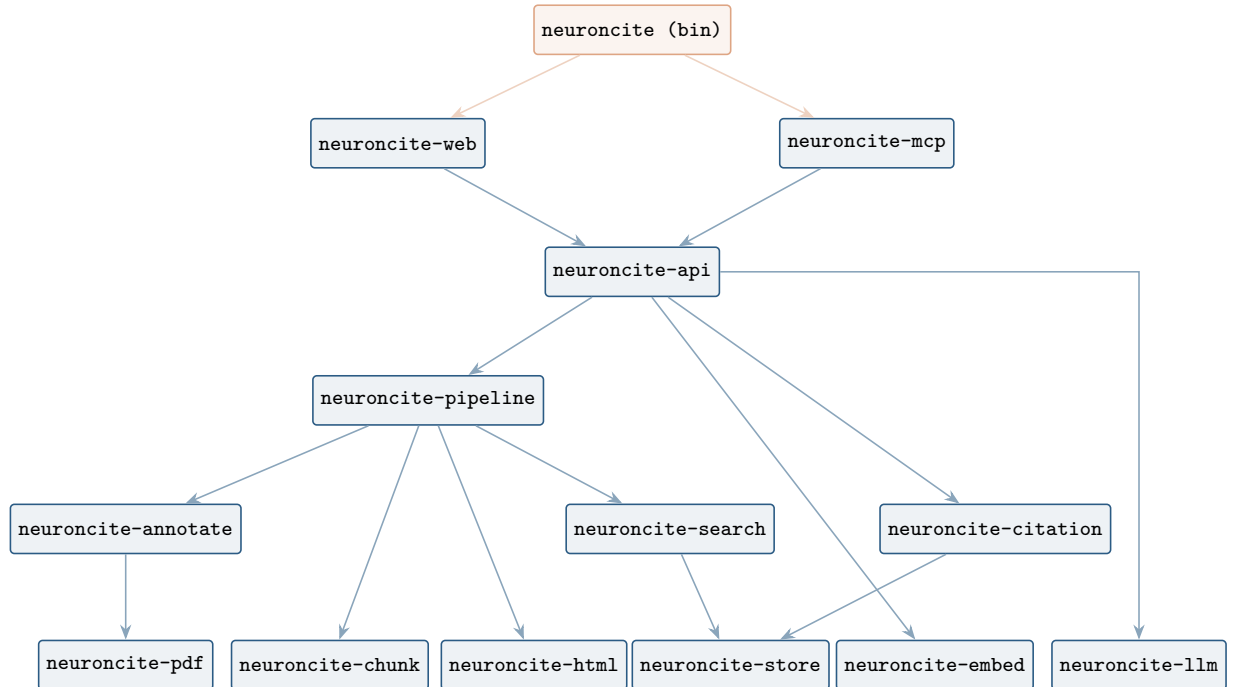
Table 2: Workspace crates and their responsibilities.

Module Path	Visibility	Responsibility
<code>src/main.rs</code>	private	Entry point. Invokes the three PDF generators and writes the output files into <code>tests/PDFs/ocr/</code> .
<code>src/xref_corruption.rs</code>	private	Generates a PDF with corrupted cross-reference table entries, an invalid <code>startxref</code> pointer, and a mismatched trailer <code>/Size</code> . Both <code>pdf-extract</code> and <code>pdfium</code> fail on this file, exercising the error path in the fallback chain.
<code>src/cid_range_panic.rs</code>	private	Generates a PDF with a malformed CID-range CMap encoding in a Type0 font. Triggers a panic in <code>pdf-extract</code> ’s CMap parser while <code>pdfium</code> extracts text via a fallback Helvetica font. Contains finance-domain content (“trading”, “stock”, “returns”).
<code>src/type1_font_panic.rs</code>	private	Generates a PDF with an unterminated Type1 FontFile stream. Triggers a panic in <code>pdf-extract</code> ’s <code>type1_encoding_parser</code> while <code>pdfium</code> extracts text via Helvetica. Contains forecasting-domain content (“forecast”, “accuracy”, “error”).

Table 3: Internal module structure of `neuroncite-testgen`.

2.2 Dependency Graph

The following diagram shows the compile-time dependency relationships between crates. Arrows point from the dependent crate to the dependency it consumes.



All library crates except neuroncite-annotate, neuroncite-citation, and neuroncite-llm depend directly on neuroncite-core (edges omitted).

Figure 1: Crate dependency graph (transitive reduction). Arrows point from the consuming crate to its dependency. To reduce visual clutter, only non-redundant edges are shown: if crate A depends on B and B depends on C, the implied edge $A \rightarrow C$ is omitted. Most library crates depend directly on `neuroncite-core` for shared domain types; `neuroncite-annotate` and `neuroncite-citation` obtain core types transitively through `neuroncite-pdf` and `neuroncite-store` respectively, and `neuroncite-llm` has zero workspace-internal dependencies. These edges are replaced by the annotation band at the bottom. The full set of direct Cargo.toml dependencies per crate is documented in the crate overview table above.

2.3 Internal Module Structure

The following tables define the internal file and module layout for each library crate. Every crate follows a shared structural convention: `src/lib.rs` serves as the module root that declares submodules and re-exports the crate’s public API. It contains primarily module declarations and re-exports; non-trivial logic is placed in submodules. Private modules (declared as `mod`, not `pub mod`) are not addressable by downstream crates; only items that are re-exported or otherwise exposed through public signatures form the crate’s public API. The sole exception is `neuroncite-core`, where all modules are `pub mod` because it serves as the shared type vocabulary imported by every other crate.

Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Declares submodules and re-exports all public types, traits, configuration structs, and error variants.
<code>src/types.rs</code>	public	Defines the domain value types: <code>PageText</code> , <code>Chunk</code> , <code>EmbeddedChunk</code> , <code>Citation</code> , <code>SearchResult</code> , <code>IndexConfig</code> , <code>IndexProgress</code> , <code>ModelInfo</code> , <code>ModelManifest</code> , <code>PoolingStrategy</code> , and <code>EmbeddingModelConfig</code> .
<code>src/traits.rs</code>	public	Declares the <code>EmbeddingBackend</code> , <code>ChunkStrategy</code> , and <code>Reranker</code> trait contracts that downstream crates implement or consume.
<code>src/config.rs</code>	public	Contains application-wide and session-level configuration structs deserialized from TOML files and CLI arguments.
<code>src/offset.rs</code>	public	Provides the shared document-offset-to-page resolution algorithm. Accepts a slice of per-page byte counts and a document-level UTF-8 byte offset, and returns the corresponding page number and page-local byte offset. This module is consumed by both <code>neuroncite-chunk</code> (for computing <code>page_start/page_end</code> during chunking) and <code>neuroncite-search</code> (for citation generation), avoiding duplication of the offset mapping logic.

Module Path	Visibility	Responsibility
<code>src/math.rs</code>	public	Canonical mathematical utility functions shared across all crates. Provides <code>cosine_similarity(a, b)</code> which computes the cosine similarity between two <code>f32</code> slices using <code>f64</code> accumulation for numerical stability. Returns 0.0 for zero-magnitude vectors. Used by <code>neuroncite-search</code> (reranking) and <code>neuroncite-api</code> (verification). 7 unit tests.
<code>src/paths.rs</code>	public	Centralized storage path resolution for the application. Computes the base directory (<code>Documents/NeuronCite</code>), runtime directory for driver DLLs (ORT, pdfium, Tesseract), model cache directory for ONNX embedding files, and per-indexed-path database directories under <code>indexes/</code> . Provides <code>sanitize_path</code> to convert filesystem paths into safe directory names and <code>canonicalize_directory</code> to resolve a directory to its absolute canonical form (stripping the <code>\\?\</code> UNC prefix on Windows). The canonical path is the single source of truth for all session creation, search, and deletion operations, preventing duplicate sessions when MCP clients pass temporary or non-canonical paths.
<code>src/disk.rs</code>	public	Cross-platform disk space detection utility. Provides <code>available_disk_space</code> (bytes free on the volume containing a given path) and <code>check_disk_space</code> (pre-flight check that errors below 500 MB, warns below 1 GB, and passes otherwise). Uses PowerShell <code>Get-PSDrive</code> on Windows and <code>df</code> on Unix. Consumed by <code>neuroncite-pdf</code> and <code>neuroncite-embed</code> before large runtime downloads.
<code>src/error.rs</code>	public	Defines the <code>NeuronCiteError</code> enum with <code>thiserror</code> -derived variants covering all failure categories across the application.

Module Path	Visibility	Responsibility
src/time.rs	public	Centralized UNIX timestamp helper. Provides <code>unix_timestamp_secs()</code> -> <code>i64</code> that returns the current system time as seconds since the UNIX epoch, falling back to zero if the system clock is before the epoch. Replaces duplicated <code>SystemTime</code> boilerplate across the store and citation crates.

Table 4: Internal module structure of `neuroncite-core`.
All modules are `pub mod` because this crate is the shared type vocabulary.

Module Path	Visibility	Responsibility
src/lib.rs	public	Declares submodules and re-exports the file discovery function, page extraction entry point, and <code>PdfError</code> .
src/discovery.rs	private	Recursively traverses a directory tree, collects PDF file paths with cycle detection via visited-path sets, and returns them in deterministic sorted order.
src/extract/mod.rs	private	Dispatches page extraction to the configured backend and implements the automatic fallback chain (<code>pdf-extract</code> → <code>pdfium</code> → <code>OCR</code>).
src/extract/pdf_extract.rs	public	Wraps the <code>pdf-extract</code> crate to parse PDF content streams and reconstruct text layout on a per-page basis.
src/extract/pdfium.rs	public	Wraps the <code>pdfium-render</code> crate for extraction from PDFs with multi-column layouts, complex tables, and non-standard font encodings.
src/pdfium_binding.rs	public	Locates the pdfium shared library at runtime by searching the executable directory, the current working directory, the auto-download cache, and system library paths. Falls back to auto-downloading pdfium from GitHub. Used by both the pdfium extraction backend and the OCR module.

Module Path	Visibility	Responsibility
<code>src/deps.rs</code>	public	Download logic for optional runtime dependencies (pdfium shared library, Tesseract binary, tessdata language packs). Downloads portable distributions and caches them in <code><Documents>/NeuronCite/runtime/</code> . Uses PowerShell on Windows and curl/tar on Linux/macOS. Public so the web frontend’s dependency doctor page can call the download and probe functions.
<code>src/ocr.rs</code>	public	Renders pages to raster images at adaptive DPI via pdfium, writes them to temporary files, and invokes the Tesseract CLI binary as a subprocess. Tesseract must be installed via the system package manager or the Doctor panel’s install button.
<code>src/quality.rs</code>	public	Evaluates extracted text against quality heuristics (alphabetic ratio, replacement character density, line break density, unique token ratio) to decide backend fallback.
<code>src/error.rs</code>	private	Defines <code>PdfError</code> with variants for I/O failures, backend extraction failures, OCR failures, dependency download failures, and missing runtime dependencies.

Table 5: Internal module structure of `neuroncite-pdf`.

Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Declares submodules and re-exports a factory function that returns a boxed <code>ChunkStrategy</code> from a strategy identifier string.
<code>src/normalize.rs</code>	public	Applies deterministic text preprocessing: whitespace collapsing, line-break hyphenation repair, and Unicode NFC normalization.

Module Path	Visibility	Responsibility
<code>src/page.rs</code>	public	Implements ChunkStrategy for page-based chunking where each PDF page becomes one chunk with its full page text.
<code>src/word_window.rs</code>	public	Implements ChunkStrategy for sliding word-window chunking with configurable window size and word overlap.
<code>src/token_window.rs</code>	public	Implements ChunkStrategy for sliding token-window chunking using the model's tokenizer to measure boundaries against the maximum sequence length. The tokenizer instance (from the tokenizers crate) is injected into the strategy constructor by the caller (the API handler or binary crate), which obtains it from the embedding backend. This avoids a compile-time dependency from neuroncite-chunk to neuroncite-embed .
<code>src/sentence.rs</code>	public	Implements ChunkStrategy for sentence-based chunking using unicode-segmentation with abbreviation-aware boundary suppression.
<code>src/abbreviations.rs</code>	public	Contains the static abbreviation whitelist (<i>et al.</i> , <i>Fig.</i> , <i>Eq.</i> , etc.) consumed by the sentence chunker to suppress false breaks.
<code>src/offset.rs</code>	public	Tracks document-level UTF-8 byte offsets during chunking. Delegates the byte-offset-to-page-number mapping to neuroncite_core::offset and adds chunk-specific bookkeeping (accumulating offsets across pages, computing page_start / page_end for each chunk).
<code>src/error.rs</code>	public	Defines ChunkError with variants for normalization failures, invalid parameter combinations, and tokenizer initialization errors.

Module Path	Visibility	Responsibility
src/property_tests.rs	private	Proptest-based property tests covering the Page, WordWindow, and Sentence chunking strategies and normalization invariants. Contains 20 tests. Compiled only under <code>#[cfg(test)]</code> .

Table 6: Internal module structure of `neuroncite-chunk`.

Module Path	Visibility	Responsibility
src/lib.rs	public	Conditionally declares backend modules via <code>#[cfg(feature)]</code> attributes and re-exports the backend registry, shared utilities, and <code>EmbedError</code> .
src/registry.rs	public	Iterates over all compiled-in backends at runtime, collects their <code>ModelInfo</code> lists, and provides a unified model and backend selection interface.
src/normalize.rs	public	Normalizes embedding vectors to unit length (L2 normalization) so that cosine similarity reduces to a dot product at search time.
src/tokenize.rs	public	Wraps the HuggingFace <code>tokenizers</code> crate for shared tokenization, padding, and attention mask construction across all backends.
src/cache.rs	public	Manages the platform-specific model cache directory (<code>%LOCALAPPDATA%\neuroncite\models\</code> on Windows, <code>~/.cache/neuroncite/models/</code> on Linux). Downloads model weights and tokenizer files from Hugging Face Hub on first use, verifies SHA-256 checksums against the <code>models.lock</code> manifest, resolves pinned revisions for reproducible builds, and provides a local file system path to the cached model directory for backend initialization. Supports registration of local model files via <code>file://</code> source URLs.

Module Path	Visibility	Responsibility
<i>ONNX Runtime backend – compiled under feature backend-ort</i>		
<code>src/ort/mod.rs</code>	public	Backend module root; declares submodules and re-exports OrtEmbedder and OrtReranker . Public to allow downstream crates access to model config functions.
<code>src/ort/embedder.rs</code>	public	Implements EmbeddingBackend for ONNX Runtime with CUDA execution provider selection and batched tensor inference.
<code>src/ort/reranker.rs</code>	public	Implements Reranker for ONNX Runtime cross-encoder models with batched (query, document) pair scoring.
<code>src/ort/session.rs</code>	public	Manages ONNX session lifecycle: execution provider negotiation, GPU memory configuration, and thread pool options.

<code>src/error.rs</code>	public	Defines EmbedError with variants for model loading, tokenization, GPU initialization, and inference failures.
---------------------------	--------	--

Table 7: Internal module structure of **neuroncite-embed**. Backend modules are conditionally compiled via Cargo feature flags. The `#[cfg(feature = "...")]` gate is applied to the `mod` declaration in `lib.rs`; submodules within each backend are implicitly gated.

Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Declares submodules and re-exports the connection pool builder, repository types, index management functions, workflow types, and StoreError .
<code>src/pool.rs</code>	private	Creates and configures the r2d2 connection pool with WAL mode, foreign key enforcement, busy timeout, and SQLite PRAGMA tuning.
<code>src/schema.rs</code>	private	Contains all DDL statements for tables, indexes, triggers, and the FTS5 virtual table; runs versioned schema migrations on startup.

Module Path	Visibility	Responsibility
<code>src/manifest.rs</code>	private	Reads and writes the <code>models.lock</code> TOML file, recording model revisions, checksums, and tokenizer checksums for index reproducibility. This module is the sole owner of the <code>models.lock</code> file on disk. The <code>neuroncite-embed</code> cache module queries <code>neuroncite-store</code> through a <code>ModelManifest</code> value (defined in <code>neuroncite-core</code>) to verify checksums; it does not access the lock file directly.
<code>src/error.rs</code>	public	Defines <code>StoreError</code> with variants for SQLite failures, HNSW serialization errors, manifest parsing errors, and connection pool exhaustion.
<i>Repository layer (<code>repo/</code>) – CRUD operations for SQLite tables</i>		
<code>src/repo/mod.rs</code>	public	Declares repository submodules and re-exports all repository types for use by downstream crates.
<code>src/repo/session.rs</code>	public	Provides CRUD operations for the <code>index_session</code> table, including configuration-based session lookup, unique constraint enforcement, <code>find_sessions_by_directory</code> (returns all sessions whose <code>directory_path</code> matches a given canonical path), and <code>delete_sessions_by_directory</code> (deletes all matching sessions; ON DELETE CASCADE removes child records).
<code>src/repo/file.rs</code>	public	Provides CRUD operations for the <code>indexed_file</code> table with two-stage change detection (mtime/size pre-filter, then SHA-256 hash verification). Provides <code>find_file_by_session_path</code> to look up an existing file record by session ID and file path, used by the change detection step in the indexing pipeline.
<code>src/repo/page.rs</code>	public	Provides CRUD operations for the <code>page</code> table, including bulk insert of normalized page text and per-page <code>byte_count</code> computation.

Module Path	Visibility	Responsibility
<code>src/repo/chunk.rs</code>	public	Provides CRUD operations for the <code>chunk</code> table, including transactional bulk insert with embedding blobs and batch deletion by file ID.
<code>src/repo/aggregate.rs</code>	public	Reusable SQL aggregate query functions shared by statistics and discovery endpoints. Provides <code>all_session_aggregates</code> (per-session file, page, chunk, byte counts), <code>file_chunk_stats_by_session</code> , <code>file_page_stats_by_session</code> , <code>browse_chunks</code> (paginated chunk browsing), <code>find_files_by_path_pattern</code> (cross-session file search), and <code>session_quality_data</code> (extraction quality per file).
<code>src/repo/annotation_status.rs</code>	public	Repository functions for the <code>annotation_quote_status</code> table. Tracks per-quote progress during annotation pipeline execution. Functions: <code>insert_pending_quotes</code> (bulk insert at job start), <code>update_quote_status</code> (per-quote update), <code>list_quote_statuses</code> (paginated query), <code>count_quote_statuses_by_status</code> (aggregate counts).
<code>src/repo/diff.rs</code>	public	Session comparison functions. Contains <code>FileDiff</code> and <code>SessionDiff</code> types. The <code>diff_sessions</code> function compares two sessions by <code>file_path</code> , classifying files into <code>only_in_a</code> , <code>only_in_b</code> , <code>identical</code> , and <code>changed</code> categories using SHA-256 hash, page count, and file size.

Module Path	Visibility	Responsibility
src/repo/web_source.rs	public	CRUD operations for the <code>web_source</code> table that stores HTML-specific metadata linked 1:1 to <code>indexed_file</code> rows. Provides <code>insert_web_source</code> , <code>get_web_source_by_file</code> , <code>get_web_source_by_url</code> , <code>list_web_sources</code> (by session), <code>list_web_sources_all</code> , and <code>delete_web_source</code> .
<i>Index layer (<code>index/</code>) – search index construction and access</i>		
src/index/mod.rs	private	Declares index submodules and re-exports the HNSW management functions, FTS5 operations, and external embedding storage interface.
src/index/hnsw.rs	public	Builds, serializes, deserializes, and rebuilds the HNSW index; tracks orphan counts. The built index is inserted into the per-session <code>ArcSwap</code> map in <code>AppState</code> via <code>insert_hnsw()</code> .
src/index/fts.rs	public	Manages the FTS5 virtual table <code>chunk_fts</code> : four synchronization triggers (INSERT, DELETE, soft-delete, un-delete of <code>is_deleted</code>), and the optimize maintenance command.
src/index/external.rs	public	Manages the memory-mapped external embeddings file via <code>memmap2</code> for session-scoped embedding storage in large collections.
<i>Workflow layer (<code>workflow/</code>) – job tracking and idempotency</i>		
src/workflow/mod.rs	private	Declares workflow submodules and re-exports job and idempotency repository types.
src/workflow/job.rs	public	Provides CRUD operations for the <code>job</code> table: state transitions, progress updates, and 24-hour retention cleanup.
src/workflow/idempotency.rs	public	Provides CRUD operations for the <code>idempotency</code> table: key lookup, response storage, and expiration cleanup aligned with the job retention window.

Module Path	Visibility	Responsibility
Table 8: Internal module structure of <code>neuroncite-store</code>. Modules are organized into three internal layers: <code>repo/</code> for SQLite table CRUD operations, <code>index/</code> for search index file management (HNSW, FTS5, external embeddings), and <code>workflow/</code> for background job tracking and idempotency. All three layers share the same <code>r2d2</code> connection pool configured in <code>pool.rs</code>.		
Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Declares submodules and re-exports the <code>SearchPipeline</code> entry point, search configuration options, <code>SearchError</code> , and refinement functions (<code>generate_sub_chunks</code> , <code>apply_refinement</code> , <code>parse_divisors</code>).
<code>src/pipeline.rs</code>	private	Orchestrates the full search pipeline: receives the pre-computed query vector, runs parallel retrieval (HNSW + FTS5), fusion, deduplication, reranking, and citation assembly.
<code>src/vector.rs</code>	private	Executes HNSW approximate nearest neighbor queries with configurable <code>ef_search</code> and filters orphaned vector labels against the <code>chunk</code> table.
<code>src/keyword.rs</code>	private	Executes FTS5 BM25 keyword queries, converts the raw BM25 scores to 1-indexed rank positions, and returns ranked chunk IDs.
<code>src/fusion.rs</code>	private	Merges vector and keyword result lists using Reciprocal Rank Fusion with configurable <code>k</code> parameter and optional BM25 must-match filtering.
<code>src/dedup.rs</code>	private	Removes exact-duplicate results by <code>content_hash</code> and near-duplicate results by SimHash Hamming distance comparison.

Module Path	Visibility	Responsibility
src/citation.rs	private	Generates <code>Citation</code> objects from chunk metadata. Delegates document-level UTF-8 byte offset resolution to <code>neuroncite_core::offset</code> and assembles the formatted citation string from the resolved page numbers and text excerpts.
src/refine.rs	public	Sub-chunk refinement post-processing. Splits each search result chunk into token-based sub-windows at multiple scales (controlled by divisor values), embeds the sub-chunks, and replaces the chunk content with the highest-scoring sub-section when it exceeds the original cosine similarity. Exports <code>RefinementConfig</code> , <code>generate_sub_chunks</code> , <code>apply_refinement</code> , and <code>parse_divisors</code> .
src/error.rs	private	Defines <code>SearchError</code> with variants for query embedding failures, HNSW query errors, FTS5 query errors, chunk metadata lookup failures, and refinement errors.

Table 9: Internal module structure of `neuroncite-search`.

Module Path	Visibility	Responsibility
src/lib.rs	public	Declares submodules and re-exports the server builder function, <code>AppState</code> , DTO types, and <code>ApiError</code> .

Module Path	Visibility	Responsibility
<code>src/state.rs</code>	private	Defines <code>AppState</code> with three composable sub-structs: <code>AuthState</code> (bearer token hash, rate-limit counters, shutdown nonce), <code>IndexState</code> (HNSW index map, vector dimension, progress tracker), and <code>SseChannels</code> (three <code>tokio::sync::broadcast::Sender<String></code> fields for progress, citation, and source events, initialized unconditionally in <code>SseChannels::new()</code>). Implements the <code>PipelineContext</code> trait from <code>neuroncite-pipeline</code> .
<code>src/router.rs</code>	private	Constructs the axum <code>Router</code> with all <code>/api/v1/</code> route registrations, middleware layers, and the OpenAPI JSON endpoint.
<i>Request handlers</i>		
<code>src/handlers/mod.rs</code>	private	Declares handler submodules for each endpoint group.
<code>src/handlers/health.rs</code>	public	Reports application version, build features, GPU availability, and optional dependency status.
<code>src/handlers/index.rs</code>	public	Validates indexing parameters, checks the idempotency key, enforces concurrent job policy, and orchestrates the indexing pipeline by scheduling CPU-bound work via <code>spawn_blocking</code> and GPU embedding via the GPU worker channel.
<code>src/handlers/search.rs</code>	public	Delegates search and hybrid search requests to the search pipeline and serializes the ranked results.
<code>src/handlers/verify.rs</code>	public	Computes keyword overlap (Jaccard) and semantic similarity (cosine) to produce a weighted evidence verdict.
<code>src/handlers/jobs.rs</code>	public	Queries job state, lists active and historical jobs within the retention window, and triggers cooperative cancellation.

Module Path	Visibility	Responsibility
<code>src/handlers/sessions.rs</code>	public	Lists index session configurations, performs cascading deletion of individual sessions (with HNSW eviction), and provides a <code>delete_sessions_by_directory</code> handler that deletes all sessions for a given canonical directory path (also evicting their HNSW graphs from the in-memory map).
<code>src/handlers/documents.rs</code>	public	Retrieves stored page text from the <code>page</code> table by indexed file ID and page number.
<code>src/handlers/backends.rs</code>	public	Returns the list of compiled-in embedding backends with their available models, reranker models, and runtime capabilities (e.g., GPU support).
<code>src/handlers/shutdown.rs</code>	public	Triggers the global <code>CancellationToken</code> for graceful shutdown and returns 202 Accepted (headless mode only).
<code>src/handlers/annotate.rs</code>	public	Handles <code>POST /api/v1/annotate:</code> validates input data (CSV or JSON), creates an annotate job with serialized parameters in <code>params_json</code> , and returns the job ID for progress tracking.
<code>src/handlers/citation.rs</code>	public	Six citation verification handlers: <code>create</code> (parses LaTeX/BibTeX, matches PDFs, assigns batches, inserts citation rows), <code>claim</code> (atomic batch claiming with stale reclaim), <code>submit</code> (accepts verification results, auto-completes job), <code>status</code> (aggregates counts, verdicts, alerts), <code>export</code> (generates annotation CSV, corrections JSON, summary report, triggers annotation pipeline), and <code>auto_verify</code> (starts a background citation agent that uses a local LLM via the <code>LlmBackend</code> trait to verify all pending rows autonomously).

Module Path	Visibility	Responsibility
<code>src/handlers/chunks.rs</code>	public	Handles GET <code>/api/v1/sessions/{id}/files/{fid}/chunks</code> : paginated chunk browsing for a file with optional page-number filtering.
<code>src/handlers/quality.rs</code>	public	Handles GET <code>/api/v1/sessions/{id}/quality</code> : extraction quality report with per-file flags and extraction method distribution summary.
<code>src/handlers/compare.rs</code>	public	Handles POST <code>/api/v1/files/compare</code> : cross-session file comparison by path or name pattern.
<code>src/handlers/discover.rs</code>	public	Handles POST <code>/api/v1/discover</code> : directory-level overview with filesystem scan, session listing, and unindexed PDF detection.
<i>Middleware</i>		
<code>src/middleware/mod.rs</code>	public	Declares middleware submodules for authentication, CORS, and security headers.
<code>src/middleware/auth.rs</code>	public	Extracts and validates the Bearer token from the Authorization header; bypassed when the server is bound to localhost.
<code>src/middleware/cors.rs</code>	public	Configures CORS response headers: omitted for localhost binding, wildcard or explicit origin list for LAN binding.
<code>src/middleware/security_headers.rs</code>	public	Tower middleware layer that appends defensive response headers (<code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , <code>Content-Security-Policy</code>) to every response.
<code>src/middleware/api_version.rs</code>	public	Tower middleware layer that appends <code>X-API-Version: v1</code> to every response. Registered in <code>router.rs</code> alongside <code>SecurityHeadersLayer</code> so that every endpoint advertises the active API version without per-handler boilerplate.
<i>Service Layer</i>		

Module Path	Visibility	Responsibility
<code>src/service/mod.rs</code>	public	Module declaration for the service layer, which holds business logic shared between REST handlers and MCP handlers. Centralising shared logic here prevents duplication across the two handler trees.
<code>src/service/index.rs</code>	public	Provides the shared <code>resolve_vector_dimension</code> function used by both the REST index handler and the MCP index handler. Returns the vector dimension for a given model identifier by querying the model catalog, eliminating the previous duplication between the two handler trees.
<i>Indexing Pipeline</i>		
<code>src/indexer/mod.rs</code>	public	Thin shim re-exporting the complete public API from <code>neuroncite-pipeline::indexer</code> . Sub-module shims (<code>batch</code> , <code>pipeline</code> , <code>stdio</code>) preserve qualified path resolution for intra-crate callers.
<code>src/indexer/batch.rs</code>	public	Glob re-export shim for <code>neuroncite_pipeline::indexer::batch</code> . Exposes batch size computation, length-sorted permutation, and byte conversion utilities under the <code>crate::indexer::batch</code> path so that test modules inside <code>neuroncite-api</code> continue to resolve without modification.
<code>src/indexer/pipeline.rs</code>	public	Glob re-export shim for <code>neuroncite_pipeline::indexer::pipeline</code> . Exposes PDF extraction, chunking, embedding, and database insertion functions under the <code>crate::indexer::pipeline</code> path for intra-crate callers.

Module Path	Visibility	Responsibility
<code>src/indexer/stdio.rs</code>	public	Glob re-export shim for <code>neuroncite_pipeline::indexer::stdio</code> . Exposes stdout suppression utilities and the MCP writer flag under the <code>crate::indexer::stdio</code> path for intra-crate callers.
<code>src/html_indexer.rs</code>	public	Two-phase indexing pipeline for HTML web pages, mirroring the PDF indexer architecture. Phase 1 reads cached HTML from disk, splits content into heading-based sections via <code>neuroncite-html</code> , applies the configured chunk strategy, and computes a SHA-256 content hash. Phase 2 embeds chunk text and stores vectors, page records, and <code>web_source</code> metadata in the database. Provides <code>extract_and_chunk_html</code> (CPU-bound), <code>embed_and_store_html</code> (synchronous), and <code>embed_and_store_html_async</code> (via <code>WorkerHandle</code>).
<code>src/executor.rs</code>	public	Thin wrapper shim that re-exports the background job executor from <code>neuroncite-pipeline</code> . Provides <code>spawn_job_executor</code> , <code>load_all_session_hnsw</code> , <code>ensure_hnsw_for_session</code> , and <code>ensure_hnsw_for_session_async</code> with <code>Arc<AppState></code> signatures, coercing to <code>Arc<dyn PipelineContext></code> internally.
<i>GPU Worker</i>		
<code>src/worker.rs</code>	private	Thin shim re-exporting <code>WorkerHandle</code> and <code>spawn_worker</code> from <code>neuroncite_pipeline::worker</code> . All intra-crate paths (<code>crate::worker::WorkerHandle</code>) resolve through this shim without modification.
<code>src/dto.rs</code>	private	Defines JSON request and response structs with <code>serde</code> serialization and <code>utoipa</code> OpenAPI schema annotations.

Module Path	Visibility	Responsibility
<code>src/openapi.rs</code>	private	Assembles the <code>utoipa</code> OpenAPI 3.1 specification from all annotated handler functions, DTO types, and error schemas.
<code>src/agent.rs</code>	public	Autonomous citation verification agent driven by a local LLM via the <code>LlmBackend</code> trait. Claims batches from the citation job, prompts the LLM to generate search queries, executes the search pipeline (embedding + HNSW + FTS5), reads page content, prompts the LLM for a structured verdict (JSON), and submits results back to the database. Emits SSE events for per-row phase transitions, streaming reasoning tokens, and aggregate job progress. Runs as a fire-and-forget <code>tokio::spawn</code> background task started by the <code>auto_verify</code> handler.
<code>src/error.rs</code>	private	Defines <code>ApiError</code> and implements axum <code>IntoResponse</code> to map each variant to the appropriate HTTP status code and JSON error body.
<code>src/util.rs</code>	private	Internal path validation utilities. <code>validate_path_containment</code> verifies that a user-supplied file path resolves to a location within one of the allowed session root directories, rejecting directory traversal attempts. Used by the annotate and citation handlers.
<i>Test Infrastructure</i>		
<code>src/test_support.rs</code>	private	Shared test utilities compiled only under <code>#[cfg(test)]</code> : stub embedding backend, stub reranker, vector helpers, AppState constructors with in-memory database, and Axum request dispatching helpers.

Table 10: Internal module structure of `neuroncite-api`.

Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Declares the four public submodules (<code>context</code> , <code>executor</code> , <code>indexer</code> , <code>worker</code>) and re-exports the primary public API items. The <code>test_support</code> module is declared <code>pub(crate)</code> and compiled only under <code>#[cfg(test)]</code> .
<code>src/context.rs</code>	public	Defines the <code>PipelineContext</code> trait with 12 methods providing access to the database pool, GPU worker handle, HNSW index map, progress tracker, configuration, and SSE channels. Decouples the pipeline from the concrete <code>AppState</code> type in <code>neuroncite-api</code> and enables independent testing via stub implementations.
<code>src/executor.rs</code>	public	Background job executor: <code>spawn_job_executor</code> drives the job queue loop, <code>load_all_session_hnsw</code> rebuilds all HNSW indices from stored embeddings on startup, and <code>ensure_hnsw_for_session</code> loads or rebuilds the HNSW index for a single session on demand. Generic over <code>Arc<dyn PipelineContext></code> .
<code>src/worker.rs</code>	public	GPU worker management: <code>WorkerHandle</code> holds priority channels for embedding and reranking requests, and <code>spawn_worker</code> starts the dedicated worker thread that owns the <code>EmbeddingBackend</code> and processes requests via message passing.
<i>Indexing Pipeline</i>		
<code>src/indexer/mod.rs</code>	public	Re-exports <code>ExtractionResult</code> , <code>ExtractedFile</code> , and <code>FileIndexResult</code> and all public items from the <code>batch</code> , <code>pipeline</code> , and <code>stdio</code> sub-modules so that callers import from a single path.

Module Path	Visibility	Responsibility
<code>src/indexer/batch.rs</code>	public	Adaptive batch size computation (<code>compute_batch_size</code>) based on sequence length, length-sorted permutation (<code>length_sorted_permutation</code> , <code>restore_original_order</code>) for throughput, and byte-vector conversion utilities (<code>f32_slice_to_bytes</code> , <code>bytes_to_f32_vec</code>).
<code>src/indexer/pipeline.rs</code>	public	Two-phase indexing pipeline: <code>extract_and_chunk_file</code> (Phase 1, CPU-bound, parallelizable via rayon) reads and chunks a PDF; <code>embed_and_store_file</code> (Phase 2, synchronous) and <code>embed_and_store_file_async</code> (Phase 2, routes via <code>WorkerHandle</code>) embed chunks and persist vectors, page records, and file metadata to SQLite.
<code>src/indexer/stdio.rs</code>	public	Stdout suppression for parallel rayon PDF extraction on Windows: <code>suppress_stdout</code> redirects the CRT file descriptor for stdout to <code>/dev/null</code> (or <code>NUL</code> on Windows), <code>restore_stdout</code> undoes the redirect. <code>StdoutSuppressionGuard</code> provides RAII-based suppression. Used to silence noisy third-party PDF crates during background indexing without affecting the MCP stdio transport.
<i>Test Infrastructure</i>		
<code>src/test_support.rs</code>	private	<code>StubBackend</code> test double that implements <code>EmbeddingBackend</code> with deterministic fixed-dimension embeddings. Used by pipeline unit tests in <code>executor.rs</code> and <code>worker.rs</code> to avoid loading real ONNX models. Compiled only under <code>#[cfg(test)]</code> .

Table 11: Internal module structure of neuroncite-pipeline.

Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Declares submodules and re-exports <code>build_web_router</code> , <code>WebState</code> , and <code>open_browser</code> . Contains the router assembly that merges the existing API router with web-specific handlers, SSE endpoints, and embedded static assets.
<code>src/assets.rs</code>	public	Embedded static file serving via <code>rust-embed</code> . The compiled SolidJS frontend from <code>frontend/dist/</code> is baked into the binary at compile time. Serves files with correct MIME types and falls back to <code>index.html</code> for SPA client-side routing.
<code>src/sse.rs</code>	public	Server-Sent Event endpoints for real-time streaming. Registers six SSE streams: <code>/logs</code> (log messages), <code>/progress</code> (indexing progress), <code>/jobs</code> (job state transitions), <code>/models</code> (model operations), <code>/citation</code> (citation verification events), and <code>/sources</code> (source fetching events). Each stream is backed by a <code>tokio::sync::broadcast</code> channel in <code>WebState</code> . Includes a 15-second keep-alive ping interval.
<code>src/broadcast_layer.rs</code>	public	Tracing <code>Layer</code> implementation that forwards log events to a <code>tokio::sync::broadcast</code> channel for SSE log streaming to the web frontend. Each log event is formatted as a JSON string with level, target, and message fields.
<code>src/browser.rs</code>	public	Cross-platform browser launcher that opens the default browser to the server URL after startup via the <code>opener</code> crate. Prints the URL to the console as fallback when the browser cannot be opened.

Module Path	Visibility	Responsibility
<code>src/native_window.rs</code>	public	Native OS window creation via <code>tao</code> (event loop) and <code>wry</code> (WebView). Shows a transparent PNG splash screen (via <code>splash.rs</code>) during startup, then transitions to a platform-native application window (WebView2 on Windows, WebKit on macOS/Linux) pointing at the local Axum server URL. The <code>tao</code> event loop runs on the main thread while the Axum server runs in a background <code>tokio</code> thread. Compiled only when the <code>gui</code> feature is enabled.
<code>src/splash.rs</code>	public	Falls back to browser launch on failure. Transparent splash screen rendering via platform-native APIs. Embeds a pre-rendered PNG (<code>assets/splash.png</code>) via <code>include_bytes!</code> and renders it onto a <code>tao</code> window using Win32 layered windows (<code>UpdateLayeredWindow</code>) on Windows, <code>NSImageView</code> on macOS, and Cairo surfaces on Linux. Compiled only when the <code>gui</code> feature is enabled. Called from <code>native_window.rs</code> during startup to display a frameless transparent splash while the Axum server initializes.
<i>Web-specific REST handlers under <code>/api/v1/web/</code></i>		
<code>src/handlers/mod.rs</code>	public	Declares handler submodules and assembles the Axum router for all web-specific endpoints.
<code>src/handlers/browse.rs</code>	public	File system browsing and recursive document discovery. Provides directory listing (<code>/browse</code>) with drive enumeration on Windows, native OS directory picker (<code>/browse/native</code>), native file picker (<code>/browse/native-file</code>), and document scanning (<code>/scan-documents</code>) with metadata collection.

Module Path	Visibility	Responsibility
<code>src/handlers/models.rs</code>	public	Model catalog endpoint returning embedding and reranker models with cached/active status from the static registry in <code>neuroncite-embed</code> . Includes placeholder endpoints for model download and activation.
<code>src/handlers/doctor.rs</code>	public	Dependency probe handlers that check for <code>pdfium</code> (shared library), <code>tesseract</code> (CLI executable), and <code>poppler/pdftotext</code> (CLI executable). Includes a placeholder install endpoint for <code>pdfium</code> .
<code>src/handlers/model_doctor.rs</code>	public	Model health diagnostics and repair handlers. <code>model_diagnostics</code> returns file-level cache state for all models in the static registry. <code>repair_model</code> purges a broken model cache directory and re-downloads all files from HuggingFace.
<code>src/handlers/config.rs</code>	public	Configuration reading endpoint that serializes the <code>AppConfig</code> from <code>neuroncite-core</code> for browser display.
<code>src/handlers/mcp.rs</code>	public	MCP server registration status check, install, and uninstall handlers. Reads the Claude Desktop configuration file to determine registration status and spawns the <code>neuroncite mcp install/uninstall</code> subprocess for registration changes.
<code>src/handlers/ollama.rs</code>	public	Ollama proxy handlers for the web frontend. Seven endpoints: <code>ollama_status</code> (connectivity check with version info), <code>list_ollama_models</code> (installed model list via <code>/api/tags</code>), <code>list_running_models</code> (currently loaded models), <code>ollama_catalog</code> (available model catalog), <code>pull_model</code> (download a model), <code>delete_model</code> (remove a model), and <code>show_model</code> (model metadata and parameters).

Module Path	Visibility	Responsibility
<code>src/handlers/setup.rs</code>	public	First-run setup dialog state handlers. <code>get_setup_status</code> checks whether the <code>.setup_complete</code> marker file exists in the NeuronCite data directory and returns the <code>is_first_run</code> flag with the data directory path. <code>mark_setup_complete</code> creates the marker file to suppress the welcome dialog on future launches.
<code>src/handlers/update.rs</code>	public	Update check handler. Queries the GitHub Releases API to determine whether a newer version of NeuronCite is available. Returns the current version, latest version, an <code>update_available</code> flag, and a link to the GitHub release page. Does not download or install updates.
<code>src/middleware.rs</code>	public	Security response header middleware for the web router. Applies <code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: DENY</code> , and <code>Referrer-Policy: strict-origin-when-cross-origin</code> to all responses served through the web interface.

Table 12: Internal module structure of `neuroncite-web`.

Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Declares submodules and re-exports the public API: <code>HtmlError</code> , <code>WebMetadata</code> , <code>FetchResult</code> , <code>HtmlSection</code> , <code>CrawlConfig</code> , <code>fetch</code> functions, <code>parse</code> functions, and <code>crawl</code> functions.
<code>src/error.rs</code>	public	Defines <code>HtmlError</code> with variants for HTTP failures, URL parsing, I/O errors, HTML parsing, crawling, caching, invalid arguments, and regex compilation.

Module Path	Visibility	Responsibility
src/types.rs	public	Domain types: WebMetadata (URL, canonical URL, title, Open Graph tags, author, language, domain, HTTP status, fetch timestamp), FetchResult (URL, raw HTML, metadata, cache path), HtmlSection (section index, heading text, heading level, content, byte offsets), and CrawlConfig (start URL, max depth, domain filter, URL pattern, max pages, request delay, sitemap mode, boilerplate stripping).
src/fetch.rs	public	HTTP fetching with disk caching and URL normalization. build_http_client constructs a request::Client with a 30-second timeout, 10-redirect limit, and NeuronCite User-Agent. fetch_url downloads a page and writes the raw HTML to a SHA-256-named cache file; when the HTTP response is a redirect, the content is written under both the final URL hash and the original URL hash so that callers passing either URL to cache_path_for_url find the cached file. fetch_urls fetches multiple URLs sequentially with a configurable delay. cache_path_for_url normalizes the URL via normalize_cache_url before SHA-256 hashing, stripping tracking parameters (UTM, fbclid, gclid, msclkid), error parameters (error, code), and fragment identifiers to produce stable cache keys independent of dynamic query strings. default_cache_dir returns <code>~/.neuroncite/html_cache/</code> .

Module Path	Visibility	Responsibility
src/parse.rs	public	HTML text extraction and section splitting. <code>extract_visible_text</code> strips all tags and returns visible text preserving paragraph structure. <code>extract_main_content</code> applies a readability algorithm that scores DOM nodes by text density and penalizes elements with nav/sidebar/footer/ad class names. <code>extract_metadata</code> reads <code><title></code> , <code><meta></code> , <code><link rel="canonical"></code> , and Open Graph tags. <code>split_into_sections</code> segments content at H1/H2 heading boundaries, producing <code>HtmlSection</code> values with byte offsets. <code>sections_to_pages</code> converts sections into <code>PageText</code> values with 1-based page numbers. <code>extract_links</code> collects and deduplicates <code><a href></code> URLs, resolving relative paths.
src/crawl.rs	public	BFS website crawling and sitemap parsing. <code>crawl</code> performs breadth-first traversal from a start URL, following links up to <code>max_depth</code> with same-domain filtering and optional regex URL matching. Respects <code>max_pages</code> and <code>delay_ms</code> between requests. <code>fetch_sitemap</code> downloads and parses <code>sitemap.xml</code> to discover URLs. <code>normalize_url</code> standardizes URLs for deduplication (lowercase scheme/host, remove fragment, remove trailing slash, sort query parameters).

Module Path	Visibility	Responsibility
src/download.rs	public	PDF download and URL classification. <code>classify_url</code> sends an HTTP HEAD request and inspects the Content-Type header and URL path extension to determine whether a URL serves a PDF or an HTML page (returns <code>UrlSourceType::Pdf</code> or <code>Html</code>). <code>download_pdf</code> downloads a file via GET, validates the Content-Type and %PDF- magic bytes, and writes the content to disk. <code>sanitize_filename</code> replaces filesystem-invalid characters and truncates to 200 characters.
src/ssrf.rs	public	Server-Side Request Forgery (SSRF) protection module. <code>validate_url_no_ssrf</code> parses a URL and rejects requests targeting internal network addresses: IPv4 loopback (127.0.0.0/8), RFC 1918 private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), link-local (169.254.0.0/16), IPv6 loopback (::1), IPv6 link-local (fe80::/10), and IPv4-mapped IPv6 addresses embedding blocked ranges. Called at the entry point of <code>fetch_url</code> , <code>crawl</code> , <code>download_pdf</code> , and <code>fetch_sitemap</code> . 44 unit tests covering all blocked ranges, boundary addresses, and IPv4-mapped IPv6 bypass attempts.
src/resolve.rs	public	Multi-source DOI resolution chain for obtaining direct open-access PDF URLs. Queries three free academic APIs in sequence: Unpaywall (requires email, no account), Semantic Scholar, and OpenAlex, with a 10-second timeout per API call. Short-circuits on first success. Falls back to <code>https://doi.org/{doi}</code> when all APIs fail. Pure parser functions (<code>parse_unpaywall_response</code> , <code>parse_semantic_scholar_response</code> , <code>parse_openalex_response</code>) are testable without network access. 21 unit tests.

Module Path	Visibility	Responsibility
Table 13: Internal module structure of <code>neuroncite-html</code>. The crate depends only on <code>neuroncite-core</code> for shared types and uses <code>request</code> for HTTP, <code>scraper</code> (html5ever) for DOM parsing, and <code>url</code> for URL manipulation.		
Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Declares submodules and gates the crate with <code>#![forbid(unsafe_code)]</code> and <code>#![deny(warnings)]</code> .
<code>src/protocol.rs</code>	public	Defines JSON-RPC 2.0 message types: <code>JsonRpcRequest</code> , <code>JsonRpcResponse</code> , <code>JsonRpcError</code> . Provides serialization/deserialization for the wire format.
<code>src/transport.rs</code>	public	Line-based stdin/stdout reader and writer. <code>StdioReader</code> reads complete lines from stdin, <code>StdioWriter</code> serializes responses to stdout. Both operate on synchronous I/O; the server loop drives them sequentially.
<code>src/tools.rs</code>	public	Static MCP tool definitions. Each <code>ToolDefinition</code> contains the tool name, a human-readable description, and a JSON Schema for the input parameters. The <code>all_tools()</code> function returns the complete catalog of forty-three tools (core search, indexing, annotation, and citation tools plus HTML web scraping tools).
<code>src/dispatch.rs</code>	public	Routes <code>tools/call</code> requests to handler functions. Maps each tool name string to the corresponding handler in the <code>handlers</code> module. Returns a JSON-RPC error for unrecognized tool names.
<code>src/server.rs</code>	public	The main server loop. Performs the MCP <code>initialize</code> handshake, responds to <code>tools/list</code> with tool definitions, dispatches <code>tools/call</code> to the dispatch layer, and handles <code>notifications/cancelled</code> and shutdown.

Module Path	Visibility	Responsibility
src/registration.rs	public	Install/uninstall/status logic for the MCP server entry in Claude Code's configuration file (<code>~/.claude/claude_mcp_settings.json</code>). Reads/writes JSON with the server command and arguments.
<i>Tool handlers</i>		
src/handlers/mod.rs	public	Declares handler submodules. All handlers share the same signature: <code>async fn(&Arc<AppState>, &serde_json::Value) -> Result<Value, String></code> .
src/handlers/search.rs	public	Handles <code>neuroncite_search</code> : loads the HNSW index, executes vector/hybrid search, and returns ranked passages with citations and scores.
src/handlers/index.rs	public	Handles <code>neuroncite_index</code> : creates a session and job, spawns the indexing pipeline (PDF extraction, chunking, embedding), and returns the job ID.
src/handlers/batch_search.rs	public	Handles <code>neuroncite_batch_search</code> : executes multiple search queries in a single call with shared HNSW guard and database connection.
src/handlers/files.rs	public	Handles <code>neuroncite_files</code> : lists all indexed PDF files in a session with <code>file_id</code> , path, page count, and size.
src/handlers/sessions.rs	public	Handles <code>neuroncite_sessions</code> (list all sessions), <code>neuroncite_session_delete</code> (cascade delete by session ID or by directory path, with HNSW eviction), and <code>neuroncite_session_update</code> (set or clear the session label).
src/handlers/jobs.rs	public	Handles <code>neuroncite_jobs</code> (list all jobs) and <code>neuroncite_job_status</code> (retrieve a single job's state, progress, and timestamps).
src/handlers/export.rs	public	Handles <code>neuroncite_export</code> : formats the most recent search results as Markdown, BibTeX, or CSL-JSON citation output.

Module Path	Visibility	Responsibility
<code>src/handlers/models.rs</code>	public	Handles <code>neuroncite_models</code> : lists all supported embedding models with their dimensions, pooling strategies, and download cache status.
<code>src/handlers/common.rs</code>	public	Shared utility functions for content handlers: <code>truncate_content</code> (truncates at a UTF-8 boundary at 100 KB) and <code>content_part_to_json</code> (builds the JSON response object for a single content part with part number, extraction backend, and optional truncation metadata). Used by both <code>content.rs</code> and <code>batch_content.rs</code> .
<code>src/handlers/content.rs</code>	public	Handles <code>neuroncite_content</code> : retrieves the full text content of a specific part from the database for context expansion around search results. A “part” is the logical content unit of the source format: a page in PDFs, a heading-based section in HTML. Validates parameters before any DB access. Enriches HTML sources with <code>web_source</code> metadata.
<code>src/handlers/system.rs</code>	public	Handles <code>neuroncite_doctor</code> and <code>neuroncite_health</code> : returns runtime diagnostics including GPU availability, loaded backend, database statistics, and feature flags.
<code>src/handlers/annotate.rs</code>	public	Handles <code>neuroncite_annotate</code> : validates annotation input (CSV or JSON), creates an annotate job with serialized parameters, and returns the job ID and quote count.
<code>src/handlers/citation.rs</code>	public	Five citation verification tool handlers: <code>handle_create</code> (parses LaTeX/BibTeX, matches PDFs via Jaro-Winkler, assigns batches, creates job), <code>handle_claim</code> (atomic batch claiming), <code>handle_submit</code> (accepts results, auto-completes), <code>handle_status</code> (counts, verdicts, alerts), and <code>handle_export</code> (generates output files, triggers annotation).

Module Path	Visibility	Responsibility
<code>src/handlers/chunks.rs</code>	public	Handles <code>neuroncite_chunks</code> : browses chunks of a file with pagination and optional page-number filtering. Returns chunk content, page range, word count, and byte count without requiring a search query.
<code>src/handlers/quality.rs</code>	public	Handles <code>neuroncite_quality_report</code> : generates text extraction quality flags per file (incomplete extraction, OCR-heavy, low text density, empty pages). Classifies files by extraction method (native, OCR, mixed).
<code>src/handlers/compare.rs</code>	public	Handles <code>neuroncite_file_compare</code> : finds the same PDF across different sessions by path pattern, compares chunk counts, content hashes, and chunking strategies to evaluate configuration differences.
<code>src/handlers/discover.rs</code>	public	Handles <code>neuroncite_discover</code> : scans a directory for PDFs, queries all sessions for that directory, computes unindexed files and per-session aggregate statistics.
<code>src/handlers/inspect.rs</code>	public	Handles <code>neuroncite_inspect_annotations</code> : reads annotation properties from a PDF file via <code>pdfium</code> . Returns type, <code>stroke_color (/C)</code> , <code>fill_color (/IC)</code> , alpha, and bounding rectangle per annotation.
<code>src/handlers/ batch_content.rs</code>	public	Handles <code>neuroncite_batch_content</code> : retrieves content parts from multiple files in a single call with partial failure reporting. Enforces limits on batch size (10 requests) and total parts (20). Truncates large content at 100 KB. Enriches HTML sources with <code>web_source</code> metadata.

Module Path	Visibility	Responsibility
<code>src/handlers/compare_search.rs</code>	public	Handles <code>neuroncite_compare_search</code> : runs the same query against two sessions and returns side-by-side results with per-session average vector scores for comparing search quality across different chunking strategies or embedding models.
<code>src/handlers/reindex.rs</code>	public	Handles <code>neuroncite_reindex_file</code> : re-indexes a single PDF file within an existing session. Deletes old file record (CASCADE), re-extracts, re-chunks, re-embeds, and rebuilds the HNSW index for the session.
<code>src/handlers/index_add.rs</code>	public	Handles <code>neuroncite_index_add</code> : incrementally adds or updates specific PDF files in an existing session. Uses two-stage change detection (mtime/size + SHA-256) to skip unchanged files. Extracts, chunks, embeds, and rebuilds the HNSW index once at the end.
<code>src/handlers/preview_chunks.rs</code>	public	Handles <code>neuroncite_preview_chunks</code> : stateless chunking preview that extracts pages from a single PDF, applies the requested strategy, and returns the first N chunks without database writes.
<code>src/handlers/multi_search.rs</code>	public	Handles <code>neuroncite_multi_search</code> : searches across 2–10 index sessions simultaneously. Embeds the query once, runs the search pipeline per session, and merges results into a single ranked list with session_id attribution.
<code>src/handlers/reranker.rs</code>	public	Handles <code>neuroncite_reranker_load</code> : loads a cross-encoder reranker model at runtime and hot-swaps it into the GPU worker via <code>ArcSwap</code> . Activates the <code>rerank</code> parameter in search tools without server restart.

Module Path	Visibility	Responsibility
src/handlers/ annotate_status.rs	public	Handles <code>neuroncite_annotate_status</code> : returns per-quote progress for annotation jobs including status (pending/matched/not_found/error), match method, page number, and PDF filename. Validates the job exists and is an annotate job. Supports pagination (limit/offset). Queries the <code>annotation_quote_status</code> table.
src/handlers/remove.rs	public	Handles <code>neuroncite_annotation_remove</code> : parses parameters (<code>pdf_path</code> , <code>mode</code> , <code>output_path</code> , <code>colors</code> , <code>pages</code> , <code>dry_run</code>). Delegates to <code>neuroncite_annotate::remove_highlights</code> via <code>spawn_blocking</code> . Runs in a blocking task because <code>lopdf</code> performs synchronous file I/O.
src/handlers/ session_diff.rs	public	Handles <code>neuroncite_session_diff</code> : compares two index sessions to report file differences. Validates both sessions exist. Returns four categories: <code>only_in_a</code> , <code>only_in_b</code> , <code>identical</code> , <code>changed</code> . Delegates to <code>neuroncite_store::diff_sessions</code> .
src/handlers/ text_search.rs	public	Handles <code>neuroncite_text_search</code> : performs literal substring search within indexed PDF page text stored in the database. Validates the file exists. Returns matching pages with context. Delegates to <code>neuroncite_store::search_page_text</code> .
<i>HTML web scraping and indexing tool handlers</i>		
src/handlers/ html_fetch.rs	public	Handles <code>neuroncite_html_fetch</code> : fetches one or more URLs via HTTP GET, caches raw HTML to disk, extracts metadata (title, author, Open Graph tags). Accepts <code>urls</code> array or single <code>url</code> parameter.

Module Path	Visibility	Responsibility
src/handlers/ html_crawl.rs	public	Handles <code>neuroncite_html_crawl</code> : crawls a website from a start URL with configurable depth, same-domain filtering, URL pattern matching, and sitemap-based discovery. Returns metadata and cache paths for all fetched pages.
<i>Citation source acquisition</i>		
src/handlers/ source_fetch.rs	public	Handles <code>neuroncite_citation_fetch_sources</code> : parses a BibTeX file, extracts <code>url</code> and <code>doi</code> fields, classifies each URL as PDF or HTML via HEAD request, downloads PDFs to disk, fetches HTML pages via the cache pipeline, and indexes all acquired files into the specified session. Filters out bot-detection and Cloudflare challenge pages via <code>check_blocked_page</code> (word count below 50 plus title/content pattern match). Operates in four phases: fetch and classify (with blocked page filtering), PDF indexing (extract/chunk/embed via the PDF pipeline), HTML indexing (section splitting/chunk/embed via <code>html_indexer</code>), rebuild HNSW.
src/handlers/ bib_report.rs	public	Handles <code>neuroncite_bib_report</code> : parses a BibTeX file, checks file existence in the output directory, and generates CSV and XLSX report files listing all entries with <code>cite_key</code> , title, author, year, link type (URL/DOI/none), download status (exists/missing/no_link), and the existing filename. The XLSX workbook includes a formatted data sheet with status-colored rows, badge column, and a summary sheet with aggregate statistics.

Table 14: Internal module structure of `neuroncite-mcp`.

Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Module root, re-exports <code>annotate_pdfs</code> , <code>parse_input</code> , <code>detect_format</code> , <code>inspect_pdf</code> , <code>inspect_pdf_with_pdfium</code> , <code>AnnotateConfig</code> , <code>AnnotationDetail</code> , <code>InspectionResult</code> , <code>InputFormat</code> , and <code>InputRow</code> .
<code>src/error.rs</code>	public	<code>AnnotateError</code> enum with variants covering input parsing, PDF discovery, quote location, annotation creation, save, and verification failures.
<code>src/types.rs</code>	public	Core types: <code>AnnotateConfig</code> , <code>InputRow</code> , <code>InputFormat</code> , <code>Color</code> , <code>MatchMethod</code> , <code>MatchResult</code> , <code>PreCheckResult</code> , <code>PostCheckResult</code> .
<code>src/input.rs</code>	private	CSV and JSON input parsing with automatic format detection. Validates required fields (title, author, quote) and optional fields (color as <code>#RRGGBB</code> hex, comment text).
<code>src/resolve.rs</code>	public	Fuzzy PDF filename matching. Extracts title and author from filenames via regex, computes Jaro-Winkler similarity with weighted scoring ($0.6 \times \text{title} + 0.4 \times \text{author}$), and groups input rows by matching PDF file. Threshold: 0.80. Public because the MCP annotate handler calls <code>match_pdfs_to_quotes</code> directly for dry-run validation.
<code>src/locate.rs</code>	private	5-stage text location pipeline. Stage 1: exact <code>str::find()</code> on pdfium-extracted page text. Stage 2: normalized match (Unicode NFC, soft-hyphen removal, whitespace collapse). Stage 3: fuzzy match via Levenshtein sliding window ($\geq 90\%$ threshold). Stage 3.5: fallback extraction — re-extracts page text using the neuroncite-pdf multi-backend pipeline and re-attempts stages 1–3 on the fresh text. Stage 4: OCR fallback via Tesseract hOCR bounding boxes (feature-gated).

Module Path	Visibility	Responsibility
src/annotate.rs	private	Creates pdfium highlight and text annotations. Merges per-character bounding boxes into line-level rectangles. Uses <code>set_bounds()</code> , <code>set_stroke_color()</code> (writes the <code>/C</code> entry for PDF viewer display), and <code>set_fill_color()</code> (writes the <code>/IC</code> entry for compatibility).
src/appearance.rs	public	lopdf-based post-processing that injects <code>/AP</code> (Appearance Stream) entries into highlight annotations. pdfium-render creates highlights with <code>/Rect</code> , <code>/QuadPoints</code> , <code>/C</code> , <code>/IC</code> but without <code>/AP</code> . Most PDF viewers require the <code>/AP</code> <code>/N</code> Form XObject to render the colored overlay. Generates compliant Form XObject appearance streams with Multiply blend mode and 50% opacity. Runs after pdfium save and before post-check verification.
src/inspect.rs	public	Read-only PDF annotation inspector. Scans all annotations via pdfium and extracts type, <code>stroke_color (/C)</code> , <code>fill_color (/IC)</code> , alpha, and bounding rectangle. Used by the <code>neuroncite_inspect_annotations</code> MCP tool for agent-driven color verification after annotation.
src/verify.rs	public	Pre-check (validates bounding box coverage $\geq 95\%$) and post-check (verifies annotation count and area within $\pm 15\%$ tolerance after save).
src/report.rs	public	<code>AnnotationReport</code> , <code>PdfReport</code> , <code>QuoteReport</code> , <code>ReportSummary</code> , and <code>UnmatchedInput</code> structs. JSON-serializable report saved to the output directory.
src/pipeline.rs	private	Pipeline orchestration: resolves PDFs, iterates matched groups, locates quotes, creates annotations, saves results, and generates the final report.

Module Path	Visibility	Responsibility
<code>src/remove.rs</code>	public	Removes highlight annotations from PDFs using <code>lopdf</code> (pure Rust). Supports three removal modes: <code>All</code> (all highlights), <code>ByColor</code> (matching specific hex colors), <code>ByPage</code> (specific page numbers). Two-pass algorithm: pass 1 collects annotation targets by filtering on <code>/Subtype=Highlight</code> and mode criteria, pass 2 removes matching annotation <code>ObjectIds</code> from page <code>/Annots</code> arrays and deletes orphaned objects. Supports <code>dry_run</code> mode and separate <code>output_path</code> . Depends on <code>inspect.rs</code> helpers (<code>read_subtype_str</code> , <code>read_color_entry</code> , etc.) for reading annotation properties.

Table 15: Internal module structure of `neuroncite-annotate`.

Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Module root, re-exports all primary types: <code>Verdict</code> , <code>CorrectionType</code> , <code>CitationRow</code> , <code>CitationRowInsert</code> , <code>BatchClaim</code> , <code>SubmitEntry</code> , <code>CitationJobParams</code> , <code>Alert</code> , <code>ExportSummary</code> . Declares submodules.
<code>src/error.rs</code>	public	<code>CitationError</code> enum with variants covering TeX/Bib file reading, unresolved cite-keys, database errors, JSON serialization, CSV generation, and I/O failures.

Module Path	Visibility	Responsibility
<code>src/types.rs</code>	public	Data types for all pipeline stages: <code>Verdict</code> (6 variants with color mapping), <code>CorrectionType</code> (4 variants), <code>RawCitation</code> , <code>BibEntry</code> , <code>CitationRow</code> (19-column DB row), <code>CitationRowInsert</code> , <code>BatchClaim</code> , <code>SubmitEntry</code> , <code>PassageRef</code> , <code>LatexCorrection</code> , <code>StatusCounts</code> , <code>BatchCounts</code> , <code>StatusReport</code> , <code>ExportResult</code> , <code>ExportSummary</code> , <code>CitationJobParams</code> .
<code>src/latex.rs</code>	public	LaTeX citation parser. Extracts <code>\cite</code> , <code>\citep</code> , <code>\citet</code> , <code>\autocite</code> , <code>\parencite</code> , and <code>\textcite</code> commands with line numbers, anchor words, section headings, and group IDs for multi-key citations.
<code>src/bibtex.rs</code>	public	BibTeX parser. Extracts entries by cite-key with author, title, year, abstract, and keywords. Handles brace-delimited and quote-delimited field values with case-insensitive field names.
<code>src/batch.rs</code>	public	Batch assignment. Groups nearby citations by section proximity into batches of configurable size. Citations sharing a <code>group_id</code> (from the same <code>\cite{a,b,c}</code> command) are kept together.
<code>src/db.rs</code>	public	SQLite CRUD operations for the <code>citation_row</code> table: insert rows, claim the next pending batch (with 5-minute stale reclaim), submit batch results, count rows and batches by status, aggregate verdicts, and build alert entries from flagged rows.
<code>src/export.rs</code>	public	Export functions: builds annotation CSV (verdict-colored), corrections JSON sorted by <code>tex_line</code> , and a summary report JSON with statistics and alert counts.

Table 16: Internal module structure of `neuroncite-citation`.

Module Path	Visibility	Responsibility
<code>src/lib.rs</code>	public	Declares submodules and re-exports the <code>LlmBackend</code> trait, type definitions, error types, and the <code>OllamaBackend</code> implementation. Defines the <code>LlmBackend</code> trait with <code>chat_completion</code> (blocking) and <code>chat_completion_streaming</code> (token-by-token callback) methods returning <code>Pin<Box<dyn Future>></code> for object safety.
<code>src/types.rs</code>	public	Data structures for LLM communication: <code>ChatRole</code> (System, User, Assistant), <code>ChatMessage</code> , <code>LlmResponse</code> (with optional token counts), and <code>LlmConfig</code> (base URL, model name, temperature, max tokens).
<code>src/error.rs</code>	public	Error enumeration for LLM operations: <code>ConnectionFailed</code> , <code>RequestFailed</code> (with HTTP status and body), <code>ParseError</code> , <code>SchemaViolation</code> , <code>ModelNotFound</code> , and <code>Timeout</code> .
<code>src/ollama.rs</code>	public	Concrete <code>LlmBackend</code> implementation for the Ollama HTTP API. <code>chat_completion</code> sends POST <code>/api/chat</code> with <code>stream: false</code> . <code>chat_completion_streaming</code> sends POST <code>/api/chat</code> with <code>stream: true</code> and reads NDJSON line-by-line via <code>request::Response::bytes_stream()</code> , invoking the <code>on_token</code> callback per chunk. <code>list_models</code> queries GET <code>/api/tags</code> for installed model metadata.

Table 17: Internal module structure of `neuroncite-llm`.

3 Core Abstractions

The `neuroncite-core` crate defines all shared types and trait contracts that the other crates depend on. It contains primarily data structures, trait signatures, configuration types, and a unified error enumeration, with no domain-specific processing logic.

3.1 Domain Types

Domain Types	
PageText	Represents the extracted text of a single PDF page. Fields: source file path, page number (one-indexed), the raw text content as a string, and the extraction backend that produced it (<code>pdf_extract</code> , <code>pdfium</code> , or <code>ocr</code>).
Chunk	A text fragment produced by a chunking strategy. Fields: source file path, page number start (one-indexed), page number end (equal to page number start for single-page chunks, greater for chunks spanning page boundaries), chunk index within the document (zero-indexed, sequential across all pages), document text offset start and end (UTF-8 byte positions within the concatenated, normalized full-document text), the text content, and a SHA-256 content hash for deduplication.
EmbeddedChunk	A chunk together with its dense vector embedding. Fields: all fields from Chunk plus the embedding as a vector of 32-bit floats.
Citation	A structured reference to the source of a search result. Fields: source file path, file display name (without directory prefix), page number start, page number end (for chunks spanning page boundaries), document-level UTF-8 byte offset start and end (resolved to page-local positions for display), and a pre-formatted citation string (e.g., “pagan1996.pdf, p.17: <i>“Financial econometrics has emerged...”</i> ”).
SearchResult	A single ranked result returned by a search query. Fields: final ranking score (the RRF score from hybrid fusion, or the reranker score if reranking was applied – this is the primary sort key), vector similarity score (diagnostic, included for transparency), BM25 rank position (diagnostic, included for transparency; SQLite FTS5 <code>bm25()</code> returns a lower-is-better score, so this field stores the rank position rather than the raw BM25 value), reranker score (if reranking was applied), source file path, page number start, page number end, chunk index, the text content, and a Citation instance.
IndexConfig	Captures the full configuration of an indexing session. Fields: directory path, embedding model name, chunking strategy

	identifier, chunk size, chunk overlap, maximum words per chunk (sentence-based strategy), OCR language string, embedding storage mode (<code>sqlite_blob</code> or <code>mmap_file</code>), and the vector dimensionality. All fields that participate in the session uniqueness constraint (see Section 9.3) are present in this type.
IndexProgress	Reports the current state of an ongoing indexing operation. Fields: total number of files discovered, number of files processed so far, total number of chunks created, and a boolean indicating whether the operation is complete.
PoolingStrategy	Enum that determines how the final embedding vector is extracted from a transformer model’s output tensor. Three variants: CLS (extract the [CLS] token at position 0, used by BERT-family models), MeanPooling (attention-weighted mean over all non-padding tokens, used by Qwen3-Embedding and similar decoder models), LastToken (extract the embedding at the last non-padding position, reserved for causal decoder models).
EmbeddingModelConfig	Static configuration for a single embedding model. Encapsulates all parameters that differ between encoder (BERT, GTE-Large) and decoder (Qwen) architectures: model identifier, display name, vector dimension, maximum sequence length (512 for standard BERT, 8192 for GTE-Large and decoder models), pooling strategy, whether the model requires <code>token_type_ids</code> input tensor, whether the ONNX model requires explicit <code>position_ids</code> (for rotary position embeddings), and optional instruction prefix strings (<code>query_prefix</code>, <code>document_prefix</code>) for decoder models that use instruction-tuned embeddings.
ModelInfo	Describes an available embedding model. Fields: model identifier string, display name, vector dimensionality, and the backend that provides it.
ModelManifest	Tracks a downloaded model for reproducibility and offline operation. Fields: model identifier string, revision hash (the specific Hugging Face commit), SHA-256 checksum of the model weights file, source URL, license identifier (e.g., “MIT”, “Apache-2.0”), and a local file system path hint pointing to the cached model directory (re-resolved on each launch; not a hard requirement for portability). A manifest file (<code>models.lock</code>) in the <code>.neuroncite/</code> directory records all models used by the current index, analogous to <code>Cargo.lock</code> for Rust dependencies.

3.2 The Embedding Backend Trait

EmbeddingBackend Trait

The central abstraction for all machine-learning backends. The `ort` backend provides an implementation of this trait. The trait is object-safe and uses dynamic dispatch so that the web server and API can hold a reference to the active backend without knowing its concrete type at compile time.

Required methods:

name	Returns the human-readable name of the backend (e.g., “ONNX Runtime”).
vector_dimension	Returns the dimensionality of the embedding vectors produced by the currently loaded model (e.g., 384 for <code>bge-small-en-v1.5</code>).
load_model	Loads a specific model by identifier string. This may involve downloading model weights on first use and initializing a GPU session.
embed_batch	Accepts a slice of text strings and returns a vector of embedding vectors. This is the primary hot path and must leverage GPU parallelism.
embed_single	Convenience method that embeds a single text string. Delegates to <code>embed_batch</code> with a one-element slice.
supports_gpu	Returns whether this backend was compiled with GPU support and whether a compatible GPU is currently available.
available_models	Returns a list of <code>ModelInfo</code> structures describing all models this backend can load.

3.3 The Reranker Trait

Reranker Trait

Abstracts over cross-encoder models that score (query, document) pairs. Unlike the `EmbeddingBackend` trait, which produces independent vector representations, a reranker jointly attends to both query and document text and outputs a single relevance score. This produces higher-quality relevance judgments than cosine similarity alone, at the cost of being unable to pre-compute document representations.

Required methods:

name	Returns the human-readable name of the reranker model.
rerank_batch	Accepts a query string and a slice of candidate document texts. Returns a vector of relevance scores (one per candidate), where higher scores indicate stronger relevance. The reranker processes all pairs on the GPU in a single batched forward pass.
load_model	Loads a cross-encoder model by identifier string.

The embedding backend (`ort`) provides a corresponding reranker implementation

behind the same feature flag. The reranker is an optional component: if no reranker model is configured, the search pipeline skips the reranking stage and returns results ranked by the hybrid score alone.

3.4 The Chunk Strategy Trait

ChunkStrategy Trait

Abstracts over different text splitting approaches. Each strategy receives a list of `PageText` values (the full extracted text of a document, page by page) and produces a list of `Chunk` values.

Required method:

chunk Accepts a slice of `PageText` and returns a vector of `Chunk`. The implementation determines how text is split and what metadata is attached. Each chunk records its UTF-8 byte offset range within the concatenated, normalized full-document text, and the page range it spans.

Provided strategies:

Page-based Each PDF page becomes one chunk. The simplest strategy, suitable for documents where page boundaries carry semantic meaning.

Word-window A sliding window of configurable word count with configurable overlap. Parameters: *window_size* (number of words per chunk) and *overlap* (number of words shared between consecutive chunks).

Token-window A sliding window measured in model tokens rather than whitespace-separated words. Uses the tokenizer of the active embedding model to count tokens, ensuring that each chunk fits within the model's maximum sequence length. Parameters: *token_limit* and *token_overlap*. Produces deterministic chunk boundaries for a given model and text.

Sentence-based Splits text at sentence boundaries using the `unicode-segmentation` crate. Groups consecutive sentences until a configurable maximum word count is reached. Uses an abbreviation whitelist to avoid false sentence breaks.

4 PDF Extraction Pipeline

The `neuroncite-pdf` crate is responsible for three tasks: discovering PDF files in a directory tree, extracting their textual content through multiple backends, and triggering OCR for pages where text extraction fails.

4.1 File Discovery

Given a root directory path, the crate recursively traverses all subdirectories and collects every file with a `.pdf` extension (case-insensitive). The traversal uses Rust’s standard library directory iterator and filters by extension. Symbolic links are followed, with cycle detection via a visited-inode set (on Linux) or canonicalized-path set (on Windows) to prevent infinite loops caused by circular symlinks. Unreadable directories are skipped with a warning logged to the progress channel, but do not abort the overall discovery.

The result is an ordered list of absolute file paths, sorted alphabetically for deterministic processing order.

4.2 Extraction Backends

The crate supports two text extraction backends. The user can configure which backend to use, or leave the default automatic selection.

Primary Backend: `pdf-extract`

The `pdf-extract` crate parses PDF content streams and reconstructs text layout. It is fast, pure Rust, and requires no external dependencies. It handles the majority of well-formed PDF documents correctly. This is the default backend.

Fallback Backend: `pdfium`

The `pdfium-render` crate provides Rust bindings to Google’s PDFium library (the rendering engine used in Chromium). PDFium handles complex PDF features that `pdf-extract` struggles with: multi-column layouts, tables with cell boundaries, non-standard font encodings, and PDFs produced by certain commercial tools. The pdfium backend requires the PDFium shared library to be present at runtime. On Windows, this is `pdfium.dll`; on Linux, `libpdfium.so`. The library is bundled alongside the executable or downloaded on first use.

Automatic backend selection: When set to “auto” mode, the system first attempts extraction with `pdf-extract`. The extracted text is evaluated against multiple quality heuristics to decide whether to fall back to pdfium:

- **Alphabetic ratio:** If the ratio of alphabetic characters to total characters falls below a configurable threshold (default 0.3), the text is likely garbled due to encoding issues.

- **Replacement character ratio:** If the ratio of Unicode replacement characters (U+FFFD) and other common substitution markers to total characters exceeds a configurable threshold (default 0.05), the font encoding was not decoded correctly.
- **Line break density:** If the number of line break characters per total character count exceeds a configurable threshold (default 0.15), the layout reconstruction fragmented continuous text into single-character or single-word lines.
- **Unique token ratio:** If the ratio of unique whitespace-separated tokens to total tokens falls below a configurable threshold (default 0.05), the text consists of repetitive garbage output rather than natural language.

If any heuristic fails, the page is re-extracted using pdfium. This catches cases where `pdf-extract` returns garbled output due to encoding issues, fragmented multi-column layouts, or non-standard font mappings.

4.3 Automatic OCR Fallback

For pages where both text extraction backends produce insufficient text (fewer than 20 non-whitespace characters), the system assumes the page contains scanned or image-based content and triggers OCR:

1. The page is rendered to a raster image (PNG) at an adaptive DPI setting using pdfium’s rendering capabilities (see below).
2. The rendered PNG image is written to a temporary file and passed to the Tesseract CLI binary as a subprocess (`tesseract <file> stdout -l <lang>`). Tesseract is located on the system PATH or in the Doctor panel’s download cache. It must be installed beforehand via the system package manager or the Doctor panel’s install button (UB-Mannheim installer on Windows, AppImage on Linux).
3. The OCR output replaces the empty extraction result.
4. The `PageText` is annotated with extraction backend `ocr` so that downstream components can distinguish OCR text from directly extracted text.

Adaptive DPI: The render resolution for OCR is selected based on the page dimensions and content density. Pages with physical dimensions smaller than 150 × 200 mm (approximately A5 or smaller) are rendered at 400 DPI to capture fine detail. All other pages use the default of 300 DPI. The user can override this behavior by setting a fixed DPI value in the session configuration.

OCR language configuration: Tesseract requires trained language data files (tessdata) for each language it recognizes. The OCR language is configurable per session through the web frontend settings or the API `/api/v1/index` request field `ocr_language` (default: `eng`). Multiple languages can be specified as a `+`-separated string (e.g., `eng+deu` for English and German). The web frontend provides a dropdown populated with all `.traineddata` files found in the Tesseract data directory.

OCR Dependency

OCR requires the Tesseract engine and trained language data files to be installed on the system. If Tesseract is not available, the OCR step is skipped and the page

is included with empty text. The web frontend displays a warning listing pages that could not be processed and the reason (missing Tesseract binary or missing language data).

4.4 Text Extraction Output

Each page produces one `PageText` value containing:

- The absolute path to the source PDF file
- The one-indexed page number
- The extracted text as a UTF-8 string
- The extraction backend identifier (`pdf_extract`, `pdfium`, or `ocr`)

Pages that yield empty text after all backend attempts are included in the output with an empty string. This preserves correct page numbering for subsequent chunking and result display.

4.5 Error Handling

If a PDF file cannot be opened (corrupted, password-protected, or unsupported format), the error is recorded and the file is skipped. Processing continues with the remaining files. The final extraction summary reports the count of successfully processed files and the count of skipped files with their error reasons.

5 PDF Annotation Pipeline

The `neuroncite-annotate` crate provides a pipeline for highlighting quoted text passages in PDF documents and adding popup comment annotations. It operates standalone without requiring an indexed session. The pipeline takes structured input (CSV or JSON) containing paper titles, authors, and verbatim quotes, resolves them to PDF files in a source directory, locates the quoted passages, and creates pdfium annotations on the matching regions.

5.1 Architecture Overview

The annotation pipeline consists of six sequential stages:

1. **Input Parsing:** CSV or JSON input is parsed into a list of `InputRow` structures. Format detection is automatic: if the first non-whitespace byte is `[` or `{`, JSON is assumed; otherwise CSV. Required fields: `title`, `author`, `quote`. Optional: `color` (`#RRGGBB` hex, default `#FFFF00` yellow) and `comment` (popup text).
2. **PDF Resolution:** Input rows are matched to PDF files via Jaro-Winkler similarity on extracted title and author metadata from filenames. The filename pattern `Title (Author, Year).pdf` is decomposed via regex. The combined score uses weights $0.6 \times \text{title_score} + 0.4 \times \text{author_score}$ with a minimum threshold of 0.80. Multiple quotes can target the same PDF.
3. **Text Location:** A 5-stage pipeline locates each quote in the PDF text (described below).
4. **Annotation Creation:** Highlight annotations with configurable color and optional popup text annotations are created via pdfium-render's annotation API.
5. **Verification:** Pre-check validates bounding box coverage; post-check verifies annotations in the saved file.
6. **Report Generation:** A JSON report documents per-PDF and per-quote results, saved as `annotate_report.json` in the output directory.

5.2 5-Stage Text Location Pipeline

For each quote, the pipeline iterates all pages of the resolved PDF and attempts progressively more tolerant matching strategies. The first successful match terminates the search.

Stage 1: Exact Match

Performs `str::find()` on the pdfium-extracted page text. If the verbatim quote string is found, per-character bounding boxes are extracted from the pdfium `PdfPageText` API via `loose_bounds()`.

Stage 2: Normalized Match

Both the quote and page text are normalized: Unicode NFC, soft hyphen (U+00AD) removal, hyphenated line break joining (`-\\n`), whitespace collapse to single spaces, and lowercase conversion. If the normalized quote is found in the normalized page, the match position is mapped back to original character indices for bounding box extraction.

Stage 3: Fuzzy Match

Slides a window of approximately $\pm 10\%$ of the quote length across the page text. For each window, the normalized Levenshtein similarity is computed. The best-scoring window above the 0.90 threshold is accepted. A pre-filter skips pages where fewer than 30% of the quote's words appear. Pages exceeding 100,000 characters or quotes exceeding 500 characters are skipped to prevent excessive computation.

Stage 3.5: Fallback Extraction

When pdfium text extraction yields no match in stages 1–3, re-extracts page text using the neuroncite-pdf multi-backend pipeline (poppler, mupdf, pdfium fallback chain). Stages 1–3 are re-attempted on the freshly extracted text. This captures content that pdfium's primary extraction missed due to encoding differences between PDF text extraction backends.

Stage 4: OCR Fallback (feature-gated)

Requires the `ocr` feature flag to be enabled. Renders the page to a PNG image, invokes Tesseract with hOCR output (`-c tesseract_create_hocr=1`), and parses the resulting HTML for word-level bounding boxes. The OCR text is concatenated from hOCR words and matched using the Stage 2 normalized match algorithm. Pixel bounding boxes are converted to PDF coordinate space using the render DPI: $\text{pdf_x} = \text{pixel_x} \times 72/\text{DPI}$, with Y-axis inversion for PDF's bottom-up coordinate system.

5.3 Annotation Creation

Annotations are created using pdfium-render's annotation API:

- **Highlight annotations:** Per-character bounding boxes are merged into line-level rectangles (characters on the same line with vertical midpoints within half the character height). Each line rectangle produces one highlight annotation via `create_highlight_annotation()`, configured with `set_bounds()` and `set_fill_color()` for semi-transparent color (`alpha = 128`).
- **Text annotations:** If a comment is specified, a popup text annotation is created via `create_text_annotation()` positioned at the top-left corner of the first bounding box.

5.4 Verification

- **Pre-check:** Before annotation creation, validates that at least 95% of characters in the match range have valid (non-degenerate) bounding boxes. Computes the total highlight area in square points.
- **Post-check:** After saving the annotated PDF, the file is re-opened and highlight annotations on the target page are counted. The annotation count must match expectations and the total area must be within $\pm 15\%$ of the pre-check value.

5.5 Job Integration

Annotation jobs use the existing job executor infrastructure. The `job` table's `params_json` column stores serialized job parameters (source directory, output directory, input data, default color). The executor's `find_next_queued_job` query accepts both `index` and `annotate` job kinds.

The annotation pipeline runs synchronously within a `spawn_blocking` task. Progress updates (done PDFs / total PDFs) are written to the job table for polling.

5.6 Interface Exposure

The annotation feature is accessible through all three interfaces:

- **REST API:** `POST /api/v1/annotate` accepts an `AnnotateRequest` body and returns an `AnnotateResponse` with the job ID. Status code 202 (Accepted).
- **MCP:** The `neuroncite_annotate` tool accepts `source_directory`, `output_directory`, `input_data`, and optional `default_color`. Returns a job ID for progress tracking.
- **CLI:** The `annotate` subcommand runs the pipeline synchronously with `--input`, `--source-dir`, `--output-dir`, and `--default-color` flags. Prints the JSON report to stdout.

6 Citation Verification Pipeline

The `neuroncite-citation` crate provides a multi-agent citation verification system. A main agent creates a job from a $\text{\LaTeX}$ /BibTeX file pair. The deterministic backend parses citations, groups them by proximity, and stores them as `citation_row` records. Sub-agents claim batches of related citations, verify them against indexed PDFs, and submit structured results with verdicts, passages, and optional $\text{\LaTeX}$ corrections.

6.1 Architecture Overview

The citation verification workflow consists of five sequential phases:

1. **Job Creation (`create`):** Parses the `.tex` file for citation commands (`\cite`, `\citep`, `\citet`, `\autocite`, `\parencite`, `\textcite`), resolves cite-keys against the `.bib` file, matches cited works to indexed PDFs via Jaro-Winkler similarity (threshold 0.65), assigns nearby citations to batches (respecting group boundaries), and inserts all `citation_row` records.
2. **Batch Claiming (`claim`):** Sub-agents atomically claim the lowest pending batch. Stale batches (claimed more than 5 minutes ago) are reclaimed to pending status before selection.
3. **Result Submission (`submit`):** Each sub-agent submits verification results for its batch: verdict, passages found, reasoning, confidence score, and optional $\text{\LaTeX}$ corrections. The job auto-completes when all rows are done or failed.
4. **Status Monitoring (`status`):** Aggregates counts by row status, batch status, verdict distribution, and flagged alerts for immediate escalation.
5. **Export (`export`):** Generates three output files (annotation CSV, corrections JSON, summary report) and triggers the `neuroncite-annotate` pipeline with verdict-colored highlights.

6.2 Database Schema

The `citation_row` table stores all citation verification data. It contains 21 columns covering cite-key identity, source matching, batch assignment, and sub-agent results. Foreign key references the `job` table with cascade delete. Four indexes cover job-scoped queries by status, batch, and group.

Column	Type	Description
<code>id</code>	INTEGER PRIMARY KEY AUTOINCREMENT	Auto-incrementing row identifier.
<code>job_id</code>	TEXT NOT NULL	Foreign key to <code>job.id</code> . Cascade delete removes all citation rows when the parent job is deleted.

Column	Type	Description
<code>group_id</code>	INTEGER NOT NULL	Identifies all cite-keys originating from the same <code>\cite{a,b,c}</code> command. Citations sharing a <code>group_id</code> are never split across batches.
<code>cite_key</code>	TEXT NOT NULL	The BibTeX cite-key as it appears in the $\LaTeX$ source (e.g., “fama1993”).
<code>author</code>	TEXT NOT NULL	Author field from the resolved BibTeX entry.
<code>title</code>	TEXT NOT NULL	Title field from the resolved BibTeX entry.
<code>year</code>	TEXT	Publication year from the BibTeX entry. NULL if the entry has no year field.
<code>tex_line</code>	INTEGER NOT NULL	1-based line number of the <code>\cite</code> command in the $\LaTeX$ source file.
<code>anchor_before</code>	TEXT NOT NULL	Text fragment preceding the citation command, used by sub-agents to understand the citing context.
<code>anchor_after</code>	TEXT NOT NULL	Text fragment following the citation command.
<code>section_title</code>	TEXT	Heading of the $\LaTeX$ section containing the citation. NULL if no section heading precedes the cite.
<code>matched_file_id</code>	INTEGER	Foreign key to <code>indexed_file.id</code> . The PDF matched via Jaro-Winkler similarity (threshold 0.65). NULL if no matching PDF was found in the corpus.
<code>bib_abstract</code>	TEXT	Abstract field from the BibTeX entry, if present. Provides additional context for sub-agent verification.
<code>bib_keywords</code>	TEXT	Keywords field from the BibTeX entry, if present.
<code>tex_context</code>	TEXT	Surrounding $\LaTeX$ source text (approximately 200 characters) centered on the citation command’s line. Provides sub-agents with the prose context in which the citation appears, beyond the single-line anchor fragments.
<code>batch_id</code>	INTEGER	Batch number assigned during the batching phase. Sub-agents claim entire batches atomically. NULL before batch assignment.
<code>status</code>	TEXT NOT NULL	Row status: “pending”, “claimed”, “done”, or “failed”. Default: “pending”.
<code>flag</code>	TEXT	Alert flag: NULL (normal), “critical”, or “warning”. Flagged rows are surfaced in the status endpoint for immediate escalation.

Column	Type	Description
<code>claimed_at</code>	INTEGER	Unix epoch seconds when a sub-agent claimed the batch containing this row. NULL while unclaimed. Used for stale-batch reclamation (threshold: 5 minutes).
<code>result_json</code>	TEXT	Serialized JSON containing the sub-agent’s verdict, passages found, reasoning, confidence score, and optional <code>\LaTeX</code> correction. NULL until submission.
<code>co_citation_json</code>	TEXT	Serialized JSON array of co-citation metadata for cite-keys that share the same <code>\cite{a,b,c}</code> command (identified by <code>group_id</code>). Each entry records the companion cite-key, its verdict, and confidence. NULL for singleton citations that appear alone in their cite command.

Table 18: Schema of the `citation_row` table. Each row represents one cite-key from a `\cite{}` command in a `\LaTeX` document.

Indexes:

- `idx_citation_row_job` on (`job_id`) – filters all citation rows belonging to a specific verification job.
- `idx_citation_row_status` on (`job_id`, `status`) – supports the claim and status endpoints that count rows by status within a job.
- `idx_citation_row_batch` on (`job_id`, `batch_id`) – supports the claim endpoint that atomically selects all rows in a batch.
- `idx_citation_row_group` on (`job_id`, `group_id`) – verifies that cite-groups sharing a `group_id` are never split across batches.

6.3 Cite Groups and Batching

A multi-key citation command like `\cite{a,b,c}` produces three `citation_row` entries sharing the same `group_id`. The batch assignment algorithm sorts citations by (`section_title`, `tex_line`), groups them into batches of configurable size (default 5), and enforces the constraint that citations sharing a `group_id` are never split across batches. If a group would exceed the batch size, the batch extends rather than splitting the group.

6.4 Verdict Types

Seven verdict types capture the sub-agent’s assessment:

supported The cited source contains a passage supporting the claim. Color: `#00AA00` (green).

partial	The source partially supports the claim. Color: #FFD700 (gold).
unsupported	The source exists but does not support the claim. Color: #FF4444 (red).
not_found	No relevant passage found in any indexed PDF. Color: #AAAAAA (gray).
wrong_source	The claim is correct but attributed to the wrong source. Color: #FF8800 (orange).
unverifiable	The cited source is not in the indexed corpus. Color: #CCCCCC (light gray).
peripheral_match	The claim text was found in the cited source, but exclusively in non-substantive sections (table of contents, bibliography, index, foreword, appendix, glossary). Color: #FFC107 (amber).

6.5 Interface Exposure

The citation verification feature is accessible through two interfaces:

- **REST API:** Ten endpoints under `/api/v1/citation/`: POST create, POST claim, POST submit, GET `{job_id}/status`, POST `{job_id}/export`, GET `{job_id}/rows`, POST `{job_id}/auto-verify`, POST bib-report, POST fetch-sources, and POST parse-bib.
- **MCP:** Eight tools: `neuroncite_citation_create`, `neuroncite_citation_claim`, `neuroncite_citation_submit`, `neuroncite_citation_status`, `neuroncite_citation_export`, `neuroncite_citation_rows`, `neuroncite_citation_retry`, and `neuroncite_citation_fetch`.

7 Chunking Strategies

The `neuroncite-chunk` crate implements the `ChunkStrategy` trait with four concrete strategies. The user selects the active strategy through the web frontend or API before starting an indexing operation. Every chunk records its UTF-8 byte offset range (start and end positions) within the concatenated, normalized full-document text and the page range it spans (page start and page end), enabling precise citation generation even for chunks that cross page boundaries. The concatenation rule that joins pages into a single document string is defined in Section 9.5 (concatenated document text definition) and is shared between the chunking pipeline and the storage layer.

7.1 Page-Based Chunking

The simplest strategy. Each `PageText` is converted directly into one `Chunk`. The `chunk_index` is the sequential document-level position (0 through `page_count - 1`), and the content is the full text of the page.

When to use: Documents where page boundaries carry semantic meaning, such as textbooks with self-contained pages or papers with clear section breaks per page.

7.2 Word-Window Chunking

A sliding window approach that produces overlapping chunks of a fixed word count.

Parameters:

- *window_size*: The number of words per chunk (e.g., 300).
- *overlap*: The number of words shared between consecutive chunks (e.g., 50).

The algorithm concatenates all pages of a document into a single word sequence, then slides a window of *window_size* words forward by *window_size - overlap* words at each step. Each window becomes one chunk. The chunk records its UTF-8 byte offset range within the concatenated document text. The source page range (`page_start`, `page_end`) is determined by mapping the document-level byte offsets back to page boundaries using the `page.byte_count` column. Chunks near page boundaries may span two or more pages.

When to use: General-purpose strategy suitable for most documents. The overlap ensures that information spanning a window boundary is still captured in at least one chunk.

7.3 Token-Window Chunking

A sliding window approach measured in model tokens rather than whitespace-separated words. This strategy addresses the mismatch between word counts and the actual token budget consumed by the embedding model: technical documents containing formulas, abbreviations, URLs, and hyphenated compounds produce significantly more tokens per word than plain prose, causing word-based chunks to exceed the model's maximum sequence length.

Parameters:

- *token_limit*: The maximum number of tokens per chunk (e.g., 256). This must be less than or equal to the model’s maximum input sequence length minus 2 (to account for the [CLS] and [SEP] special tokens).
- *token_overlap*: The number of tokens shared between consecutive chunks (e.g., 32).

The algorithm loads the tokenizer of the active embedding model (the same `tokenizers` crate instance used by the embedding backend). It concatenates all pages of a document into a single normalized string, tokenizes the full text, then slides a window of *token_limit* tokens forward by *token_limit* – *token_overlap* tokens at each step. The chunk content is extracted by slicing the original normalized document string using the byte offset mappings returned by the tokenizer (the `offsets` field of each token), rather than by calling the tokenizer’s `decode()` method. This guarantees that the chunk text is a verbatim substring of the normalized document text, preserving exact byte offsets for citation generation. The tokenizer’s `decode()` method is not used for content extraction because it may alter whitespace or apply internal normalization that would make the chunk text diverge from the stored page text. Document-level byte offsets are taken directly from the tokenizer’s offset mappings, and the page range is resolved using the `page.byte_count` column, preserving the ability to generate precise citations even for chunks that span page boundaries.

When to use: Documents with mixed content (prose, code snippets, mathematical notation, bibliographic references) where word-based counting produces inconsistent chunk sizes relative to the model’s token budget. This strategy guarantees that every chunk fits within the model’s context window.

7.4 Sentence-Based Chunking

Splits text at sentence boundaries and groups consecutive sentences until a maximum word count is reached.

Parameters:

- *max_words*: The maximum number of words per chunk (e.g., 250).

Sentence boundary detection uses the `unicode-segmentation` crate, which implements the Unicode Standard Annex #29 sentence segmentation algorithm. This handles Unicode punctuation, quotation marks, and non-Latin scripts correctly.

To avoid false sentence breaks at abbreviations common in academic text, the splitter maintains a whitelist of known abbreviations: “et al.”, “Fig.”, “Eq.”, “Dr.”, “Prof.”, “Inc.”, “Ltd.”, “vs.”, “i.e.”, “e.g.”, “cf.”, “approx.”, “Vol.”, “No.”, “pp.”, and “Sect.”. When a period is followed by a whitelisted abbreviation pattern, the sentence boundary is suppressed and the abbreviation is kept within the current sentence.

Sentences are accumulated into a buffer until adding the next sentence would exceed *max_words*, at which point the buffer is flushed as a chunk and a new accumulation begins.

When to use: Documents with flowing prose where sentence integrity matters for semantic coherence, such as academic papers and reports.

7.5 Chunk Boundary Determinism

All chunking strategies produce deterministic output: the same PDF file processed with the same strategy and parameters always yields identical chunks (same text content, same byte offsets, same content hashes). This property is critical for incremental re-indexing, where the system relies on content hashes to detect whether a file’s chunks have changed. To ensure determinism, text normalization (whitespace collapsing, hyphenation repair at line breaks, and Unicode NFC normalization) is applied as a fixed preprocessing step before the chunking strategy runs.

7.6 Context Window for Display

When the web frontend displays a search result, the user can expand the result to view surrounding context beyond the chunk boundaries. The context window uses a two-tier strategy: it first loads the adjacent chunks (± 1 by `chunk_index`) from the `chunk` table. If adjacent chunks are not available (the matched chunk is the first or last in the document), the system falls back to loading $\pm N$ sentences (configurable, default 3) from the stored page text in the `page` table (see Section 9.5). The surrounding context is rendered in a visually distinct style (lighter color) to distinguish it from the matched chunk text.

8 Embedding Backends

The `neuroncite-embed` crate provides three implementations of the `EmbeddingBackend` trait and three corresponding implementations of the `Reranker` trait, each gated behind a Cargo feature flag. Multiple backends can be compiled into the same binary simultaneously. At runtime, the user selects which backend to use from a dropdown that lists only the compiled-in options.

8.1 Backend: ONNX Runtime (`backend-ort`)

ONNX Runtime via the `ort` Crate

Feature flag: `backend-ort`

Description: Uses the `ort` Rust crate, which provides safe bindings to Microsoft's ONNX Runtime. Models are loaded from ONNX format files. The runtime supports multiple execution providers, with CUDA being the primary target for GPU acceleration.

Supported embedding models: Both BERT-family encoder models and decoder-based LLM embedding models exported to ONNX format. Each model is described by an `EmbeddingModelConfig` struct that defines its architecture-specific parameters: pooling strategy, maximum sequence length, input tensor layout, and optional instruction prefixes. The built-in model catalog contains:

- **Encoder models (BERT architecture, CLS pooling):**
 - BAAI/bge-small-en-v1.5 (384 dimensions, 33M parameters, 512 tokens)
 - BAAI/bge-base-en-v1.5 (768 dimensions, 110M parameters, 512 tokens)
 - sentence-transformers/all-MiniLM-L6-v2 (384 dimensions, 22M parameters, 512 tokens)
- **Decoder models (LLM architecture, mean pooling):**
 - Qwen/Qwen3-Embedding-0.6B (1024 dimensions, 600M parameters, 8192 tokens, mean pooling)
 - Qwen/Qwen3-Embedding-4B (2560 dimensions, 4B parameters, 8192 tokens, mean pooling)
 - Qwen/Qwen3-Embedding-8B (4096 dimensions, 8B parameters, 8192 tokens, mean pooling)
- **Extended encoder models (BERT++ with RoPE, CLS pooling):**
 - Alibaba-NLP/gte-large-en-v1.5 (1024 dimensions, 434M parameters, 8192 tokens, CLS pooling)
- **Multilingual encoder models (CLS pooling):**
 - BAAI/bge-m3 (1024 dimensions, 568M parameters, 8192 tokens, CLS pooling)

Pooling strategies: The backend supports three pooling strategies, selected per model via `PoolingStrategy`:

- **CLS** – Extracts the [CLS] token at sequence position 0 (BERT-family and GTE-Large).

- **MeanPooling** – Computes the attention-weighted mean over all non-padding tokens (Qwen3-Embedding and similar decoder models).
- **LastToken** – Extracts the embedding at the last non-padding position (reserved for causal decoder models; no model in the current catalog uses this strategy).

Position IDs: Decoder-based ONNX models with rotary position embeddings (RoPE) require an explicit `position_ids` input tensor. The `needs_position_ids` field in `EmbeddingModelConfig` controls whether the inference pipeline generates a sequential `[0, 1, 2, ..., seq_len-1]` tensor per batch element. Encoder models have absolute position embeddings baked into the model weights and do not need this tensor.

Instruction prefixes: Decoder-based embedding models use instruction prefixes to distinguish query embeddings from document embeddings. The `query_prefix` and `document_prefix` fields in `EmbeddingModelConfig` control text prepended before tokenization. Encoder models leave these fields empty.

Input tensor layout: BERT-family models receive three input tensors (`input_ids`, `attention_mask`, `token_type_ids`), while decoder models receive only `input_ids` and `attention_mask`. The `uses_token_type_ids` flag in `EmbeddingModelConfig` controls this.

Download redirects: Some decoder models have their ONNX weights hosted in community conversion repositories rather than the original model repository. The `DOWNLOAD_REPO_REDIRECTS` mapping translates canonical model IDs to alternate HuggingFace repository IDs for download. For example, `Qwen/Qwen3-Embedding-0.6B` downloads from `zhiqing/Qwen3-Embedding-0.6B-ONNX`.

Supported reranker models: The built-in reranker catalog contains four cross-encoder models exported to ONNX format:

- `cross-encoder/ms-marco-MiniLM-L-6-v2`
- `cross-encoder/ms-marco-MiniLM-L-12-v2`
- `cross-encoder/ms-marco-electra-base`
- `BAAI/bge-reranker-v2-m3` (downloads from `mogolloni/bge-reranker-v2-m3-onnx`)

GPU acceleration: Via the CUDA execution provider. Requires the NVIDIA CUDA toolkit and cuDNN libraries to be installed on the system. Falls back to the CPU execution provider if CUDA is not available.

Tokenization: Uses the `tokenizers` crate from Hugging Face to tokenize input text according to the model’s vocabulary before passing token IDs to the ONNX session. The tokenizer is model-agnostic and handles both WordPiece (BERT) and BPE/SentencePiece (decoder models) through the unified `tokenizer.json` format.

Batch processing: Multiple texts are tokenized and padded to the same sequence length, then passed as a single batched tensor to the ONNX session for parallel GPU inference. The maximum sequence length is model-dependent (512 for BERT, 8192 for decoder models).

8.2 Backend Selection at Runtime

At application startup, the binary queries each compiled-in backend for its availability (whether the required GPU drivers are present). The web frontend’s model management page displays a catalog of all models reported by all available backends. When the user activates a model, the corresponding backend is loaded and becomes the globally active model used for both indexing and search operations.

8.2.1 Global Model Selection

The embedding model selection is global: the same model is used for indexing and for encoding search queries. This is necessary because search requires that query vectors and indexed chunk vectors inhabit the same embedding space. If a user indexes a session with model A and then switches the active model to B, the search results would be meaningless because the query vector (produced by model B) is compared against stored vectors (produced by model A) in different embedding spaces.

The web frontend tracks this as a *model mismatch*: it stores the `active_model_id` (the currently loaded model) and the `session_model_name` (the model stored in the active session’s metadata). When these differ, warnings are displayed in the relevant UI sections.

8.2.2 Model Manager Panel

The Model Manager panel (`panels/models.rs`) provides a dedicated interface for model inspection and switching. It displays:

- **System information:** GPU device name and CUDA execution provider availability.
- **Active model:** Display name and vector dimension of the currently loaded model.
- **Mismatch warning:** When the session model differs from the active model, an orange warning describes the specific mismatch.
- **Model catalog table:** Each row shows the model’s display name, vector dimension, maximum sequence length, pooling strategy, cache status, active status, and an “Activate” button. The button is disabled when the model is already active, a switch is in progress, or the model is not cached.

Model switching follows the one-shot dispatch pattern: the panel sets `pending_model_switch` to `Some(model_id)`, which is consumed by `dispatch_actions()` on the next frame. The worker loads the model on a background thread and responds with `ModelSwitched`, which updates all global model state fields and the catalog active flags.

9 Storage Layer

The `neuroncite-store` crate manages all persistent data. It combines three storage mechanisms: an SQLite database for structured metadata and text content, an FTS5 virtual table for keyword search, and an HNSW index file for approximate nearest neighbor vector search.

9.1 Storage Location

All persistent data is stored in a centralized, platform-specific application data directory rather than alongside the indexed documents. The base directory is resolved with the following priority:

- **Windows:** %APPDATA%\NeuronCite, then %USERPROFILE%\Documents\NeuronCite
- **Linux:** \$XDG_DATA_HOME/NeuronCite, then \$HOME/Documents/NeuronCite
- **macOS:** \$HOME/Documents/NeuronCite
- **Fallback:** ./NeuronCite (relative to working directory)

Within this base directory, per-indexed-path storage is placed under `indexes/<sanitized-path>/`, where the sanitized path replaces colons with underscores and path separators with double dashes (e.g., `D:\Papers\Finance` becomes `D_--Papers--Finance`). The resulting directory structure is:

```
Documents/NeuronCite/
+-- runtime/                (ONNX Runtime DLLs, pdfium, Tesseract)
+-- models/                 (downloaded ONNX embedding model files)
+-- indexes/
    +-- <sanitized-path>/
        +-- index.db         (SQLite database)
        +-- hnsw_<session_id>/ (HNSW index directory per session)
        +-- exports/         (exported search results)
```

Each per-path directory contains:

- `index.db` – the SQLite database
- `hnsw_{session_id}/` – one HNSW index directory per session
- `models.lock` – the model manifest (see Section 16)
- `api_token.txt` – bearer token for LAN-mode authentication (created only when LAN binding is active)
- `exports/` – directory for exported search results (Markdown, BibTeX, CSL-JSON)
- `embeddings_{session_id}.bin` – external embedding storage file (created only when external storage mode is active)

Centralizing storage under the application data directory avoids cluttering source document trees with hidden directories. The model manifest (`models.lock`) records model identifiers and checksums for reproducibility; the `local_path` field is a diagnostic hint that is re-resolved from the model cache on each application launch.

9.2 Connection Pooling

The SQLite database is accessed through an `r2d2` connection pool backed by `r2d2_sqlite`. The pool maintains multiple connections to the same database file, enabling concurrent read access from the API server and the web frontend without contention.

Pool configuration:

- **Maximum connections:** 8 (1 writer, 7 readers).
- **WAL mode:** SQLite is configured in Write-Ahead Logging mode, which allows concurrent readers while a single writer is active.
- **Foreign keys:** `PRAGMA foreign_keys = ON` is set on every connection at initialization time. All foreign key constraints use explicit `ON DELETE CASCADE` rules so that deleting a session or file automatically removes all dependent rows.
- **Busy timeout:** 5000 ms. If a connection cannot acquire the write lock within this period, the operation returns a busy error. The `busy_timeout` PRAGMA causes SQLite to sleep and retry internally for up to 5000 ms before returning `SQLITE_BUSY`.
- **Journal size limit:** The WAL file is checkpointed automatically when it exceeds 64 MB.

Write operations (inserting chunks, creating sessions) acquire a connection from the pool and hold it only for the duration of the transaction. Read operations (loading chunk metadata for search results, listing sessions) use separate connections that do not block writers.

9.3 Table: `index_session`

Tracks each unique combination of directory, model, and chunking configuration. This table is the key to incremental indexing: before starting a new indexing operation, the system checks whether a session with the same configuration already exists.

Column	Type	Description
<code>id</code>	INTEGER PRIMARY KEY	Auto-incrementing session identifier.
<code>directory_path</code>	TEXT NOT NULL	Absolute path to the indexed directory.
<code>model_name</code>	TEXT NOT NULL	Identifier of the embedding model used.
<code>chunk_strategy</code>	TEXT NOT NULL	Strategy identifier (e.g., “word”, “token”, “sentence”, “page”).
<code>chunk_size</code>	INTEGER	Window size in words or token limit, depending on strategy. NULL for page-based chunking.
<code>chunk_overlap</code>	INTEGER	Overlap in words or tokens, depending on strategy. NULL for page-based and sentence-based chunking.

Column	Type	Description
<code>max_words</code>	INTEGER	Maximum words per chunk for sentence-based chunking. NULL for other strategies.
<code>ocr_language</code>	TEXT NOT NULL	Tesseract language string used during indexing (e.g., “eng”, “eng+deu”). Default: “eng”.
<code>vector_dimension</code>	INTEGER NOT NULL	Dimensionality of the embedding vectors.
<code>embedding_storage_mode</code>	TEXT NOT NULL	Storage mode for embedding vectors: “sqlite_blob” (default) or “mmap_file” (external).
<code>schema_version</code>	INTEGER NOT NULL	Database schema version at the time of session creation. Enables automated migrations when the application is upgraded.
<code>app_version</code>	TEXT NOT NULL	Application version string (e.g., “0.3.1”) that created the session. Recorded for diagnostic and compatibility purposes.
<code>hnsw_total</code>	INTEGER NOT NULL	Total number of vectors in the HNSW graph for this session. Updated after each indexing operation and after each full rebuild.
<code>hnsw_orphans</code>	INTEGER NOT NULL	Running count of orphaned vectors (vectors whose corresponding <code>chunk</code> row has been physically deleted). Incremented during incremental re-indexing when chunks are removed. Reset to zero after a full HNSW rebuild. The orphan ratio (<code>hnsw_orphans</code> / <code>hnsw_total</code>) is used to determine when a rebuild is recommended.
<code>last_rebuild_at</code>	INTEGER	Unix epoch seconds (UTC) of the most recent full HNSW rebuild for this session. NULL if no rebuild has been performed.
<code>created_at</code>	INTEGER NOT NULL	Unix epoch seconds (UTC) of session creation. Formatted as ISO 8601 for display in the web frontend.
<code>label</code>	TEXT	Optional human-readable label for the session. NULL by default. Set via the <code>neuroncite_session_update</code> MCP tool or the web frontend.
<code>tags</code>	TEXT	JSON array of user-defined tag strings (e.g., <code>["finance","2024"]</code>). NULL when no tags are set. Added to existing databases by the <code>ensure_columns</code> repair step.

Column	Type	Description
<code>metadata</code>	TEXT	JSON object of arbitrary key-value pairs (e.g., <code>{"source": "manual"}</code>). NULL when no metadata is set. Added to existing databases by the <code>ensure_columns</code> repair step.

Table 19: Schema of the `index_session` table.

Unique constraint: Because `chunk_size`, `chunk_overlap`, and `max_words` are nullable (NULL for strategies that do not use them), a plain `UNIQUE` constraint would not prevent duplicates—in SQLite, `NULL != NULL`, so two rows with identical non-null columns and matching NULLs are considered distinct. The uniqueness invariant is therefore enforced via an expression-based unique index that substitutes a sentinel value for NULL:

```
CREATE UNIQUE INDEX idx_session_unique ON index_session(
  directory_path,
  model_name,
  chunk_strategy,
  COALESCE(chunk_size, -1),
  COALESCE(chunk_overlap, -1),
  COALESCE(max_words, -1),
  ocr_language,
  embedding_storage_mode
);
```

The index includes `ocr_language` and `embedding_storage_mode` because both affect the session’s stored data and must be part of the identity check.

An indexing request that matches an existing session triggers an incremental update (see Section 9.4) rather than a full re-index.

9.4 Table: `indexed_file`

Tracks each individual file processed within a session (PDF or HTML). This single table combines file identity, content hash, and filesystem metadata for incremental change detection. The `source_type` column distinguishes PDF files (default) from HTML web pages. HTML sources store the local cache file path in `file_path` and link to a `web_source` row (see Section 9.10) for URL and metadata. The system first compares `mtime` and `size` against the current filesystem values as a fast pre-filter. Only if either value differs does the system compute a SHA-256 content hash and compare it against `file_hash`. This two-stage check avoids expensive hash computation for unchanged files.

Unique constraint: A composite unique index on (`session_id`, `file_path`) ensures one tracking entry per file per session.

Incremental re-indexing procedure:

Column	Type	Description
<code>id</code>	INTEGER PK	Auto-incrementing file identifier.
<code>session_id</code>	INTEGER NN	Foreign key to <code>index_session.id</code> with ON DELETE CASCADE.
<code>file_path</code>	TEXT NN	Absolute path to the PDF file.
<code>file_hash</code>	TEXT NN	SHA-256 hash of the file contents at the time of last indexing.
<code>mtime</code>	INTEGER NN	Last modification timestamp (Unix epoch seconds) at the time of last indexing. Used as a fast pre-filter for change detection.
<code>size</code>	INTEGER NN	File size in bytes at the time of last indexing. Used together with <code>mtime</code> as a fast pre-filter.
<code>page_count</code>	INTEGER NN	Number of pages with extracted text. This is the count of pages that were successfully processed by the text extraction pipeline.
<code>pdf_page_count</code>	INTEGER	Structural page count from the PDF page tree (via <code>lopdf::Document::get_pages().len()</code>). Independent of how many pages had extractable text. NULL for files indexed before this column was added. When <code>page_count < pdf_page_count</code> , extraction was incomplete (some pages may lack extractable text or failed during processing).
<code>created_at</code>	INTEGER NN	Unix epoch seconds (UTC) when the file was first indexed in this session.
<code>updated_at</code>	INTEGER NN	Unix epoch seconds (UTC) when the file was last re-indexed (equal to <code>created_at</code> if never re-indexed).
<code>source_type</code>	TEXT NN	Content source type: “pdf” for PDF files, “html” for web pages. DEFAULT ‘pdf’ for backward compatibility with existing rows. HTML sources store the cache file path in <code>file_path</code> and link to a <code>web_source</code> row for URL and metadata.

PK = PRIMARY KEY, NN = NOT NULL.

Table 20: Schema of the `indexed_file` table. Combines file identity, content hash, and filesystem metadata in a single table for incremental change detection. The `source_type` column distinguishes PDF and HTML sources.

1. Scan the directory for all PDF files currently present.
2. Load all `indexed_file` rows for the active session.
3. **Deleted files:** Files present in `indexed_file` but absent from the directory scan are removed. Deleting the `indexed_file` row cascades to all dependent `page` and `chunk` rows via `ON DELETE CASCADE`. The FTS5 triggers remove the corresponding full-text entries. The old HNSW vectors become temporary orphans within the current indexing run but are eliminated when the HNSW graph is rebuilt from all active vectors at the end of the run (see Section 9.12).
4. **Modified files:** Files where `mtime` or `size` differ from `indexed_file` are re-hashed.
 - *Content changed* (`file_hash` differs): All existing `page` and `chunk` rows belonging to that file are physically deleted (`DELETE FROM chunk WHERE file_id = ?` and `DELETE FROM page WHERE file_id = ?`). The `AFTER DELETE` trigger on `chunk` removes the corresponding FTS5 entries. The file is then re-extracted, re-chunked, and re-embedded. The new pages and chunks are inserted into SQLite, FTS5, and the HNSW index. The `indexed_file` row is updated with the new `file_hash`, `mtime`, `size`, and `updated_at` values.
 - *Content unchanged* (`file_hash` is the same): Only `mtime`, `size`, and `updated_at` are updated in `indexed_file`. No re-extraction, re-chunking, or re-embedding occurs. This avoids redundant work when filesystem metadata changes without actual content modification (e.g., a file copy or backup restore that preserves content but updates timestamps).
5. **New files:** Files present in the directory scan but absent from `indexed_file` are fully processed (extract, chunk, embed, store).
6. **Unchanged files:** Files where `mtime` and `size` match `indexed_file` are skipped entirely. However, the `indexed_file.updated_at` timestamp is refreshed to record that the file was confirmed present during this indexing run.

9.5 Table: page

Stores the normalized text content of each PDF page together with the extraction backend that produced it. The text stored in `content` has undergone the same deterministic normalization applied before chunking (whitespace collapsing, line-break hyphenation repair, Unicode NFC normalization). This guarantees that document-level byte offsets stored in the `chunk` table reference the exact text stored in `page.content`, making offset-to-page resolution and passage highlighting reliable by construction. This table serves three purposes: (1) it enables the web frontend to display full page text for context windows and passage highlighting without re-extracting the PDF, (2) it records which extraction backend (`pdf_extract`, `pdfium`, or `ocr`) produced each page’s text, enabling diagnostics and quality auditing, and (3) it provides the reference text against which document-level UTF-8 byte offsets in the `chunk` table can be resolved back to specific pages.

Unique constraint: `UNIQUE(file_id, page_number)` ensures one row per page per file.

Concatenated document text definition: The full-document text used for chunking and offset computation is formed by concatenating each page’s normalized content in page-number order, separated by a single newline character (`\n`) between consecutive pages. Pages with empty content contribute zero bytes, but the inter-page newline separator byte

Column	Type	Description
<code>id</code>	INTEGER PK	Auto-incrementing page identifier.
<code>file_id</code>	INTEGER NN	Foreign key to <code>indexed_file.id</code> with ON DELETE CASCADE.
<code>page_number</code>	INTEGER NN	One-indexed page number within the PDF.
<code>content</code>	TEXT NN	The extracted text content of the page. Empty string if the page produced no text after all extraction attempts (stored as NOT NULL DEFAULT "" to avoid NULL-handling complexity in offset computation and FTS indexing).
<code>backend</code>	TEXT NN	The extraction backend that produced this page’s text: “pdf_extract”, “pdfium”, or “ocr”.
<code>byte_count</code>	INTEGER NN	Number of UTF-8 bytes in <code>content</code> (0 if the page produced no extractable text). Used for fast document-offset-to-page mapping without loading the full text. Byte counts enable direct <code>&str</code> slicing in Rust without $O(n)$ character index conversion.

PK = PRIMARY KEY, NN = NOT NULL.

Table 21: Schema of the `page` table. Stores per-page extracted text and the backend that produced it.

is still inserted. Formally, for a document with pages $p_1, p_2, \dots, p_n$, the concatenated text is $p_1 \parallel '\n' \parallel p_2 \parallel '\n' \parallel \dots \parallel '\n' \parallel p_n$, where $\parallel$ denotes string concatenation and each p_i is the normalized text stored in `page.content`. The `byte_count` column stores the number of UTF-8 bytes in the page’s content only (excluding the separator). The offset mapping accounts for the $n - 1$ separator bytes by adding 1 for each page boundary crossed.

Document-offset-to-page mapping: The `byte_count` column enables resolving a document-level byte offset (as stored in `chunk.doc_text_offset_start`) to a specific page number and page-local byte offset. The mapping iterates over pages in order, accumulating `byte_count` values plus one separator byte per page boundary until the target offset falls within the current page’s range. The algorithm is implemented in `neuroncite-core/src/offset.rs` and is shared between the chunking pipeline (for computing `page_start/page_end`) and the search pipeline (for citation generation). This computation does not require loading the full page text. Because all offsets and counts use UTF-8 bytes, the resolution maps directly to Rust `&str` slice indices with $O(1)$ per page.

9.6 Table: `chunk`

Stores the actual text chunks and their embedding vectors. Embeddings are stored as binary blobs containing the raw little-endian IEEE 754 single-precision (f32) floating point values. For a 384-dimensional model, each blob is $384 \times 4 = 1536$ bytes.

The embedding blob is kept in SQLite as a backup for HNSW index rebuilding. The primary search path reads vectors from the HNSW index file, not from SQLite.

Column	Type	Description
<code>id</code>	INTEGER PRIMARY KEY AUTOIN- CREMENT	Strictly monotonic chunk identifier. AUTOINCREMENT prevents SQLite from reusing rowids of deleted rows. This is critical because the HNSW index stores <code>chunk.id</code> as the vector label: without AUTOINCREMENT , a stale HNSW vector from a crash-recovery orphan could map to a newly inserted row with a recycled id, producing semantically incorrect search results (old vector, new text). See Section 9.12.
<code>file_id</code>	INTEGER NOT NULL	Foreign key to <code>indexed_file.id</code> ON DELETE CASCADE.
<code>session_id</code>	INTEGER NOT NULL	Foreign key to <code>index_session.id</code> ON DELETE CASCADE (denormalized for fast session-wide queries).
<code>page_start</code>	INTEGER NOT NULL	One-indexed start page in the source PDF. For chunks contained within a single page, page_start equals page_end .
<code>page_end</code>	INTEGER NOT NULL	One-indexed end page in the source PDF. Greater than page_start for chunks that span page boundaries (word-window and token-window strategies).
<code>chunk_index</code>	INTEGER NOT NULL	Zero-indexed position within the document (sequential across all pages).
<code>doc_text_offset_start</code>	INTEGER NOT NULL	Start UTF-8 byte offset within the concatenated, normalized full-document text. Together with <code>doc_text_offset_end</code> and the <code>page.byte_count</code> values, this offset can be resolved to a specific page and page-local position for citation generation.
<code>doc_text_offset_end</code>	INTEGER NOT NULL	End UTF-8 byte offset (exclusive) within the concatenated, normalized full-document text.
<code>content</code>	TEXT NOT NULL	The text content of the chunk.
<code>embedding</code>	BLOB	Dense vector as raw little-endian f32 bytes (backup copy; primary search uses HNSW file). NULL when external embedding storage mode is active (see below).
<code>ext_offset</code>	INTEGER	Byte offset into the external embeddings file. NULL when using SQLite blob storage mode.
<code>ext_length</code>	INTEGER	Byte length of the embedding vector in the external embeddings file. NULL when using SQLite blob storage mode.

Column	Type	Description
<code>content_hash</code>	TEXT NOT NULL	SHA-256 hash of the text content.
<code>simhash</code>	INTEGER	64-bit SimHash locality-sensitive hash of the chunk text. Pre-computed at index time using FNV-1a per-token hashing with a 64-bit bitwise counter array. Used by the deduplication stage to detect near-duplicate chunks via Hamming distance comparison without loading the full text content from the database. NULL for chunks indexed before this column was introduced; the dedup stage falls back to on-the-fly computation from the stored content when NULL.
<code>is_deleted</code>	INTEGER NOT NULL	Deletion flag (0 = active, 1 = excluded). During incremental re-indexing, old chunks are physically deleted rather than flagged. This column is reserved for manual chunk exclusion (e.g., user-initiated hiding of individual results). Flagged chunks are excluded from search results. Default: 0.
<code>created_at</code>	INTEGER NOT NULL	Unix epoch seconds (UTC) when this chunk was created.

Table 22: Schema of the `chunk` table. The `embedding`, `ext_offset`, and `ext_length` columns are always present in the schema; exactly one storage mode is active per session (determined by `index_session.embedding_storage_mode`).

Unique constraint: `UNIQUE(file_id, chunk_index)` ensures that no duplicate chunks exist for the same file. Since `file_id` is already session-specific (via `indexed_file.session_id`), this constraint is sufficient to prevent duplicates caused by interrupted re-indexing operations.

Indexes:

- `idx_chunk_session` on (`session_id`) – for loading all chunk metadata of a session during result assembly.
- `idx_chunk_file` on (`file_id`) – for deleting chunks when a file is re-indexed (also serves the `ON DELETE CASCADE` path).
- `idx_chunk_hash` on (`content_hash`) – for deduplication checks.
- `idx_chunk_active` – a partial index defined as:

```
CREATE INDEX idx_chunk_active ON chunk(session_id) WHERE is_deleted = 0;
```

This partial index covers only active (non-deleted) chunks and is used by all search queries to exclude flagged chunks without scanning excluded rows.

9.7 Table: job

Persists all background jobs (indexing, HNSW rebuild) so that job state survives process restarts and clients can query historical jobs. A periodic cleanup task deletes rows whose `finished_at` timestamp is older than 24 hours.

Column	Type	Description
<code>id</code>	TEXT PRIMARY KEY	UUID v4 assigned at job creation.
<code>kind</code>	TEXT NOT NULL	Job type: “index” or “hnsw_rebuild”.
<code>session_id</code>	INTEGER	Foreign key to <code>index_session.id</code> . NULL for cross-session operations such as database <code>VACUUM</code> . All indexing and HNSW rebuild jobs have a non-NULL <code>session_id</code> .
<code>state</code>	TEXT NOT NULL	Current state: “queued”, “running”, “completed”, “failed”, or “canceled”.
<code>progress_done</code>	INTEGER NOT NULL	Number of items processed so far.
<code>progress_total</code>	INTEGER NOT NULL	Total number of items to process. Zero if the total is not yet known.
<code>error_message</code>	TEXT	Human-readable error message if <code>state</code> is “failed”. NULL otherwise.
<code>created_at</code>	INTEGER NOT NULL	Unix epoch seconds (UTC) when the job was created.
<code>started_at</code>	INTEGER	Unix epoch seconds when the job began executing. NULL while queued.
<code>finished_at</code>	INTEGER	Unix epoch seconds when the job reached a terminal state. NULL while running.
<code>params_json</code>	TEXT	Serialized JSON parameters for the job. Used by annotation jobs to persist input data and configuration. NULL for indexing and HNSW rebuild jobs.

Table 23: Schema of the `job` table. Rows are deleted 24 hours after `finished_at`.

Indexes:

- `idx_job_state_active` – a partial index defined as:

```
CREATE INDEX idx_job_state_active ON job(state, kind, created_at) WHERE
state IN ('queued', 'running');
```

Covers only active jobs and accelerates the concurrent job policy check (`has_active_job`) without scanning terminal-state rows.

9.8 Table: `idempotency`

Stores idempotency keys and their associated responses. Entries expire after 24 hours, matching the job retention window.

Column	Type	Description
<code>key</code>	TEXT PRIMARY KEY	The client-supplied idempotency key string.
<code>job_id</code>	TEXT NOT NULL	Foreign key to <code>job.id</code> . The job that was created for the original request.
<code>session_id</code>	INTEGER NOT NULL	Foreign key to <code>index_session.id</code> . The session referenced by the original response.
<code>created_at</code>	INTEGER NOT NULL	Unix epoch seconds (UTC) when the entry was inserted. Entries older than 24 hours are deleted by the same cleanup task that evicts expired jobs.

Table 24: Schema of the `idempotency` table. Entries expire 24 hours after `created_at`.

Indexes:

- `idx_idempotency_created` on `idempotency(created_at)` — supports the periodic cleanup query that deletes entries older than 24 hours. Without this index, each cleanup scan would require a full table scan of the `idempotency` table.

9.9 Table: `annotation_quote_status`

Tracks per-quote progress during annotation pipeline execution. Each row corresponds to one quote from the annotation input (CSV or JSON). The pipeline inserts all quotes as “pending” at job start and updates each row as the quote is located (or not found) in its target PDF. The MCP `neuroncite_annotate_status` tool queries this table to report annotation progress to clients.

Column	Type	Description
<code>id</code>	INTEGER PRIMARY KEY AUTOINCREMENT	Auto-incrementing row identifier.
<code>job_id</code>	TEXT NOT NULL	Foreign key to <code>job.id</code> with <code>ON DELETE CASCADE</code> . All quote status rows are deleted when the parent annotation job is deleted.
<code>title</code>	TEXT NOT NULL	Title field from the annotation input row, used to identify the target PDF.

Column	Type	Description
author	TEXT NOT NULL	Author field from the annotation input row.
quote_excerpt	TEXT NOT NULL	Truncated excerpt of the quote text from the input row. Stored for display purposes in status queries.
status	TEXT NOT NULL	Quote processing status: “pending”, “matched”, “not_found”, or “error”. Default: “pending”.
match_method	TEXT	The location method that found the quote: “exact”, “normalized”, “fuzzy”, “fallback_extract”, or “ocr”. NULL while pending or if not found.
page	INTEGER	1-based page number where the quote was located. NULL while pending or if not found.
pdf_filename	TEXT	Filename of the PDF that the quote was matched to. NULL if the quote could not be resolved to a PDF.
updated_at	INTEGER NOT NULL	Unix epoch seconds (UTC) when the row was last modified. Set on insert and updated on each status change.

Table 25: Schema of the `annotation_quote_status` table. Each row tracks the annotation status of one input quote within an annotation job.

Indexes:

- `idx_ann_quote_job` on `(job_id)` – filters all quote status rows belonging to a specific annotation job.

9.10 Table: `web_source`

Stores HTML-specific metadata for web pages indexed via the `neuroncite-html` pipeline. Each row is linked 1:1 to an `indexed_file` row with `source_type = 'html'`. The table captures the original URL, HTTP response metadata, Open Graph tags, and optionally the raw HTML bytes for re-extraction.

Column	Type	Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	Auto-incrementing identifier.

Column	Type	Description
file_id	INTEGER NOT NULL UNIQUE	Foreign key to <code>indexed_file.id</code> with <code>ON DELETE CASCADE</code> . Each web source is linked to exactly one indexed file row.
url	TEXT NOT NULL	The fetched URL (after redirect resolution).
canonical_url	TEXT	Canonical URL from <code><link rel="canonical"></code> or the <code>og:url</code> Open Graph tag. NULL when absent.
title	TEXT	Page title from the <code><title></code> tag.
meta_description	TEXT	Content of <code><meta name="description"></code> .
language	TEXT	Language code from the <code><html lang></code> attribute or Content-Language HTTP header.
og_image	TEXT	Open Graph image URL (<code>og:image</code>).
og_title	TEXT	Open Graph title (<code>og:title</code>).
og_description	TEXT	Open Graph description (<code>og:description</code>).
author	TEXT	Author from <code><meta name="author"></code> .
published_date	TEXT	Publication date from <code><meta name="date"></code> or <code>article:published_time</code> .
domain	TEXT NOT NULL	Domain name extracted from the URL (e.g., “docs.example.com”).
fetch_timestamp	INTEGER NOT NULL	Unix epoch seconds (UTC) when the page was fetched.
http_status	INTEGER NOT NULL	HTTP response status code (e.g., 200, 301).
content_type	TEXT	Content-Type response header value.
raw_html	BLOB	Raw HTML bytes of the fetched page. Stored for potential re-extraction with different parsing settings. NULL when storage is disabled.

Table 26: Schema of the `web_source` table. Each row stores HTML-specific metadata for one indexed web page, linked to `indexed_file` via the `file_id` foreign key.

Indexes:

- `idx_web_source_url` on (`url`) – lookup by original URL for deduplication checks and cache hit detection.
- `idx_web_source_domain` on (`domain`) – filtering indexed pages by domain for the `neuroncite_files` tool (source type and domain filtering).

9.11 Full-Text Search Index (FTS5)

A virtual table `chunk_fts` provides BM25-based keyword search over chunk content. The FTS5 table mirrors the text content of the `chunk` table and is kept in sync through four SQLite triggers that fire on INSERT, DELETE, soft-delete, and un-delete operations.

Definition	Details
Virtual table Tokenizer	<code>chunk_fts</code> USING fts5(<code>content</code> , <code>content=chunk</code> , <code>content_rowid=id</code>) <code>unicode61</code> with <code>remove_diacritics=2</code> and <code>tokenchars='-_'</code> for accent-insensitive matching that preserves hyphenated terms (e.g., “state-of-the-art”) and underscored identifiers (e.g., “Eq_12”) as single tokens. The dot character (.) is deliberately excluded from <code>tokenchars</code> because including it would cause “word.” and “word” to be indexed as distinct tokens, reducing recall for ordinary prose queries. Abbreviations such as “et al.” are handled via query-time preprocessing (trailing punctuation stripping) rather than tokenizer configuration. <i>Limitation:</i> Because hyphenated terms are indexed as single tokens, a query for “art” does not match the indexed token “state-of-the-art”. Users must include the full hyphenated form for an exact match. The hybrid search mode mitigates this limitation through the vector search component, which captures semantic similarity regardless of tokenization
Ranking	BM25 (built into FTS5)

Table 27: FTS5 virtual table configuration.

The FTS5 index enables the keyword search component of the hybrid retrieval pipeline. Given a query string, FTS5 returns chunk IDs ranked by BM25 relevance. These results are combined with HNSW vector search results through Reciprocal Rank Fusion (see Section 10).

Trigger correctness: The FTS5 external content table relies on SQLite triggers to stay synchronized with the `chunk` table. Because FTS5 external content tables use a special “delete” command (not a standard SQL DELETE), the triggers must use the FTS5-specific syntax for removals. The triggers handle four cases:

- `chunk_fts_insert` — AFTER INSERT on `chunk`: Inserts the text into the FTS5 index via `INSERT INTO chunk_fts(rowid, content) VALUES (new.id, new.content)`.
- `chunk_fts_delete` — AFTER DELETE on `chunk`: Removes the FTS5 entry using the FTS5 delete command:
`INSERT INTO chunk_fts(chunk_fts, rowid, content) VALUES('delete', old.id, old.content)`.
This special `VALUES('delete', ...)` syntax is the correct way to remove entries from an FTS5 external content table; a standard `DELETE FROM chunk_fts` would fail or produce ghost hits.
- `chunk_fts_update_deleted` — AFTER UPDATE OF `is_deleted` on `chunk` (soft-delete): When `is_deleted` changes from 0 to 1, the trigger removes the FTS5 entry using the same delete command:

```
INSERT INTO chunk_fts(chunk_fts, rowid, content) VALUES('delete', old.id, old.content).
```

This ensures that soft-deleted chunks do not appear in BM25 search results.

- **chunk_fts_update_undelated** — AFTER UPDATE OF `is_deleted` on `chunk` (un-delete): When `is_deleted` changes from 1 back to 0, the trigger re-inserts the chunk's content into the FTS5 index via `INSERT INTO chunk_fts(rowid, content) VALUES (new.id, new.content)`. Without this trigger, un-deleted chunks would be permanently lost from the FTS5 index and invisible to keyword search.

FTS5 maintenance: Over time, the FTS5 index accumulates fragmented data structures that degrade query performance. The `fts5` module provides an `optimize` command (`INSERT INTO chunk_fts(chunk_fts) VALUES('optimize')`) that merges all internal b-tree segments into a single segment. The web frontend exposes a maintenance page with an “Optimize Search Index” button that runs this command. The button also triggers an SQLite `VACUUM` on the main database to reclaim space freed by deleted chunks and compacted WAL checkpoints. Because `VACUUM` requires exclusive database access, the maintenance operation waits until all active queries and indexing jobs have completed. The web frontend disables the button and displays a progress indicator while `VACUUM` is running.

9.12 HNSW Index File

Each index session has a corresponding HNSW (Hierarchical Navigable Small World) index file stored as a binary file in the `.neuroncite/` directory. The HNSW index is built and managed by the `hnsw_rs` crate.

Index parameters:

- **M** (max connections per layer): 16. Controls the graph density and recall quality.
- **ef_construction**: 200. Number of candidate neighbors considered during index construction. Higher values produce a more accurate graph at the cost of slower indexing.
- **ef_search**: 100 (default, configurable per query). Number of candidates explored during search. Higher values improve recall at the cost of latency.
- **Distance metric**: Cosine similarity (vectors are pre-normalized to unit length, so inner product is used as the distance function).

The HNSW index stores the SQLite `chunk.id` as the label for each vector. This establishes a direct, stable mapping: when HNSW returns the top candidate labels, each label is the primary key of the corresponding row in the `chunk` table, which is then queried to retrieve the text content and metadata for result assembly. No separate ID translation layer is needed.

Index persistence: The HNSW graph is serialized to disk after indexing completes. On subsequent application launches, the index is memory-mapped from the file, avoiding the cost of rebuilding the graph. If the index file is corrupted or missing, it is rebuilt from the active embedding storage source for the session: from `chunk.embedding` blobs when the session uses `sqlite_blob` mode, or from the memory-mapped external file

(`embeddings_{session_id}.bin` via `chunk.ext_offset / chunk.ext_length`) when the session uses `mmap_file` mode. The `index_session.embedding_storage_mode` column determines which source is used.

Orphan handling: Under normal operation, the build-then-swap strategy (see indexing stage 8) eliminates orphans at the end of each indexing run: the new HNSW graph is built exclusively from active vectors (`is_deleted = 0`), so vectors belonging to deleted or modified files are simply not included. However, orphaned vectors can appear in the persisted graph if the application crashes after chunk rows have been deleted but before the new HNSW graph is serialized to disk. As a safety net, the search result assembly stage looks up the corresponding `chunk` row by primary key for each HNSW candidate label. If the row no longer exists (crash-recovery orphan) or has `is_deleted = 1` (manual chunk exclusion), the candidate is silently skipped. This ensures that stale HNSW vectors never appear in results regardless of the application’s shutdown history.

Periodic full rebuild: The system tracks the ratio of orphaned vectors (vectors whose corresponding `chunk` row has been physically deleted) to total vectors in the HNSW index. The orphan count and total vector count are persisted in the `index_session` table columns `hnsw_orphans` and `hnsw_total` (see Section 9.3). The orphan count is incremented each time chunks are deleted during incremental re-indexing and reset to zero after each successful HNSW rebuild (which occurs at the end of every indexing run under normal operation). The `last_rebuild_at` column records the timestamp of the most recent rebuild. If the orphan ratio exceeds a configurable threshold (default: 20%), or if the absolute number of orphaned vectors exceeds a configurable limit (default: 10 000)—conditions that indicate a prior crash interrupted the rebuild—the web frontend displays a “Rebuild recommended” indicator. The user can trigger a manual HNSW rebuild from the maintenance panel, which reads all active embedding vectors (where `is_deleted = 0`) from the session’s active storage source—the `chunk.embedding` column in `sqlite_blob` mode or the external `embeddings_{session_id}.bin` file in `mmap_file` mode—and constructs a fresh HNSW graph containing only active vectors.

Non-blocking rebuild via double-buffering: The rebuild constructs a separate, new HNSW index object without acquiring any lock on the current index. Search queries continue to read the old index concurrently via the shared `Arc`. Once the new index is fully built and serialized to a temporary file, the swap is performed by atomically replacing the `Arc` pointer through the `ArcSwap` primitive (a single atomic store, not a mutex-based lock). This ensures that search queries are never blocked during the rebuild; the pointer swap completes in nanoseconds.

9.13 Embedding Blob Format

The embedding vector is stored as a contiguous byte sequence with no header or metadata. The format is:

$$[\text{f32}_{LE,0}, \text{f32}_{LE,1}, \dots, \text{f32}_{LE,d-1}]$$

where d is the vector dimensionality and LE denotes little-endian byte order. This allows zero-copy deserialization in Rust by reinterpreting the byte slice as a slice of `f32` values, provided the platform is little-endian. All x86/x64 targets and all AArch64 targets in their

standard (little-endian) configuration satisfy this requirement. ARM processors configured in big-endian mode (rare in practice) would require byte-swapping during deserialization.

9.14 External Embedding Storage (Optional)

For large collections (millions of chunks), storing all embedding blobs inside the SQLite database file increases the database size significantly and slows down backup, `VACUUM`, and WAL checkpoint operations. An optional external storage mode offloads the raw embedding bytes to a separate memory-mapped file while SQLite retains only a lightweight reference.

External storage layout: All embedding vectors for one session are stored contiguously in a single flat binary file named `.neuroncite/embeddings_{session_id}.bin`. Each vector occupies exactly $d \times 4$ bytes at a fixed offset. The `chunk` table always contains the `embedding` (BLOB, nullable), `ext_offset` (INTEGER, nullable), and `ext_length` (INTEGER, nullable) columns regardless of the active storage mode. The `index_session.embedding_storage_mode` column determines which set of columns is populated: in “`sqlite_blob`” mode, `embedding` contains the vector and `ext_offset/ext_length` are NULL; in “`mmap_file`” mode, `ext_offset` and `ext_length` reference the external file and `embedding` is NULL. This stable schema avoids migrations when switching storage modes between sessions.

Access pattern: The embeddings file is opened via memory-mapped I/O (`mmap2` crate). Reading a vector for HNSW rebuild is a zero-copy slice into the mapped region. Appending new vectors during incremental indexing extends the file and updates the mapping. When files are deleted from the index, the corresponding vector slots in the embeddings file become unused. Compaction is triggered at the end of an indexing run when the ratio of unused bytes to total file size exceeds 20% (matching the HNSW orphan ratio threshold). The compaction step rewrites the file with only the active vectors and updates the `ext_offset` values in the `chunk` table within a single SQLite transaction. The compaction runs after the HNSW rebuild in stage 8 of the indexing pipeline, so the new graph references the compacted offsets.

SQLite PRAGMA tuning: For large databases regardless of storage mode, the following PRAGMAs are set at connection initialization:

- `PRAGMA mmap_size = 268435456` (256 MB) – enables memory-mapped I/O for the database file, reducing read system calls.
- `PRAGMA cache_size = -65536` (64 MB) – increases the page cache to keep frequently accessed chunk metadata in memory.
- `PRAGMA temp_store = MEMORY` – stores temporary tables and indices in memory rather than on disk.

10 Search and Retrieval

The `neuroncite-search` crate implements the full retrieval pipeline: approximate nearest neighbor search via HNSW, keyword search via BM25, result fusion, and optional cross-encoder reranking.

10.1 Vector Search (HNSW)

Given a query embedding vector, the HNSW index returns the top N most similar chunk IDs with their cosine similarity scores in $O(\log n)$ time, where n is the total number of indexed chunks. This is the primary retrieval mechanism.

The search parameter `ef_search` controls the trade-off between recall and latency. The default value of 100 provides high recall (typically $> 95\%$ of the true top-K results) with sub-millisecond query times for collections of up to several million chunks.

10.2 Keyword Search (BM25)

FTS5 provides BM25-ranked keyword search over chunk text. This catches cases where the query contains specific terms (author names, technical abbreviations, exact phrases) that share little semantic overlap with the embedding space but match lexically.

The keyword search accepts the raw query string and returns chunk IDs ranked by BM25 relevance. SQLite FTS5’s `bm25()` function returns a lower-is-better score (typically negative). Since Reciprocal Rank Fusion operates on rank positions rather than raw scores, the BM25 values are used only for ordering the keyword result list; the raw BM25 scores are not combined arithmetically with cosine similarity scores. The `SearchResult` object stores the BM25 rank position (1-indexed, where 1 is the best BM25 match) as a diagnostic field for transparency.

10.3 Hybrid Search via Reciprocal Rank Fusion

The hybrid search mode combines vector and keyword results using Reciprocal Rank Fusion (RRF). RRF merges two ranked lists without requiring score normalization:

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)}$$

where $\mathcal{R}$ is the set of rankers (vector search and BM25), $\text{rank}_r(d)$ is the rank of document d in ranker r ’s result list, and k is a configurable constant that dampens the influence of high-rank positions.

RRF parameter k : The default value is $k = 60$, which provides balanced fusion between vector and keyword results. The user can select from three presets in the web frontend or set a custom value via the API:

- **Balanced** ($k = 60$): Equal weight to both retrieval methods. Suitable for general-purpose queries.
- **Semantic-heavy** ($k = 10$): Low damping amplifies the contribution of top-ranked vector results, favoring semantic similarity.
- **Lexical-heavy** ($k = 200$): High damping flattens rank differences in both lists, giving more relative weight to BM25 exact matches.

BM25 must-match filter: An optional mode where candidates are required to appear in the BM25 result list to be included in the final fusion output. When enabled, the vector search results serve only to re-rank BM25 matches rather than introducing candidates that have no lexical overlap with the query. This mode is effective for queries targeting specific author names, variable identifiers, citation keys, or exact technical terms where lexical presence is a prerequisite for relevance.

The procedure:

1. Run HNSW vector search, retrieve top N_{vec} candidates (default: 100).
2. Run FTS5 BM25 keyword search, retrieve top N_{kw} candidates (default: 100).
3. If BM25 must-match is enabled, discard any vector candidate that does not also appear in the BM25 result list.
4. Compute the RRF score for each remaining candidate that appears in at least one list.
5. Sort all candidates by descending RRF score.
6. **Deduplicate:** Remove near-duplicate results based on `content_hash`. If two or more chunks share the same hash (identical text, e.g., repeated headers, footers, or OCR duplicates), retain only the highest-scoring instance. For near-duplicates that are not byte-identical (minor OCR variations), the system reads the pre-computed SimHash fingerprint from the `chunk.simhash` column and merges chunks whose Hamming distance falls below a configurable threshold (default: 3 bits out of 64). If a chunk's `simhash` is NULL (legacy data indexed before the column was introduced), the fingerprint is computed on-the-fly from the stored text content.
7. Take the top K_{rerank} candidates (default: 30) for the optional reranking stage.

10.4 Cross-Encoder Reranking

If a reranker model is configured, the top candidates from the fusion stage are reranked using a cross-encoder that jointly processes each (query, chunk) pair. The cross-encoder produces a relevance score that is more accurate than cosine similarity or BM25 alone, because it attends to the interaction between query and document tokens rather than comparing independent representations.

The reranking procedure:

1. Take the top K_{rerank} candidates from the fusion stage.
2. For each candidate, construct the input pair (query text, chunk text) and pass the batch to the reranker.
3. The reranker returns a relevance score for each pair.
4. Sort by descending reranker score.
5. Return the top K results (the user-configured final result count).

If no reranker is configured, the top K results from the fusion stage are returned directly.

10.5 Cosine Similarity

The cosine similarity between two vectors $\mathbf{a}$ and $\mathbf{b}$ of dimension d is computed as:

$$\text{cosine_sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} = \frac{\sum_{i=0}^{d-1} a_i b_i}{\sqrt{\sum_{i=0}^{d-1} a_i^2} \cdot \sqrt{\sum_{i=0}^{d-1} b_i^2}}$$

For maximum throughput, the implementation pre-normalizes all stored vectors to unit length at indexing time. This reduces the similarity computation at search time to a simple dot product:

$$\text{cosine_sim}(\hat{\mathbf{a}}, \hat{\mathbf{b}}) = \hat{\mathbf{a}} \cdot \hat{\mathbf{b}} = \sum_{i=0}^{d-1} \hat{a}_i \hat{b}_i$$

where $\hat{\mathbf{a}} = \mathbf{a}/\|\mathbf{a}\|$ is the unit-normalized vector. This dot product can be further accelerated using SIMD intrinsics on x86 platforms.

10.6 Citation Generation

After the final ranking, a `Citation` object is generated for each result. The page range is derived directly from the chunk's `page_start` and `page_end` columns. The citation includes:

- The source file display name (filename without directory path)
- The page range (`page_start` through `page_end`; displayed as a single page number when both are equal)
- A text excerpt (first 200 characters of the chunk, with ellipsis)
- A pre-formatted reference string for copy-paste:
`filename.pdf, p. 17: "excerpt..."` or
`filename.pdf, pp. 17-18: "excerpt..."` for multi-page chunks

11 REST API Design

The `neuroncite-api` crate provides a versioned HTTP REST API via `axum`. All endpoints use the `/api/v1/` prefix to enable backwards-compatible evolution. The server binds to `127.0.0.1:3030` by default and runs on the `tokio` runtime alongside the web frontend (or standalone in headless mode). An OpenAPI 3.1 specification is generated at compile time by the `utoipa` crate from the `axum` handler type signatures and is served at `GET /api/v1/openapi.json`.

11.1 Shared Application State

The API server and the web frontend share a common application state wrapped in an atomic reference-counted pointer (`Arc`). This state contains:

- An `r2d2` connection pool to the SQLite database (multiple concurrent reader connections, serialized writer access)
- A `WorkerHandle` (defined in `neuroncite-api/src/worker.rs`) that encapsulates the dual-priority channel interface to the GPU Worker thread. API handlers and web request handlers submit embedding and reranking jobs through typed methods on `WorkerHandle` (`embed_query`, `embed_batch`, `rerank_batch`) rather than interacting with raw channels directly. The embedding backend and reranker model are owned exclusively by the GPU Worker and accessed only through this handle (see Section 13.1 for details). No application-level mutex protects the GPU resources.
- A per-session map of HNSW indices (as an `ArcSwap<HashMap<i64, Arc<HnswIndex>>>`), allowing lock-free concurrent reads during search. Each session has its own HNSW graph keyed by `session_id`. When a session's index is built or rebuilt, the map is cloned, the entry is inserted or replaced, and the `ArcSwap` is atomically updated without blocking concurrent readers. On startup, all sessions with stored embeddings are loaded into the map by `load_all_session_hnsw()`.
- The current index progress (as an atomic structure)
- A global `CancellationToken` for graceful process shutdown (triggers server stop, GPU Worker exit, and runtime drain). Each indexing job additionally receives a per-job `CancellationToken` (a child of the global token) that allows individual job cancellation via the `POST /api/v1/jobs/{id}/cancel` endpoint without stopping the server

11.2 Endpoint Overview

Method	Path	Description
POST	<code>/api/v1/annotate</code>	Accepts a JSON body with quotes and annotation parameters. Creates an annotation job that highlights matching passages in a PDF file. Returns the job ID and quote count.

Method	Path	Description
POST	/api/v1/annotate/from-file	Accepts a file path to a CSV or JSON quote list. Creates an annotation job from the file contents. Returns the job ID and quote count.
GET	/api/v1/backends	Lists all compiled-in embedding backends with their available models, reranker models, and GPU capability status.
POST	/api/v1/citation/bib-report	Generates a CSV or XLSX report from a BibTeX file. Summarizes entries with metadata fields, missing fields, and structural diagnostics.
POST	/api/v1/citation/claim	Claims a batch of citation rows for sub-agent verification. Uses an atomic transaction to prevent concurrent claim of the same rows.
POST	/api/v1/citation/create	Creates a citation verification job from LaTeX and BibTeX file paths. Parses the LaTeX source, extracts citation references, and matches them against BibTeX entries. Returns the job ID.
POST	/api/v1/citation/fetch-sources	Downloads source documents referenced in a BibTeX file. Resolves DOIs and URLs, fetches PDFs, and stores them locally. Returns a status report per entry.
POST	/api/v1/citation/parse-bibtex	Parses a BibTeX file and returns the structured entry data without creating a verification job. Used for preview and validation.
POST	/api/v1/citation/submit	Submits verification results for a batch of citation rows. Accepts per-row verdicts, confidence scores, and supporting evidence text.
POST	/api/v1/citation/{job_id}/trigger-verify	Triggers automatic verification of all pending citation rows in a job. Runs the verification pipeline without agent interaction.
POST	/api/v1/citation/{job_id}/export	Exports citation verification results for a job as CSV, XLSX, or JSON. Includes per-row verdicts and confidence scores.
GET	/api/v1/citation/{job_id}/rows	Returns citation rows for a job with optional status filtering. Includes row data, current verification status, and assigned agent.

Method	Path	Description
GET	/api/v1/citation/{job_id}	Returns job status with verdict distribution and flagged alerts. Includes counts for each verdict category and completion percentage.
POST	/api/v1/discover	Scans a directory for supported files (PDF, HTML) and reports the indexing status for each file (indexed, not indexed, stale).
GET	/api/v1/documents/{id}/pages/{n}	Returns the full text of page <i>n</i> of the indexed file with the given ID.
POST	/api/v1/files/compare	Compares the same file across different index sessions. Reports differences in chunk count, page count, and embedding dimensions.
GET	/api/v1/health	Returns a health check response including the application version, compiled feature flags, GPU availability, active backend, and the status of optional dependencies (pdfium, tesseract).
POST	/api/v1/index	Starts an indexing operation. Returns immediately with a <code>job_id</code> and <code>session_id</code> . Indexing runs in the background. Supports an optional <code>idempotency_key</code> header to prevent duplicate job creation on retried requests.
GET	/api/v1/jobs	Lists all jobs stored in the database (active and completed within the 24-hour retention window), ordered by creation time descending.
GET	/api/v1/jobs/{id}	Returns the state of a background job: <code>queued</code> , <code>running</code> , <code>completed</code> , <code>failed</code> , or <code>canceled</code> . Includes progress counters, start time, and finish time.
POST	/api/v1/jobs/{id}/cancel	Requests cooperative cancellation of a running job. The current batch completes before the cancellation takes effect.
GET	/api/v1/openapi.json	Returns the OpenAPI 3.1 specification for the entire API. Generated at compile time by utoipa.

Method	Path	Description
POST	/api/v1/search	Accepts a query string, a session identifier, and a top-K parameter. Returns a ranked list of matching chunks with scores and citations.
POST	/api/v1/search/hybrid	Performs hybrid search combining vector similarity and BM25 keyword matching. Accepts additional parameters: <code>use_fts</code> (boolean), <code>use_vector</code> (boolean), <code>use_reranker</code> (boolean), <code>bm25_must_match</code> (boolean), <code>rrf_k</code> (integer, optional).
POST	/api/v1/search/multi	Searches across multiple index sessions simultaneously. Merges results from all specified sessions and deduplicates by content hash.
GET	/api/v1/sessions	Lists all index sessions with their configurations and creation timestamps.
DELETE	/api/v1/sessions/{id}	Deletes an index session and all associated data (chunks, FTS5 entries, HNSW file). Also evicts the session's HNSW graph from the in-memory map.
POST	/api/v1/sessions/delete-by-directory	Accepts a JSON body with a <code>directory</code> field. Canonicalizes the path, deletes all sessions whose <code>directory_path</code> matches, evicts their HNSW graphs from the in-memory map, and returns the list of deleted session IDs.
POST	/api/v1/sessions/{id}/optimize	Runs FTS5 optimization on the session's full-text search index, merging internal b-tree segments into a single segment. Returns 202 Accepted; the operation runs in the background.
POST	/api/v1/sessions/{id}/rebuild	Triggers a full HNSW index rebuild from the session's active embedding storage. Returns 202 Accepted with a <code>job_id</code> .
POST	/api/v1/shutdown	Initiates a graceful shutdown of the server (see Section 13.3). Available only when the server runs in headless mode (<code>neuroncite serve</code>). Returns 202 Accepted immediately; the shutdown sequence proceeds asynchronously.
POST	/api/v1/verify	Accepts a claim string and a list of chunk IDs or texts. Returns a heuristic verification result (see Section 11.6).

Method	Path	Description
GET	/api/v1/sessions/{id}/quality	Returns text extraction quality metrics for all files in a session: extraction method distribution, page count mismatches, empty pages, and per-file quality flags.
GET	/api/v1/sessions/{session_id}/files/{file_id}/chunks	Returns paginated chunks for a file within a session. Each chunk includes content, page range, word count, and byte count.

Table 28: REST API v1 endpoints. All paths are prefixed with /api/v1/.

11.3 Request and Response Formats

All request and response bodies use JSON (`application/json`). Every response includes a top-level `api_version` field (string, e.g., “v1”) so that clients can verify compatibility. Errors are returned as JSON objects with an `error` field containing a human-readable message and a `code` field containing a machine-readable error identifier.

Indexing request fields: `directory` (string), `model` (string), `chunk_strategy` (string), `chunk_size` (integer, optional), `chunk_overlap` (integer, optional), `ocr_language` (string, optional, default “eng”).

Indexing response fields: `job_id` (string, UUID), `session_id` (integer). The job ID is used to poll job status via `GET /api/v1/jobs/{id}`.

Job status response fields: `job_id` (string), `state` (string, one of “queued”, “running”, “completed”, “failed”, “canceled”), `progress` (object with `files_total`, `files_done`, `chunks_created`), `started_at` (ISO 8601 string, null if queued), `finished_at` (ISO 8601 string, null if not finished), `error` (string, null unless state is “failed”).

Health response fields: `status` (string, “ok”), `version` (string, application version), `api_version` (string, “v1”), `build_features` (array of strings, e.g., [“backend-ort”]), `gpu_available` (boolean), `active_backend` (string or null), `pdfium_available` (boolean), `tesseract_available` (boolean), `bind_address` (string, the effective host and port the server is listening on).

Search request fields: `query` (string), `session_id` (integer), `top_k` (integer, default 10).

Hybrid search request extends the search request with: `use_fts` (boolean, default true), `use_vector` (boolean, default true), `use_reranker` (boolean, default false), `bm25_must_match` (boolean, default false – when true, only candidates that appear in the BM25 result list are included in the fusion output, see Section 10), and `rrf_k` (integer, optional – overrides the configured RRF fusion parameter k for this query; if omitted, the server-wide default from the configuration file is used).

Verify request fields: `claim` (string), `chunk_ids` (array of integers) or `texts` (array of strings).

Search response: An array of result objects, each containing `score` (float – the final ranking score, either RRF or reranker), `vector_score` (float – cosine similarity, diagnostic), `bm25_rank` (integer – the 1-indexed rank position in the BM25 result list, diagnostic; null if the chunk did not appear in BM25 results), `reranker_score` (float, if reranked), `source_file` (string), `page_start` (integer), `page_end` (integer), `chunk_index` (integer), `content` (string), and `citation` (object with `formatted` string and `page_start/page_end` integers).

11.4 Idempotency and Job Model

Long-running operations (indexing, HNSW rebuild) are modeled as first-class **Job** objects rather than anonymous background tasks. This provides three benefits: (1) multiple concurrent operations can be tracked independently, (2) clients can poll specific jobs by ID rather than relying on a single global status endpoint, and (3) retried requests do not create duplicate work.

Job lifecycle: A job transitions through the states `queued` → `running` → `completed` (or `failed` or `canceled`). Each job has a UUID assigned at creation time. Jobs are persisted in the SQLite `job` table (see Section 9.7), which means they survive process restarts. The `GET /api/v1/jobs` endpoint returns jobs from the database, not from process memory. Completed and failed jobs are retained in the database for 24 hours after their `finished_at` timestamp; a periodic cleanup task deletes expired rows on server startup and every 60 minutes thereafter.

Idempotency: The `POST /api/v1/index` endpoint accepts an optional **Idempotency-Key** HTTP header (a client-chosen string, typically a UUID). If a request arrives with an idempotency key that matches an entry in the `idempotency` table (see Section 9.8) and the entry has not expired, the server returns the stored response (the same `job_id` and `session_id`) without creating a new job. Idempotency entries share the same 24-hour retention window as job records, so the two guarantees are consistent. This prevents duplicate indexing operations when clients retry after network timeouts.

Concurrent job policy: At most one indexing job per session can be in the `queued` or `running` state at any time. If a client submits an index request for a session that already has an active job, the server responds with `409 Conflict` and includes the existing `job_id` in the response body so the client can poll the running job instead. Search requests are not subject to this constraint and execute concurrently without limit. HNSW rebuild jobs are also limited to one per session; a rebuild request while a rebuild is running returns the existing rebuild job's ID.

11.5 API Versioning Policy

The API uses URL-based versioning with the prefix `/api/v1/`. The versioning policy follows these rules:

- **Backwards-compatible changes** (adding fields to response objects, adding optional

request fields, adding endpoints) are made within the current version without incrementing the version number.

- **Breaking changes** (removing fields, changing field types, removing endpoints, changing endpoint semantics) require a new version prefix (e.g., `/api/v2/`). The previous version remains available for at least two minor application releases after the new version is introduced.
- **Deprecation:** Deprecated endpoints include a `X-Deprecated: true` response header and a `sunset` field in the response body indicating the version in which the endpoint will be removed.

11.6 Claim Verification Endpoint

The `/api/v1/verify` endpoint provides a heuristic evidence-matching service that determines whether a given claim is supported by a set of source passages. It does not use a generative language model and produces no generated text. The verification logic operates entirely through deterministic scoring:

1. **Keyword overlap:** Compute the Jaccard similarity between the claim’s token set and each candidate passage’s token set. Tokens are lowercased and stemmed using a lightweight Porter stemmer.
2. **Semantic similarity:** Embed the claim using the active embedding model and compute cosine similarity against each candidate passage’s stored embedding.
3. **Combined score:** Weight the keyword overlap and semantic similarity (default: 0.3 keyword, 0.7 semantic) into a single evidence score per passage.
4. **Verdict:** If the highest evidence score exceeds a configurable upper threshold (default: 0.75), the verdict is “supports”. If it falls between the upper and a lower threshold (default: 0.45), the verdict is “partial”. Below the lower threshold, the verdict is “not_supported”.
5. **Quote highlighting:** The response includes the substring of the top-scoring passage with the highest local token overlap with the claim, extracted as a character-offset range. This substring is the “matching quote” shown in the web frontend.

The response contains: `verdict` (string – the overall verdict based on the highest-scoring passage), `evidence_score` (float – the highest combined score among all candidate passages), `matching_quote` (string – the highlighted substring from the top-scoring passage), `source_file` (string – the file containing the top-scoring passage), `page_start` (integer), `page_end` (integer), and `per_passage_scores` (array of objects, one per candidate passage, each containing `chunk_id`, `keyword_score`, `semantic_score`, and `combined_score`). When multiple passages are provided, the verdict is determined by the single highest-scoring passage, not by aggregation across all passages.

11.7 Network Binding and Access Control

The API server binds to `127.0.0.1:3030` by default, restricting access to the local machine. This binding is sufficient for the primary use case where the web frontend and API run on the same host. The port is configurable via the `--port` CLI flag, the `port` field in the configuration file, or the `NEURONCITE_PORT` environment variable (precedence: CLI flag >

environment variable > config file > default). If the configured port is already in use, the server tries the next five consecutive ports before failing with an error. The effective bind address and port are reported in the `/api/v1/health` response and printed to stdout on startup.

LAN binding: An optional configuration flag `bind_address` allows the server to listen on `0.0.0.0:3030` for access from other machines on the local network. When LAN binding is enabled, the server requires a bearer token for authentication. The token is generated at startup, displayed in the web frontend’s status area (in web mode), and written to `.neuroncite/api_token.txt`. The token is *not* printed to stdout by default to avoid leaking credentials into shell histories or CI logs. The `--print-token` CLI flag explicitly opts in to printing the token to stdout on startup; this is intended for scripted environments that capture the token programmatically. All API requests must include an `Authorization: Bearer <token>` header; requests without a valid token receive a 401 Unauthorized response. The token is excluded from log output to prevent accidental exposure.

CORS: When LAN binding is enabled, the server sets permissive CORS headers (`Access-Control-Allow-Origin: *`) to allow browser-based clients on the local network to access the API. When bound to localhost, CORS headers are omitted. As an alternative to the wildcard origin, the configuration file accepts a `cors_origins` list (e.g., `["http://192.168.1.0:3000"]`) that restricts allowed origins to specific addresses. When `cors_origins` is set, the server uses those values instead of the wildcard.

Private subnet restriction: For deployments where the server should be reachable from the local network but not from the public internet, the `bind_address` field can be set to a private subnet IP (e.g., `192.168.1.100`) rather than `0.0.0.0`. This limits the listening interface to a single network adapter and avoids exposing the API on public-facing interfaces, adding a network-level access boundary in addition to the bearer token.

12 Data Flow

12.1 Indexing Pipeline

The indexing pipeline transforms a directory of PDF files into searchable embeddings stored in SQLite and an HNSW index. It consists of eight sequential stages. For incremental re-indexing of an existing session, stages 3–8 operate only on new and modified files:

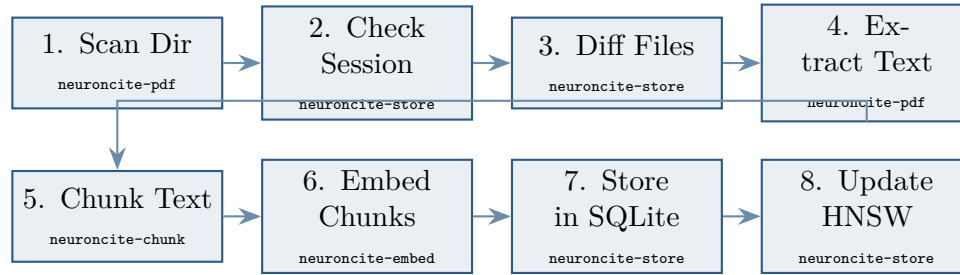


Figure 2: The eight stages of the indexing pipeline. Stage 3 (Diff Files) compares the current directory contents against `indexed_file` to identify new, modified, and deleted files. Unchanged files are skipped.

1. **Scan Directory:** Recursively discover all PDF files in the selected directory and its subdirectories.
2. **Check Session:** Query the SQLite database for an existing `index_session` with matching directory, model, and chunk configuration. If found, load the existing HNSW index and proceed to the incremental diff stage (step 3). If not, create a new session record and process all discovered files.
3. **Diff Against `indexed_file`:** Compare the discovered file list against the `indexed_file` table for this session. Classify each file as *new* (not present in `indexed_file`), *modified* (`mtime` or `size` changed and SHA-256 hash differs), *deleted* (present in `indexed_file` but absent from the directory scan), or *unchanged* (meta-data matches). Delete the `indexed_file` rows for deleted files (cascading to chunks and pages). For modified files, physically delete their old `page` and `chunk` rows (the `AFTER DELETE` trigger on `chunk` removes the corresponding FTS5 entries; the old HNSW vectors are temporary orphans until the graph rebuild in step 8). Skip unchanged files entirely.
4. **Extract Text:** For each new or modified PDF file, extract text page by page using the configured backend (pdf-extract, pdfum, or auto with OCR fallback). Compute a SHA-256 hash of the file contents. Store extracted page text in the `page` table. Record or update the file in the `indexed_file` table.
5. **Chunk Text:** Apply the selected chunking strategy to the extracted pages, producing a list of `Chunk` values for each file. Each chunk records its document-level UTF-8 byte offsets and the page range it spans.
6. **Embed Chunks:** Pass all chunk texts through the selected embedding backend in batches. The backend processes each batch on the GPU and returns the corresponding embedding vectors. Each vector is normalized to unit length.

7. **Store in SQLite:** Write each chunk’s text, metadata, and embedding blob to the `chunk` table and the FTS5 index. This is done in a single SQLite transaction for performance (bulk insert).
8. **Update HNSW Index:** Build a new HNSW graph from all active embedding vectors of the session (existing chunks where `is_deleted = 0`, plus the newly created chunks). The new graph is constructed on a background thread without affecting the current search index. Once the build completes, the session’s entry in the per-session HNSW map is atomically replaced via `insert_hnsw()`, and the old graph is dropped. This approach eliminates orphan accumulation for sessions that are re-indexed: each indexing run produces a clean graph. For sessions where files are only added (no deletions or modifications), the rebuild overhead is acceptable because the HNSW construction cost is dominated by the GPU embedding time that has already occurred. Serialize the graph to the session’s index file in the `.neuroncite/` directory. Record the model manifest in `models.lock`.

12.2 Search Pipeline

The search pipeline transforms a natural-language query into a ranked list of relevant document passages with citations. It consists of five stages:

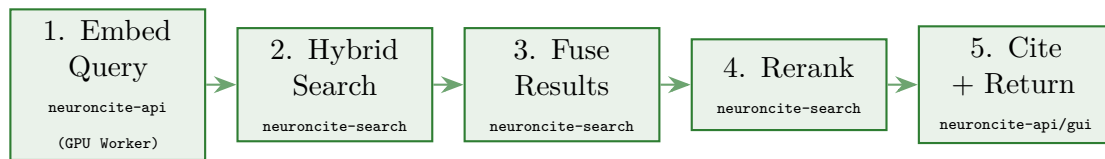


Figure 3: The five stages of the search pipeline.

1. **Embed Query:** The query string is passed through the same embedding model that was used for indexing, producing a single query vector normalized to unit length.
2. **Hybrid Search:** Two parallel searches are executed:
 - *Vector:* The HNSW index is queried with the query vector, returning the top N_{vec} candidates with cosine similarity scores.
 - *Keyword:* The FTS5 index is queried with the raw query string, returning the top N_{kw} candidates with BM25 scores.
 If hybrid mode is disabled, only the vector search is executed.
3. **Fuse Results:** The two candidate lists are merged using Reciprocal Rank Fusion (RRF). Candidates appearing in both lists receive a higher combined score. The top K_{rerank} candidates proceed to the next stage.
4. **Rerank (optional):** If a cross-encoder reranker is configured, the fused candidates are re-scored by jointly encoding each (query, chunk) pair. The reranker score replaces the RRF score as the primary ranking criterion. If no reranker is configured, this stage is skipped.
5. **Cite and Return:** For each of the final top-K results, a `Citation` is generated from the chunk metadata (source file, page range, document-level byte offsets resolved to page-local positions via the `page.byte_count` column). The `SearchResult` objects are returned to the GUI or API caller.

13 Concurrency Model

The application must run an HTTP server, serve the web frontend, and potentially long-running GPU computations simultaneously without blocking any of them.

13.1 Thread Architecture

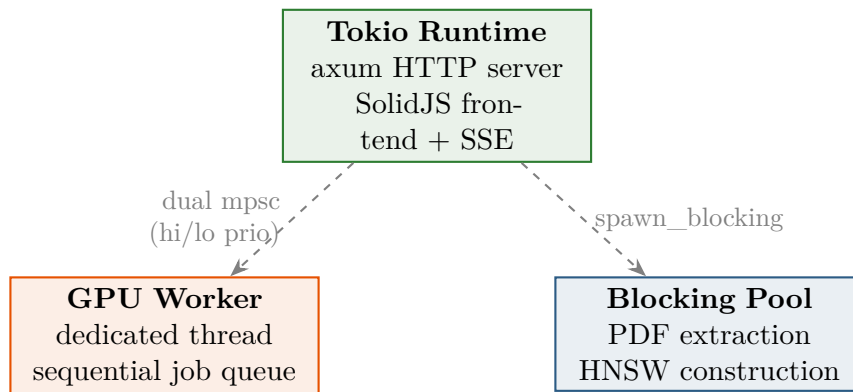


Figure 4: Thread architecture overview. The GPU Worker runs on a dedicated thread and processes embedding and reranking jobs from two channels (high-priority for search, low-priority for indexing).

Tokio Multi-Threaded Runtime

The tokio runtime hosts the axum HTTP server, serves the embedded SolidJS frontend, manages SSE broadcast channels for real-time updates, and coordinates all asynchronous background tasks (indexing, searching, model operations).

GPU Worker Thread

A single dedicated thread that owns the embedding backend and the reranker model. The thread is spawned and managed by the `worker` module in `neuroncite-api` (see `src/worker.rs` in the module structure table). It reads jobs from two separate bounded `tokio::sync::mpsc` channels—a high-priority channel for search-time jobs (query embedding, reranking) and a low-priority channel for indexing-time jobs (batch embedding). The worker’s receive loop uses `tokio::select!` biased toward the high-priority channel: on each iteration it drains all pending high-priority jobs before polling the low-priority channel. This ensures that interactive queries are serviced with minimal latency even while a large indexing operation is in progress, without requiring a custom priority queue. Each job carries a oneshot response channel through which the GPU worker returns the result to the caller. The `WorkerHandle` struct provides the public interface (`embed_query`, `embed_batch`, `rerank_batch`) that API handlers and web request handlers use to submit jobs. This architecture eliminates the need for a `Mutex` around the GPU resources: the worker thread is the sole owner, and all access is serialized through the handle’s channels.

Blocking Thread Pool

CPU-bound operations (PDF text extraction, HNSW index construction, SHA-256

hashing) are dispatched to tokio’s blocking thread pool via `spawn_blocking`. This prevents these operations from starving the async executor. GPU operations are not dispatched here – they go through the dedicated GPU Worker Thread.

Cancellation: Each long-running job (batch embedding during indexing) checks a `CancellationToken` between batches. When the user cancels an indexing operation through the GUI, the token is triggered, and the GPU worker discards the remaining batches for that job. The cancellation is cooperative – the current batch runs to completion before the cancellation takes effect.

13.2 Shared State Synchronization

- **SQLite connection pool:** Managed by `r2d2`. The pool handles concurrent access internally. No additional application-level mutex is needed for database operations. WAL mode allows concurrent readers alongside a single writer. The `busy_timeout` PRAGMA of 5000 ms causes SQLite to sleep and retry internally when the write lock is held by another connection; if the lock is not released within 5000 ms, SQLite returns `SQLITE_BUSY`. The GUI treats a busy-timeout error as a transient condition and displays “Database busy, retrying...” rather than a fatal error, retrying the operation up to three times with exponential backoff.
- **HNSW index map:** Stored in an `ArcSwap<HashMap<i64, Arc<HnswIndex>>>`. Each session has its own HNSW graph keyed by `session_id`. Multiple search queries read the current map concurrently via a load of the `Arc` pointer (lock-free). Index construction and rebuilds build a new HNSW graph into a separate object without affecting the current map. Once the new graph is complete, the map is cloned, the session’s entry is inserted, and the `ArcSwap` is atomically updated. Session deletion removes the entry from the map via `remove_hnsw()` or `remove_hnsw_many()`. On startup, `load_all_session_hnsw()` populates the map from all sessions with stored embeddings in the database.
- **Embedding backend and reranker:** Owned exclusively by the GPU Worker thread. No mutex is needed – all access is mediated through the `WorkerHandle` interface (defined in `neuroncite-api/src/worker.rs`). The GUI and API call typed methods on the handle (`embed_query`, `embed_batch`, `rerank_batch`) and await the response on a oneshot channel. This avoids holding a `std::sync::Mutex` across `.await` points, which would risk deadlocks in the async context.
- **Index progress:** Reported through a `tokio::sync::watch` channel carrying `IndexProgress` values. The background task sends an updated progress snapshot after each file; the GUI calls `try_recv()` on the watch receiver each frame to read the latest snapshot without blocking. The watch channel retains only the most recent value, making it suitable for high-frequency progress updates where intermediate values can be skipped.

13.3 Graceful Shutdown

The application implements a coordinated shutdown sequence that ensures data integrity and releases resources in the correct order. Shutdown is triggered by three events: (1) closing the GUI window, (2) sending `SIGINT` (Ctrl+C) or `SIGTERM` to the process in headless

mode, or (3) calling `POST /api/v1/shutdown` (only available in headless mode, protected by the bearer token when LAN binding is active).

Shutdown sequence:

1. The global `CancellationToken` is triggered, signaling all background tasks to stop accepting new work.
2. The axum HTTP server stops accepting new connections and waits up to 5 seconds for in-flight requests to complete. After the grace period, remaining connections are dropped.
3. The GPU Worker thread finishes its current batch (if any), then exits its job processing loop. No new jobs are accepted after the cancellation token fires.
4. Any in-progress HNSW serialization completes. If serialization has not started for a modified graph, the system performs a final serialization to prevent data loss.
5. The SQLite connection pool is drained: all checked-out connections are returned, pending WAL checkpoints are flushed, and the pool is closed.
6. The tokio runtime shuts down, joining all spawned tasks.
7. The process exits with code 0.

Forced shutdown: If the graceful shutdown does not complete within 15 seconds, the process exits with code 3. This timeout prevents indefinite hangs caused by unresponsive GPU operations or stuck I/O.

14 Cargo Feature Flags

Feature flags control which embedding backends are compiled into the binary. This allows users to build a minimal binary with only the backend they need, or a full binary with all three.

14.1 Feature Definitions

Embedding backend features are defined on the `neuroncite-embed` crate and forwarded through the workspace root and the binary crate. The `web`, `mcp`, `pdfium`, and `ocr` features are defined on the binary crate and `neuroncite-pdf`, respectively.

14.2 Build Commands

14.3 Conditional Compilation

Inside the `neuroncite-embed` crate, each backend module is wrapped in a `#[cfg(feature = "...")]` attribute. A backend registry function iterates over all compiled-in backends and collects their `ModelInfo` lists. The GUI and API only see backends that were actually compiled in.

Feature	Effect
<code>backend-ort</code> (default)	Compiles the ONNX Runtime backend. Pulls in the <code>ort</code> and <code>ndarray</code> crates. Provides both embedding and reranking via ONNX models. Requires ONNX Runtime shared libraries at runtime for GPU execution.
<code>web</code> (default)	Compiles the <code>neuroncite-web</code> crate and pulls in <code>rust-embed</code> , <code>mime_guess</code> , and <code>opener</code> . Enables the <code>neuroncite web</code> CLI subcommand that starts an Axum server with the embedded SolidJS frontend and opens a native OS window (when the <code>gui</code> feature is also enabled) or the default browser. The <code>backend-ort</code> feature is forwarded to <code>neuroncite-web</code> via optional dependency syntax. Defined on the binary crate.
<code>pdfium</code> (default)	Enables the <code>pdfium-render</code> backend in <code>neuroncite-pdf</code> for PDFs with multi-column layouts, complex tables, and non-standard font encodings. Requires the pdfium shared library at runtime. Defined on <code>neuroncite-pdf</code> , forwarded by the binary crate.
<code>ocr</code> (default)	Enables Tesseract-based OCR fallback for image-heavy pages in <code>neuroncite-pdf</code> . Tesseract is invoked as a CLI subprocess and must be installed separately via the system package manager or the Doctor panel's install button. Implies the <code>pdfium</code> feature for page rendering to raster images. Defined on <code>neuroncite-pdf</code> , forwarded by the binary crate.
<code>mcp</code> (default)	Compiles the <code>neuroncite-mcp</code> crate and enables the <code>mcp serve</code> , <code>mcp install</code> , <code>mcp uninstall</code> , and <code>mcp status</code> CLI subcommands. The MCP crate is an optional dependency of the binary crate (<code>dep:neuroncite-mcp</code>). When disabled, all MCP-related CLI subcommands are hidden and the binary has no MCP protocol code. The <code>backend-ort</code> feature is forwarded to <code>neuroncite-mcp</code> via the <code>neuroncite-mcp?/backend-ort</code> optional dependency syntax. Defined on the binary crate.
<code>gui</code> (default)	Compiles the native OS window module in <code>neuroncite-web</code> using <code>tao</code> for the event loop and <code>wry</code> for the WebView. When enabled, the <code>web</code> subcommand opens a dedicated application window instead of a browser tab. The feature is forwarded from the binary crate to <code>neuroncite-web</code> via <code>neuroncite-web?/gui</code> . When disabled or when native window creation fails at runtime, the <code>opener</code> -based browser launch is used as fallback. Defined on <code>neuroncite-web</code> , forwarded by the binary crate.

Table 29: Feature flag definitions and their effects.

Command	Result
<code>cargo build --release</code>	Builds with default features (backend-ort, web, gui, mcp, pdfium, ocr).
<code>cargo build --release --all-features</code>	Builds with all features enabled.

Table 30: Example build commands for different backend combinations.

15 Error Handling

15.1 Strategy

The project follows the standard Rust error handling convention:

- **Library crates** (core, pdf, chunk, embed, store, search) use the `thiserror` crate to define typed error enumerations. Each crate has its own error type with variants for each failure mode.
- **Application crates** (api, gui, binary) use the `anyhow` crate for error aggregation, converting library errors into a unified error type with context messages.

15.2 Error Propagation

Library errors propagate upward through the `Result` type using the `?` operator. At the API boundary, errors are mapped to appropriate HTTP status codes:

Error Category	HTTP Status
Invalid request parameters	400 Bad Request
Session or resource not found	404 Not Found
Concurrent index job for the same session	409 Conflict
PDF extraction or embedding failure	500 Internal Server Error

Table 31: Error-to-HTTP-status mapping.

In the GUI, errors are displayed as dismissable notification banners at the top of the results panel. The error message includes the error chain (original cause and context) for debugging.

16 Model Management

The system downloads, caches, and tracks embedding and reranker models. Model management ensures reproducibility (the same model version produces the same embeddings), supports offline operation, and allows users to register locally stored models.

16.1 Model Cache

Model weights are cached in a platform-specific directory:

- **Windows:** %LOCALAPPDATA%\neuroncite\models\
- **Linux:** ~/.cache/neuroncite/models/

Each model is stored in a subdirectory named by its identifier and revision hash (e.g., BAAI-bge-small-en-v1.5-abc123/). This allows multiple revisions of the same model to coexist.

16.2 Model Manifest (`models.lock`)

Each `.neuroncite/` directory contains a `models.lock` file that records the exact model versions used to build the index. This file serves the same purpose as `Cargo.lock`: it pins the model revision so that the index can be rebuilt identically.

The manifest contains one entry per model used in the directory’s index sessions. Each entry records:

- **Model identifier:** The Hugging Face model ID (e.g., “BAAI/bge-small-en-v1.5”).
- **Revision:** The specific commit hash on Hugging Face Hub.
- **SHA-256 checksum:** Hash of the model weights file for integrity verification.
- **Tokenizer checksum:** SHA-256 hash of the tokenizer vocabulary file (`tokenizer.json`). The token-window chunking strategy depends on the tokenizer to produce deterministic chunk boundaries. Pinning the tokenizer checksum ensures that the same model revision with the same tokenizer always produces identical chunks, even if the tokenizer file is re-downloaded.
- **Source URL:** The URL from which the model was downloaded.
- **License:** The model’s license identifier.
- **Local path:** Absolute path to the cached model directory on the machine that last wrote the manifest. This field is a diagnostic hint used to locate the model cache without re-scanning the file system. On a different machine, the application re-resolves the model by identifier and revision from the local cache directory, ignoring an invalid `local_path`.

16.3 Download and Verification

On first use of a model, the system:

1. Resolves the latest revision from Hugging Face Hub (or uses a pinned revision if specified in `models.lock`).

2. Downloads the model weights and tokenizer files.
3. Computes the SHA-256 checksum and verifies it against the manifest (if present).
4. Writes or updates the `models.lock` entry.
5. Stores the files in the model cache directory.

The download progress is shown in the GUI's progress bar.

16.4 Offline Operation

If the model is already cached, no network access is required. If the model is not cached and the system is offline, the application displays an error message identifying the missing model and its expected cache path, so the user can manually place the files.

16.5 Local Model Registration

Users can register locally stored model files (ONNX format) through the GUI settings or the API. A registered model is added to the `models.lock` manifest with a `file://` source URL and its computed checksum. This enables use of custom or proprietary models without requiring Hugging Face Hub access.

17 Build and Deployment

17.1 Build Output

The final build artifact is a single executable file. On Windows, this is `neuroncite.exe`. The binary contains:

- The SolidJS web frontend (embedded via rust-embed, served by axum)
- The axum HTTP server
- The SQLite engine (statically linked via rusqlite’s bundled feature)
- The HNSW index implementation
- The selected embedding backend(s) and reranker(s)
- All tokenizer vocabularies (embedded as assets or downloaded on first use)

17.2 Runtime Dependencies

The runtime dependencies depend on the build configuration:

Configuration	Runtime Requirements
CPU-only (any backend, <code>pdf-extract</code> only)	None for the core embedding and search pipeline. GPU acceleration is unavailable, but all core functionality works on CPU. PDFium and Tesseract dependencies are listed separately below and apply regardless of CPU or GPU mode.
<code>backend-ort</code> + GPU	ONNX Runtime shared libraries (<code>onnxruntime.dll</code> / <code>libonnxruntime.so</code>), NVIDIA CUDA toolkit, and cuDNN.
PDF extraction (<code>pdfium</code>)	PDFium shared library (<code>pdfium.dll</code> / <code>libpdfium.so</code>). Required if the pdfium extraction backend is used or if OCR fallback is enabled (OCR uses pdfium for page-to-image rendering).
OCR	Tesseract OCR engine, trained language data files, and the PDFium shared library (for page rendering). Only required if OCR fallback is triggered.

Table 32: Runtime dependencies by configuration.

17.3 Distribution Profiles

The “single binary” claim holds for the CPU-only build, but GPU-enabled builds and PDF extraction via pdfium require external shared libraries. To prevent user confusion,

the project defines four explicit distribution profiles:

Profile	Contents and Dependencies
Portable CPU	A single <code>neuroncite.exe</code> (or <code>neuroncite</code> on Linux) with no external dependencies. All processing runs on CPU. Embedding inference is slower but fully self-contained. Suitable for machines without a GPU.
GPU NVIDIA (ort)	The binary plus bundled ONNX Runtime shared libraries (<code>onnxruntime.dll</code> / <code>libonnxruntime.so</code>) and a cuDNN redistribution. Requires the NVIDIA CUDA toolkit to be installed on the system. Distributed as a ZIP archive with a directory layout that places all DLLs alongside the binary.

Table 33: Distribution profiles. Each profile targets a specific hardware and dependency configuration.

All profiles optionally include PDFium (`pdfium.dll` / `libpdfium.so`) for the pdfium extraction backend and Tesseract language data files for OCR. These optional components are detected at runtime, and missing components degrade gracefully (fallback to `pdf-extract` or skipping OCR).

17.4 Dependency Doctor

The GUI includes a **Dependency Doctor** panel accessible from the settings menu. On application startup, and on demand through a “Check Dependencies” button, the system probes for all optional runtime dependencies and reports their status:

- **CUDA toolkit:** Checks for `nvcc` on the system PATH and reports the detected version. If missing, displays the NVIDIA download URL.
- **cuDNN:** Checks for `cuda64_8.dll` (Windows) or `libcudnn.so` (Linux) in the library search path. Reports version if found.
- **ONNX Runtime:** Checks for `onnxruntime.dll` / `libonnxruntime.so` adjacent to the binary or on the library path. If missing and the ort backend is compiled in, offers a download button that fetches the correct version from the ONNX Runtime GitHub releases.
- **PDFium:** Checks for `pdfium.dll` / `libpdfium.so` adjacent to the binary. If missing, offers a download button that fetches the library from the pdfium-binaries GitHub releases.
- **Tesseract:** Checks for the `tesseract` binary on the system PATH and for the `TESSDATA_PREFIX` environment variable. Reports available language data files.
- **GPU driver:** Queries the GPU vendor and driver version via the platform’s graphics API (DXGI on Windows, Vulkan on Linux). Reports whether the driver is compatible with the compiled-in backends.

Each dependency displays one of three states: “Found (version X.Y)”, “Missing (required for Z)”, or “Not needed (feature not compiled in)”. Missing dependencies that have a known download source display an actionable download button.

17.5 Continuous Integration

A CI pipeline builds and tests the application across a matrix of:

- Operating systems: Windows, Linux
- Feature combinations: ort-only, all-features
- Rust toolchain: stable

Tests run with the CPU execution provider to avoid requiring GPU hardware in CI.

18 Command-Line Interface

The binary supports a set of subcommands parsed by the `clap` crate. When the binary is invoked without any subcommand, it defaults to web mode (equivalent to `neuroncite web`). All subcommands that produce output write machine-readable JSON to stdout by default. A `--format text` flag switches to human-readable tabular output for interactive terminal use.

18.1 Subcommands

18.2 Exit Codes

18.3 JSON Output Contract

CLI subcommands that correspond to API endpoints produce JSON output matching the API response schema where the semantics are equivalent. For example, the output of `neuroncite search` matches the response body of `POST /api/v1/search`, and `neuroncite doctor` matches `GET /api/v1/health`. The `neuroncite index` subcommand is an exception: because it runs synchronously (blocking until indexing completes), its output is a completion result containing `session_id`, `chunks_created`, `files_processed`, and `elapsed_seconds`—not the asynchronous `job_id` response returned by `POST /api/v1/index`. Clients switching between CLI and API invocation must account for this difference in the index operation.

Progress output (during `neuroncite index`) is written to stderr as newline-delimited JSON (NDJSON) objects, one per processed file. This separation ensures that stdout contains only the final result object, while stderr provides real-time progress updates that can be consumed by monitoring tools or discarded.

Two flags control progress output behavior:

- `--quiet` – suppresses all stderr progress output. Only the final JSON result is written to stdout. Useful for scripts that do not need progress reporting and want to avoid parsing or redirecting stderr.
- `--progress mode` – selects the progress output format. Accepted values: `ndjson` (default, one JSON object per file on stderr) and `none` (equivalent to `--quiet`). Future versions may add additional formats (e.g., a compact single-line counter).

When `--quiet` is combined with `--progress ndjson`, the `--quiet` flag takes precedence and progress output is suppressed.

18.4 Model Management Subcommands

The `neuroncite models` subcommand group provides LLM agents and automation scripts with programmatic access to the embedding model catalog, local cache state, download operations, integrity verification, and GPU/system capability detection. All five nested

Subcommand	Description
web	Starts the Axum HTTP server with the embedded SolidJS frontend and opens a native OS window (with the <code>gui</code> feature) or the default browser (without <code>gui</code>). This is the default mode when no subcommand is given (double-click behavior on Windows). Accepts <code>--port</code> , <code>--bind</code> , and <code>--config</code> flags.
serve	Starts the API server in headless mode without a browser window. The server runs until terminated by <code>SIGINT</code> (Ctrl+C), <code>SIGTERM</code> , or a <code>/api/v1/shutdown</code> request. Accepts <code>--port</code> , <code>--bind</code> , <code>--print-token</code> , and <code>--config</code> flags.
index	Indexes a directory from the command line without starting a persistent HTTP server. Internally, this subcommand spawns a tokio runtime and initializes the embedding backend (including GPU resources if available) for the duration of the indexing operation; both are torn down on completion. Accepts <code>--directory</code> , <code>--model</code> , <code>--strategy</code> , <code>--chunk-size</code> , <code>--overlap</code> , and <code>--ocr-language</code> flags. Outputs a JSON object with the <code>session_id</code> and indexing statistics on completion.
search	Executes a single search query against an existing session. Accepts <code>--directory</code> , <code>--session-id</code> , <code>--query</code> , <code>--top-k</code> , <code>--hybrid</code> , and <code>--rerank</code> flags. Outputs a JSON array of <code>SearchResult</code> objects.
doctor	Runs the Dependency Doctor checks and reports the status of all optional runtime dependencies (CUDA, cuDNN, ONNX Runtime, PDFium, Tesseract, GPU driver). Outputs a JSON object with each dependency’s status.
sessions	Lists all index sessions in the specified directory’s <code>.neuroncite/</code> database. Accepts a <code>--directory</code> flag. Outputs a JSON array of session summary objects.
export	Exports search results from a previous search to Markdown, BibTeX, or CSL-JSON. Accepts <code>--directory</code> , <code>--session-id</code> , <code>--query</code> , <code>--format</code> (markdown, bibtex, csl-json), and <code>--output</code> (file path, defaults to stdout).
annotate	Runs the PDF annotation pipeline from the command line. Accepts <code>--input</code> (CSV or JSON quote file), <code>--pdf</code> (source PDF path), and <code>--output</code> (annotated PDF path) flags. Outputs a JSON object with the annotation results on completion.
models	Model management subcommand group for LLM agent control. Provides five nested subcommands (<code>list</code> , <code>info</code> , <code>download</code> , <code>verify</code> , <code>system</code>) that expose the embedding model catalog, cache status, download operations, integrity verification, and GPU/system capability detection. All output is machine-readable JSON by default. See Section 18.4 for the full schema.
mcp	MCP server management subcommand group. Provides four nested subcommands: <code>serve</code> (start the MCP stdio server), <code>install</code> (register in Claude Code MCP settings), <code>uninstall</code> (remove registration), and <code>status</code> (report registration status).
version	Prints the application version, build features, and Git commit hash as a JSON object (or a single-line text format).

Table 34: CLI subcommands. All subcommands support the global flags `--format` (json or text), `--config` (path to configuration file), and `--log-level` (trace, debug, info, warn,

Code	Meaning
0	The operation completed successfully.
1	Application error (indexing failure, search failure, database error, GPU initialization failure). The error message is written to stderr as a JSON object with error and code fields.
2	Usage error (invalid arguments, missing required flags, unknown subcommand). Printed by clap with a help message.
3	Shutdown timeout (the graceful shutdown did not complete within 15 seconds).

Table 35: CLI exit codes. Scripts and agents can rely on these codes for control flow.

subcommands produce JSON to stdout by default and write errors to stderr as JSON objects with **error** and **code** fields.

18.4.1 **models list**

Lists all supported embedding models with their configuration parameters and local cache status. The JSON output contains a **models** array where each entry includes **model_id**, **display_name**, **vector_dimension**, **max_seq_len**, **pooling** (one of “cls”, “mean-pooling”, “last-token”), **uses_token_type_ids**, **query_prefix**, **document_prefix**, **cached** (boolean), and **cache_path** (present only when cached). Two summary fields **total** and **cached_count** provide the aggregate counts. The invariant **cached_count** $\leq$ **total** always holds.

18.4.2 **models info <model-id>**

Returns the full configuration and per-file status for a single model identified by its Hugging Face model ID (e.g., **BAAI/bge-small-en-v1.5**). In addition to the fields from **models list**, the response includes a **files** array listing each expected file with **filename**, **exists** (boolean), and **size_bytes** (present only when the file exists). If the model ID is not in the catalog, the command exits with code 1 and writes a JSON error object with **code** set to **model_not_found**.

18.4.3 **models download <model-id>**

Downloads the model files from Hugging Face Hub to the local cache. If the model is already cached, the command returns immediately with **status** set to **already_cached**. For a fresh download, the response includes **status** set to **downloaded**, the **cache_path**, and **elapsed_seconds** measuring the wall-clock download time. Download progress is written to stderr. An unknown model ID produces exit code 1 with a **model_not_found** error.

18.4.4 **models verify <model-id>**

Checks file existence and computes SHA-256 checksums for all expected files of the specified model. The response contains a **files** array with per-file **filename**, **exists**, and **sha256**

(present when the file exists) fields. A top-level `valid` boolean is `true` only when all expected files exist. An unknown model ID produces exit code 1.

18.4.5 `models system`

Reports system capabilities relevant to embedding model execution. The response contains a `gpu` object with three detection layers: `nvidia_detected` (whether an NVIDIA GPU is present, detected via `nvidia-smi`), `cuda_available` (whether the ONNX Runtime CUDA execution provider can initialize), and `gpu_ort_runtime` (whether the locally cached ORT runtime DLL was built with GPU support). A `backends` array lists the compiled-in embedding backend names. The `cache_dir` field provides the absolute path to the model cache directory, and `cache_disk_usage_bytes` reports the total disk usage of all cached models. This information enables agents to determine whether GPU acceleration is available before initiating indexing operations.

19 Operational Concerns

19.1 Configuration File

Application-wide settings are stored in a TOML configuration file. The configuration file is loaded from the following locations in order of precedence (first found wins):

1. Path specified by the `--config` CLI flag
2. Path specified by the `NEURONCITE_CONFIG` environment variable
3. `.neuroncite/config.toml` in the current working directory
4. Platform-specific user configuration directory:
 - Windows: `%APPDATA%\neuroncite\config.toml`
 - Linux: `~/.config/neuroncite/config.toml`

If no configuration file is found, the application uses built-in defaults. CLI flags and environment variables override the corresponding configuration file values.

Configuration fields:

19.2 Logging

The application uses the `tracing` crate for structured, leveled logging. Log output is configured as follows:

- **Stderr:** In both GUI and headless mode, log messages at the configured level and above are written to stderr in a human-readable format (`tracing-subscriber` compact formatter). In GUI mode, the log level defaults to `warn` to avoid cluttering the terminal that launched the application. In headless mode, the default level is `info`.
- **Log file:** When the `log_file` configuration field is set or the `--log-file` CLI flag is provided, log messages are additionally written to the specified file in JSON format (one JSON object per log event). The log file uses append mode and is safe for concurrent writes from multiple threads. Log rotation is left to external tools (e.g., `logrotate` on Linux).
- **GUI log panel:** In GUI mode, the last 500 log messages at `info` level and above are displayed in a scrollable log panel accessible from the settings menu. This enables diagnosing issues without leaving the application.

Log fields: Each log event includes the timestamp (ISO 8601), level, target module, thread name, span context (e.g., the current job ID or file being processed), and the message. Sensitive data (bearer tokens, file system paths containing usernames) is redacted in log output.

19.3 Storage Location for Operational Files

Field	Type	Description
port	integer	API server port (default: 3030).
bind_address	string	Bind address (default: “127.0.0.1”).
log_level	string	Minimum log level for stderr output: trace, debug, info, warn, error. Default: “info” in headless mode, “warn” in GUI mode (to avoid cluttering the terminal that launched the application). An explicit value in the config file or <code>--log-level</code> CLI flag overrides the mode-specific default.
log_file	string	Path to log file. If set, logs are written to this file in addition to stderr (default: null, no file logging).
default_model	string	Default embedding model identifier (default: “BAAI/bge-small-en-v1.5”).
default_strategy	string	Default chunking strategy (default: “word”).
default_chunk_size	integer	Default chunk size (default: 300).
default_overlap	integer	Default chunk overlap (default: 50).
ocr_language	string	Default Tesseract language (default: “eng”).
ef_search	integer	Default HNSW ef_search parameter (default: 100).
rrf_k	integer	Default RRF fusion parameter k (default: 60).
cors_origins	array	TOML array of allowed CORS origin strings for LAN mode (e.g., <code>["http://192.168.1.0:3000"]</code>). If empty or unset, all origins are allowed via wildcard. Ignored when bound to localhost (default: []).
simhash_threshold	integer	Maximum Hamming distance (out of 64 bits) below which two chunks are considered near-duplicates during deduplication (default: 3).
orphan_ratio_threshold	float	HNSW orphan ratio above which a rebuild is recommended (default: 0.2).

Table 36: Configuration file fields with their types and defaults.

File	Location
Database and HNSW index files	<base>/indexes/<sanitized-path>/. Storage is centralized under a single root directory (<base>), not inside the indexed directory itself. The path sanitization converts absolute paths to flat names (e.g., D:\Papers\Finance becomes D_--Papers--Finance).
Base directory (<base>)	Windows: %APPDATA%\NeuronCite (fall-back: %USERPROFILE%\Documents\NeuronCite). Linux: \$XDG_DATA_HOME/NeuronCite (fall-back: ~/Documents/NeuronCite). macOS: ~/Documents/NeuronCite.
Runtime dependencies	<base>/runtime/ (ONNX Runtime DLLs, pdfium shared library, Tesseract executable and tessdata).
Model cache	<base>/models/ (downloaded ONNX embedding and reranker model files from HuggingFace).
Configuration file	User-specified via <code>--config <path></code> CLI flag. No default location; <code>AppConfig::default()</code> is used when no file is provided. The file format is TOML.
Log file (if configured)	User-specified path.
Bearer token (LAN mode)	Generated at runtime and printed to stdout when <code>--print-token</code> is passed to <code>neuroncite serve</code> . Not persisted to disk.
Export files	<base>/indexes/<sanitized-path>/exports/.

Table 37: File system locations for operational files. All paths are relative to the centralized base directory (<base>) resolved by `neuroncite_core::paths::base_dir()`.

20 Agent Integration and Python Client

The application is designed to be operated by automated agents (LLM-based or script-based) in addition to human users. This section describes the integration contract and the reference Python client.

20.1 Integration Modes

An automated agent can interact with NeuronCite through four interfaces, ordered by recommendation:

1. **MCP stdio server (recommended for Claude Code):** The agent runtime spawns `neuroncite mcp serve` as a subprocess and communicates via JSON-RPC 2.0 over stdin/stdout. Claude Code auto-starts the MCP server based on the registration entry in `~/.claude/claude_mcp_settings.json`. This mode provides the tightest integration with AI assistants that support the Model Context Protocol.
2. **HTTP API (recommended for general use):** The agent starts the server via `neuroncite serve` (or relies on an already-running instance) and sends HTTP requests to `http://127.0.0.1:3030/api/v1/`. This mode supports concurrent operations, job tracking, and long-lived sessions.
3. **CLI subcommands:** The agent invokes `neuroncite index`, `neuroncite search`, etc. as subprocess calls and parses the JSON output from stdout. This mode requires no running server and is suitable for one-shot operations in pipelines.
4. **Python client library:** A thin Python wrapper around the HTTP API that provides typed method signatures and handles server lifecycle management.

20.2 Server Lifecycle for Agents

Agents that use the HTTP API need a running server. The recommended pattern:

1. Start the server as a subprocess: `neuroncite serve --port 3030`
2. Wait for the server to become ready by polling `GET /api/v1/health` until it returns HTTP 200.
3. Perform indexing and search operations via the API.
4. Shut down the server by sending `SIGINT` to the subprocess or (if in headless mode) by calling `POST /api/v1/shutdown`.

The Python client library automates this lifecycle (see below).

20.3 Python Client Library

The repository contains a reference Python client in the `clients/python/` directory, installable via `pip install -e clients/python/`. The client provides a minimal, typed interface for all API operations.

Client architecture:

- **Class `NeuronCiteClient`:** Wraps all HTTP API calls. Accepts `base_url` (default: `http://127.0.0.1:3030`) and an optional `token` for LAN-mode authentication. 33 public methods organized by domain:
 - **System:** `health()`, `backends()`, `shutdown()`.
 - **Indexing:** `index()`.
 - **Search:** `search()`, `hybrid_search()`, `multi_search()`.
 - **Verification:** `verify()`.
 - **Jobs:** `get_job()`, `wait_for_job()`, `cancel_job()`, `list_jobs()`.
 - **Sessions:** `list_sessions()`, `delete_session()`, `delete_sessions_by_directory()`, `optimize_session()`, `rebuild_index()`.
 - **Documents:** `get_page()`, `quality_report()`, `compare_files()`, `discover()`.
 - **Annotation:** `annotate()`, `annotate_from_file()`.
 - **Citation:** `citation_create()`, `citation_claim()`, `citation_submit()`, `citation_status()`, `citation_rows()`, `citation_export()`, `citation_auto_verify()`, `citation_fetch_sources()`, `citation_parse_bib()`, `citation_bib_report()`.
- **Class `NeuronCiteServer`:** Manages the server subprocess. The constructor accepts the path to the `neuroncite` binary and optional configuration (port, bind address, log level). The `start()` method launches the subprocess and blocks until the health endpoint responds. The `stop()` method sends `SIGINT` and waits for the process to exit. Implements the context manager protocol (`__enter__`/`__exit__`) for use in `with` statements.
- **Data classes:** Typed dataclasses defined in the `neuroncite.models` module for all response structures (`IndexResponse`, `SearchResult`, `JobStatus`, `HealthResponse`, `CitationCreateResponse`, etc.). These mirror the Rust DTOs in `neuroncite-api/src/dto.rs`.

Usage example:

Python Client Usage

```
from neuroncite import NeuronCiteServer, NeuronCiteClient

with NeuronCiteServer("./neuroncite.exe", port=3030) as server:
    client = NeuronCiteClient(base_url=server.url)

    # Index a directory
    job = client.index(
        directory="C:/papers/econometrics",
        model="BAAI/bge-small-en-v1.5",
        chunk_strategy="word",
        chunk_size=300,
        overlap=50,
    )
    result = client.wait_for_job(job.job_id)
```

```
# Search
results = client.search(
    session_id=result.session_id,
    query="financial econometrics methodology",
    top_k=5,
)
for r in results:
    print(r.citation.formatted)
```

20.4 OpenAPI Specification

The OpenAPI 3.1 specification is served at `GET /api/v1/openapi.json` and is generated at compile time from the Rust handler type signatures using the `utoipa` crate. The specification describes all endpoints, request bodies, response schemas, and error responses. Agents can use this specification to auto-generate client bindings in any language, validate requests before sending them, or feed the specification to an LLM as context for tool-use prompting.

The specification file is also checked into the repository at `docs/openapi.json` for offline reference and CI-based schema validation.

20.5 LLM Agent Tool-Use Contract

For LLM agents that use function-calling or tool-use interfaces, each API endpoint maps to a single tool definition. The tool descriptions, parameter schemas, and return value schemas are derived from the OpenAPI specification. The recommended approach is to provide the LLM with the OpenAPI specification as context and let the LLM construct HTTP requests directly.

Key design decisions for agent compatibility:

- All responses are flat JSON objects or arrays (no nested pagination wrappers). Agents can iterate over results directly.
- Enumerated string fields (e.g., job state, chunk strategy) use lowercase snake_case identifiers that are unambiguous and copy-paste-safe.
- Error responses include a machine-readable `code` field (e.g., “session_not_found”, “index_in_progress”) that agents can match against without parsing human-readable error messages.
- The `/api/v1/health` endpoint returns enough information for an agent to determine which features are available before attempting to use them.
- The `neuroncite models` CLI subcommand group (Section 18.4) provides agents with model catalog discovery, cache status, download control, integrity verification, and GPU capability detection without requiring a running HTTP server. Agents that operate in CLI-only mode (no `serve` process) can use these subcommands to inspect and prepare the embedding environment before executing `index` or `search` operations.

21 Model Context Protocol Server

The `neuroncite-mcp` crate implements a Model Context Protocol server that enables AI assistants (Claude Code, and other MCP-compatible clients) to access all NeuronCite operations through a standardized tool-use interface. The server communicates via JSON-RPC 2.0 over stdin/stdout, following the MCP specification version derived from the workspace crate version (`CARGO_PKG_VERSION`).

21.1 Protocol Architecture

The MCP server processes requests through a layered pipeline:

1. **Transport layer** (`transport.rs`): Reads complete JSON lines from stdin via `StdioReader` and writes serialized responses to stdout via `StdioWriter`. All diagnostic logging is directed to stderr to keep the stdout channel clean for protocol messages.
2. **Protocol layer** (`protocol.rs`): Defines the JSON-RPC 2.0 message types. `JsonRpcRequest` contains the method name, params object, and request ID. `JsonRpcResponse` wraps either a result value or a `JsonRpcError` with code and message fields.
3. **Server loop** (`server.rs`): Drives the request lifecycle. Performs the MCP initialize handshake (exchanging protocol version and server capabilities), responds to `tools/list` with the tool catalog, dispatches `tools/call` requests to handlers, and handles `notifications/cancelled` messages.
4. **Dispatch layer** (`dispatch.rs`): Maps each tool name string (e.g., `"neuroncite_search"`) to its handler function. Returns a JSON-RPC method-not-found error for unrecognized tool names.
5. **Handler layer** (`handlers/`): Each handler bridges one MCP tool call to the NeuronCite subsystems. Handlers receive the shared `AppState` and deserialized parameters, execute the operation, and return a `serde_json::Value` for the response content.

21.2 Tool Catalog

The server exposes forty-three tools, each corresponding to one NeuronCite operation:

Tool Name	Description
<code>neuroncite_annotate</code>	Highlights text passages in PDFs with color-coded annotations. Accepts a CSV or JSON quote list, creates an annotation job, and returns the job ID and quote count.
<code>neuroncite_annotate_status</code>	Returns per-quote progress for a running annotation job, including completion percentage and individual quote match status.

Tool Name	Description
<code>neuroncite_annotation_remove</code>	Removes highlight annotations from PDFs by color or page range. Accepts a file path and filter criteria (color, page number).
<code>neuroncite_batch_content</code>	Retrieves content from multiple indexed documents in one call. Accepts a list of file IDs and page ranges, returning the concatenated text for each requested segment.
<code>neuroncite_batch_search</code>	Executes multiple search queries in a single call against the same session. Each query runs the full hybrid pipeline (vector + BM25 via RRF, deduplication, optional reranking). Returns results keyed by caller-provided query ID. Supports session-level <code>file_ids</code> and <code>min_score</code> filtering. Each query group includes <code>score_summary</code> and each result includes <code>relevance</code> labels and <code>page_numbers</code> .
<code>neuroncite_bib_report</code>	Generates a CSV or XLSX report from a BibTeX file. Summarizes entries with metadata fields, missing fields, and structural diagnostics.
<code>neuroncite_chunks</code>	Browses chunks of an indexed file with pagination. Accepts a file ID, offset, and limit. Returns chunk text, byte offsets, and page ranges.
<code>neuroncite_citation_claim</code>	Claims a batch of citation rows for sub-agent verification. Uses an atomic transaction to prevent concurrent agents from processing the same rows.
<code>neuroncite_citation_create</code>	Creates a citation verification job from LaTeX and BibTeX files. Parses the LaTeX source, extracts citation references, and matches them against BibTeX entries.
<code>neuroncite_citation_export</code>	Exports citation verification results as CSV, XLSX, and JSON. Includes per-row verdicts, confidence scores, and flagged alerts.
<code>neuroncite_citation_fetch_sources</code>	Downloads source documents referenced in a BibTeX file. Resolves DOIs and URLs, fetches PDFs, and stores them locally for subsequent indexing.
<code>neuroncite_citation_retry</code>	Resets citation rows or failed batches back to pending status, allowing re-verification by agents.
<code>neuroncite_citation_rows</code>	Reads citation rows with optional status filtering. Returns the row data, current verification status, and assigned agent information.
<code>neuroncite_citation_status</code>	Returns job status with verdict distribution and flagged alerts. Includes counts for each verdict category and overall completion percentage.
<code>neuroncite_citation_submit</code>	Submits verification results for a batch of citation rows. Accepts per-row verdicts, confidence scores, and supporting evidence text.
<code>neuroncite_compare_search</code>	Compares search quality between two index sessions. Runs the same query against both sessions and reports result overlap and score differences.

Tool Name	Description
<code>neuroncite_content</code>	Retrieves text content from an indexed document by page number. Supports single-page retrieval via <code>page_number</code> or contiguous range retrieval via <code>start_page/end_page</code> (maximum 20 pages per request).
<code>neuroncite_discover</code>	Scans a directory for supported files (PDF, HTML) and reports the indexing status for each file (indexed, not indexed, stale).
<code>neuroncite_doctor</code>	Checks system capabilities: GPU availability, CUDA version, OCR engine status (Tesseract), pdfium library presence, and ONNX Runtime backend.
<code>neuroncite_export</code>	Exports search results as Markdown, BibTeX, or CSL-JSON formatted citations.
<code>neuroncite_file_compare</code>	Compares the same file across different index sessions. Reports differences in chunk count, page count, and embedding dimensions.
<code>neuroncite_files</code>	Lists all indexed files within a session with <code>file_id</code> , path, display name, <code>page_count</code> (extracted pages), <code>pdf_page_count</code> (structural pages), and size. Includes a <code>warning</code> field when extraction is incomplete (<code>page_count < pdf_page_count</code>). Enables direct use of <code>neuroncite_content</code> and <code>file_ids</code> parameter in search without a prior search.
<code>neuroncite_health</code>	Returns server health status including the API version string, GPU availability flag, and loaded embedding backend identifier.
<code>neuroncite_html_crawl</code>	Crawls a website via BFS link-following with configurable depth. Discovers linked pages, fetches their content, and indexes each page into the specified session.
<code>neuroncite_html_fetch</code>	Fetches web pages, caches the raw HTML on disk, and extracts metadata (title, description, canonical URL, language).
<code>neuroncite_index</code>	Indexes a directory of PDF files. Creates a session and spawns the indexing pipeline (extraction, chunking, embedding). Returns the job ID for progress monitoring.
<code>neuroncite_index_add</code>	Incrementally adds or updates files in an existing session. Accepts a list of file paths and processes only files that are missing or have changed since the last indexing run.
<code>neuroncite_inspect_annotations</code>	Inspects annotation properties in a PDF file. Lists all highlight annotations with their color, page number, quad points, and associated text.
<code>neuroncite_job_status</code>	Retrieves the status of a specific job by UUID, including progress and error messages.
<code>neuroncite_job_cancel</code>	Requests cooperative cancellation of a running job by UUID. The current batch completes before cancellation takes effect.

Tool Name	Description
<code>neuroncite_jobs</code>	Lists all jobs with their state, progress counters, and timestamps.
<code>neuroncite_models</code>	Lists all supported embedding models with their vector dimensions, pooling strategies, and download cache status.
<code>neuroncite_multi_search</code>	Searches across multiple index sessions simultaneously. Merges results from all specified sessions and deduplicates by content hash.
<code>neuroncite_preview_chunks</code>	Previews the chunking strategy results without writing to the database. Accepts a file path and strategy parameters, returns the resulting chunks for inspection.
<code>neuroncite_quality_report</code>	Generates an index quality report for a session. Analyzes chunk distribution, embedding coverage, and extraction completeness across all files.
<code>neuroncite_reindex_file</code>	Re-extracts and re-embeds a single file within an existing session. Deletes the old file data and runs the full pipeline on the file again.
<code>neuroncite_reranker_load</code>	Loads a cross-encoder reranker model at runtime. Downloads the model if not cached and initializes the inference session on the GPU Worker thread.
<code>neuroncite_search</code>	Performs semantic and keyword hybrid search across indexed PDF documents. Returns ranked passages with citations (source file, <code>file_id</code> , page numbers) and relevance scores (RRF, vector cosine similarity, optional cross-encoder reranker score). Supports file-scoped search via <code>file_ids</code> parameter, relevance filtering via <code>min_score</code> , and includes <code>relevance</code> labels (“high”, “medium”, “low”, “marginal”), <code>score_summary</code> statistics, and <code>page_numbers</code> arrays in each result.
<code>neuroncite_session_delete</code>	Deletes sessions and all associated data (files, pages, chunks, embeddings) via cascading delete. Accepts either a <code>session_id</code> (integer) to delete a single session or a <code>directory</code> (string) to delete all sessions for a given directory path. Also evicts the corresponding HNSW graphs from the in-memory map.
<code>neuroncite_session_diff</code>	Compares two sessions to find file differences. Reports files present in one session but not the other, and files with differing metadata.
<code>neuroncite_session_update</code>	Updates session metadata. Supports setting or clearing the human-readable <code>label</code> field.
<code>neuroncite_sessions</code>	Lists all index sessions with their metadata (directory, model, chunk strategy, creation timestamp).

Tool Name	Description
<code>neuroncite_text_search</code>	Performs keyword-only BM25 search via the FTS5 full-text index, bypassing vector similarity. Returns ranked passages with BM25 scores and source citations.

Table 38: MCP tool catalog. Each tool maps to one NeuronCite operation accessible through the `tools/call` JSON-RPC method.

21.3 Registration and Lifecycle

The registration module (`registration.rs`) manages the MCP server entry in Claude Code’s configuration file at `~/.claude/claude_mcp_settings.json`. The configuration follows the structure expected by Claude Code for auto-starting MCP servers:

MCP Configuration Entry

```
{
  "mcpServers": {
    "neuroncite": {
      "command": "/path/to/neuroncite",
      "args": ["mcp", "serve"]
    }
  }
}
```

The CLI provides four MCP subcommands:

neuroncite mcp serve Starts the MCP stdio server. Initializes the `AppState` (connection pool, GPU worker, per-session HNSW map via `load_all_session_hnsw()`) and enters the server loop. Runs until stdin is closed or a shutdown notification is received.

neuroncite mcp install

Writes the `neuroncite` entry into the MCP settings file. Creates the `.claude` directory and settings file if they do not exist. If an entry named “`neuroncite`” already exists, it is overwritten with the current executable path.

neuroncite mcp uninstall

Removes the `neuroncite` entry from the MCP settings file. If the file does not exist or does not contain a `neuroncite` entry, the operation is a no-op.

neuroncite mcp status

Reports the current registration status: whether the server is registered, the configured executable path and arguments, and the settings file location.

22 Testing Strategy

This section defines the testing requirements, infrastructure, and the complete test catalog for the NeuronCite project. Every test listed here specifies a verifiable acceptance criterion that must hold before the corresponding feature is considered implemented. The catalog is divided into unit tests (isolated per crate), integration tests (cross-crate pipelines with real PDF files), property-based tests (randomized invariant checks), regression tests (edge cases and failure modes), and performance benchmarks.

22.1 Testing Requirements and Toolchain

The following requirements apply to the entire workspace and are enforced by the CI pipeline:

- **Clippy with deny-all-warnings:** Every crate compiles under `cargo clippy --workspace --all-targets --all-features -- -D warnings`. No Clippy lint is suppressed globally; individual suppressions require an inline `#[allow(clippy:...)]` attribute with a comment explaining the rationale.
- **Rustfmt enforcement:** All source files conform to the project's `rustfmt.toml` configuration. The CI pipeline runs `cargo fmt --all -- --check` and fails on any formatting deviation.
- **No unsafe code:** The crates `neuroncite-core`, `neuroncite-chunk`, `neuroncite-search`, `neuroncite-api`, and `neuroncite-mcp` declare `#![forbid(unsafe_code)]`. The crates `neuroncite-pdf`, `neuroncite-embed`, and `neuroncite-store` permit limited `unsafe` blocks for FFI boundaries and zero-copy deserialization; each such block carries a `// SAFETY:` comment documenting the invariant.
- **Test isolation:** Every test function creates its own temporary directory (via `tempfile::TempDir`) and its own SQLite database. Tests do not share mutable state and can run in parallel (`cargo test` default behavior) without interference.
- **Deterministic test order:** Tests must not depend on execution order. The CI pipeline runs the test suite twice per build—once with default ordering and once with reverse ordering (via `--test-threads=1 -Z unstable-options --shuffle`)—to detect hidden order dependencies.
- **Feature-flag-aware compilation:** Tests that exercise a specific embedding backend are gated behind `#[cfg(feature = "...")]` attributes matching the backend's feature flag. The CI matrix compiles and tests each feature combination separately (see Section 22.7).
- **Code coverage threshold:** Line coverage across the workspace must remain above 80%. Coverage is measured by `cargo-llvm-cov` with branch coverage enabled. Coverage reports are generated per CI run and archived as build artifacts.
- **Documentation tests:** All public API items have doc comments with at least one usage example. These examples are compiled and executed as part of `cargo test --doc`.
- **Compilation warnings:** The workspace builds with `#![deny(warnings)]` in all

crates. Any compiler warning fails the build.

22.2 Test Infrastructure

22.2.1 Synthetic PDF Generation

Integration tests require real PDF files with known content so that extraction, chunking, and search results can be verified against expected values. The test infrastructure generates synthetic PDFs programmatically at test time using a shared helper module (`tests/common/pdf_gen.rs`). This module provides functions to create PDF files with the following characteristics:

- **Plain text PDF:** A well-formed PDF with a specified number of pages, each containing a deterministic text block. The text is injected via PDF content stream operators so that `pdf-extract` can recover it faithfully.
- **Multi-column PDF:** A PDF with two-column layout per page, used to verify the `pdfium` fallback path.
- **Image-only PDF:** A PDF where each page contains a rendered image of text (no text layer), used to trigger the OCR fallback path.
- **Mixed-content PDF:** A PDF combining text pages, image pages, and pages with mathematical notation, used for end-to-end pipeline tests.
- **Corrupt PDF:** A deliberately malformed byte sequence with a `.pdf` extension, used to verify error handling for unreadable files.
- **Empty PDF:** A valid PDF with zero pages, used to verify boundary handling.
- **Unicode PDF:** A PDF containing CJK characters, Arabic script, diacritics, and mathematical symbols, used to verify Unicode handling across the full pipeline.

All synthetic PDFs are created in the test's temporary directory and are deleted automatically when the `TempDir` guard is dropped at the end of the test.

22.2.2 OCR Fallback Test Fixtures

The OCR integration tests (`T-OCR-*`) require PDFs that trigger specific failure modes in the `pdf-extract` crate, forcing the extraction pipeline through its fallback chain. These test fixtures are generated by the standalone `neuroncite-testgen` crate and committed to the repository under `tests/PDFs/ocr/`. Each fixture is a small (1–3 KB) self-contained PDF with no third-party copyright restrictions. The three fixtures are:

- **Cross-reference corruption** (`synthetic_statistics_xref_corruption.pdf`): A valid 3-page PDF with statistics domain text whose serialized bytes are post-processed to corrupt the xref table offsets, the `startxref` pointer, and the trailer `/Size` value. The triple corruption eliminates all anchor points that both `pdf-extract` and `pdfium`'s xref repair algorithm rely on. Both backends fail; only OCR can extract text.
- **CID-range panic** (`synthetic_finance_cid_range.pdf`): A multi-page PDF containing a Type0 (CID) font with a deliberately malformed CMap encoding stream. The CMap includes `INVALID_TOKEN` where the `pom` parser expects a hex string or integer, causing `adobe_cmap_parser::get_byte_mapping().unwrap()` to panic.

A standard Helvetica font carries the visible finance domain text, which pdfium extracts after the fallback chain catches the panic.

- **Type1 font panic** (`synthetic_forecasting_type1_encoding.pdf`): A multi-page PDF containing a Type1 font whose `/FontFile` stream holds an unterminated PostScript string literal. The pom parser in `type1-encoding-parser` consumes to EOF without finding the closing parenthesis and returns `Err`, triggering `.expect("failed to parse")`. A standard Helvetica font carries the visible forecasting domain text, which pdfium extracts after the fallback chain catches the panic.

To regenerate the fixtures after modifying the generator code: `cargo run -p neuroncite-testgen`.

22.2.3 Test Database Lifecycle

Each test that requires database access creates a fresh SQLite database in a temporary directory. The database schema is initialized by calling the same migration function used in production (`neuroncite_store::schema::migrate`). After the test completes, the temporary directory (including the database file) is deleted. No shared database is used across tests.

22.2.4 Deterministic Embedding Stub

Unit tests that require embedding vectors but do not test the embedding backend itself use a deterministic stub implementation of the `EmbeddingBackend` trait. The stub maps each input text to a fixed vector derived from a SHA-256 hash of the text, normalized to unit length. This produces consistent, reproducible vectors without requiring a GPU or model files. The stub is defined in `tests/common/stub_embed.rs` and is shared across all crate-level test suites.

22.2.5 Temporary Directory Management

Every test that creates files, databases, or index structures uses the `tempfile` crate to create an isolated temporary directory. The `TempDir` handle is held for the lifetime of the test; when the handle is dropped (at test completion or panic), the directory and all contents are removed. This prevents test artifacts from accumulating on the file system.

22.3 Unit Test Catalog

The following tables define all required unit tests organized by crate. Each test has a unique identifier, a description of the behavior under test, and the acceptance criterion that must hold for the test to pass.

ID	Description	Acceptance Criterion
<i>Crate: <code>neuroncite-annotate</code> – PDF highlight annotation and text location pipeline</i>		

ID	Description	Acceptance Criterion
T-ANN-BUG-B-001	When <code>cached_pages</code> is <code>None</code> and <code>live_extracted</code> is <code>None</code> (native-text PDF), <code>effective_cached</code> must be <code>None</code> .	This verifies the <code>or()</code> logic correctly falls through when neither cache is available.
T-ANN-BUG-B-002	When <code>cached_pages</code> is <code>None</code> but <code>live_extracted</code> is <code>Some</code> , <code>effective_cached</code> must resolve to the live-extracted data.	This is the path taken for scanned PDFs not in the DB cache.
T-ANN-BUG-B-003	When <code>cached_pages</code> is <code>Some</code> (DB cache hit), it takes precedence over <code>live_extracted</code> .	This is the fast path: no live extraction is performed when the DB cache covers the PDF.
T-ANN-CANCEL-001	<code>annotate_pdfs_with_cancel</code> returns <code>AnnotateError::Canceled</code> when the cancel callback returns <code>true</code> immediately.	The pipeline does not process any PDFs because the cancel check fires before the first PDF.
T-ANN-CANCEL-002	<code>annotate_pdfs_with_cancel</code> with <code>cancel_check</code> returning <code>false</code> behaves identically to <code>annotate_pdfs</code> .	The pipeline completes and returns an <code>AnnotationReport</code> .
T-ANN-CANCEL-003	The <code>annotate_pdfs</code> wrapper function delegates to <code>annotate_pdfs_with_cancel</code> with a never-cancel callback.	Verifies backward compatibility of the original API.
T-ANN-CANCEL-004	The <code>AnnotateError::Canceled</code> variant correctly stores and reports the completed and total PDF counts.	—

ID	Description	Acceptance Criterion
T-ANN-CANCEL-005	The cooperative cancel flag can be shared via <code>Arc<AtomicBool></code> , matching the pattern used by the executor.	This test verifies that the cancel callback correctly reads the flag and that the pipeline respects it.
T-ANN-DEF001-001	Snapshot directory creation and file copy.	Verifies that <code>tempfile::tempdir</code> creates a usable directory and <code>fs::copy</code> preserves file content.
T-ANN-DEF001-002	Snapshot only copies files that exist in the prior directory.	Non-existent files are silently skipped.
T-ANN-DEF001-003	The <code>effective_prior_dir</code> selection logic prefers the snapshot directory over the original <code>prior_output_directory</code> .	—
T-ANN-DEF001-004	When <code>prior_output_directory</code> is <code>None</code> (non-append mode), no snapshot is created and <code>effective_prior_dir</code> is <code>None</code> .	—
T-ANN-DEF001-005	Multiple files are correctly snapshotted.	Each file from the prior directory is independently copied to the snapshot directory.
T-ANN-PARALLEL-001	<code>annotate_pdfs_with_cancel</code> accepts progress and <code>cancel_check</code> callbacks that are <code>Send + Sync</code> , as required by the rayon parallel implementation.	This test verifies that Arc-backed closures (the pattern used by the executor) compile and work correctly.
T-ANN-PREFILTER-001	When <code>cached_pages</code> is <code>None</code> (no indexed session), all pages are returned regardless of quote content.	Pre-filtering is impossible without cached text.

ID	Description	Acceptance Criterion
T-ANN-PREFILTER-002	When a quote exactly matches cached text on a specific page, only that page is included in the result.	—
T-ANN-PREFILTER-003	Case-insensitive matching catches quotes with different casing than the cached text.	—
T-ANN-PREFILTER-004	Multiple quotes matching different pages produce a sorted, deduplicated set of all matched pages.	—
T-ANN-PREFILTER-005	Multiple quotes matching the SAME page produce a single entry (deduplication via HashSet).	—
T-ANN-PREFILTER-006	When no quote matches any cached page (neither substring nor word-overlap), all pages are returned as a fallback for Stage 4's last-resort full-document search.	—
T-ANN-PREFILTER-007	Word-overlap heuristic ($\geq 40\%$) catches fuzzy matches where the exact substring does not appear but enough words are shared.	This handles OCR text variants (ligatures, "rn" -> "m").
T-ANN-PREFILTER-008	Word-overlap heuristic is skipped for short quotes (2 words or fewer) to avoid false positives.	Only substring matching applies.

ID	Description	Acceptance Criterion
T-ANN-PREFILTER-009	Word-overlap threshold boundary: exactly 40% overlap (2 out of 5 words) must be included.	—
T-ANN-PREFILTER-010	Word-overlap threshold boundary: 39% overlap (below threshold) must NOT be included.	—
T-ANN-PREFILTER-011	Empty rows list returns all pages (nothing to pre-filter for, so the fallback triggers).	—
T-ANN-PREFILTER-012	Empty cached pages (empty HashMap) returns all pages.	No text to match against, so all pages need OCR.
T-ANN-PREFILTER-013	Result pages are sorted in ascending order regardless of the order they were matched or the HashMap iteration order.	—
T-ANN-PREFILTER-014	page_count=0 produces an empty result (edge case: a PDF with zero pages should not cause a panic or infinite loop).	—
T-ANN-PREFILTER-015	page_count=1 with a match returns a single-element vector.	—
T-ANN-PREFILTER-016	The channel-based timeout pattern used by the extract_block returns the hOCR result within the timeout period.	Verifies that recv_timeout works correctly for the happy path.

ID	Description	Acceptance Criterion
T-ANN-PREFILTER-017	The channel-based timeout pattern returns <code>RecvTimeoutError::Timeout</code> when the hOCR batch exceeds the timeout.	Verifies the timeout fallback path.
T-ANN-PREFILTER-018	The channel-based timeout pattern returns <code>RecvTimeoutError::Disconnected</code> when the hOCR thread panics (sender dropped without sending).	—
T-ANN-PREFILTER-019	The <code>total_steps</code> counter increases by the number of targeted OCR pages (not all pages).	For 35 total pages with 5 matching, <code>total_steps</code> increases by 5.
T-ANN-PREFILTER-020	After hOCR completes, <code>on_quote_done</code> is called with the number of completed pages.	This advances the progress counter by the OCR page count.
T-ANN-PREMATCH-001	<code>pre_match_quotes</code> finds exact matches in cached text.	—
T-ANN-PREMATCH-002	<code>pre_match_quotes</code> uses page hint to search the hinted page first, even when it appears later in the <code>HashMap</code> .	—
T-ANN-PREMATCH-003	<code>pre_match_quotes</code> returns empty map when no cached text is available for any PDF.	—

ID	Description	Acceptance Criterion
T-ANN-PREMATCH-004	pre_match_quotes finds normalized matches (whitespace differences between cached text and quote).	—
T-ANN-PREMATCH-005	pre_match_quotes finds fuzzy matches when text has minor differences (anchor-word algorithm).	—
T-ANN-PREMATCH-006	pre_match_quotes handles multiple PDFs with multiple quotes each, producing correct (pdf_idx, quote_idx) keys.	—
T-ANN-PROGRESS-001	When zero PDFs match, the progress callback is not invoked.	The handler sets its own initial progress_total (based on quote count) before the pipeline runs, so the pipeline must not overwrite it with (0, 0). Regression test for ISSUE-014 where a no-match annotation job showed progress_total=0 instead of the handler's initial value.
T-ANN-RETEST5-001	The total_quotes calculation sums all quote rows across all matched PDFs.	This is the denominator for per-quote progress.
T-ANN-RETEST5-002	The atomic quote_done counter increments by 1 for each quote processed.	After processing N quotes, the counter equals N.
T-ANN-RETEST5-003	When a PDF fails to load, all its quotes are counted as done in a single batch increment.	This prevents the progress counter from stalling at the failed PDF.
T-ANN-RETEST5-004	Progress callbacks fire with monotonically increasing done values.	Concurrent atomic increments from parallel PDF workers must produce a strictly increasing sequence when observed.

ID	Description	Acceptance Criterion
T-ANN-RETEST5-005	The OS thread dispatch for pre-extraction avoids rayon pool saturation.	This tests the pattern where <code>extract_pages</code> is called on a separate OS thread and the result is sent back via channel.
T-ANN-RETEST5-006	When the OS thread panics or terminates without sending a result, the channel <code>recv</code> returns <code>Err</code> .	The pre-extraction returns <code>None</code> and the pipeline continues without cached text.
T-ANN-RETEST5-007	The <code>on_quote_done</code> callback is invoked exactly once per quote, even when the <code>page-get</code> fails and the loop uses <code>continue</code> .	This verifies that the progress counter does not skip quotes.
T-ANN-RETEST5-008	With zero matched PDFs (<code>total_steps=0</code>), the initial progress callback is not fired to avoid overwriting the handler's initial <code>progress_total</code> .	—
T-ANN-V3-001	<code>pre_match_quotes</code> at the 0.75 threshold catches agent-reformulated text that differs by ~20% from the cached text.	The quote uses "market hypothesis" while the cached text says "markets hypothesis", and includes minor word insertions. With the old 0.85 threshold this would be rejected; at 0.75 it matches.
T-ANN-V3-002	<code>pre_match_quotes</code> rejects text that shares too few words even at the lowered 0.75 threshold.	Completely unrelated content must not match.
T-ANN-V3-003	<code>QuoteReport</code> with status <code>"page_level_match"</code> is counted as a matched quote in the <code>matched_count</code> filter logic.	This verifies the filter condition used in <code>process_single_pdf</code> .

ID	Description	Acceptance Criterion
T-ANN-V3-004	QuoteReport for page-level fallback has the correct field values: status "page_level_match", match_method "page_level", pre_check_passed false, chars_matched 1, and stages_tried populated.	—
T-ANN-V3-005	page_level fallback comment is built correctly.	When the input row has a comment, it is prepended with "[page-level]". When no comment is provided, the default message is used.
T-ANN-V3-006	The matched_count filter correctly classifies PDF status as "partial" when some quotes are matched and some are not.	—
T-ANN-V3-007	The matched_count filter produces "success" when all quotes are matched (mix of matched + page_level_match).	—
T-ANN-V3-008	pre_match_quotes with page hint and fuzzy match at the lowered threshold.	The hint page is searched first and the fuzzy algorithm finds the reformulated text on that specific page.
T-ANNOTATE-001	Parses a CSV with all 5 columns present.	—
T-ANNOTATE-002	Parses a CSV with only required columns (title, author, quote).	Optional columns color and comment are absent.
T-ANNOTATE-003	Parses a JSON array input.	—
T-ANNOTATE-004	Parses a JSON wrapper object with "annotations" key.	—

ID	Description	Acceptance Criterion
T-ANNOTATE-005	Auto-detects CSV format from content.	—
T-ANNOTATE-006	Auto-detects JSON format from content starting with '['.	—
T-ANNOTATE-007	Rejects input with an empty title field.	—
T-ANNOTATE-008	Rejects input with an invalid hex color value.	—
T-ANNOTATE-009	Parses CSV containing Unicode characters in the quote field (e.g., German umlauts, accented characters).	—
T-ANNOTATE-010	Parses inline JSON (as would be sent by an API request body) with multiple rows.	—
T-ANNOTATE-011	Rejects empty input (no rows).	—
T-ANNOTATE-012	JSON with leading whitespace is still detected as JSON format.	—
T-ANNOTATE-020	Exact title and author match against a filename following the "(Author, Year)" convention.	—
T-ANNOTATE-021	Matching is case-insensitive.	—
T-ANNOTATE-022	Partial author matches work (e.g., input "Fama" matches filename "Fama & French").	—
T-ANNOTATE-025	Filenames without year in parentheses are still parsed.	—

ID	Description	Acceptance Criterion
T-ANNOTATE-026	Bracketed annotations like "[Book]" are stripped before metadata extraction.	—
T-ANNOTATE-027	Filenames without parenthesized author return None.	—
T-ANNOTATE-028	The normalize function collapses multiple spaces and strips punctuation.	—
T-ANNOTATE-029	Filenames with parenthesized subtitles before the author group.	The greedy regex must match the LAST parenthesized group as the author, not the first. This was the root cause of Shapiro & Wilk not being matched: "Complete Samples" was incorrectly captured as the author instead of "Shapiro & Wilk".
T-ANNOTATE-030	normalize_text collapses whitespace and removes soft hyphens.	—
T-ANNOTATE-031	normalize_text joins hyphenated line breaks.	—
T-ANNOTATE-032	normalize_text removes Unicode soft hyphens.	—
T-ANNOTATE-033	map_normalized_to_original returns correct indices for a simple case without normalization differences.	—
T-ANNOTATE-034	fuzzy_match_score returns a high score (≥ 0.90) for text with minor differences (a few character substitutions).	—

ID	Description	Acceptance Criterion
T-ANNOTATE-035	fuzzy_match_score returns a low score (< 0.90) for completely unrelated text.	—
T-ANNOTATE-036	fuzzy_match_score handles empty inputs without panicking.	Returns 0.0 for empty quote or empty page text.
T-ANNOTATE-040	Pre-check passes when all characters have valid bounding boxes.	—
T-ANNOTATE-041	Pre-check fails when too many characters lack bounding boxes (below 95% threshold).	—
T-ANNOTATE-045	Pre-check with zero total chars returns passed=false.	—
T-ANNOTATE-050	Report JSON serialization produces valid JSON.	—
T-ANNOTATE-051	compute_summary calculates correct aggregate counts.	—
T-ANNOTATE-052	excerpt truncates strings longer than the limit.	—
T-ANNOTATE-053	excerpt returns the full string when it fits.	—
T-ANNOTATE-054	QuoteReport with stages_tried serializes the array into the JSON output.	—
T-ANNOTATE-055	QuoteReport with fuzzy_score serializes the score into the JSON output.	—

ID	Description	Acceptance Criterion
T-ANNOTATE-056	QuoteReport with both stages_tried and fuzzy_score set to None omits both fields from JSON output.	—
T-ANNOTATE-060	merge_boxes_into_lines—merges two horizontally adjacent boxes on the same line.	—
T-ANNOTATE-061	merge_boxes_into_lines—splits boxes on different lines into separate rectangles.	—
T-ANNOTATE-062	merge_boxes_into_lines—handles a single box.	—
T-ANNOTATE-063	merge_boxes_into_lines—handles empty input.	—
T-ANNOTATE-064	Color parsing and RGB extraction.	—
T-ANNOTATE-065	Authors with "Last, First" comma format are preserved.	When only one comma appears and the part after it is not a 4-digit year, the entire content is treated as the author string.
T-ANNOTATE-066	Two authors separated by & with year correctly parsed.	—
T-ANNOTATE-067	Three authors with commas and ampersand, plus bracketed annotation.	Verifies that bracket stripping, rfind for last paren, and rfind for last comma all work together.
T-ANNOTATE-068	strip_subtitle extracts the main title before a colon separator.	Colon is the standard academic subtitle separator.
T-ANNOTATE-069	strip_subtitle extracts the main title before an em-dash separator.	—

ID	Description	Acceptance Criterion
T-ANNOTATE-070	End-to-end test of create_highlight_annotation.	Uses the production function to create highlights on a blank PDF page, saves the PDF to disk, then reopens it with the inspect module and verifies: 1. The highlight annotation exists in the saved file 2. The stroke color (/C entry) matches the requested color 3. The QuadPoints (/QuadPoints entry) are present (quad_points_count >= 1) This is the critical test that catches invisible highlights: without QuadPoints, the highlight annotation exists as a PDF object (with color metadata and working comments) but is invisible in PDF viewers.
T-ANNOTATE-071	End-to-end test with multiple colors on the same page.	Creates two sets of highlights with different colors via the production function, saves the PDF, and verifies both colors and QuadPoints are present in the saved file.
T-ANNOTATE-072	Appearance stream injection on a blank PDF.	Creates highlight annotations with pdfium-render, saves the PDF, injects appearance streams via the lopdf post-processing module, and verifies that the /AP /N Form XObject is present in the saved file. This is the core regression test for the invisible-highlight bug where annotations had /Rect, /QuadPoints, /C, /IC but no /AP.
T-ANNOTATE-073	Appearance stream injection on a loaded (non-blank) PDF.	Loads a real PDF from the test fixtures, creates highlight annotations on its first page, saves, injects appearance streams, and verifies that the /AP entry is present. This catches issues that only manifest with PDFs that have existing content (fonts, images, complex page trees) as opposed to freshly-created blank pages.
T-ANNOTATE-074	Color preserved in appearance stream after round-trip.	Creates a highlight with a specific color, saves with pdfium, injects appearance streams, and verifies that the /C color array in the annotation dictionary matches the requested color AND that the appearance stream content contains the corresponding RGB values.

ID	Description	Acceptance Criterion
T-ANNOTATE-075	Full pipeline test on a real PDF from the test fixtures.	Loads a real academic paper, creates highlights, saves, injects appearance streams, and inspects with pdfium to verify that highlight annotations are visible (have color AND QuadPoints AND appearance streams).
T-ANNOTATE-076	create_page_level_match produces a single bounding box covering the content area (page minus 72pt margins) with Match-Method::PageLevel.	
T-ANNOTATE-077	create_page_level_match clamps margins to 10% on small pages.	
T-ANNOTATE-078	create_page_level_match and create_highlight_annotations together produce a visible full-page highlight in a saved PDF.	
T-ANNOTATE-079	is_duplicate_highlight — returns true when a candidate rectangle overlaps an existing highlight by more than 90% of its area.	
T-ANNOTATE-080	is_duplicate_highlight — returns false when the candidate does not overlap any existing highlight.	
T-ANNOTATE-081	is_duplicate_highlight returns true for nearly-identical rectangles that differ by minor floating-point rounding (within 90% overlap).	This simulates the case where two annotation runs produce slightly different bounding boxes for the same text region.

ID	Description	Acceptance Criterion
T-ANNOTATE-082	is_duplicate_highlight handles partial overlap below the 90% threshold.	Two rectangles sharing only 50% of their area must not be flagged as duplicates.
T-ANNOTATE-083	is_duplicate_highlight — handles an empty existing list.	
T-ANNOTATE-084	collect_existing_highlight_rects returns an empty Vec for a page with no annotations.	
T-ANNOTATE-085	create_highlight_annotation skips duplicate highlights when the page already has a highlight at the same position.	Regression test for DEF-5 (append=true duplicate highlights).
T-ANNOTATE-090	The deadline guard in find_via_fallback_extraction returns None when the deadline has already passed.	This verifies that the function does not attempt the expensive extract_pages() call when insufficient time remains. Regression test for BUG-003: Before the fix, find_via_fallback_extraction did not accept or check a deadline parameter. When a non-matching quote was searched in a scanned PDF, the function would call extract_pages() (which triggers OCR) without any timeout, causing the entire annotation pipeline to hang indefinitely. This test verifies the guard logic using the elapsed-deadline path (cached_pages = None, deadline already expired). Without the fix, the function would attempt to open the PDF and call extract_pages, which would fail/hang. With the fix, it returns None immediately.
T-ANNOTATE-091	Deadline with sufficient time remaining passes the guard check.	Verifies that the 5-second margin is correctly computed.
T-ANNOTATE-092	Deadline exactly at the 5-second boundary is treated as insufficient.	The guard condition is <code>remaining < 5s</code> , so exactly 4.999s should skip extraction.

ID	Description	Acceptance Criterion
T-ANNOTATE-100	StageTracker with all stages attempted and no timeout produces diagnostics for every stage.	—
T-ANNOTATE-101	StageTracker with skipped stages (native text PDF) produces "skipped" diagnostics for fallback and OCR.	—
T-ANNOTATE-102	StageTracker with timeout produces a timeout diagnostic.	—
T-ANNOTATE-103	StageTracker with zero fuzzy pages and all skipped produces the "skipped on all pages" diagnostic.	—
T-ANNOTATE-104	StageTracker default state produces diagnostics with zero page counts, reflecting no pages processed.	—
T-ANNOTATE-105	find_fuzzy returns (None, 0.0) for empty quote.	—
T-ANNOTATE-106	The anchor-word algorithm returns 0.0 for text where most content words differ.	Only 2 of 8 anchor words ("brown", "dog") match between the quote and page text, which is below the 40% cluster threshold. The algorithm correctly avoids wasting CPU on Levenshtein scoring for clearly unrelated passages.
T-ANNOTATE-107	find_fuzzy returns a score $\geq$ threshold for a match with minor differences.	—
T-ANNOTATE-108	MatchResult from a fuzzy match carries the fuzzy_score field when constructed with Some(score).	—

ID	Description	Acceptance Criterion
T-ANNOTATE-109	MatchResult from a non-fuzzy match has fuzzy_score None.	—
T-ANNOTATE-110	The OCR fuzzy threshold is lower than the regular fuzzy threshold to accommodate character-level OCR recognition errors (ligature misreads, character substitutions).	—
T-ANNOTATE-111	Text with severe OCR errors (most distinctive words mangled) returns 0.0 from the anchor-word algorithm because fewer than 40% of anchor words survive the OCR errors.	This is correct: heavily OCR-degraded text is handled by Stage 4 (hOCR word-level bounding boxes from Tesseract) rather than text-level fuzzy matching. Stage 4 uses the hOCR word boxes directly and matches against the clean hOCR word output, which preserves word boundaries even when individual characters are misread.
T-ANNOTATE-112	Completely unrelated text falls below the OCR fuzzy threshold, preventing false-positive matches in OCR stage.	—
T-ANNOTATE-113	find_word_range_for_match maps a normalized position at the start of the word sequence to the correct word range.	
T-ANNOTATE-114	find_word_range_for_match maps a mid-text position spanning two words to the correct word range.	

ID	Description	Acceptance Criterion
T-ANNOTATE-115	find_word_range_for_match handles a match that spans the entire concatenated text, returning all words.	
T-ANNOTATE-116	find_word_range_for_match handles a single-word input.	
T-ANNOTATE-117	The OCR fuzzy threshold is between 0.70 and 0.85, providing enough tolerance for OCR artifacts without being too permissive.	—
T-ANNOTATE-118	normalize_text handles OCR-typical artifacts: ligature errors produce different Unicode characters but the normalization still collapses whitespace and lowercases text.	—
T-ANNOTATE-200	extract_anchor_words filters stopwords and short words, sorts by length descending, and deduplicates.	—
T-ANNOTATE-201	extract_anchor_words returns at most 8 words.	—
T-ANNOTATE-202	extract_anchor_words strips punctuation from word boundaries.	—
T-ANNOTATE-203	extract_anchor_words returns empty for a quote made entirely of stopwords and short words.	—
T-ANNOTATE-204	find_word_positions locates multiple occurrences.	—

ID	Description	Acceptance Criterion
T-ANNOTATE-205	find_word_positions	— returns empty for absent word.
T-ANNOTATE-206	cluster_anchor_positions	— groups nearby positions and merges overlapping clusters.
T-ANNOTATE-207	cluster_anchor_positions	— returns empty when too few distinct anchors appear in any window.
T-ANNOTATE-208	find_anchor_fuzzy	— returns exact match with score 1.0 when the quote appears verbatim on the page.
T-ANNOTATE-209	find_anchor_fuzzy	— returns None for completely unrelated text.
T-ANNOTATE-210	find_anchor_fuzzy	— handles text with a few word substitutions (agent reformulation scenario).
T-ANNOTATE-211	find_anchor_fuzzy	— handles text where the agent omitted a section (e.g., [...] in the middle of the passage).
T-ANNOTATE-212	find_anchor_fuzzy	— short-circuits via normalized substring search within the candidate window, producing score 1.0 for text that matches after normalization (whitespace differences).

ID	Description	Acceptance Criterion
T-ANNOTATE-213	fuzzy_match_score returns 0.0 for empty inputs.	—
T-ANNOTATE-214	fuzzy_match_score returns high score for text with one word difference embedded in a larger page.	—
T-ANNOTATE-220	InputRow deserializes from JSON without the optional page field (backward compatibility with existing CSV/JSON inputs).	—
T-ANNOTATE-221	InputRow deserializes from JSON with the optional page field set to a valid page number.	—
T-ANNOTATE-222	InputRow deserializes from CSV without a page column (the field defaults to None via <i>serde</i>).	—
T-ANNOTATE-223	InputRow deserializes from CSV with a page column.	—
T-ANNOTATE-224	Filenames with multiple parenthesized groups.	The regex must capture the LAST parenthesized group as the author, regardless of how many parenthesized sections appear in the title.
T-ANNOTATE-225	Filename with a single author group and no subtitle parentheses.	Verifies that the greedy regex does not break the common case.
T-ANNOTATE-226	Filename with a bracketed annotation AND parenthesized subtitle.	Both stripping and greedy matching must work together.

ID	Description	Acceptance Criterion
T-ANNOTATE-227	Filename with ampersand in the author field.	—
T-ANNOTATE-228	Multi-author names with commas are captured in full.	Regression test for the bug where the regex <code>[^,)]+</code> stopped at the first comma, truncating "McNeil, Frey & Embrechts" to just "McNeil". The <code>rfind</code> -based approach splits on the LAST comma, correctly identifying the year and preserving the full author string.
T-ANNOTATE-229	<code>FallbackExtractMatchMethod</code> variant serializes to "fallback_extract" in JSON (<code>snake_case</code>).	—
T-ANNOTATE-230	<code>MatchMethod::CrossPage</code> serializes to "cross_page" via <code>serde snake_case</code> renaming and "crosspage" via <code>Debug</code> lowercasing.	
T-ANNOTATE-231	Cross-page matching via concatenated adjacent page texts.	Verifies the core text concatenation and matching logic used by Stage 3+ (cross-page matching). Simulates two adjacent pages where a sentence spans the page boundary.
T-ANNOTATE-231b	Text with moderate OCR errors (most words intact, a few substitutions) produces a non-zero score from the anchor-word algorithm.	When enough anchor words survive ($\geq 40\%$), clusters form and the Levenshtein scoring produces a meaningful similarity measure.
T-ANNOTATE-232	Cross-page matching with normalized text.	Tests that the pipeline's <code>normalize_text</code> function collapses extra whitespace at the page boundary, allowing a quote with regular spacing to match text that has irregular whitespace from PDF extraction across two pages.

ID	Description	Acceptance Criterion
T-ANNOTATE-233	strip_subtitle extracts the main title before a hyphen-with-spaces separator.	—
T-ANNOTATE-234	strip_subtitle returns None when no subtitle separator is present.	—
T-ANNOTATE-235	strip_subtitle does not match hyphens without spaces.	This prevents false positives on compound words like "Bid-Ask".
T-ANNOTATE-236	BibTeX title with subtitle matches a filename that contains only the main title.	Regression test for NEW-5 where the citation export annotation failed because "Quantitative Risk Management: Concepts, Techniques and Tools" scored below the 0.80 Jaro-Winkler threshold against the filename "Quantitative Risk Management".
T-ANNOTATE-237	Title without subtitle still matches correctly (no regression from the subtitle stripping logic).	—
T-ANNOTATE-238	strip_subtitle returns None for empty main title (colon at start of string).	—
T-ANNOTATE-239	All MatchMethod variants survive JSON roundtrip.	—
T-ANNOTATE-240	find_via_fallback_extraction with an empty cached_pages HashMap behaves identically to None (falls through to live extraction).	This verifies the <code>.filter(c !c.is_empty())</code> guard correctly handles the empty map case.
T-ANNOTATE-241	Cached page texts are sorted by page number before iteration.	Verifies that the fallback extraction stage processes pages in deterministic order regardless of HashMap insertion order.

ID	Description	Acceptance Criterion
T-ANNOTATE-242	Cached page texts with matching content are found by the normalized matching logic.	This verifies that the cached texts are correctly passed through the normalization and substring search pipeline.
T-ANNOTATE-243	Cached page texts with fuzzy-matching content produce a score above the FUZZY_THRESHOLD.	Verifies that fuzzy matching works correctly against cached text with minor differences.
T-ANNOTATE-BUG-D-001	FALLBACK_FUZZY_THRESHOLD is strictly below FUZZY_THRESHOLD	The fallback threshold is strictly below FUZZY_THRESHOLD because OCR-extracted text from older scanned documents contains systematic character-level errors (ligature misreads, rn->m substitutions) that produce similarity scores in the 0.80-0.89 range where the standard fuzzy threshold (0.90) rejects them. Tukey (1977) and Newey & West (1987) are known affected PDFs. The fallback threshold must be below 0.90 to accept these matches, and above or equal to OCR_FUZZY_THRESHOLD (0.75) to maintain consistent threshold ordering across the pipeline stages.
T-ANNOTATE-BUG-D-002	FALLBACK_FUZZY_THRESHOLD accepts text with moderate OCR errors that FUZZY_THRESHOLD rejects.	FALLBACK_FUZZY_THRESHOLD ~85% similarity to the page text must pass the fallback threshold but fail the standard threshold. This simulates the scenario where a scan has character substitutions that reduce the Levenshtein similarity below 0.90 but above 0.80.
T-ANNOTATE-BUG-D-003	FALLBACK_FUZZY_THRESHOLD rejects text that is clearly unrelated to the quote (low similarity), preventing false positive matches.	

ID	Description	Acceptance Criterion
T-ANNOTATE-ERR-001	QuoteNotFound error message includes the quote excerpt but not the stages_tried data (stages_tried is a structured field, not part of the Display format).	—
T-ANNOTATE-ERR-002	QuoteNotFound error carries stages_tried data accessible via pattern matching.	—
T-ANNOTATE-ERR-003	QuoteNotFound with empty stages_tried (for edge cases like empty quotes where no stages are attempted).	—
T-ANNOTATE-RETEST5-001	When Stage 3.5 identified a page (fallback_page is Some) and hOCR words are available for that page, the fallback constructs word-level bounding boxes from ALL hOCR words on the page.	This tests the pixel-to-PDF coordinate conversion formula: $\text{pdf_x} = \text{pixel_x} * 72.0 / \text{render_dpi}$ $\text{pdf_y} = \text{page_height} - (\text{pixel_y} * 72.0 / \text{render_dpi})$
T-ANNOTATE-RETEST5-002	The fallback produces one bounding box per hOCR word.	This tests that N words produce exactly N bounding boxes, and that the MatchResult uses MatchMethod::Ocr.
T-ANNOTATE-RETEST5-003	When fallback_page is None (Stage 3.5 did not identify a page), the hOCR word fallback is skipped entirely.	The function returns None.

ID	Description	Acceptance Criterion
T-ANNOTATE-RETEST5-004	When fallback_page is Some but the hOCR results for that page are empty (Tesseract returned no words), the fallback is skipped.	This happens for blank/empty scanned pages.
T-ANNOTATE-RETEST5-005	When fallback_page refers to a page that has no hOCR results (the page was not included in the OCR batch), the fallback is skipped.	This guards against HashMap key misses.
T-ANNOTATE-RETEST5-006	The scale factor conversion from pixel to PDF coordinates is correctly computed for different DPI values.	At 300 DPI: scale = $72/300 = 0.24$ At 400 DPI: scale = $72/400 = 0.18$
T-ANNOTATE-RETEST5-007	The fallback_page parameter is correctly derived from the Stage 3.5 fallback_result.	When fallback_result is Some, the page_number is extracted. When None, fallback_page is None.
T-ANNOTATE-RETEST5-008	Y-axis inversion is correct for PDF coordinate system.	PDF origin is bottom-left (Y increases upward), pixel origin is top-left (Y increases downward). For a page_height of 792pt: pdf_y_bottom = page_height - (pixel_y_bottom * scale) pdf_y_top = page_height - (pixel_y_top * scale) The resulting pdf_y_bottom < pdf_y_top (bottom is lower than top).
T-ANNOTATE-V3-PERF-001	find_anchor_fuzzy completes within 100ms for a long quote on a large page.	Regression test against the $O(n^2)$ sliding Levenshtein window that caused multi-minute hangs. With the 3-scale centered comparison, the function makes at most 3 Levenshtein calls per cluster instead of ~6,000.
T-ANNOTATE-V3-REPORT-001	compute_summary counts "page_level_match" quotes as matched.	These are full-page highlights created when text location failed but the page number was known.

ID	Description	Acceptance Criterion
T-ANNOTATE-V3-REPORT-002	compute_summary classifies a PDF with "page_level_match" and "not_found" as "partial".	—
T-ANNOTATE-V3-REPORT-003	post_check_passed=None is omitted from JSON.	—
T-ANNOTATE-V3-REPORT-004	page_level_match serializes correctly in JSON.	—
T-ANNOTATE-V3-REPORT-005	All three match statuses together produce correct summary counts.	—
T-APPEAR-001	build_appearance_xobject generates a valid Form XObject stream with the correct BBox, color, and graphics state.	—
T-APPEAR-002	parse_color_array correctly extracts RGB from a PDF color array with Real values.	—
T-APPEAR-003	parse_color_array returns None for missing key.	—
T-APPEAR-004	parse_color_array handles Integer color values (some PDF generators use Integer 0/1 instead of Real 0.0/1.0).	—
T-APPEAR-005	annotation_needs_appearance returns true for a Highlight annotation dictionary without /AP.	—
T-APPEAR-006	annotation_needs_appearance returns false for a Highlight annotation that already has /AP.	—

ID	Description	Acceptance Criterion
T-APPEAR-007	annotation_needs_appearances (Text annotations). returns false for non-Highlight annotations (e.g.	
T-APPEAR-008	inject_appearance_streams round-trip test.	Creates a minimal valid PDF with a highlight annotation lacking /AP, runs the injection, and verifies the /AP entry was added.
T-APPEAR-009	inject_appearance_streams is idempotent – calling it twice does not add a second /AP or modify the existing one.	
T-APPEAR-010	read_rect_and_color — falls back to /IC when /C is absent.	
T-APPEAR-011	read_rect_and_color — uses yellow default when both /C and /IC are absent.	
T-APPEAR-012	promote_inline_annotations converts inline Dictionary objects inside /Annots arrays to indirect References.	Simulates the serialization format that pdfium-render produces when saving PDFs.
T-APPEAR-013	promote_inline_annotations is a no-op when all annotations in the /Annots array are already indirect References.	
T-APPEAR-014	inject_appearance_streams works end-to-end on a PDF with inline annotation dictionaries (simulating pdfium output).	Verifies the full pipeline: promote -> detect -> inject -> save.

ID	Description	Acceptance Criterion
T-APPEAR-015	promote_inline_annotations handles mixed arrays containing both inline Dictionary annotations and indirect References.	
T-APPEAR-016	promote_inline_annotations is a no-op for pages without any /Annots entry.	
T-APPEAR-017	rects_overlap_90_percent returns true for identical rects.	
T-APPEAR-018	rects_overlap_90_percent returns false for non-overlapping rects (completely disjoint).	
T-APPEAR-019	rects_overlap_90_percent returns false for partial overlap below 90% threshold.	
T-APPEAR-020	rects_overlap_90_percent returns true for slightly offset rects with > 90% overlap.	
T-APPEAR-021	rects_overlap_90_percent returns false for zero-area rects.	
T-APPEAR-022	merge_prior_annotations copies a highlight annotation from a prior PDF into a new PDF.	End-to-end test using temporary files.
T-APPEAR-023	merge_prior_annotations deduplicates annotations.	When the new PDF already has a highlight at the same position as the prior PDF, the prior annotation is not copied (90% overlap).
T-APPEAR-DEF001-001	Merge from a snapshot file restores prior annotations into a new (overwritten) PDF.	Simulates the full DEF-001 scenario: prior PDF has 1 red highlight, new PDF has 1 blue highlight at a different position, merge adds the red highlight from the snapshot into the new PDF.

ID	Description	Acceptance Criterion
T-APPEAR-DEF001-002	Without the snapshot fix, merging from the overwritten output file produces 0 merged annotations (the overwritten file has no prior annotations to merge).	This test demonstrates the bug scenario.
T-INSPECT-001	annotation_type_to_string returns "highlight" for the Highlight variant.	
T-INSPECT-002	annotation_type_to_string returns "text" for the Text variant.	
T-INSPECT-003	annotation_type_to_string returns "unknown" for annotation types without an explicit mapping.	
T-INSPECT-004	color_to_hex produces uppercase hex with leading '#'.	—
T-INSPECT-005	color_to_hex handles pure black (all zero RGB).	—
T-INSPECT-006	color_to_hex handles pure white (all 255 RGB).	—
T-INSPECT-007	inspect_pdf returns an error for a nonexistent file path.	—
T-INSPECT-008	inspect_pdf with page_filter=0 returns an out-of-range error.	Page numbers are 1-indexed, so 0 is invalid.
T-INSPECT-009	inspect_pdf on a blank PDF (one page, no annotations) returns an InspectionResult with zero annotations.	—

ID	Description	Acceptance Criterion
T-INSPECT-010	Round-trip test that creates a highlight annotation with a specific color (setting /Rect, /QuadPoints, /C, and /IC entries as the production annotate.rs does), saves the PDF, then inspects it to verify all entries are physically present in the saved file.	—
T-INSPECT-011	Round-trip with multiple colors on the same page.	Creates two highlight annotations (red and green) with /Rect, /QuadPoints, /C, and /IC entries, saves, then verifies both colors and QuadPoints appear in the inspection result.
T-INSPECT-012	Page filter restricts results to the specified page.	Creates highlights on pages 1 and 2 (with /Rect, /QuadPoints, /C, /IC), then verifies that filtering to page 2 only returns the annotation from page 2.
T-INSPECT-013	Page filter with a page number exceeding the document's page count returns an error.	—
T-INSPECT-014	unique_colors is sorted alphabetically and deduplicated.	Creates three highlights (with /Rect, /QuadPoints, /C, /IC) using only two distinct colors to verify deduplication.
T-INSPECT-015	Text annotations (popup notes) are reported with type "text" and do not contribute to highlight_count or unique_colors.	Text annotations have quad_points_count == 0 because QuadPoints are only applicable to highlight annotations.
T-INSPECT-016	annotation_type_to_string covers all explicitly mapped annotation types.	

ID	Description	Acceptance Criterion
T-INSPECT-017	Round-trip through lopdf AP injection.	Creates a pdfium PDF with a yellow highlight annotation, runs <code>inject_appearance_streams</code> (the lopdf post-processing step from the production annotation pipeline that converts inline annotation dicts to indirect object references and adds <code>/AP</code> appearance streams), then calls <code>inspect_pdf</code> . This test is the primary regression guard for DEFECT-004: before the fix, <code>inspect_pdf</code> called pdfium's <code>FPDF_Annot_GetAttachmentPoints</code> C API on lopdf-modified PDFs, triggering a Windows SEH access violation that killed the MCP server process and produced "MCP error -32000: Connection closed" on every <code>neuroncite_inspect_annotations</code> call.
T-INSPECT-018	A PDF written by pdfium only (without lopdf post-processing) is correctly readable through <code>inspect_pdf</code> 's lopdf code path.	Verifies that lopdf handles pdfium's inline annotation dict format (annotations stored as inline dicts directly inside the <code>/Annots</code> array, as opposed to indirect references).
T-INSPECT-019	<code>read_quad_points_count</code> returns 0 for an annotation dictionary without a <code>/QuadPoints</code> entry.	
T-INSPECT-020	<code>read_quad_points_count</code> returns 2 for a <code>/QuadPoints</code> array with 16 elements (two groups of 8 floats each, where each group defines one highlighted line region).	
T-INSPECT-021	<code>read_color_entry</code> returns <code>None</code> for an absent color entry.	—

ID	Description	Acceptance Criterion
T-INSPECT-022	read_color_entry returns the correct uppercase hex string for a 3-element RGB array [1.0, 0.0, 0.0] (pure red).	—
T-REMOVE-001	RemoveMode::All removes all highlight annotations from a PDF with 3 highlights across 2 pages.	Verifies annotations_removed=3 and both pages are in pages_affected.
T-REMOVE-002	RemoveMode::ByColor removes only highlights matching the specified color.	Red highlights are removed, yellow highlights survive.
T-REMOVE-003	RemoveMode::ByPage removes only highlights on the specified page.	Highlights on page 1 are removed, page 2 highlights are untouched.
T-REMOVE-004	dry_run mode reports what would be removed but does not modify the source file.	The output file is not written.
T-REMOVE-005	When output_path differs from the source, the source file is preserved unchanged while the output has fewer annotations.	—
T-REMOVE-006	ByColor with a non-existent color results in zero removals.	The output PDF is identical to the input (except for lopdf re-serialization).
T-REMOVE-007	ByColor comparison is case-insensitive.	"#ff0000" matches a highlight with color "#FF0000".
T-REMOVE-008	RemoveMode::All on a PDF with zero annotations results in zero removals and an empty pages_affected list.	—

ID	Description	Acceptance Criterion
T-REMOVE-009	ByPage with a page number that exceeds the document's page count results in zero removals (no error).	—
T-REMOVE-010	Multiple colors in ByColor filter.	Both red and blue highlights are removed, green survives.
<i>Crate: neuroncite-api – REST API server and GPU worker</i>		
T-CIT-MATCH-001	BUG-003 regression: <code>find_best_file_match</code> returns the correct file even when the BibTeX year (2000) differs from the filename year (1997).	Before the fix, year mismatch caused hard rejection and the function returned None. The fix applies a penalty factor instead, so a strong title match can overcome a year discrepancy.
T-CIT-MATCH-002	When the BibTeX year matches the filename year, no penalty is applied and the match score is higher than the year-mismatch case.	—
T-CIT-MATCH-003	When the BibTeX entry has no year field, no year penalty is applied regardless of whether the filename contains a year.	—
T-CIT-MATCH-004	When the file lookup is empty, returns None.	—
T-CIT-MATCH-005	When both author and title are empty, returns None.	—

ID	Description	Acceptance Criterion
T-CIT-MATCH-006	The function correctly picks the best matching file when multiple candidates exist, preferring the one with higher token overlap even when another has a matching year.	—
T-CIT-MATCH-007	The <code>YEAR_MISMATCH_PENALTY</code> constant is between 0 (exclusive) and 1 (exclusive) – it must penalize but not zero out the score.	Uses a runtime binding to avoid clippy's <code>assertions_on_constants</code> lint.
T-CIT-MATCH-008	Regression – a BibTeX entry for a stock-returns paper must not match "Comparing measures of sample skewness and kurtosis.pdf".	The previous overlap-coefficient algorithm accepted this because the shared 3-letter token "and" and the word "sample" together reached the 0.30 threshold. After switching to Jaccard similarity with minimum token length 4, both false matches are rejected.
T-CIT-MATCH-009	Regression – "The econometrics of financial markets.pdf" must not match cont2000 ("Herd Behavior and Aggregate Fluctuations in Financial Markets").	The old overlap coefficient gave score $2/3 = 0.67$ because the 3-token file shared "financial" and "markets". Jaccard gives $2/12 = 0.17$, which is below threshold, so only the correct Cont/Bouchaud file is returned.
T-CIT-MATCH-010	Regression – "The econometrics of financial markets.pdf" must not match a short-title book entry whose only meaningful token is "econometrics".	The old overlap coefficient gave score $1/\min(2,3) = 0.5$. Jaccard gives $1/(2+3-1) = 0.25$, which is below threshold.

ID	Description	Acceptance Criterion
T-CIT-MATCH-011	Regression – "The Akaike Information Criterion Background Derivation Properties Application Interpretation and Refinements.pdf" (the cavanaugh2019 file) must not be flagged as a match for akaike1973 ("Information Theory and an Extension of the Maximum Likelihood Principle").	The two titles share only "akaike" and "information" as 4+ char tokens. Jaccard = $2/15 = 0.13 < 0.30$ threshold.
T-CIT-MATCH-012	Regression – find_all_disk_file_matches must not return more than one result when only one file genuinely matches the BibTeX entry.	Before the fix, shared stopwords ("and", "the") and common domain tokens caused a second unrelated file to exceed the overlap threshold, triggering a false "duplicate" status in the UI.
T-CORS-001	is_loopback recognizes all three canonical loopback address representations (IPv4, IPv6, hostname).	Any other value is treated as a non-loopback address.
T-CORS-002	build_cors_layer constructs a layer without panicking for each of the three configuration branches: localhost (permissive), LAN without origins (restrictive default), and LAN with explicit origins.	This test verifies that config.bind_address is the sole input used for CORS policy selection.

ID	Description	Acceptance Criterion
T-CORS-003	Documents the fix for the CLI-bind divergence: if the caller omits updating <code>config.bind_address</code> before calling <code>build_cors_layer</code> , a server bound to 0.0.0.0 would be evaluated as localhost (default value).	This test asserts that the <code>is_loopback</code> check uses the actual bind address, NOT the default config value, i.e., the caller has the responsibility to override <code>config.bind_address</code> with the CLI value before calling this function.
T-SEC-010	<code>escape_like_pattern</code> escapes the backslash escape character itself, then SQL LIKE wildcards <code>%</code> and <code>_</code> .	—
T-SEC-011	<code>escape_like_pattern</code> handles empty input.	—
T-SEC-012	<code>escape_like_pattern</code> handles input that is entirely metacharacters.	—
T-SEC-013	The compare handler wraps <code>file_path</code> with a leading <code>%</code> for suffix matching, with the path content escaped.	This test verifies the pattern construction logic without needing a database connection.
T-SHUT-001	Verifies that the error returned when shutdown is called in GUI mode (<code>state.headless == false</code>) maps to HTTP 400 Bad Request.	The utoipa annotation on the shutdown handler declares <code>status = 400</code> for this branch. This test confirms that <code>ApiError::BadRequest</code> — the variant the handler returns — produces exactly HTTP 400, so the annotation and the implementation agree. The previous annotation incorrectly declared <code>status = 403</code> (Forbidden), which would have caused API clients and the generated OpenAPI spec to expect the wrong status code.

ID	Description	Acceptance Criterion
T-SHUT-002	Verifies that a nonce mismatch (wrong value or wrong length) maps to HTTP 403 Forbidden, not HTTP 400 or any other code.	The handler uses <code>ApiError::Forbidden</code> when the provided nonce does not match the startup nonce. This test confirms that variant maps to exactly HTTP 403 so API clients receive a distinct status from the GUI-mode rejection path (which returns HTTP 400).
T-SOURCE-SSE-001	Regression – <code>build_source_sse_payload</code> must produce flat JSON where <code>cite_key</code> and <code>status</code> are at the top level.	Before the fix, the closures wrapped the entry under a nested "data" field, causing the frontend <code>isSourceEntryUpdate</code> type guard to reject every event (it checks for <code>cite_key</code> and <code>status</code> at the top level). The progress bar stayed at zero and the results list never updated during a fetch.
T-SOURCE-SSE-002	Regression – <code>build_source_sse_payload</code> must flatten a <code>download-status</code> entry the same way it flattens a <code>DOI-resolve</code> entry.	Covers the <code>send_source_event</code> closure used during Phase 2 (downloading).
T-SOURCE-SSE-003	<code>build_source_sse_payload</code> must not modify the original entry (it clones before inserting the event field).	This matters because the same entry value is pushed into the HTTP response results vector after being passed to <code>send_source_event</code> ; HTTP results must not carry the "event" discriminator field.
T-UTIL-001	Loopback bind with empty allowlist permits all paths.	The returned <code>PathBuf</code> matches the input path (no canonicalization is performed in the loopback fast-path).
T-UTIL-002	Non-loopback bind with empty allowlist rejects all paths.	—
T-UTIL-003	Path within an allowed root is permitted.	The returned <code>PathBuf</code> is the canonicalized form of the input path.
T-UTIL-004	Path outside all allowed roots is rejected.	—
T-UTIL-005	Nonexistent path is rejected when roots are configured.	—

ID	Description	Acceptance Criterion
<i>Crate: <code>neuroncite-chunk</code> – Text chunking strategies</i>		
T-CHK-001	Given <code>n</code> pages, page-based chunking produces exactly <code>n</code> chunks.	—
T-CHK-002	The <code>chunk_index</code> values form a contiguous sequence from 0 to <code>n-1</code> .	—
T-CHK-003	Each chunk (except the last) contains exactly <code>window_size</code> whitespace-separated words.	—
T-CHK-004	The last <code>overlap</code> words of chunk <code>i</code> are identical to the first <code>overlap</code> words of chunk <code>i+1</code> .	—
T-CHK-005	The last chunk contains between 1 and <code>window_size</code> words (inclusive).	—
T-CHK-006	A chunk whose word range spans a page boundary has <code>page_start</code> < <code>page_end</code> .	—
T-CHK-007	Each chunk, when tokenized by the model’s tokenizer, produces at most <code>token_limit</code> tokens.	—
T-CHK-008	The last <code>token_overlap</code> tokens of chunk <code>i</code> match the first <code>token_overlap</code> tokens of chunk <code>i+1</code> (verified via tokenizer output).	—

ID	Description	Acceptance Criterion
T-CHK-009	Each chunk's content is a verbatim (byte-exact) substring of the concatenated, normalized document text at the offsets <code>[doc_text_offset_start, doc_text_offset_end)</code> .	—
T-CHK-010	No sentence boundary inside chunk interior (sentences are not split mid-sentence).	Each chunk contains only complete sentences.
T-CHK-011	No chunk exceeds <code>max_words</code> whitespace-separated words.	—
T-CHK-012	Text containing "et al." followed by a lowercase word does not produce a sentence break at the period after "al".	—
T-CHK-013	Multiple spaces, tabs, and form feeds are collapsed to a single space character.	—
T-CHK-014	A word hyphenated at a line break ("exam-\{nple") is joined to "example" in the normalized output.	—
T-CHK-015	The decomposed sequence <code>"e\{u{0301}"</code> (e + combining acute accent) is normalized to the precomposed form <code>"\{u{00e9}"</code> (e-acute).	—

ID	Description	Acceptance Criterion
T-CHK-016	Resolving <code>doc_text_offset_start</code> through cumulative page byte counts yields the correct <code>page_start</code> value.	Test setup: three pages with byte counts [80, 120, 60]. Concatenated text layout: <code>p1(80) + '\{\}n'(1) + p2(120) + '\{\}n'(1) + p3(60)</code> . An offset of 100 falls within page 2 (page 2 starts at byte 81).
T-CHK-017	Processing the same PDF with the same strategy and parameters twice produces identical chunk lists (same text, same offsets, same hashes).	—
T-CHK-018	The <code>content_hash</code> of each chunk equals the SHA-256 hex digest of the chunk's content bytes.	—
T-CHK-019	The factory function creates a token strategy when given valid <code>chunk_size</code> and tokenizer JSON parameters.	—
T-CHK-020	The factory function rejects a token strategy when the tokenizer JSON is malformed (cannot be deserialized).	—
T-CHK-021	The factory function rejects a token strategy when <code>token_overlap >= token_limit</code> .	—
T-CHK-022	The factory function rejects a token strategy with zero <code>chunk_size</code> .	—
T-CHK-023	End-to-end test of the token strategy created via the factory function.	Verifies that chunking produces non-empty results with valid page ranges and byte offsets.

ID	Description	Acceptance Criterion
T-CHK-024	Token strategy with default overlap (None) produces non-overlapping chunks.	When overlap is not specified, the factory defaults to 0.
T-CHK-025	Token strategy determinism – identical input produces identical output when using the same tokenizer.	—
T-CHK-026	The word strategy zero <code>chunk_size</code> is rejected.	—
T-CHK-027	The sentence strategy zero <code>max_words</code> is rejected.	—
T-CHK-028	The sentence strategy uses explicit <code>max_words</code> when provided.	Regression test for NEW-1: verifies that explicit <code>max_words</code> overrides the default and is not ignored.
T-CHK-029	The sentence strategy defaults to 256 when <code>chunk_size</code> is provided but <code>max_words</code> is not.	The <code>chunk_size</code> parameter is mapped to <code>max_words</code> by the <code>preview_chunks</code> handler, but the factory function itself only looks at <code>max_words</code> . Verifies the default still applies.
T-CHK-030	Spacing diaeresis (U+00A8) after a vowel is converted to a combining diaeresis, then NFC composes "u" + combining diaeresis into the precomposed "ue" (U+00FC).	This is the primary fix for German umlauts extracted from PDFs with separate diacritical mark glyphs.
T-CHK-031	All three German umlaut vowels (a, o, u) are correctly composed from spacing diaeresis sequences in a single text block.	—

ID	Description	Acceptance Criterion
T-CHK-032	Spacing diaeresis that does not follow a letter is preserved as-is, because it may be intentional punctuation or mathematical notation.	—
T-CHK-033	Spacing diaeresis after a digit or punctuation is preserved, since only letter + diaeresis sequences represent decomposed umlauts.	—
T-CHK-034	Latin ligature ff (U+FB00) is decomposed into "ff".	—
T-CHK-035	Latin ligatures fi (U+FB01) and fl (U+FB02) are decomposed.	—
T-CHK-036	Latin ligatures ffi (U+FB03) and ffl (U+FB04) are decomposed.	—
T-CHK-037	Combined diacritical mark repair and ligature decomposition in a single text block.	Tests the full normalization pipeline with a realistic German technical text containing both artifact types.
T-CHK-038	Spacing acute accent (U+00B4) after a vowel is converted to combining acute, then NFC composes the precomposed form.	—
T-CHK-039	Spacing cedilla (U+00B8) after 'c' is converted to combining cedilla, then NFC composes c-cedilla (U+00E7).	—

ID	Description	Acceptance Criterion
T-CHK-040	Full pipeline test with hyphenation repair, whitespace collapsing, diacritical mark repair, ligature decomposition, and NFC normalization applied to realistic German PDF extraction output.	—
<i>Crate: neuroncite-citation – LaTeX citation verification pipeline</i>		
T-CIT-001	Single <code>\{cite{key}</code> extraction.	Verifies that a single citation command produces one RawCitation with correct fields. The <code>anchor_before</code> captures all words before the citation on the same line.
T-CIT-002	Multi-key <code>\{cite{a,b,c}</code> produces 3 entries with the same <code>group_id</code> .	Each entry has the correct <code>cite_key</code> .
T-CIT-003	All supported citation command variants are recognized.	—
T-CIT-004	Commented lines (starting with <code>%</code>) are skipped.	—
T-CIT-005	Section tracking sets <code>section_title</code> correctly.	Citations after a <code>\{section{}</code> heading inherit that section’s title. A subsequent <code>\{subsection{}</code> updates the <code>section_title</code> .
T-CIT-006	Anchor extraction captures up to 15 words from the same line.	With the extended anchor context, all available words on the line are captured (up to the <code>MAX_ANCHOR_WORDS</code> limit).
T-CIT-007	Citation with optional argument <code>\{cite[p.~42]{key}</code> .	The optional argument is consumed by the regex and the cite-key is correctly extracted.
T-CIT-008	Empty .tex content produces an empty vector.	—

ID	Description	Acceptance Criterion
T-CIT-009	Citation at the start of a line (no anchor_before) and at the end of a line (no anchor_after).	—
T-CIT-010	Line numbers are 1-indexed and correct across multiple lines.	—
T-CIT-011	Single BibTeX entry with all fields populated.	—
T-CIT-012	Entry without abstract or keywords fields.	Those fields must be None in the parsed BibEntry.
T-CIT-013	Multiple entries parsed correctly.	—
T-CIT-014	Brace-delimited and quote-delimited values are both correctly parsed.	—
T-CIT-015	Multi-line field values are collapsed into single-line strings with normalized whitespace.	—
T-CIT-016	Author names with LaTeX accent commands are preserved as-is without interpretation.	—
T-CIT-017	Empty .bib content produces an empty HashMap.	—
T-CIT-018	Case-insensitive field names.	"Author" and "author" produce the same result.
T-CIT-019	Basic batching respects batch_size.	With 7 citations and batch_size=3, the first batch has 3 entries, the second has 3, and the third has 1.
T-CIT-020	Citations in different sections get different batches when batch_size allows.	Section boundaries act as natural batch boundaries.

ID	Description	Acceptance Criterion
T-CIT-021	group_id members are never split across batches.	A 3-member group at the batch boundary stays in one batch.
T-CIT-022	A single group that exceeds batch_size occupies one batch.	—
T-CIT-023	A single citation gets batch_id 0.	—
T-CIT-024	Case-insensitive url and doi field names.	"URL" and "DOI" in uppercase are recognized the same as "url" and "doi".
T-CIT-025	The first test (t_cit_011) with all fields now includes url and doi as None since that entry does not have them.	—
T-CIT-026	claim_next_batch returns None when all batches are done.	—
T-CIT-027	Two sequential claims take different batches (atomic).	—
T-CIT-028	Stale batch reclaim after timeout.	A claimed batch older than STALE_TIMEOUT_SECS is reset to pending and can be claimed again.
T-CIT-029	submit_batch_results updates status and stores result_json.	—
T-CIT-030	submit rejects row that is not in claimed status.	—
T-CIT-031	submit rejects row that belongs to a different job.	—
T-CIT-032	count_by_status returns correct counts.	—
T-CIT-033	count_verdicts aggregates from result_json.	—
T-CIT-034	list_flagged_rows returns only rows with non-NULL flag.	—

ID	Description	Acceptance Criterion
T-CIT-035	progress_done updates correctly after submit.	—
T-CIT-036	Annotation CSV has correct columns.	—
T-CIT-037	Colors map correctly to verdicts.	—
T-CIT-038	Corrections JSON sorted by tex_line.	—
T-CIT-039	Summary report contains all statistics.	—
T-CIT-040	reset_failed_rows resets all failed rows when no batch_ids filter.	—
T-CIT-041	reset_failed_rows with specific batch_ids only resets those batches.	—
T-CIT-042	reset_failed_rows returns (0, 0) when no failed rows exist.	—
T-CIT-043	reset_failed_rows clears result_json and flag fields.	—
T-CIT-044	list_all_rows returns rows in all statuses.	—
T-CIT-045	list_rows_filtered with status filter returns only matching rows.	—
T-CIT-046	list_rows_filtered with pagination returns correct slice.	—
T-CIT-047	claim_specific_batch claims the requested batch directly.	—
T-CIT-048	claim_specific_batch returns None for already claimed batch.	—

ID	Description	Acceptance Criterion
T-CIT-049	claim_specific_batch retries failed batches by resetting failed rows to pending before claiming.	—
T-CIT-050	submit rejects wrong_source verdict without other_source_list.	—
T-CIT-051	submit rejects partial verdict with confidence > 0.80.	—
T-CIT-052	submit accepts partial verdict with confidence <= 0.80.	—
T-CIT-053	submit rejects invalid flag values.	—
T-CIT-054	submit stores empty flag as NULL in the database.	—
T-CIT-055	co_citation_json is stored and retrieved correctly.	—
T-CIT-056	co_citation_info returns default when JSON is NULL.	—
T-CIT-057	reset_rows_by_ids resets only the specified rows and leaves other rows in their original status.	—
T-CIT-058	reset_rows_by_ids with empty slice returns (0, 0) without modifying any rows.	—
T-CIT-059	reset_rows_by_flags resets only rows with matching flag values and leaves rows with other flags or no flag unchanged.	—

ID	Description	Acceptance Criterion
T-CIT-060	reset_rows_by_ids returns correct batches_affected count when resetting rows across multiple batches.	—
T-CIT-061	reset_rows_by_flags clears result_json and flag columns so the sub-agent starts with a clean slate.	—
T-CIT-062	submit rejects supported verdict with confidence below 0.50.	Under the meta-confidence model, a supported verdict with confidence below the 0.50 floor contradicts the requirement that the agent must be at least moderately certain the source supports the claim.
T-CIT-063	submit accepts supported verdict with confidence at the floor boundary (0.50).	—
T-CIT-064	submit rejects not_found verdict with confidence below 0.70.	Corpus absence is a near-binary determination, so the meta-confidence floor is higher (0.70) than for interpretive verdicts.
T-CIT-065	submit rejects unsupported verdict with confidence below 0.50.	—
T-CIT-066	submit accepts unsupported verdict with high confidence (0.85).	Under meta-confidence, a high value on unsupported means the agent is very certain the source does not support the claim.
T-CIT-067	submit rejects unverifiable verdict with confidence below 0.50.	The agent must be at least moderately certain the claim cannot be checked.
T-CIT-068	passage_location is empty string for missing passages.	When fewer than 3 passages exist, the missing passage_location columns are empty strings in the CSV.
T-CIT-069	row_to_columns returns exactly 38 elements.	—

ID	Description	Acceptance Criterion
T-CIT-070	All 16 PassageLocation variants serialize to their snake_case label and round-trip through serde correctly.	—
T-CIT-071	PassageRef with passage_location serializes and deserializes correctly through serde_json, including through result_json storage.	—
T-CIT-072	Full-detail JSON export includes passage_location in passages.	—
T-CIT-073	Excel workbook generates valid bytes with passage_location columns.	Verifies the workbook contains all 38 columns including the three passage_location columns.
T-CIT-074	EXPORT_COLUMNS contains passage_N_location at correct indices.	Verifies the column ordering: passage_N_location appears after each passage_N_score, before the next passage group.
T-CIT-075	SubmitEntry with all fields populated serializes and deserializes correctly, including the passage_location field on passages and the other_source_list, better_source array, and co-citation fields.	—
T-CIT-076	Annotation CSV includes passage_location in the comment field when passages have location metadata.	—

ID	Description	Acceptance Criterion
T-CIT-077	Data CSV with rows having no passages produces empty passage_location columns.	—
T-CIT-078	Excel with all 7 verdict types and passage_location produces valid output without errors.	—
T-CIT-089	Anchor extraction is limited to MAX_ANCHOR_WORDS (25) words.	When more than 25 words precede a citation, only the last 25 are captured.
T-CIT-091	tex_context captures surrounding lines up to ~200 chars.	A citation on a short line expands to include adjacent lines for context.
T-CIT-092	tex_context handles a single-line document.	—
T-CIT-093	extract_trailing_words returns empty string for empty input.	—
T-CIT-094	extract_leading_words strips trailing punctuation from the last word only.	—
T-CIT-095	extract_tex_context builds context from surrounding lines.	—
T-CIT-096	Rows without passages produce no annotation CSV entry.	—
T-CIT-097	Rows without latex_correction produce no corrections entry.	—
T-CIT-098	Full-detail JSON contains all verification fields for done rows.	—

ID	Description	Acceptance Criterion
T-CIT-099	Full-detail JSON includes pending rows with null result.	—
T-CIT-100	Entry with url field.	The url field is extracted and stored in the BibEntry struct for source acquisition.
T-CIT-101	Entry with doi field.	The doi field is extracted for resolving source URLs via https://doi.org/{doi} .
T-CIT-102	Entry with both url and doi fields.	Both are extracted independently and can coexist in the same BibEntry.
T-CIT-103	Entry without url and doi fields.	Both fields are None when absent from the BibTeX source.
T-CIT-104	URL field with quote-delimited value.	The parser handles both brace-delimited and quote-delimited url values.
T-CIT-107	insert_citation_rows stores all fields correctly.	—
T-CIT-108	claim_next_batch returns the lowest pending batch.	—
T-CIT-109	Data CSV has 38 columns in correct order.	—
T-CIT-110	Data CSV sorts by tex_line ascending.	—
T-CIT-111	Excel workbook generates valid bytes.	—
T-CIT-112	Excel workbook handles multiple verdicts with different row colors.	—
T-CIT-113	Verdict row_bg_hex and badge_hex return valid hex strings.	—
T-CIT-114	other_source_list serialized as JSON in data CSV.	—
T-CIT-115	better_source serialized as JSON array in data CSV.	—

ID	Description	Acceptance Criterion
T-CIT-116	passage_location appears in data CSV at the correct column positions. <i>Crate: neuroncite-core – Shared domain types, traits, and configuration</i>	Columns 26, 30, 34 (0-indexed) are passage_1_location, passage_2_location, passage_3_location respectively.
T-CORE-001	PageText construction with valid fields.	All fields are accessible after construction and match the input values.
T-CORE-002	Chunk content_hash determinism.	Two chunks with identical text content produce identical SHA-256 hashes. Two chunks with different content produce different hashes.
T-CORE-003	EmbeddedChunk vector dimension equals the vector_dimension field of the associated IndexConfig.	—
T-CORE-004	Citation formatted string contains the file display name, page number, and a text excerpt enclosed in quotation marks.	—
T-CORE-005	SearchResult ordering.	A vector of SearchResult sorted by descending score produces a monotonically non-increasing sequence.
T-CORE-006	IndexConfig TOML roundtrip.	Serializing an IndexConfig to TOML and deserializing it back produces a value equal to the original.
T-CORE-007	Every variant of NeuronCiteError produces a non-empty Display string that does not contain the variant name verbatim (the message is human-readable, not a raw enum variant name).	—

ID	Description	Acceptance Criterion
T-CORE-008	ModelManifest TOML roundtrip preserves all fields including the tokenizer checksum.	—
T-CORE-009	Single-page document of 100 bytes.	Resolving offset 50 returns page 1 with page-local offset 50.
T-CORE-010	Multi-page document with pages [80, 120, 60] bytes.	The concatenated text is: $p1(80) + '\{\}n'(1) + p2(120) + '\{\}n'(1) + p3(60)$ Total length = $80 + 1 + 120 + 1 + 60 = 262$ bytes. Offset 81 is the first byte of page 2 (80 bytes for page 1 + 1 separator). Expected result: page 2, page-local offset 0.
T-CORE-011	Last byte of the document returns the last page with the correct page-local offset.	Pages [80, 120, 60]: total = $80 + 1 + 120 + 1 + 60 = 262$. Last byte offset = 261 (0-indexed). Accumulated before page 3: $80 + 1 + 120 + 1 = 202$. Page-local offset = $261 - 202 = 59$. Page 3.
T-CORE-020	Default InferenceCapabilities returns conservative CPU values.	—
T-CORE-021	InferenceCapabilities Clone produces an independent copy.	—
T-CORE-022	InferenceCapabilities Debug format includes all fields.	—
T-CORE-023	InferenceCapabilities supports all known EP names.	—
T-CORE-023a	Regression test for $O(\log n)$ binary search correctness.	Verifies that a 200-page document resolves correctly at page boundaries and mid-page offsets. The old $O(n)$ linear scan would produce the same results but in $O(n)$ time; the prefix-sum + partition_point approach achieves $O(\log n)$.
T-CORE-024	InferenceCapabilities handles edge case memory values.	—

ID	Description	Acceptance Criterion
T-CORE-024a	SearchResult accesses <code>source_file</code> , <code>page_start</code> , <code>page_end</code> exclusively through the Citation field.	Verifies that the SearchResult struct does not contain its own duplicated copies of these fields, and that all values are reachable through <code>result.citation.*</code> .
T-CORE-050	PoolingStrategy Display trait produces stable string representations used for logging and configuration display.	—
T-CORE-051	PoolingStrategy equality comparison.	Two identical variants compare as equal, different variants compare as unequal.
T-CORE-052	PoolingStrategy JSON roundtrip.	Serializing to JSON and deserializing back produces the original variant.
T-CORE-053	EmbeddingModelConfig TOML roundtrip preserves all fields including the instruction prefix strings.	—
T-CORE-054	EmbeddingModelConfig JSON roundtrip preserves all fields.	—
T-CORE-055	<code>compute_file_hash</code> produces the correct SHA-256 digest for a known byte sequence.	Two files with identical content produce the same hash; files with different content produce different hashes.
T-CORE-056	<code>compute_file_hash</code> returns an error for a non-existent path.	—
T-CORE-057b	Separator-boundary offset maps to the end of the preceding page.	The inter-page ‘ <code>\{n}</code> ’ separator at position <code>page_end</code> is treated as the exclusive-end offset of the previous page. The old $O(n)$ implementation had a latent underflow bug on separator offsets; the $O(\log n)$ version handles this correctly.
T-CORE-058c	Single-page exclusive-end boundary.	For a single page of 100 bytes, offset 100 (one past end) maps to (1, 100).

ID	Description	Acceptance Criterion
T-CORE-058d	Returns Err when doc_offset exceeds document length.	—
T-CORE-058e	Returns Err when byte_counts is empty.	—
T-CORE-059b	SearchResult JSON roundtrip.	Serializing to JSON and back preserves all fields, confirming that removing the duplicated fields does not break the serde contract.
T-CORE-MATH-001	Identical vectors produce cosine similarity of 1.0.	Two equal non-zero vectors are perfectly aligned, so their cosine is 1.0.
T-CORE-MATH-002	Orthogonal vectors produce cosine similarity of 0.0.	Two vectors whose dot product is zero are perpendicular.
T-CORE-MATH-003	Opposite vectors (negated) produce cosine similarity of -1.0.	A vector and its negation point in exactly opposite directions.
T-CORE-MATH-004	A zero-magnitude vector (all elements 0.0) returns similarity 0.0 because the denominator is zero.	—
T-CORE-MATH-005	Vectors of different lengths return 0.0 because cosine similarity is undefined for vectors in different-dimensional spaces.	—
T-CORE-MATH-006	Single-element vectors.	Cosine similarity between two positive scalars is 1.0 (same direction on the number line).
T-CORE-MATH-007	Empty slices return 0.0 because there are no dimensions to compare.	—

ID	Description	Acceptance Criterion
T-DSK-001	<code>available_disk_space</code> returns <code>Some</code> for a valid local path.	The current working directory always resides on a valid filesystem, so this test validates that the platform-specific implementation produces a parseable u64 value.
T-DSK-002	<code>available_disk_space</code> returns <code>None</code> for a nonexistent path rather than panicking.	Verifies the fail-open behavior documented in the function contract.
T-DSK-003	<code>check_disk_space</code> returns <code>Ok</code> for a valid local path.	Since the test environment has more than 500 MB free, this validates the happy path.
T-DSK-004	<code>check_disk_space</code> returns <code>Ok</code> for a nonexistent path (fail-open behavior).	When disk space cannot be determined, the check must not block the operation.
T-DSK-005	Threshold constants are correctly ordered.	The error threshold must be strictly below the warning threshold.
T-DSK-006	On Windows, drive-letter paths produce a valid space reading.	Validates that the first character extraction and PowerShell invocation work for standard drive-letter paths.
T-DSK-007	On Windows, a path without a drive letter (e.g., a relative path or UNC-like prefix) does not panic.	The function should either return <code>Some</code> (if the fallback <code>DriveInfo</code> strategy succeeds) or <code>None</code> .
T-PATH-001	<code>sanitize_path</code> replaces Windows drive colon with underscore.	—
T-PATH-002	<code>sanitize_path</code> handles Unix absolute paths by stripping the leading slash.	—
T-PATH-003	<code>sanitize_path</code> handles complex Windows paths with multiple directory levels.	—
T-PATH-004	<code>sanitize_path</code> handles paths with trailing separator.	—

ID	Description	Acceptance Criterion
T-PATH-005	sanitize_path produces a non-empty string for any non-empty input path.	—
T-PATH-006	base_dir returns a path containing "NeuronCite".	—
T-PATH-007	runtime_dir, models_dir, indexes_dir are subdirectories of base_dir.	—
T-PATH-008	index_dir_for_path returns a path under indexes_dir.	—
T-PATH-009	Two different input paths produce different index directories.	—
T-PATH-010	sanitize_path produces a string that contains no path separators (forward slash or backslash), making it safe to use as a single directory name component.	—
T-PATH-011	canonicalize_directory returns Ok with a valid path for an existing directory and the result does not start with the Windows <code>\{\}\{\}?\{\}</code> extended-length prefix.	—
T-PATH-012	canonicalize_directory returns Err for a path that does not exist on disk.	The error propagates to callers instead of silently returning the unverified input path.

ID	Description	Acceptance Criterion
T-PATH-013	canonicalize_directory — produces consistent results for the same directory across multiple calls (idempotent when the path already has no \{\}\{\}?\{\} prefix).	
T-PATH-014	strip_extended_length_prefix removes the \{\}\{\}?\{\} prefix from a Windows extended-length path.	
T-PATH-015	strip_extended_length_prefix returns the input unchanged when no prefix is present.	
T-PATH-016	Stripping the \{\}\{\}?\{\} prefix from both sides of a path comparison produces equality for the same underlying filesystem path.	This is the core logic that fixes BUG-007 where the discover handler compared \{\}\{\}?\{\}D:\{\}...\{\}paper.pdf (from the database) against D:\{\}...\{\}paper.pdf (from read_dir) and the comparison always failed.
T-PATH-017	A HashSet comparison works correctly after stripping the \{\}\{\}?\{\} prefix from both indexed and filesystem paths.	This tests the exact pattern used in the discover handler's unindexed PDF detection.
T-TIME-001	unix_timestamp_secs — returns a positive value on any system with a properly configured clock.	
<i>Crate: neuroncite-embed – Embedding backend abstraction and ML framework integrations</i>		
T-CACHE-FILES-001	model_expected_files returns the local filenames for a known model.	Verifies that the function returns Some with a vector containing both "model.onnx" and "tokenizer.json" for a model that exists in the MODEL_FILES manifest.

ID	Description	Acceptance Criterion
T-CACHE-FILES-002	model_expected_files returns None for an unknown model identifier that is not present in the MODEL_FILES manifest.	—
T-CACHE-MANIFEST-001	Every model ID in MODEL_FILES has a non-empty file list containing at least model.onnx and tokenizer.json.	—
T-CACHE-REDIRECT-001	Verifies that the download redirect mapping resolves canonical model IDs to their community ONNX repository IDs.	Models without a redirect mapping should resolve to themselves.
T-CACHE-RERANK-001	The cross-encoder model is present in MODEL_FILES.	Without this entry, download_model() returns "no download manifest" and the reranker handler cannot auto-download the model.
T-CACHE-RERANK-002	The cross-encoder model manifest contains the required ONNX model file and tokenizer.	OrtReranker::resolve_model_paths looks for "model.onnx" and "tokenizer.json" in the cache directory.
T-CACHE-RERANK-003	The cross-encoder canonical repo ships ONNX weights directly at onnx/model.onnx, so no DOWN-LOAD_REPO_REDIRECT is needed.	The previous Xenova redirect was removed because the canonical repo now contains ONNX exports and the Xenova repo became inaccessible.

ID	Description	Acceptance Criterion
T-CACHE-RERANK-004	The cross-encoder remote path in MODEL_FILES uses the "onnx/model.onnx" prefix, which matches the canonical repository structure.	OrtReranker expects a flat "model.onnx" in the cache directory, which is achieved by the (remote_path, local_name) mapping.
T-CACHE-RERANK-005	model_expected_files returns the correct local filenames for the cross-encoder model, matching what OrtReranker looks for in the cache directory.	—
T-CACHE-RERANK-006	All DOWN-LOAD_REPO_REDIRECTS entries reference a canonical model ID that is also in MODEL_FILES.	A redirect without a MODEL_FILES entry would be unreachable dead code.
T-CACHE-RERANK-007	The MiniLM-L12-v2 model is present in MODEL_FILES.	Without this manifest entry, download_model() returns "no download manifest" and the reranker handler cannot auto-download the model.
T-CACHE-RERANK-008	The MiniLM-L12-v2 manifest contains the required ONNX model file and tokenizer.	OrtReranker::resolve_model_paths looks for "model.onnx" and "tokenizer.json" in the local cache directory.
T-CACHE-RERANK-009	The MiniLM-L12-v2 remote path in MODEL_FILES uses the "onnx/" prefix, matching the Xenova community repository structure where ONNX weights are stored under an "onnx/" subdirectory.	—

ID	Description	Acceptance Criterion
T-CACHE-RERANK-010	The MiniLM-L12-v2 canonical repo ships ONNX weights directly at <code>onnx/model.onnx</code> , so no <code>DOWN-LOAD_REPO_REDIRECT</code> is needed.	The previous Xenova redirect was removed because the canonical repo now contains ONNX exports and the Xenova repo became inaccessible.
T-CACHE-RERANK-011	The Electra-Base model is present in <code>MODEL_FILES</code> .	Without this entry, <code>download_model()</code> cannot locate the file manifest and returns an error before any download is attempted.
T-CACHE-RERANK-012	The Electra-Base manifest contains the required ONNX model file and tokenizer.	<code>OrtReranker::resolve_model_paths</code> expects both <code>"model.onnx"</code> and <code>"tokenizer.json"</code> to be present in the cache directory before the model can be loaded.
T-CACHE-RERANK-013	The Electra-Base canonical repo ships ONNX weights directly at <code>onnx/model.onnx</code> , so no <code>DOWN-LOAD_REPO_REDIRECT</code> is needed.	The previous Xenova redirect was removed because the Xenova/ms-marco-electra-base repository became private (HTTP 401 Unauthorized).
T-CACHE-RERANK-014	All three cross-encoder reranker models are registered in <code>MODEL_FILES</code> .	This completeness check ensures that every model offered in the GUI catalog has a corresponding download manifest entry so that auto-download never silently fails.
T-EMB-001	L2 normalization produces a unit vector.	After normalization, the Euclidean norm of the vector must be within $1e-6$ of 1.0 for any non-zero input.
T-EMB-002	Tokenizer produces expected token IDs for known input.	Verifies that "hello world" tokenizes into the expected token IDs from the test vocabulary.
T-EMB-003	Batch padding and attention masks are correct.	When two texts of different lengths are batched, the shorter one is padded to the length of the longer one. Attention masks have 1 for real tokens and 0 for padding positions.
T-EMB-004	Registry returns one entry per compiled-in backend.	With the default <code>backend-ort</code> feature enabled, this test expects exactly one backend in the registry.

ID	Description	Acceptance Criterion
T-EMB-005	<code>embed_single</code> returns the same vector as <code>embed_batch(&[text])[0]</code> within 1e-6 tolerance.	This test uses the registry to create a backend. If no model is available in the cache, the test verifies structural correctness (both paths return the same error type) rather than numerical equality.
T-EMB-006	<code>embed_batch</code> vectors all have length equal to <code>vector_dimension()</code> .	This test verifies the structural invariant that all returned vectors have the correct dimensionality. If no model is cached, the test verifies that the error path is consistent.
T-EMB-007	<code>rerank_batch</code> returns exactly <code>candidates.len()</code> scores.	This test verifies that an unloaded reranker returns an error rather than fabricated scores. With a cached model, the test would verify that the number of scores matches the number of candidates.
T-EMB-008	Relevant pair scores higher than irrelevant.	Without a loaded model, this test verifies the error handling path.
T-EMB-009	Each backend returns at least one <code>ModelInfo</code> .	Every compiled-in backend must advertise at least one supported model.
T-EMB-010	<code>supports_gpu</code> reflects hardware availability.	This test verifies that the <code>gpu_supported</code> field is a boolean (true or false) – the actual value depends on the hardware and CUDA driver availability at test time.
T-EMB-011	Download and checksum verification using a temporary file.	Creates a temp file with known content, computes the expected SHA-256, and verifies that <code>verify_checksum</code> returns <code>true</code> for the correct digest and <code>false</code> for an incorrect digest.
T-EMB-012	Offline resolution – cached model loads without network.	Creates a fake model directory in the temp cache, then verifies that <code>is_cached</code> returns true and <code>model_dir</code> points to the correct path.
T-EMB-020	The <code>ORT_LIB_INIT</code> Mutex starts as None and does not permanently cache failures.	This was changed from <code>OnceLock</code> to <code>Mutex</code> so that transient failures (network timeout, disk full) can be retried without restarting the process. The <code>Mutex</code> state must never contain a cached <code>Err</code> ; failures leave it as <code>None</code> for retry.

ID	Description	Acceptance Criterion
T-EMB-021	The <code>CUDA_RUNTIME_READY</code> OnceLock is correctly typed as <code>OnceLock<bool></code> .	Unlike <code>ORT_LIB_INIT</code> , CUDA hardware availability does not change during the process lifetime, so OnceLock is the correct primitive for this value.
T-EMB-022	The ORT Mutex poison recovery pattern compiles and produces a usable guard.	Validates the <code>unwrap_or_else(p p.into_inner())</code> pattern used in <code>ensure_ort_library</code> for Mutex poison recovery.
T-EMB-023	<code>intra_op_threads</code> returns a value within the EP-specific upper bound.	CoreML ≤ 4 , CUDA/ROCm/DirectML ≤ 8 , CPU ≤ 16 . All EPs must return at least 1.
T-EMB-024	Score clamping guards against overflow and non-finite values.	Regression test for DEF-4: cross-encoder inference producing extreme float values ($\sim -5.5e15$) that propagated as ranking artifacts. The fix clamps scores to $[-1000, 1000]$ and replaces NaN/infinity with -1000. This test verifies the clamping logic directly on f32 $\rightarrow$ f64 conversion.
T-EMB-CFG-001	Every model in the catalog has a valid, non-empty <code>model_id</code> and a positive <code>vector_dimension</code> .	—
T-EMB-CFG-002	No duplicate model IDs exist in the catalog.	—
T-EMB-CFG-003	<code>find_model_config</code> returns <code>Some</code> for registered models and <code>None</code> for unknown models.	—
T-EMB-CFG-004	Decoder models (MeanPooling, LastToken) do not use <code>token_type_ids</code> .	BERT-based CLS models use <code>token_type_ids</code> , while XLM-RoBERTa-based CLS models (BGE-M3) do not. The test verifies decoder models never have <code>token_type_ids</code> and that the CLS/ <code>token_type_ids</code> relationship is consistent within each architecture family.

ID	Description	Acceptance Criterion
T-EMB-CFG-005	Decoder models have a non-empty <code>query_prefix</code> .	Encoder models have an empty <code>query_prefix</code> .
T-EMB-CFG-006	Decoder models in the catalog use a <code>max_seq_len</code> greater than the default BERT 512, reflecting their longer context windows.	Encoder models have a <code>max_seq_len</code> of 512 or higher (GTE-Large supports 8192).
T-EMB-CFG-007	<code>PoolingStrategy</code> Display trait produces expected strings.	—
T-EMB-CFG-008	Models with <code>needs_position_ids == true</code> do not use <code>token_type_ids</code> , and vice versa.	These two input requirements are mutually exclusive across the current catalog.
T-EMB-DIM-001	<code>OrtEmbedder::unload_embeddings()</code> exposes an unsafe default dimension.	<code>find_model_config()</code> tests documents the value so that any change to the default is a conscious decision. The dimension of an unloaded embedder must never be relied upon for session creation – callers must always use the model catalog (<code>find_model_config</code>) for authoritative dimensions.
T-EMB-DIM-002	Every model in the catalog has a <code>vector_dimension</code> that matches the known specification.	Prevents silent regression if someone changes a dimension constant in the catalog.
T-EMB-DIM-003	<code>find_model_config</code> returns the correct dimension for each model in the catalog.	This ensures that index handlers using <code>find_model_config</code> to resolve dimensions will always get the right value, independent of which model is loaded in the backend.

ID	Description	Acceptance Criterion
T-EMB-DIM-004	The unloaded embedder’s dimension (384) is a real dimension shared by multiple models in the catalog.	This test verifies that the default model (BAAI/bge-small-en-v1.5) is among those with dimension 384, and that models with different dimensions (e.g. Qwen3 at 1024) are correctly distinguished. The core safety against the unloaded-dimension bug is the catalog-based resolution in the index handlers, not the uniqueness of dimension 384.
T-EMB-F16-001	<code>convert_f16_to_f32</code> correctly converts a slice of known f16 values to their f32 equivalents.	Verifies that standard IEEE 754 half-precision values survive the round-trip within representable precision.
T-EMB-F16-002	<code>convert_f16_to_f32</code> handles an empty slice without panic.	—
T-EMB-F16-003	<code>convert_f16_to_f32</code> preserves the relative ordering of embedding-like values.	When downstream pooling computes a mean, the converted values must produce the same ranking as the original f16 data.
T-EMB-F16-004	Simulates the full output processing path (extraction + pooling) with synthetic f16 data to verify that mean-pooled embeddings from f16 output tensors produce valid, non-zero, L2-normalized vectors.	This test exercises the same code path that failed for Qwen3-Embedding before the f16 fallback was added.
T-EMB-LAZY-001	<code>supported_model_configs</code> returns the same pointer on repeated calls, confirming that the catalog is initialized once and reused without reallocation.	

ID	Description	Acceptance Criterion
T-EMB-LAZY-002	<code>find_model_config</code> returns a reference into the static catalog.	The returned reference must point into the same memory region as the catalog slice.
T-EMB-ORT-001	Unloaded embedder returns error on <code>embed_batch</code> .	—
T-EMB-ORT-002	<code>available_models</code> returns entries for all catalog models.	—
T-EMB-POOL-001	CLS pooling extracts position 0 for each batch element.	Verifies that <code>cls_pool</code> returns the first token’s hidden state from a synthetic 3-D output tensor.
T-EMB-POOL-002	Mean pooling computes attention-weighted average, excluding padding positions (<code>mask == 0</code>).	—
T-EMB-POOL-003	Mean pooling returns zero vector when all mask values are zero.	—
T-EMB-POOL-004	Last-token pooling extracts the final non-padding position.	—
T-EMB-POOL-005	Last-token pooling works when all tokens are real (no padding).	—
T-EMB-POOL-006	Mean pooling works correctly for batched inputs where different batch elements have different padding amounts.	—
T-EMB-SEQ-001	Unloaded <code>OrtEmbedder</code> returns the default <code>max_sequence_length</code> of 512 (matching the BERT-family default).	—

ID	Description	Acceptance Criterion
T-EMB-SEQ-002	Every model in the catalog has a <code>max_seq_len</code> that would produce a valid (≥ 1) batch size through the adaptive computation.	—
T-EMB-TRUNC-001	Truncation flag is set when input exceeds <code>max_length</code> .	Regression test for BUG-006 where truncation occurred silently without any indication. The <code>truncated</code> field in <code>BatchEncoding</code> now correctly reports whether each input was truncated.
T-EMB-TRUNC-002	Truncation flag is false when input fits within <code>max_length</code> .	—
T-EMB-TRUNC-003	Truncation flags are per-input in a batch.	When a batch contains both short and long inputs, only the long inputs are flagged as truncated.
T-EMB-TRUNC-004	Empty batch produces empty truncated flags vector.	—
T-GPU-001	<code>detect_nvidia_gpu</code> returns a boolean without panicking, regardless of whether an NVIDIA GPU or driver is present on the system.	This test guards against regressions where <code>nvidia-smi</code> invocation or output parsing could panic on unexpected environments (CI without GPU, Docker containers, etc.).
T-GPU-002	<code>is_gpu_ort_runtime</code> returns false when no ORT DLLs are installed (clean test environment).	This test creates a temporary directory structure mimicking the ORT cache layout and verifies that the GPU variant detection correctly distinguishes between CPU-only and GPU installations based on the presence of <code>onnxruntime_providers_cuda.dll</code> .

ID	Description	Acceptance Criterion
T-GPU-003	Verifies that <code>ort_cache_dir</code> returns a path containing the expected <code>NeuronCite/runtime/ort</code> directory structure under the user's Documents folder.	—
T-GPU-004	Verifies that <code>ensure_ort_runtime</code> logic correctly decides which variant to download based on GPU detection.	This is a structural test: it exercises the decision-making without triggering the actual ~300MB download. The test verifies that <code>detect_nvidia_gpu()</code> is consistent with <code>is_gpu_ort_runtime()</code> after a successful <code>ensure_ort_runtime()</code> call: if GPU hardware is present and ORT is cached, the GPU variant DLLs should be present.
<i>Crate: neuroncite-html – HTML web scraping, parsing, section splitting, and crawling</i>		
T-HTML-C-001	<code>normalize_url</code> handles various URL edge cases.	—
T-HTML-C-002	<code>parse_sitemap_xml</code> extracts URLs from valid sitemap XML.	—
T-HTML-C-003	<code>CrawlConfig::default</code> has correct default values.	—
T-HTML-C-004	<code>crawl</code> rejects a start URL that resolves to a private IP.	—
T-HTML-DL-001	URL path ending in <code>.pdf</code> is detected as PDF.	—
T-HTML-DL-002	URL path ending in <code>.pdf</code> with query parameters is detected.	—
T-HTML-DL-003	URL path not ending in <code>.pdf</code> is not detected.	—
T-HTML-DL-004	URL path ending in <code>.PDF</code> (uppercase) is detected.	—

ID	Description	Acceptance Criterion
T-HTML-DL-005	DOI resolver URL (no .pdf extension) is not detected as PDF.	—
T-HTML-DL-006	Invalid URL is not detected as PDF.	—
T-HTML-DL-007	sanitize_filename replaces invalid characters with underscores.	—
T-HTML-DL-008	sanitize_filename preserves valid characters.	—
T-HTML-DL-009	sanitize_filename truncates long names to 200 characters.	—
T-HTML-DL-010	sanitize_filename handles empty string.	—
T-HTML-DL-011	sanitize_filename replaces control characters.	—
T-HTML-DL-012	URL path ending in .pdf with fragment identifier is detected.	—
T-HTML-DL-013	classify_url returns Pdf for obvious .pdf URLs without making a network request (fast path via URL path analysis).	—
T-HTML-DL-014	build_source_filename produces a descriptive filename from BibTeX metadata with "Title (Author, Year)" format.	—
T-HTML-DL-015	build_source_filename handles multiple authors with "et al." suffix when "and" separator is present.	—
T-HTML-DL-016	build_source_filename falls back to cite_key when the title is empty.	—

ID	Description	Acceptance Criterion
T-HTML-DL-017	build_source_filename — handles missing year.	
T-HTML-DL-018	build_source_filename — handles missing author and year.	
T-HTML-DL-019	build_source_filename — handles "Given Family" author format (no comma), extracting the last token as family name.	
T-HTML-DL-020	build_source_filename strips LaTeX braces from author names.	The inner backslash-quote sequence from LaTeX accent commands gets sanitized (double-quotes are invalid in filenames).
T-HTML-DL-021	classify_url returns Html for a URL targeting a private IP (SSRF protection blocks the HEAD request and falls back to Html).	—
T-HTML-F-001	cache_path_for_url produces deterministic SHA-256-based paths.	The same URL always maps to the same cache file path.
T-HTML-F-002	cache_path_for_url — produces different paths for different URLs.	
T-HTML-F-003	default_cache_dir — returns a path ending with html_cache.	
T-HTML-F-008	build_http_client — returns a client without error.	

ID	Description	Acceptance Criterion
T-HTML-F-009	When the original URL differs from the final (redirected) URL,	Both must be written by <code>fetch_url</code> to enable lookups via either URL.
T-HTML-F-010	<code>cache_path_for_url</code> produces different paths for each.	—
T-HTML-F-011	<code>cache_path_for_url</code> produces the same path for URLs that differ only in stripped tracking parameters (UTM, error/code).	—
T-HTML-F-012	<code>cache_path_for_url</code> preserves meaningful query parameters that are not in the strip list.	—
T-HTML-F-013	<code>normalize_cache_url</code> strips UTM parameters but preserves other query parameters.	—
T-HTML-F-014	<code>normalize_cache_url</code> removes the query string entirely when all parameters are in the strip list.	—
T-HTML-F-015	<code>normalize_cache_url</code> removes fragment identifiers since they are client-side only and do not affect the served content.	—
T-HTML-F-016	<code>normalize_cache_url</code> handles URLs without query or fragment.	—
T-HTML-F-017	<code>normalize_cache_url</code> handles multiple cookie-error and tracking params from Nature/Springer combined URLs.	—

ID	Description	Acceptance Criterion
T-HTML-P-001	extract_visible_text	— extracts text from simple HTML.
T-HTML-P-002	extract_visible_text	— preserves paragraph breaks.
T-HTML-P-003	extract_visible_text	— ignores script and style content.
T-HTML-P-004	extract_visible_text	— handles HTML entities.
T-HTML-P-005	extract_visible_text	— handles empty HTML.
T-HTML-P-006	extract_main_content	— extracts article content.
T-HTML-P-008	extract_metadata	— extracts title from <title> tag.
T-HTML-P-009	extract_metadata	— extracts Open Graph tags.
T-HTML-P-010	extract_metadata	— extracts language from <html lang>.
T-HTML-P-011	extract_metadata	— extracts author from <meta name="author">.
T-HTML-P-012	extract_metadata	— extracts canonical URL.
T-HTML-P-013	split_into_sections	— creates one section per H1/H2 heading.
T-HTML-P-015	split_into_sections	— handles page with no headings.
T-HTML-P-019	sections_to_pages	— maps section indices to 1-based page numbers.

ID	Description	Acceptance Criterion
T-HTML-P-020	sections_to_pages sets correct ExtractionBackend.	—
T-HTML-P-021	extract_links resolves relative URLs against base URL.	—
T-HTML-P-022	extract_links deduplicates links.	—
T-HTML-P-023	extract_links ignores javascript: and mailto: URLs.	—
T-HTML-P-025	extract_visible_text handles and <hr> as line breaks.	—
T-HTML-RESOLVE-001	Unpaywall response with url_for_pdf extracts the direct PDF URL and sets is_pdf=true.	—
T-HTML-RESOLVE-002	Unpaywall response with url_for_pdf = null falls back to the url field and sets is_pdf=false.	—
T-HTML-RESOLVE-003	Unpaywall response with no best_oa_location returns None (paper is not open access).	—
T-HTML-RESOLVE-004	Unpaywall response with missing best_oa_location key returns None (unexpected schema).	—
T-HTML-RESOLVE-005	Semantic Scholar response with openAccessPdf.url extracts the PDF URL and sets is_pdf=true.	—

ID	Description	Acceptance Criterion
T-HTML-RESOLVE-006	Semantic Scholar response with <code>openAccessPdf = null</code> returns None (no open-access PDF available).	—
T-HTML-RESOLVE-007	Semantic Scholar response without the <code>openAccessPdf</code> key returns None (field absent from response).	—
T-HTML-RESOLVE-008	OpenAlex response with <code>best_oa_location.pdf_url</code> extracts the PDF URL and sets <code>is_pdf=true</code> .	—
T-HTML-RESOLVE-009	OpenAlex response without <code>best_oa_location.pdf_url</code> falls back to <code>primary_location.pdf_url</code> .	—
T-HTML-RESOLVE-010	OpenAlex response with no PDF URLs in any location returns None.	—
T-HTML-RESOLVE-011	<code>validate_url</code> accepts a well-formed https URL.	—
T-HTML-RESOLVE-012	<code>validate_url</code> accepts a well-formed http URL.	—
T-HTML-RESOLVE-013	<code>validate_url</code> rejects an empty string.	—
T-HTML-RESOLVE-014	<code>validate_url</code> rejects a data: URI.	—
T-HTML-RESOLVE-015	<code>validate_url</code> rejects a malformed URL.	—
T-HTML-RESOLVE-016	<code>resolve_doi</code> with <code>email=None</code> skips Unpaywall.	Verified by the fact that doi.org fallback is used (no network calls in unit test context would succeed for Semantic Scholar/OpenAlex).

ID	Description	Acceptance Criterion
T-HTML-RESOLVE-017	DoiSource Display trait renders the expected strings.	—
T-HTML-RESOLVE-018	doi.org fallback produces is_pdf=false because the redirect target is unknown at resolution time.	—
T-HTML-RESOLVE-019	Unpaywall parser sets is_pdf=true for url_for_pdf results, distinguishing them from landing page fallbacks.	—
T-HTML-RESOLVE-020	urlencoding encodes DOI special characters correctly.	—
T-HTML-RESOLVE-021	urlencoding encodes email special characters correctly.	—
T-SEC-008	A standard URL-matching pattern compiles within the NFA size limit imposed by RegexBuilder.	This verifies that typical user-supplied patterns for crawl filtering are not rejected by the ReDoS protection.
T-SSRF-V-001	10.0.0.1 (RFC 1918 Class A private) is blocked.	—
T-SSRF-V-002	10.255.255.255 (upper bound of 10.0.0.0/8) is blocked.	—
T-SSRF-V-003	172.16.0.1 (RFC 1918 Class B private lower bound) is blocked.	—
T-SSRF-V-004	172.31.255.255 (RFC 1918 Class B private upper bound) is blocked.	—
T-SSRF-V-005	172.15.255.255 (just below the 172.16.0.0/12 range) is allowed.	—

ID	Description	Acceptance Criterion
T-SSRF-V-006	172.32.0.0 (just above the 172.16.0.0/12 range) is allowed.	—
T-SSRF-V-007	192.168.0.1 (RFC 1918 Class C private) is blocked.	—
T-SSRF-V-008	192.168.255.255 (upper bound of 192.168.0.0/16) is blocked.	—
T-SSRF-V-009	127.0.0.1 (IPv4 loopback) is blocked.	—
T-SSRF-V-010	127.255.255.255 (upper bound of loopback range) is blocked.	—
T-SSRF-V-011	169.254.169.254 (AWS/GCP/Azure cloud metadata endpoint) is blocked.	—
T-SSRF-V-012	169.254.0.1 (link-local) is blocked.	—
T-SSRF-V-013	0.0.0.0 (unspecified) is blocked.	—
T-SSRF-V-014	0.255.255.255 (upper bound of 0.0.0.0/8) is blocked.	—
T-SSRF-V-015	::1 (IPv6 loopback) is blocked.	—
T-SSRF-V-016	fe80::1 (IPv6 link-local) is blocked.	—
T-SSRF-V-017	febf::1 (upper bound of fe80::/10 link-local range) is blocked.	—
T-SSRF-V-018	fc00::1 (IPv6 unique local address, fc00::/8 subset of fc00::/7) is blocked.	—
T-SSRF-V-019	fd00::1 (IPv6 unique local address, fd00::/8 subset of fc00::/7) is blocked.	—

ID	Description	Acceptance Criterion
T-SSRF-V-020	fdff:ffff:ffff:ffff:ffff:ffff:ffff:ffff (upper bound of fc00::/7) is blocked.	
T-SSRF-V-021	8.8.8.8 (Google Public DNS) is allowed.	—
T-SSRF-V-022	1.1.1.1 (Cloudflare DNS) is allowed.	—
T-SSRF-V-023	93.184.216.34 (example.com) is allowed.	—
T-SSRF-V-024	2001:db8::1 (documentation range, not blocked by SSRF rules) is allowed.	This range is reserved for documentation but is not a private/loopback/link-local range.
T-SSRF-V-025	URL with scheme "ftp" is rejected.	—
T-SSRF-V-026	URL with scheme "file" is rejected.	—
T-SSRF-V-027	URL with scheme "data" is rejected.	—
T-SSRF-V-028	URL targeting 127.0.0.1 is rejected.	—
T-SSRF-V-029	URL targeting 10.0.0.1 is rejected.	—
T-SSRF-V-030	URL targeting 192.168.1.1 is rejected.	—
T-SSRF-V-031	URL targeting 172.16.0.1 is rejected.	—
T-SSRF-V-032	URL targeting 169.254.169.254 (cloud metadata) is rejected.	—
T-SSRF-V-033	URL targeting 0.0.0.0 is rejected.	—
T-SSRF-V-034	URL targeting [::1] (IPv6 loopback) is rejected.	—
T-SSRF-V-035	URL targeting [fe80::1] (IPv6 link-local) is rejected.	—

ID	Description	Acceptance Criterion
T-SSRF-V-036	URL targeting [fc00::1] (IPv6 unique local) is rejected.	—
T-SSRF-V-037	A well-known public URL (google.com) passes validation.	This test requires DNS resolution to succeed, so it may be skipped in offline CI environments.
T-SSRF-V-038	Malformed URL returns an error.	—
T-SSRF-V-039	URL without host component is rejected.	—
T-SSRF-V-040	DNS rebinding defense – hostname "localhost" resolves to 127.0.0.1 and is blocked even though it is a named host.	—
T-SSRF-V-041	IPv4-mapped IPv6 address ::ffff:127.0.0.1 is blocked at the is_blocked_ip level because the mapped IPv4 is in the loopback range.	—
T-SSRF-V-042	IPv4-mapped IPv6 address ::ffff:10.0.0.1 is blocked.	—
T-SSRF-V-043	IPv4-mapped IPv6 address ::ffff:169.254.169.254 is blocked.	—
T-SSRF-V-044	IPv4-mapped IPv6 address ::ffff:8.8.8.8 is allowed (public IP).	—
T-SSRF-V-045	the request DNS resolver blocks loopback results at connect time.	—
<i>Crate: neuroncite-mcp – MCP (Model Context Protocol) server</i>		

ID	Description	Acceptance Criterion
T-CIT-079	Year mismatch causes rejection.	When the BibTeX year is "1970" and the only candidate filename contains "1993", the function must return None because the year mismatch indicates a different publication even if author/title tokens overlap. Regression test for BUG-005: Before the fix, <code>find_best_file_match</code> did not consider the year field, allowing "Fama, 1970" to match a file named "Fama, 1993" purely based on author-name token overlap.
T-CIT-080	Year match allows correct identification.	When the BibTeX year matches the filename year, the candidate is considered and returned if token overlap exceeds the threshold.
T-CIT-081	No year in BibTeX entry still matches via token overlap.	When the BibTeX year is None, the function falls back to pure token overlap without year filtering.
T-CIT-082	Filename without a year still matches when BibTeX has a year.	The year filter only rejects candidates that contain a <i>*different*</i> year, not candidates that contain no year at all.
T-CIT-083	Multiple candidates with different years selects the correct one.	Given two files from the same author but different years, the function must select the one whose year matches the BibTeX entry.
T-CIT-084	Empty file lookup returns None without panicking.	—
T-CIT-085	Empty author and title returns None without panicking.	—
T-CIT-086	Year field with extra text (e.g., "1993/04") is parsed correctly by extracting the first 4 digits.	—
T-CIT-087	Below-threshold overlap score returns None.	Two strings that share fewer than 30% of their tokens must not match.

ID	Description	Acceptance Criterion
T-CIT-088	Author-weighted matching disambiguates papers with similar generic titles.	Regression test for BUG-002 where the original overlap coefficient (without author weighting) matched "The econometrics of financial markets" by Campbell, Lo, MacKinlay to the wrong PDF because the generic title tokens "financial" and "markets" appeared in multiple filenames. The weighted algorithm prioritizes author family name matches.
T-CIT-090	Author-only matching works when the title has only stop-word-length tokens.	The algorithm falls back to the combined overlap coefficient when no author tokens are extractable, but when author tokens are present, they dominate the scoring.
T-CIT-117	handle_export rejects jobs with pending or claimed rows.	Regression test for DEF-1: export on an incomplete job previously succeeded silently and produced partial output. The fix adds a count_by_status check that returns an error when rows remain in pending or claimed status. This test verifies the validation logic by checking that the StatusCounts struct correctly detects incomplete states.
T-CIT-118	HTML page title provides meaningful token overlap for matching.	When an HTML file's web_source title is "Efficient Capital Markets: A Review of Theory and Empirical Work", find_best_file_match matches it to a BibTeX entry with the same title/author. This verifies that using the page title (as the lookup string for HTML files) produces correct matches, unlike using the URL-derived filename stem which yields "article" or a hash.
T-CIT-119	URL-derived filename stem fails to match BibTeX entries.	When the lookup string for an HTML file is the URL's file_stem (e.g., "article" from "https://journals.plos.org/.../article"), it has no token overlap with the BibTeX author/title and returns None.

ID	Description	Acceptance Criterion
T-MCP-001	JsonRpcRequest deserializes a valid initialize request with all fields present (jsonrpc, id, method, params).	—
T-MCP-002	JsonRpcRequest deserializes a notification (no id field).	—
T-MCP-003	JsonRpcResponse success serialization produces correct JSON with result present and error absent.	—
T-MCP-004	JsonRpcResponse error serialization produces correct JSON with error present and result absent.	—
T-MCP-005	All standard error codes are negative integers conforming to the JSON-RPC 2.0 specification range (-32768 to -32000).	—
T-MCP-006	read_message returns None on empty input (EOF).	—
T-MCP-007	read_message parses a valid JSON-RPC request line.	—
T-MCP-008	read_message returns an error for malformed JSON.	—
T-MCP-009	write_message produces a newline-terminated JSON string.	—

ID	Description	Acceptance Criterion
T-MCP-010	Multiple messages can be read sequentially from the same stream.	—
T-MCP-011	all_tools returns a non-empty list of tool definitions.	—
T-MCP-012	Every tool definition has a non-empty name and description.	—
T-MCP-013	Every tool name starts with the "neuroncite_" prefix.	—
T-MCP-014	Every tool's input_schema is a valid JSON object with a "type" field set to "object".	—
T-MCP-015	Tool names are unique (no duplicates in the list).	—
T-MCP-016	Dispatching an unknown tool name returns METHOD_NOT_FOUND.	—
T-MCP-017	Dispatching without a 'name' field returns IN-VALID_PARAMS.	—
T-MCP-018	The initialize handshake returns the correct protocol version and server capabilities.	—
T-MCP-019	tools/list returns all tool definitions with the correct structure (name, description, inputSchema for each tool).	—

ID	Description	Acceptance Criterion
T-MCP-020	The server loop processes a full initialize -> tools/list -> EOF sequence without errors.	—
T-MCP-021	Malformed JSON input produces a parse error response.	—
T-MCP-022	Unknown method name returns METHOD_NOT_FOUND error.	—
T-MCP-023	Install creates the config file and writes the correct entry structure including the "type": "stdio" field.	—
T-MCP-024	check_status returns registered=false when the config file does not contain a neuroncite entry.	—
T-MCP-025	config_file_path returns a path ending with .claude.json for the Claude Code target.	—
T-MCP-026	Every line written to the MCP stdout transport is valid JSON-RPC 2.0.	If tracing, ORT warnings, or any other diagnostic output leaks into stdout, the MCP client receives unparseable data. This test sends a full initialize + tools/list sequence and asserts that every output line is valid JSON containing the "jsonrpc" field.

ID	Description	Acceptance Criterion
T-MCP-027	Searching a session whose embedding dimension differs from the loaded model returns an MCP error with <code>isError=true</code> and a message containing "dimension mismatch".	This guards against silently producing meaningless cosine similarity scores when query and document embeddings have different vector sizes (e.g., bge-small 384d vs gte-large 1024d).
T-MCP-028	Searching a session whose embedding dimension matches the loaded model does NOT trigger a dimension mismatch error.	This is the positive counterpart to T-MCP-027, verifying that the validation does not produce false positives for correctly configured sessions.
T-MCP-029	Calling a nonexistent tool via <code>tools/call</code> returns <code>isError=true</code> instead of a protocol-level JSON-RPC error.	This prevents MCP clients from interpreting tool dispatch failures as transport errors, which would cause cancellation of sibling calls in parallel tool call batches. Regression test for the "sibling error" problem: when the MCP client sends parallel tool calls and one returns a protocol-level error, the client aborts all in-flight sibling calls.
T-MCP-030	A tool handler error (INTERNAL_ERROR) is returned as <code>isError=true</code> , not as a protocol-level JSON-RPC error.	This validates that the <code>isError</code> wrapping works for handler-level failures (e.g., database errors, validation errors).
T-MCP-031	Protocol-level errors (unknown JSON-RPC method like "resources/list") still return as JSON-RPC errors with an "error" field.	The <code>isError</code> wrapping only applies to <code>tools/call</code> dispatch, not to top-level method routing.

ID	Description	Acceptance Criterion
T-MCP-032	Two sequential tool calls where the first fails and the second succeeds both return independent responses.	This verifies that a failed tool call does not corrupt the server state or prevent subsequent calls from succeeding. The second call uses tools/list (synchronous dispatch) instead of a real tool call to avoid the <code>Handle::block_on</code> path that deadlocks when the tokio worker threads are contended by parallel tests.
T-MCP-033	The tool catalog contains 42 tools.	The original 46 tools were reduced to 41 by removing 5 HTML-specific duplicates, then increased to 43 by adding <code>neuroncite_bib_report</code> .
T-MCP-034	Every tool with required parameters has those parameters defined in the <code>properties</code> object.	—
T-MCP-035	<code>probe_pdfium_available</code> returns a boolean without panicking.	On a machine where pdfium has been downloaded to the runtime cache, this returns true; otherwise false. Either outcome is valid.
T-MCP-036	<code>probe_tesseract_available</code> returns a boolean without panicking.	Verifies that the function completes and the return value is consistent with the runtime cache state (including all search locations that <code>cached_tesseract_path()</code> checks in <code>neuroncite-pdf</code>).
T-MCP-037	The <code>PDFIUM_LIB_NAME</code> constant in <code>system.rs</code> matches the platform-specific shared library naming convention.	—
T-MCP-038	Both probe functions return consistent results across repeated calls (no flaky behavior from process spawning or file system races).	—

ID	Description	Acceptance Criterion
T-MCP-039	probe_tesseract_available and cached_tesseract_path (from neuroncite-pdf) must agree on Tesseract availability.	This regression test catches the BUG-1 scenario where the MCP doctor probe checked fewer filesystem locations than the actual extraction pipeline, causing a false negative "tesseract_available: false" even when Tesseract is installed. The test verifies that when Tesseract is found by cached_tesseract_path() in deps.rs, probe_tesseract_available() in system.rs also returns true. This ensures the doctor report and extraction pipeline are consistent.
T-MCP-040	parse_filename_metadata extracts title, authors, and year from a standard academic PDF filename.	
T-MCP-041	parse_filename_metadata handles filenames with bracketed annotations like "[Book]" or "[arXiv PDF]".	
T-MCP-042	parse_filename_metadata returns the full stem as title when no parenthesized author/year block exists.	
T-MCP-043	parse_filename_metadata handles multiple authors with the ampersand separator.	
T-MCP-044	parse_filename_metadata handles filenames with author-only parentheses (no year after comma).	

ID	Description	Acceptance Criterion
T-MCP-045	format_as_bibtex produces @article entries with extracted metadata and cleaned file paths (no \{\}\{\}? prefix).	Entries with both authors and year use @article; entries missing either use @misc.
T-MCP-046	format_as_csl_json produces entries with structured author names (family field) and article-journal type when authors+year present.	—
T-MCP-047	BibTeX uses @misc for entries without year.	—
T-MCP-048	CSL-JSON uses "report" type for entries without year.	The "report" type is a standard CSL type, replacing the non-standard "document" that was used before.
T-MCP-049	parse_csl_author with "Last, First" produces structured name.	—
T-MCP-050	parse_csl_author with single-token name produces family-only.	—
T-MCP-051	BibTeX uses @article for entries with both authors and year, and @misc otherwise.	Regression test for BUG-7 where all entries used @misc regardless of available metadata.
T-MCP-052	CSL-JSON uses article-journal for entries with both authors and year, and document otherwise.	Regression test for BUG-7 where all entries used the generic "document" type.

ID	Description	Acceptance Criterion
T-MCP-053	CSL-JSON author names are structured objects with "family" (and optionally "given") fields, not the deprecated "literal" format.	Regression test for BUG-7 where author names used <code>{"literal": "Name"}</code> which citation processors cannot format per style.
T-MCP-054	<code>parse_csl_author</code> handles whitespace around names correctly.	—
T-MCP-055	<code>parse_csl_author</code> with trailing comma (no given name).	—
T-MCP-056	<code>parse_filename_metadata</code> handles three or more authors separated by ampersands.	
T-MCP-057	<code>parse_filename_metadata</code> handles filenames with both parenthesized metadata and bracketed annotations.	
T-MCP-058	<code>generate_citation_key</code> produces human-readable keys from author names and year.	Single author produces "author_year", multiple authors produce "author1_author2_year".
T-MCP-059	<code>generate_citation_key</code> falls back to generic format when no author metadata is available.	—
T-MCP-060	Missing required parameters return an error.	—
T-MCP-061	Malformed <code>input_data</code> (invalid CSV) is detected during early validation before job creation.	—

ID	Description	Acceptance Criterion
T-MCP-062	dry_run mode validates input without creating a job.	—
T-MCP-063	append mode parameter is correctly parsed.	—
T-MCP-064	default values for dry_run and append are false.	—
T-MCP-065	dry_run uses resolve::match_pdfs_to_quotes (same function as the actual pipeline) to guarantee identical match decisions.	Regression test for DEF-6: dry_run previously used a simplified substring-based matching that produced different results from the Jaro-Winkler weighted matching in the actual annotation pipeline. The fix delegates to resolve::match_pdfs_to_quotes directly.
T-MCP-066	In append mode, the serialized params_json contains source_directory (not output_directory) as the source for text extraction, and prior_output_directory is set to output_directory.	Regression test for NEW-2 where append mode passed output_directory as the effective source, causing text extraction to fail on pdfium-re-linearized PDFs.
T-MCP-067	In non-append mode, prior_output_directory is null.	—
T-MCP-068	Empty query string is rejected by the export handler.	Regression test for BUG-003 where neuroncite_export accepted empty queries while neuroncite_search rejected them, causing inconsistent behavior between the two endpoints.
T-MCP-069	Non-empty query after trimming passes validation.	—
T-MCP-070	text_search handler rejects empty query.	—

ID	Description	Acceptance Criterion
T-MCP-071	text_search handler defaults case_sensitive to false.	—
T-MCP-072	neuroncite_index tool declares a chunk_strategy enum property with the four valid strategy values: page, word, token, sentence.	—
T-MCP-073	neuroncite_search declares a min_score property with type number.	This parameter was added for Issue 8 (search scoring transparency) to allow callers to filter low-relevance results.
T-MCP-074	neuroncite_session_update declares tags with type array-or-null and metadata with type object-or-null.	Verifies the exact JSON Schema types added for Issue 21 (session metadata).
T-MCP-075	RIS export produces valid RIS records with TY/ER tags.	—
T-MCP-076	RIS uses GEN type for entries without year.	—
T-MCP-077	Plain-text format includes numbered results with source info.	—
T-MCP-078	neuroncite_multi_search tool definition exists with session_ids and query as required parameters.	The session_ids property must be typed as an array of integers.
T-MCP-079	neuroncite_reranker tool definition exists with model_id as the sole required parameter.	The backend parameter must be optional.
T-MCP-080	Missing pdf_path parameter returns an error string.	—

ID	Description	Acceptance Criterion
T-MCP-081	Non-file pdf_path (a directory path) is detected before reaching the pdfium layer.	—
T-MCP-082	page_number extraction from params handles absent value (returns None, meaning scan all pages).	—
T-MCP-083	page_number extraction from params handles a valid integer.	—
T-MCP-084	All 42 tool names exist in the catalog.	—
T-MCP-085	The <code>neuroncite_reranker_longdr</code> tool description and the model_id property description both mention auto-download.	This pins the contract that the tool no longer requires pre-cached model files.
T-MCP-086	Missing pdf_path parameter returns an error string.	—
T-MCP-087	Missing mode parameter returns an error string.	—
T-MCP-088	Invalid mode string is detected during parameter parsing.	—
T-MCP-089	dry_run defaults to false when not provided.	—
T-MCP-090	output_path defaults to pdf_path when not provided.	—
T-MCP-091	by_color mode requires colors parameter.	—
T-MCP-092	by_page mode requires pages parameter.	—

ID	Description	Acceptance Criterion
T-MCP-093	Missing job_id parameter returns None from the JSON accessor.	—
T-MCP-093b	On Windows, probe_tesseract_available must check the Tesseract-OCR subdirectory inside the runtime cache.	This is the specific location where the UB-Mannheim NSIS installer places files when invoked with /D=<cache_dir>.
T-MCP-094	limit defaults to 100 and offset defaults to 0 when absent.	—
T-MCP-094c	On Windows, probe_tesseract_available must check the default Program Files installation directory.	—
T-MCP-095	limit is clamped to 500 when a larger value is provided.	—
T-MCP-096	limit is clamped to 1 when zero or negative value is provided.	—
T-MCP-097	offset is clamped to 0 when a negative value is provided.	—
T-MCP-098	split_filename_authors correctly handles comma+ampersand author separators.	The comma separates authors, NOT "Last, First" pairs.
T-MCP-099	BibTeX author field for comma-separated filenames formats each author name joined with " and " (BibTeX standard).	Regression test for the bug where "McNeil, Frey & Embrechts" was formatted as "McNeil, Frey and Embrechts", treating "Frey" as the given name of "McNeil".

ID	Description	Acceptance Criterion
T-MCP-100	<code>neuroncite_citation_rows</code> tool exists with <code>job_id</code> as the sole required parameter, and optional <code>status/offset/limit</code> properties.	
T-MCP-101	<code>neuroncite_citation_claim</code> tool includes optional <code>batch_id</code> in its properties while keeping only <code>job_id</code> as required.	
T-MCP-102	<code>neuroncite_annotate</code> tool declares optional <code>dry_run</code> and <code>append</code> boolean properties.	These parameters are not in the required array, and the description mentions both modes.
T-MCP-103	<code>neuroncite_preview_chunks</code> tool definition exists with <code>chunk_strategy</code> as the sole required parameter.	<code>chunk_strategy</code> is optional because HTML preview uses <code>url</code> or <code>file_id</code> instead. The <code>chunk_strategy</code> property must have a 4-value enum.
T-MCP-104	<code>neuroncite_index_add</code> tool definition exists with <code>session_id</code> and <code>files</code> as required parameters.	The <code>files</code> property must be typed as an array of strings.
T-MCP-105	The <code>neuroncite_multi_search</code> description mentions <code>session_id</code> to inform AI agents that each result is tagged with its source session.	—
T-MCP-106	The <code>neuroncite_reranker_load</code> description mentions <code>reranker_available</code> to inform AI agents to check health after loading.	—

ID	Description	Acceptance Criterion
T-MCP-107	The <code>neuroncite_index_add</code> description mentions change detection to inform AI agents about the skip-unchanged behavior.	—
T-MCP-108	The <code>neuroncite_preview_chunks</code> description mentions "without" database writes to clarify this is a stateless preview.	—
T-MCP-109	CSL-JSON author array for comma-separated filenames creates separate family-name objects for each author.	Regression test for the bug where "McNeil, Frey" was parsed as {"family": "McNeil", "given": "Frey"} instead of two separate authors.
T-MCP-110	BibTeX export deduplicates cite-keys when multiple results come from the same PDF.	Duplicate keys get a letter suffix (a, b, c, ...).
T-MCP-111	CSL-JSON export deduplicates IDs the same way as BibTeX.	—
T-MCP-112	<code>neuroncite_citation_create</code> tool includes optional <code>dry_run</code> and <code>file_overrides</code> in its properties.	—
T-MCP-113	<code>neuroncite_session_delete</code> has no required parameters.	The tool accepts either <code>session_id</code> or <code>directory</code> , both optional at the schema level. Validation of "at least one" is handled by the handler at runtime, not by the JSON Schema.
T-MCP-114	<code>session_diff</code> handler rejects same session IDs.	—
T-MCP-115	<code>session_diff</code> handler requires <code>session_a</code> parameter.	—

ID	Description	Acceptance Criterion
T-MCP-116	The MCP server starts and completes an initialize handshake when <code>spawn_worker</code> is called with <code>rt.enter()</code> guard instead of inside <code>rt.block_on()</code> .	This reproduces the exact pattern used by <code>run_mcp_serve</code> in the main binary: create a tokio runtime, enter its context via guard, spawn the worker, then run the synchronous server loop. The test verifies that this pattern does not panic and produces valid responses.
T-MCP-117	The <code>neuroncite_batch_search</code> tool definition exists and declares <code>session_id</code> and <code>queries</code> as required parameters.	—
T-MCP-118	The <code>neuroncite_files</code> tool definition exists and declares <code>session_id</code> as the sole required parameter.	—
T-MCP-119	The <code>neuroncite_session_update</code> tool definition exists and declares <code>session_id</code> as required, with optional <code>label</code> , <code>tags</code> , and <code>metadata</code> parameters.	—
T-MCP-120	The <code>neuroncite_search</code> tool description mentions <code>file_id</code> so AI agents know the field is available in search results.	—

ID	Description	Acceptance Criterion
T-MCP-121	The <code>neuroncite_inspect_annotations</code> tool definition exists with <code>pdf_path</code> as the sole required parameter and an optional <code>page_number</code> property.	—
T-MCP-122	<code>generate_citation_key</code> without year produces author-only key.	—
T-MCP-123	BibTeX uses 'file' field instead of 'howpublished'.	Regression test ensuring the PDF path is in the standard 'file' field that reference managers (Zotero, Mendeley, JabRef) recognize.
T-MCP-124	CSL-JSON uses standard "report" type instead of the non-standard "document" type for entries without year.	Regression test.
T-MCP-125	<code>config_file_path</code> returns a path ending with <code>claude_desktop_config.json</code> for the Claude Desktop target.	—
T-MCP-126	<code>McpTarget::from_cli_str</code> parses valid target strings and rejects unknown ones.	—
T-MCP-127	<code>McpTarget::all</code> returns both targets.	—
T-MCP-128	<code>cli_name</code> round-trips through <code>from_cli_str</code> for all targets.	—
T-MCP-129	<code>display_name</code> returns non-empty human-readable names.	—

ID	Description	Acceptance Criterion
T-MCP-130	install_at writes the correct mcpServers entry and uninstall_at removes it again.	—
T-MCP-131	install_at creates parent directories if they do not exist.	—
T-MCP-132	install preserves existing keys in the config file.	—
T-MCP-133	uninstall is a no-op when config file does not exist.	—
T-MCP-134	check_status_at returns correct config_path when the file does not exist.	—
T-MCP-135	install and uninstall on separate config files are independent – installing in one does not affect the other.	—
T-MCP-BATCH-001	A batch where all queries are invalid (missing "id" fields) returns a top-level error because there is nothing to process.	—
T-MCP-BATCH-002	A batch with one valid and one invalid query returns a successful response containing both the search results for the valid query and an error entry for the invalid query.	—

ID	Description	Acceptance Criterion
T-MCP-BATCH-003	A batch with duplicate query IDs produces per-query error entries for the duplicates while processing the first occurrence.	—
T-MCP-BATCH-004	Structural validation errors (empty queries array, missing session_id) still return top-level errors as before.	—
T-MCP-BATCH-005	A query with out-of-range top_k produces a per-query error instead of blocking the entire batch.	—
T-MCP-BATCH-006	top_k=0 produces a per-query error in batch mode.	Regression test for BUG-004: Before the fix, top_k=0 passed the upper bound check and was silently converted to 1 via <code>.max(1)</code> . After the fix, top_k=0 fails the lower bound check (<code>top_k < 1</code>) and produces a per-query error entry.
T-MCP-BATCH-007	The response includes an error_count field that reflects the number of queries that failed validation.	This allows callers to detect partial failures without iterating all result entries. Regression test for BUG B-02: Before the fix, callers had no top-level indicator that some queries in the batch had failed. Per-query error entries existed but required iterating every result to discover them.
T-MCP-BATCH-008	When all queries in a batch are valid, error_count is 0 in the response.	—
T-MCP-BATCH-009	The error_count field is present and consistent with the actual number of error entries in the results map.	This validates the field is computed before query_errors is consumed.

ID	Description	Acceptance Criterion
T-MCP-BATCH-010	Duplicate query IDs do not overwrite the valid first occurrence's result.	Regression test for ISSUE-007 where the error entry for a duplicate ID was inserted with the same key as the valid result, replacing the successful search output with a "duplicate query id" error.
T-MCP-BATCH-011	min_score values outside [0.0, 1.0] are rejected.	Regression test for ISSUE-010 where min_score=1.5 was silently accepted, filtering out all results because no cosine similarity exceeds 1.0.
T-MCP-BATCH-012	Each query entry in the batch response contains a query_analysis object with structured tokenization details.	Verifies the presence of query_word_count, max_sequence_length, was_truncated, and bm25_terms fields.
T-MCP-BATCH-013	The query_analysis section omits bm25_terms when use_fts=false in batch mode.	—
T-MCP-BATCH-014	The full_chunk_content field preservation mechanism stores original chunk text per-query before refinement.	This unit test verifies the data structure logic without requiring an actual search.
T-MCP-BCONTENT-001	Content truncation works correctly.	—
T-MCP-BCONTENT-002	content_part_to_json includes truncation metadata.	—
T-MCP-BCONTENT-003	content_part_to_json does not include truncation fields for small content.	—
T-MCP-CHUNK-001	Calling the handler without session_id returns an error because session_id is a required parameter.	—

ID	Description	Acceptance Criterion
T-MCP-CHUNK-002	Calling the handler without <code>file_id</code> returns an error because <code>file_id</code> is a required parameter.	—
T-MCP-CHUNK-003	Providing a <code>session_id</code> that does not exist in the database returns an error.	The handler validates session existence before querying chunks.
T-MCP-CHUNK-004	Providing a <code>file_id</code> that belongs to a different session returns an error.	The handler checks <code>file.session_id == requested_session_id</code> .
T-MCP-CHUNK-005	Browsing chunks returns correct pagination metadata.	Creates a session with 5 chunks, requests <code>offset=0 limit=3</code> , and verifies that <code>returned</code> is 3 and <code>total_chunks</code> is 5.
T-MCP-CHUNK-006	The <code>page_number</code> filter narrows results to chunks whose page range includes the specified page.	With each chunk spanning exactly one page (<code>page_start == page_end == chunk_index + 1</code>), filtering by <code>page_number=3</code> returns only the chunk on page 3.
T-MCP-CHUNK-007	Each chunk object in the response contains <code>word_count</code> and <code>byte_count</code> fields.	The test content "word_a word_b word_c chunk_{i}" has exactly 4 whitespace-separated words. <code>byte_count</code> equals the UTF-8 byte length of the content string.
T-MCP-CMP-001	Calling handle without <code>file_path</code> or <code>file_name_pattern</code> returns an error.	The handler requires at least one of these two parameters to construct a LIKE pattern for the database query.
T-MCP-CMP-002	When no files in the database match the given pattern, the handler returns <code>matched_files=0</code> and an empty <code>comparisons</code> array.	This verifies the early-return path for zero matches.

ID	Description	Acceptance Criterion
T-MCP-CMP-003	The same file (identical hash) indexed in two separate sessions results in <code>same_content=true</code> in the comparison output.	Both sessions contain the same file path and content hash, so the handler correctly identifies the content as identical across sessions.
T-MCP-CMP-004	The same file name indexed in two sessions with different content hashes results in <code>same_content=false</code> .	This scenario occurs when a PDF is re-scanned or modified between indexing sessions.
T-MCP-CMP-005	The <code>file_name_pattern</code> parameter supports SQL LIKE wildcards.	A pattern with <code>%</code> matches files whose paths contain the given substring. This verifies that the pattern is forwarded to <code>find_files_by_path_pattern</code> without additional wrapping.
T-MCP-COMMON-001	Content within the byte limit is returned unmodified.	—
T-MCP-COMMON-002	Content exceeding <code>MAX_CONTENT_BYTES</code> is truncated at a valid UTF-8 boundary and the truncation flag is set.	—
T-MCP-COMMON-003	Truncation respects multi-byte UTF-8 characters and does not split in the middle of a codepoint.	—
T-MCP-COMMON-004	<code>content_part_to_json</code> includes truncation metadata for large content.	—
T-MCP-COMMON-005	<code>content_part_to_json</code> does not include truncation fields when content is within the limit.	—

ID	Description	Acceptance Criterion
T-MCP-COMMON-006	Content exactly at MAX_CONTENT_BYTES is not truncated.	—
T-MCP-COMMON-007	Empty content is handled correctly.	—
T-MCP-CONTENT-001	content_part_to_json preserves metadata fields.	—
T-MCP-CONTENT-002	Truncation includes metadata for large content.	—
T-MCP-CONTENT-003	part=0 returns a validation error.	—
T-MCP-CSRCH-001	MAX_TOP_K is set to 20 for comparison queries.	—
T-MCP-DISC-001	Calling the discover handler without the required directory parameter returns an error.	The handler validates that params["directory"] is a string; an empty JSON object causes the indexing operation on serde_json::Value::Null to fail as_str() .
T-MCP-DISC-002	Passing a directory path that does not exist on the filesystem returns a valid JSON response with directory_exists set to false .	The filesystem , sessions , and unindexed_files arrays are all empty because no directory scan or session lookup can find results for a path that does not exist.
T-MCP-DISC-003	The response JSON structure contains all five required top-level fields: directory , directory_exists , filesystem , sessions , and unindexed_files .	The filesystem sub-object contains type_counts and total_size_bytes . This test validates the response schema regardless of whether the directory exists on disk.

ID	Description	Acceptance Criterion
T-MCP-DISC-004	Files indexed with the Windows extended-length prefix (<code>\{\}\{\}?{\}</code>) in their database path are correctly matched against filesystem paths that lack the prefix.	This is the regression test for BUG-007 where <code>neuroncite_discover</code> reported all PDFs as "unindexed" on Windows because indexed file paths had the <code>\{\}\{\}?{\}</code> prefix from <code>std::fs::canonicalize()</code> while filesystem paths from <code>read_dir()</code> did not. The test inserts a file into the database with a <code>\{\}\{\}?{\}</code> -prefixed path and a corresponding file on disk without the prefix, then verifies that the discover handler correctly identifies the file as indexed (not in <code>unindexed_pdfs</code>).
T-MCP-DISC-005	Files indexed without the <code>\{\}\{\}?{\}</code> prefix are also correctly detected as indexed.	This verifies that the prefix stripping does not break the common case where paths are stored without prefix.
T-MCP-DISC-006	An unindexed PDF file on disk is correctly reported in <code>unindexed_files</code> when no matching file record exists in any session.	—
T-MCP-DISC-007	A non-existent directory produces a response with a warning field explaining that the path does not exist and that <code>neuroncite_index</code> will reject it.	This addresses the inconsistency between discover (returns structured data) and index (returns error) for non-existent directories by making the behavior self-documenting. Regression test for BUG B-03: Before the fix, discover silently returned <code>directory_exists=false</code> without any explicit guidance, while index threw an error for the same path, creating a confusing UX.
T-MCP-DISC-008	An existing directory produces a response without a warning field.	The warning is only added for non-existent directories.

ID	Description	Acceptance Criterion
T-MCP-DISC-009	The warning message includes the canonicalized directory path (with Windows extended-length prefix stripped) so the caller can verify which path was checked.	—
T-MCP-ERR-001	Missing parameter error is classified as MISSING_PARAMETER with the parameter name in details.	—
T-MCP-ERR-002	Session not found error is classified as SESSION_NOT_FOUND with the session_id extracted from the message.	—
T-MCP-ERR-003	Job not found error is classified as JOB_NOT_FOUND.	—
T-MCP-ERR-004	Connection pool error is classified as CONNECTION_ERROR.	—
T-MCP-ERR-005	Parsing failure is classified as INVALID_INPUT.	—
T-MCP-ERR-006	Concurrent job error is classified as CONCURRENT_JOB.	—
T-MCP-ERR-007	Unrecognized error messages fall back to INTERNAL_ERROR.	—
T-MCP-ERR-008	All classified errors contain the three required fields: error_code, message, and details.	—

ID	Description	Acceptance Criterion
T-MCP-ERR-009	extract_number_after — extracts the session ID from typical error messages.	—
T-MCP-ERR-010	File not found error is classified as FILE_NOT_FOUND.	The classifier matches messages containing both "file" and "not found" substrings, which are produced by handlers when a requested file_id does not exist in the database.
T-MCP-ERR-011	The "invalid" substring in error messages triggers INVALID_INPUT classification.	This path is distinct from the "parsing failed" path and covers validation errors like "invalid session_id" or "invalid min_score value".
T-MCP-ERR-012	Classification precedence: "session not found" takes priority over "file not found" when both substrings are present.	The classifier checks session before file in its match chain.
T-MCP-ERR-013	extract_number_after handles large session IDs.	Database-generated AUTOINCREMENT IDs can grow to large values over time, especially after many create/delete cycles.
T-MCP-ERR-014	All 40 tool names in the dispatch match table are present in the known tools list.	Validates that every dispatch match arm has a corresponding entry and the total count is 40 (consolidated from 46).
T-MCP-ERR-015	The "reranker backend error" message produced by the reranker handler is classified as INTERNAL_ERROR (no specific classification pattern matches).	—
T-MCP-ERR-016	The "chunking strategy error" message produced by the preview_chunks handler is classified as INTERNAL_ERROR.	—

ID	Description	Acceptance Criterion
T-MCP-ERR-017	Validation error "must not be empty" is classified as INVALID_INPUT.	Regression test for NEW-3 where index_add's "files array must not be empty" fell through to INTERNAL_ERROR.
T-MCP-ERR-018	Validation error "must contain" is classified as INVALID_INPUT.	Regression test for NEW-3/NEW-4 where multi_search's "session_ids must contain at least 2 session IDs" and index_add's "files array must contain only string paths" fell through to INTERNAL_ERROR.
T-MCP-ERR-019	Validation error "must be" is classified as INVALID_INPUT.	Regression test for NEW-4 where multi_search's "min_score must be between 0.0 and 1.0" fell through to INTERNAL_ERROR.
T-MCP-ERR-020	Filesystem path validation error "does not exist" is classified as FILE_NOT_FOUND.	Regression test for NEW-3 where preview_chunks's "file does not exist: /path" fell through to INTERNAL_ERROR.
T-MCP-ERR-021	"does not exist" messages containing "session" are NOT classified as FILE_NOT_FOUND.	They fall through to the session-not-found or other branches depending on the full message content.
T-MCP-ERR-022	"query must not be empty" from multi_search is classified as INVALID_INPUT.	—
T-MCP-ERR-023	"source_directory does not exist" is classified as FILE_NOT_FOUND.	—
T-MCP-ERR-024	Extraction failure errors from reindex_file and preview_chunks are classified as INVALID_INPUT.	Regression test for DEF-005 where "extraction failed: ..." fell through to INTERNAL_ERROR.
T-MCP-EXPORT-DEDUP-001	group_results_by_document groups results by file_id and produces one DocumentGroup per source document.	

ID	Description	Acceptance Criterion
T-MCP-EXPORT-DEDUP-002	merge_page_ranges consolidates overlapping ranges into non-overlapping ranges.	—
T-MCP-EXPORT-DEDUP-003	merge_page_ranges preserves non-overlapping ranges as separate entries.	—
T-MCP-EXPORT-DEDUP-004	merge_page_ranges merges adjacent ranges where one ends at page N and the next starts at page N+1.	—
T-MCP-EXPORT-DEDUP-005	merge_page_ranges handles a single range without modification.	—
T-MCP-EXPORT-DEDUP-006	merge_page_ranges returns an empty vector for empty input.	—
T-MCP-EXPORT-DEDUP-007	BibTeX dedup produces one entry per document with consolidated page ranges using double-dash notation.	—
T-MCP-EXPORT-DEDUP-008	CSL-JSON dedup produces one entry per document with consolidated page ranges.	—
T-MCP-EXPORT-DEDUP-009	RIS dedup produces one record per document with the page range spanning all matched passages.	—

ID	Description	Acceptance Criterion
T-MCP-EXPORT-DEDUP-010	deduplicate=false preserves the per-result entry format (backward compatibility).	Three results from the same PDF produce three separate BibTeX entries.
T-MCP-EXPORT-DEDUP-011	Markdown format is unaffected by dedup logic (always per-result).	This test verifies that both results appear in the markdown output.
T-MCP-EXPORT-DEDUP-012	Plain-text format is unaffected by dedup logic (always per-result).	—
T-MCP-EXPORT-DEDUP-013	BibTeX dedup handles multiple documents correctly, producing one entry per document sorted by best score.	—
T-MCP-EXPORT-DEDUP-014	merge_page_ranges handles unsorted input correctly, sorting by start page before merging.	—
T-MCP-EXPORT-DEDUP-015	merge_page_ranges handles fully contained ranges (one range completely inside another).	—
T-MCP-EXPORT-DEDUP-016	format_bibtex_page_ranges produces correct BibTeX notation for single and multi-page ranges.	—
T-MCP-FILES-001	Calling handle without a session_id parameter returns an error indicating the required parameter is missing.	—

ID	Description	Acceptance Criterion
T-MCP-FILES-002	Providing a session_id that does not exist in the database returns a "not found" error.	The handler verifies session existence before querying files.
T-MCP-FILES-003	A session containing one file returns file_count=1 with correctly populated pages, text, and chunks sub-objects.	Verifies the structure and specific field values produced by the handler for a known data set (3 pages, 2 chunks, 1 ocr page, 1 empty page).
T-MCP-FILES-004	The session_totals fields in the response match the summed values from all per-file entries.	This test inserts two files in the same session and verifies that session_totals.total_files, total_pages, total_chunks, and total_content_bytes equal the sums of the respective per-file values.
T-MCP-FILES-005	Passing the file_id parameter returns a single-file detail response (file_count=1) containing only the specified file, rather than all files in the session.	—
T-MCP-FILES-006	Passing a file_id that belongs to a different session returns an error.	The handler verifies session ownership of the file before returning its details.
T-MCP-IDX-001	resolve_vector_dimension returns 384 for BGE-small, matching the model catalog specification.	
T-MCP-IDX-002	resolve_vector_dimension returns 1024 for Qwen3-0.6B, matching the model catalog specification.	This is the core regression test for the bug where Qwen3 sessions were stored with dimension=384.
T-MCP-IDX-003	resolve_vector_dimension returns 2560 for Qwen3-4B.	

ID	Description	Acceptance Criterion
T-MCP-IDX-004	resolve_vector_dimension returns an error for unknown model IDs.	The error message lists available models.
T-MCP-IDX-005	resolve_vector_dimension is independent of any backend state.	It always returns the catalog value regardless of what model was loaded at server startup.
T-MCP-IDX-006	resolve_vector_dimension returns consistent results for every model in the catalog.	This ensures no model is accidentally missing from the lookup path.
T-MCP-IDX-007	The full index handler creates a session with the correct vector_dimension from the model catalog, not from state.vector_dimension.	Regression test for the core bug.
T-MCP-IDX-008	The handler rejects unknown model IDs with a descriptive error instead of silently using the default dimension.	—
T-MCP-IDX-009	The "page" strategy returns null for chunk_size, chunk_overlap, and max_words in the response.	Regression test for BUG-3 where the response showed irrelevant default values (256, 32) for parameters that the page strategy ignores entirely.
T-MCP-IDX-010	The "token" strategy returns the correct chunk_size and chunk_overlap values (non-null) and null for max_words.	—
T-MCP-IDX-011	The "sentence" strategy returns max_words and null for chunk_size and chunk_overlap.	—
T-MCP-IDXADD-001	Handler rejects requests missing the session_id parameter.	Validates the parameter extraction logic.

ID	Description	Acceptance Criterion
T-MCP-IDXADD-002	Handler rejects requests missing the files parameter.	—
T-MCP-IDXADD-003	Handler rejects an empty files array.	At least one file path must be provided for the operation to be meaningful.
T-MCP-IDXADD-004	Nonexistent file paths are detected during the triage phase before any extraction is attempted.	—
T-MCP-IDXADD-005	Handler rejects non-string entries in the files array.	All entries must be valid JSON strings.
T-MCP-IDXADD-006	Module compiles and the handler returns the correct Result type.	Compilation of this module validates that all imports, types, and function signatures are correct.
T-MCP-IDXADD-007	The ChangeStatus enum variants used by the handler are accessible.	Validates that the neuroncite_store dependency exposes the required types.
T-MCP-IDXADD-008	Mixed string and non-string entries in the files array produce a count mismatch.	The handler uses filter_map to extract only string values, then compares the count to the original array length to detect non-string entries.
T-MCP-IDXADD-009	Valid files array with all string entries passes the type validation check.	—
T-MCP-IDXADD-010	Session ID is correctly extracted from params as i64.	—
T-MCP-IDXADD-011	The ChangeStatus and FileRow types are accessible from neuroncite_store.	This validates the imports used by the handler.
T-MCP-IDXADD-012	Canonicalizing a real file path produces a result that matches itself when canonicalized again (idempotent).	This is the fundamental property that DEFECT-002 fix relies on.

ID	Description	Acceptance Criterion
T-MCP-IDXADD-013	Two different string representations of the same file canonicalize to the same path.	This simulates the bulk indexer storing a canonical path and <code>index_add</code> receiving a raw user path.
T-MCP-IDXADD-014	Canonicalization fallback for nonexistent files returns the original path string.	The handler uses <code>unwrap_or_else</code> to fall back gracefully when canonicalization fails.
T-MCP-IDXADD-015	The <code>to_process</code> list stores canonical paths, not raw user paths.	This ensures that subsequent <code>find_file_by_session_path</code> lookups in Phase 2 use the same path format as the initial triage, and that stored file paths match the bulk indexer’s format.
T-MCP-IDXADD-016	Canonical path comparison is case-sensitive on case-sensitive filesystems (Linux/macOS).	On Windows, the OS-level canonicalization normalizes casing. This test verifies that the canonical path is stable for the same file.
T-MCP-IDXADD-017	Multiple files in the same directory all resolve to canonical paths that share the same parent directory prefix.	—
T-MCP-IDXADD-018	The JSON response fields <code>hnsw_rebuilt</code> and <code>total_session_vectors</code> correctly reflect the DEFECT-005 repair outcome.	When <code>ensure_hnsw_for_session</code> succeeds, <code>hnsw_rebuilt</code> is true and <code>total_session_vectors</code> contains the vector count.
T-MCP-IDXADD-019	When all files are skipped and no embeddings exist in SQLite (<code>ensure_hnsw_for_session</code> returns false), <code>hnsw_rebuilt</code> is false and <code>total_session_vectors</code> is null.	—

ID	Description	Acceptance Criterion
T-MCP-JOB-001	read_annotate_completion_summary parses a well-formed anno- tate_report.json and extracts both the comple- tion_summary and match_details arrays.	
T-MCP-JOB-002	read_annotate_completion_summary returns None when params_json is missing the output_directory field.	
T-MCP-JOB-003	read_annotate_completion_summary returns None when params_json is None entirely.	
T-MCP-JOB-004	read_annotate_completion_summary returns None when the anno- tate_report.json file does not exist on disk.	
T-MCP-JOB-005	match_details handles an empty pdfs array and empty unmatched_inputs gracefully (returns empty match_details array).	—
T-MCP-JOB-006	Annotation jobs include a session_id_note explaining that session_id is always null for annotation jobs.	—
T-MCP-MULTI-001	Handler rejects requests missing the session_ids parameter.	—

ID	Description	Acceptance Criterion
T-MCP-MULTI-002	Handler rejects an empty <code>session_ids</code> array.	—
T-MCP-MULTI-003	Handler rejects a single <code>session_id</code> .	Multi-search requires at least 2 sessions; single-session search should use <code>neuroncite_search</code> instead.
T-MCP-MULTI-004	Handler rejects <code>session_ids</code> that exceed the maximum of 10 sessions.	—
T-MCP-MULTI-005	Handler rejects requests missing the query parameter.	—
T-MCP-MULTI-006	<code>top_k</code> is clamped to the maximum of 50.	—
T-MCP-MULTI-007	Module compiles and the handler returns the correct Result type.	—
T-MCP-MULTI-008	<code>min_score</code> validation rejects values outside <code>[0.0, 1.0]</code> .	—
T-MCP-MULTI-009	<code>session_ids</code> with non-integer values are filtered out.	The handler uses <code>filter_map(as_i64)</code> and compares the count to detect non-integer entries in the <code>session_ids</code> array.
T-MCP-MULTI-010	Exactly 2 <code>session_ids</code> is the minimum accepted count.	Multi-search requires at least 2 sessions to be meaningful.
T-MCP-MULTI-011	Exactly 10 <code>session_ids</code> is the maximum accepted count.	—
T-MCP-MULTI-012	<code>top_k</code> defaults to 10 when omitted.	—
T-MCP-MULTI-013	<code>use_fts</code> defaults to true when omitted.	—
T-MCP-MULTI-014	Empty query string (after trimming) is rejected.	—
T-MCP-MULTI-015	<code>min_score</code> at boundary values (0.0 and 1.0) is accepted.	—

ID	Description	Acceptance Criterion
T-MCP-MULTI-016	per_session_k is computed as top_k * 3 to provide a generous candidate pool for cross-session merging.	—
T-MCP-PREVIEW-001	Handler rejects requests missing the file_path parameter.	The error message must follow the "missing required parameter: <name>" convention used across all handlers.
T-MCP-PREVIEW-002	Handler rejects requests missing the chunk_strategy parameter.	—
T-MCP-PREVIEW-003	Invalid chunk_strategy values are rejected.	Only "page", "word", "token", and "sentence" are accepted.
T-MCP-PREVIEW-004	Handler rejects a nonexistent file path.	The validation happens before any PDF extraction is attempted.
T-MCP-PREVIEW-005	Default limit is 5 when the parameter is omitted.	—
T-MCP-PREVIEW-006	Limit is clamped to the maximum of 50.	—
T-MCP-PREVIEW-007	Module compiles and the handler returns the correct Result type.	Compilation of this module validates that all imports, types, and function signatures are correct.
T-MCP-PREVIEW-008	Limit of zero is clamped to 1 (minimum bound).	—
T-MCP-PREVIEW-009	All four valid chunk strategy names are accepted by the validation check.	—
T-MCP-PREVIEW-010	Optional parameters default to None when omitted.	The chunk_size, chunk_overlap, and max_words parameters are passed through to neuroncite_chunk::create_strategy as Option<usize>.
T-MCP-PREVIEW-011	Explicit chunk_size and chunk_overlap values are extracted correctly from the params JSON.	—

ID	Description	Acceptance Criterion
T-MCP-PREVIEW-012	Limit is honored exactly when within the valid range.	—
T-MCP-PREVIEW-013	Sentence strategy with <code>chunk_size</code> but no <code>max_words</code> maps <code>chunk_size</code> to <code>max_words</code> .	This is the primary DEFECT-003 regression test. Before the fix, <code>chunk_size</code> was passed directly as <code>chunk_size</code> (not <code>max_words</code>), causing <code>create_strategy</code> to receive <code>max_words=None</code> and either error or use the wrong default.
T-MCP-PREVIEW-014	Sentence strategy with explicit <code>max_words</code> takes precedence over <code>chunk_size</code> .	When both parameters are provided, the explicit <code>max_words</code> value is used.
T-MCP-PREVIEW-015	Sentence strategy with only <code>max_words</code> (no <code>chunk_size</code>) passes <code>max_words</code> through directly.	—
T-MCP-PREVIEW-016	Sentence strategy with no size parameters results in all <code>None</code> values.	The chunking library applies its defaults.
T-MCP-PREVIEW-017	Word strategy passes <code>chunk_size</code> and <code>chunk_overlap</code> through without mapping.	The <code>max_words</code> parameter is ignored (set to <code>None</code>).
T-MCP-PREVIEW-018	Token strategy passes <code>chunk_size</code> and <code>chunk_overlap</code> through without mapping, same as word strategy.	—
T-MCP-PREVIEW-019	Page strategy ignores all size parameters.	The page strategy produces one chunk per PDF page regardless of any size settings.

ID	Description	Acceptance Criterion
T-MCP-PREVIEW-020	Verifies that the <code>preview_chunks</code> parameter mapping matches the <code>neuroncite_index</code> handler parameter mapping for all four strategies.	This cross-validates both handlers produce identical strategy configurations from the same input parameters, preventing the DEFECT-003 inconsistency from recurring.
T-MCP-QUAL-001	Calling the quality handler without a <code>session_id</code> parameter returns an error indicating the missing required field.	—
T-MCP-QUAL-002	Providing a <code>session_id</code> that does not exist in the database returns a "not found" error.	—
T-MCP-QUAL-003	A session containing files that have no quality issues produces a report with <code>files_with_issues=0</code> and <code>files_clean</code> equal to the number of inserted files.	A "clean" file has: - <code>page_count == pdf_page_count</code> (no incomplete extraction or mismatch) - <code>native_pages > ocr_pages</code> (not OCR-heavy) - <code>total_bytes / page_count >= 100</code> (sufficient text density) - <code>empty_pages * 10 <= page_count</code> (not many empty pages)
T-MCP-QUAL-004	A file with <code>page_count < pdf_page_count</code> triggers the <code>incomplete_extraction</code> flag.	The handler detects that not all pages from the structural PDF page tree were successfully extracted.

ID	Description	Acceptance Criterion
T-MCP-QUAL-005	A file where all pages are OCR-extracted (<code>ocr_pages > native_pages</code> , with both > 0 not required – only <code>ocr_pages > native_pages</code> and <code>ocr_pages > 0</code>) triggers the <code>ocr_heavy</code> flag.	The test inserts a file with 3 OCR pages and 1 native page. Since <code>ocr_pages</code> (3) $>$ <code>native_pages</code> (1) and <code>ocr_pages</code> $>$ 0, the flag fires.
T-MCP-QUAL-006	An empty session (no files indexed) returns an <code>extraction_summary</code> with all zero values and empty <code>quality_flags</code> .	—
T-MCP-QUAL-007	A file with empty pages (<code>byte_count = 0</code>) triggers the <code>many_empty_pages</code> flag and lists the specific empty page numbers in the details.	—
T-MCP-QUAL-008	The <code>extraction_summary</code> includes <code>total_words</code> and <code>avg_words_per_page</code> computed from the <code>total_bytes</code> using the bytes/6 heuristic.	—
T-MCP-QUAL-009	A file with very low text density (<code>total_bytes / page_count < 100</code>) triggers the <code>low_text_density</code> flag.	This indicates the extraction produced very little text per page, which can happen with scanned PDFs where OCR failed or image-heavy documents.

ID	Description	Acceptance Criterion
T-MCP-QUAL-010	A file where the extracted page count exceeds the structural PDF page count triggers the <code>page_count_mismatch</code> flag.	This can occur when the extractor produces duplicate pages or when the structural page count is inconsistent with the actual content.
T-MCP-QUAL-011	A pure-OCR file (all pages extracted via OCR, no native text) is classified as <code>ocr_required_count=1</code> in the summary.	This is distinct from <code>mixed_count</code> which requires both OCR and native pages in the same file.
T-MCP-QUAL-012	<code>avg_bytes_per_page</code> in the extraction summary is computed as <code>total_bytes / total_pages</code> (integer division).	Verifies the value is present and consistent with the inserted data.
T-MCP-QUAL-013	A file with multiple quality issues triggers all applicable flags simultaneously.	Inserts a file that has low text density AND many empty pages AND incomplete extraction, verifying all three flags appear in the same quality entry.
T-MCP-QUAL-014	Per-file recommendations are generated for each quality flag.	Each flag produces a corresponding actionable recommendation in the <code>recommendations</code> array.
T-MCP-QUAL-015	Multiple quality flags produce multiple per-file recommendations (one per flag).	Verifies the 1:1 correspondence between flags and recommendations.
T-MCP-QUAL-016	Session-level recommendations array is present in the response.	For a session with OCR files, it includes a recommendation about Tesseract.
T-MCP-QUAL-017	A clean session (no quality issues) produces an empty session-level recommendations array.	—

ID	Description	Acceptance Criterion
T-MCP-REIDX-001	Constants and types used by the handler are accessible.	Compilation of this module validates that all imports, types, and function signatures in the handler are correct.
T-MCP-REIDX-002	Canonicalizing a real file path is idempotent.	Calling canonicalize twice on the same file produces identical results.
T-MCP-REIDX-003	A raw user path and a pre-canonicalized path resolve to the same string after canonicalization.	This is the property that the DEF-002 fix relies on: the handler canonicalizes the user path before comparing it against the stored (canonical) path.
T-MCP-REIDX-004	On Windows, canonicalized paths start with the extended-length prefix.	On Unix, the canonical path is absolute.
T-MCP-REIDX-005	Canonicalization fallback for nonexistent files returns the original path.	The handler uses <code>unwrap_or_else</code> to avoid a panic when the file does not exist.
T-MCP-REIDX-006	The handler passes the canonical path to both <code>find_file_by_session_path</code> and <code>extract_and_chunk_file</code> .	This test verifies the construction logic: <code>PathBuf::from(&canonical_path)</code> preserves the canonical format.
T-MCP-RERANK-001	Handler rejects requests missing the <code>model_id</code> parameter.	Validates the parameter extraction logic.
T-MCP-RERANK-002	Handler rejects an empty <code>model_id</code> string.	The <code>model_id</code> must contain a valid Hugging Face model identifier.
T-MCP-RERANK-003	Backend name defaults to "ort" when not specified.	Validates the default value logic for the optional backend parameter.
T-MCP-RERANK-004	Module compiles and the handler returns the correct <code>Result</code> type.	Compilation of this module validates that all imports, types, and function signatures are correct.

ID	Description	Acceptance Criterion
T-MCP-RERANK-005	The <code>create_reranker</code> function from <code>neuroncite-embed</code> returns an error for invalid backend names.	Validates that the handler's error path for unknown backends is reachable.
T-MCP-RERANK-006	Explicit backend parameter overrides the default.	When the user specifies a backend, that value is used instead of "ort".
T-MCP-RERANK-007	<code>model_id</code> is extracted correctly from the params JSON.	—
T-MCP-RERANK-008	The Reranker trait from <code>neuroncite-core</code> is accessible in this module.	Validates the import chain.
T-MCP-RERANK-009	<code>Arc<dyn Reranker></code> can be constructed from <code>Box<dyn Reranker></code> .	This validates the <code>Arc::from(boxed)</code> conversion used in the handler to wrap the loaded reranker.
T-MCP-RERANK-010	The <code>create_reranker</code> function succeeds for each compiled-in backend.	Validates that at least one backend can create a reranker instance. Requires <code>backend-ort</code> because <code>list_available_backends</code> returns an empty list without a compiled-in backend.
T-MCP-RERANK-011	The cross-encoder model is present in the download manifest.	When the model is not in cache, the handler auto-downloads it. This test verifies that the download infrastructure knows the canonical model ID, so the auto-download path is reachable.
T-MCP-RERANK-012	The <code>downloaded</code> field in the handler response is a boolean.	This validates that the response JSON shape is correct for callers that inspect whether a download was triggered.
T-MCP-RERANK-013	<code>is_cached</code> returns false for a model ID that has never been downloaded.	The auto-download branch in the handler is triggered when <code>is_cached</code> returns false. This test verifies the <code>is_cached</code> API behaves correctly for the reranker model ID in a clean environment.

ID	Description	Acceptance Criterion
T-MCP-RERANK-014	download_model returns a descriptive error when the requested model ID is not in MODEL_FILES.	The error must name the model so the user can identify which model is unknown.
T-MCP-RERANK-015	download_model succeeds (i.e., does not immediately error) for the canonical cross-encoder model ID because it is in the manifest.	The actual network download is not triggered here; only the manifest lookup and directory creation path are exercised. The test verifies that the error is NOT "no download manifest for model", which was the previous failure mode before the manifest entry was added. Note: The test may return Ok (if network is available and download succeeds) or Err with a network/IO error. Both outcomes are valid here because the manifest lookup succeeded in either case.
T-MCP-RERANK-016	The handler's auto-download response shape contains all mandatory fields.	Callers (e.g., MCP AI agents) parse the response JSON and must find: model_id, backend, reranker_name, reranker_available, and downloaded.
T-MCP-RERANK-017	The backend name in the response matches the backend parameter from the request.	When no backend is specified, the default "ort" is used. This validates that the response correctly reflects the backend that was actually used.
T-MCP-RERANK-018	Whitespace-only model_id is rejected before the auto-download path is reached.	The handler trims the model_id and returns an error immediately for empty strings, so no download attempt is made for invalid input.
T-MCP-SEARCH-001	top_k=0 returns an error instead of silently returning 1 result.	Regression test for BUG-004: Before the fix, top_k=0 passed the upper bound check ($0 \leq 200$) and was silently converted to 1 via <code>.max(1)</code> . Callers expected an error or zero results but received one result, leading to incorrect behavior in pagination logic and result count assertions.
T-MCP-SEARCH-002	top_k=201 returns an error.	This value exceeds MAX_TOP_K (50), confirming the upper bound is enforced.

ID	Description	Acceptance Criterion
T-MCP-SEARCH-003	top_k=51 returns an error because the maximum was reduced from 200 to MAX_TOP_K (50) to prevent MCP client-side output truncation on large result sets.	Regression test for BUG B-01: Requesting top_k=200 for broad queries produced JSON payloads exceeding 436K characters, which caused the MCP client to truncate the response and save it to a temp file instead of delivering structured JSON.
T-MCP-SEARCH-004	top_k=50 (MAX_TOP_K) passes the validation check.	The handler continues past validation and returns a response. With no HNSW index loaded the response contains an empty result set, which confirms the validation did not reject the boundary value.
T-MCP-SEARCH-005	truncate_response_if_correctly trims the results array when the serialized payload exceeds the byte limit.	Results are trimmed from the end (lowest relevance). The response receives a truncated_from field reporting the original count. Regression test for BUG B-01: Verifies the safety mechanism that prevents oversized responses even when individual results are large.
T-MCP-SEARCH-006	truncate_response_if_no_op is a no-op when the serialized payload is already within the byte limit.	Normalized truncated_from field is added and result_count is unchanged.
T-MCP-SEARCH-007	truncate_response_if_preserves at least one result even when a single result exceeds the byte limit.	Throttled prevents returning an empty results array which callers could mistake for "no matches found".
T-MCP-SEARCH-008	min_score values outside [0.0, 1.0] are rejected.	Regression test for ISSUE-010 where min_score=1.5 was silently accepted and filtered out all results because no cosine similarity exceeds 1.0.
T-MCP-SEARCH-009	min_score within [0.0, 1.0] passes validation.	—

ID	Description	Acceptance Criterion
T-MCP-SEARCH-010	Truncation warning is generated for long queries.	Regression test for BUG-006 where queries exceeding the model's <code>max_sequence_length</code> (512 tokens) were silently truncated without any indication to the caller. The heuristic uses <code>word_count + 2</code> (accounting for [CLS] and [SEP] special tokens) as the minimum token count estimate.
T-MCP-SEARCH-011	An empty string query returns an error.	The handler rejects queries that are empty after trimming whitespace, to prevent meaningless embedding computations.
T-MCP-SEARCH-012	A whitespace-only query returns an error.	The handler trims leading/trailing whitespace before the empty check, so " " is treated the same as "".
T-MCP-SEARCH-013	A non-existent <code>session_id</code> returns a "not found" error.	The handler validates session existence before attempting the embedding or HNSW index lookup.
T-MCP-SEARCH-014	Missing <code>session_id</code> parameter returns the "missing required parameter" error.	—
T-MCP-SEARCH-015	Missing query parameter returns the "missing required parameter" error.	—
T-MCP-SEARCH-016	When the HNSW index is not loaded, the handler returns a response with an empty results array and a message indicating the index is not available.	—
T-MCP-SEARCH-020	Language mismatch warning is produced when an English embedding model receives a query with a high proportion of non-ASCII alphabetic characters (above 20% threshold).	The query "Grosse Ol Prufung Anderung" has 5 non-ASCII out of 22 alpha chars (22.7%).

ID	Description	Acceptance Criterion
T-MCP-SEARCH-021	No warning for English queries with English models.	—
T-MCP-SEARCH-022	No warning for non-English queries with multilingual models.	—
T-MCP-SEARCH-023	Short queries with one non-ASCII character do not trigger.	—
T-MCP-SEARCH-024	Exact token count upgrades truncation warning when the word-based heuristic did not fire.	Regression test for BUG-006: Queries where <code>word_count + 2 <= max_seq_len</code> (heuristic passes) but the BPE token count <code>>= max_seq_len</code> were silently truncated without any warning. The fix adds a second check using the exact token count from the tokenizer. This test verifies the logic path where the heuristic produces no warning but the exact count triggers one.
T-MCP-SEARCH-025	Exact token count does NOT generate a spurious warning when the count is below <code>max_seq_len</code> .	Complementary test to T-MCP-SEARCH-024: when the exact token count is below <code>max_seq_len</code> , no warning should be generated regardless of whether the heuristic fired.
T-MCP-SEARCH-026	Heuristic-based warning is NOT overwritten by the exact token count check.	When the heuristic already fires (very long queries), the exact-count path must not replace the heuristic message.
T-MCP-SEARCH-027	The search response contains a <code>query_analysis</code> object with structured tokenization and BM25 term information.	This test verifies the <code>query_analysis</code> section is present and contains the expected fields when a valid search is performed against an empty session (no HNSW index loaded).

ID	Description	Acceptance Criterion
T-MCP-SEARCH-028	The <code>query_analysis</code> section omits <code>bm25_</code> terms when <code>use_fts=false</code> , since no keyword search is performed.	—
T-MCP-SEARCH-029	The <code>full_chunk_content</code> field logic is correct.	When refinement produces a narrower sub-chunk, the original chunk text must be preserved. This test verifies the preservation mechanism by checking that <code>original_contents</code> stores clones before refinement.
T-MCP-SES-001	Deleting a non-existent session by ID returns an error.	This is consistent with the search and update handlers that also reject non-existent session IDs with an error, rather than silently succeeding.
T-MCP-SES-002	Deleting an existing session by ID returns <code>deleted: true</code> and the session is removed from the database.	—
T-MCP-SES-003	Double-deleting a session by ID returns an error on the second attempt.	The first delete succeeds with <code>deleted: true</code> , the second returns a "not found" error because the session no longer exists in the database.
T-MCP-SES-004	Delete without <code>session_id</code> or <code>directory</code> returns an error.	—
T-MCP-SES-005	Delete by <code>directory</code> for a non-matching path returns an error indicating no sessions were found, consistent with the delete-by-ID behavior that also errors on non-existent sessions.	—

ID	Description	Acceptance Criterion
T-MCP-SES-006	session_update rejects the literal string "null" as the label value to prevent ambiguity with JSON null.	Regression test for the bug where MCP clients serializing null as the string "null" would silently store the wrong value.
T-MCP-SES-007	session_update with JSON null correctly clears the label.	—
T-MCP-SES-008	session_update with a valid string label stores it.	—
T-MCP-SES-009	handle_list with a populated session returns aggregate fields (file_count, total_pages, total_chunks, total_content_bytes) with non-zero values.	Inserts one file, two pages, and two chunks into the session, then verifies all four aggregate fields reflect the inserted data.
T-MCP-SES-010	handle_list with an empty session (no files, pages, or chunks) returns all four aggregate fields set to 0.	The session row exists in the database but has no associated data.
T-MCP-SES-011	Deleting a session with an active (queued) job referencing it returns an error.	Regression test for BUG-001 where session deletion cascade-deleted citation rows while a citation_verify job was still running, causing the job to fail with dangling references.
T-MCP-SES-012	Deleting a session succeeds when the only jobs referencing it are completed (not queued or running).	Completed jobs do not block deletion because their data is no longer being actively processed.
T-MCP-SES-013	session_update with a valid tags array stores the JSON and returns it parsed in the response.	—
T-MCP-SES-014	session_update with JSON null tags clears the tags field.	—

ID	Description	Acceptance Criterion
T-MCP-SES-015	session_update rejects non-array tags values.	—
T-MCP-SES-016	session_update rejects tags arrays containing non-string elements.	—
T-MCP-SES-017	session_update with a valid metadata object stores and returns it.	—
T-MCP-SES-018	session_update rejects non-object metadata values.	—
T-MCP-SES-019	session_update with all three fields (label, tags, metadata) updates all of them in a single call.	—
T-MCP-SES-020	handle_list includes tags and metadata in session output.	—
T-MCP-SES-021	handle_list returns null for tags and metadata when they are not set on the session.	—
T-MCP-SES-022	session_update with empty tags array stores [].	—
T-MCP-SES-023	session_update with empty metadata object stores {}.	—
T-MCP-SES-024	Deleting by directory is blocked when an active job references one of the matching sessions.	This covers the directory-based deletion path (distinct from T-MCP-SES-011 which tests delete-by-ID).

ID	Description	Acceptance Criterion
T-MCP-SES-025	handle_list formats chunk_param_label correctly for each chunk strategy.	Verifies the four strategy-specific label formats: - "sentence" strategy uses max_words label - "page" strategy uses fixed "one chunk per page" label - "word" strategy uses chunk_size label with "words per chunk" - "token" strategy uses chunk_size label with "tokens per chunk"
T-MCP-SES-026	session_update with a non-existent session_id returns a "not found" error.	The handler validates session existence before attempting any field updates.
T-MCP-SES-027	session_update with only session_id (no label, tags, or metadata) returns the current values without modifying anything.	This is the "no-op update" path where all three optional fields are absent from the params.
T-MCP-SES-028	session_update with numeric label (non-string) returns an error.	The label field must be a string or null, not a number.
T-MCP-SRCFETCH-001	Handler rejects requests missing the bib_path parameter.	—
T-MCP-SRCFETCH-002	Handler rejects requests missing the session_id parameter.	—
T-MCP-SRCFETCH-003	Handler rejects requests missing the output_directory parameter.	—
T-MCP-SRCFETCH-004	Default delay_ms is 1000 when not specified.	—
T-MCP-SRCFETCH-005	Custom delay_ms is respected when specified.	—
T-MCP-SRCFETCH-006	URL resolution prefers the url field over DOI.	When both are present, the url field takes precedence.

ID	Description	Acceptance Criterion
T-MCP-SRCFETCH-007	DOI is resolved to a URL when no url field exists.	—
T-MCP-SRCFETCH-008	Entries without url or doi are skipped.	—
T-MCP-SRCFETCH-009	The indexed_count calculation correctly subtracts index failures from the total of downloaded + fetched sources.	—
T-MCP-SRCFETCH-010	BibTeX parsing with url and doi fields produces entries with the correct field values through the full parse pipeline.	—
T-MCP-SRCFETCH-011	Cloudflare "Just a moment" challenge page is detected as blocked.	The page has fewer than 50 words and the title matches the "just a moment" bot-detection pattern.
T-MCP-SRCFETCH-012	A legitimate short page (e.g., an abstract-only page) that has fewer than 50 words but does NOT match any bot-detection pattern passes the quality check.	This verifies that short content alone is not sufficient for the blocked classification – both criteria must be met.
T-MCP-SRCFETCH-013	A long page (≥ 50 words) with a suspect title like "Security Check" passes the quality check because the word count exceeds the MIN_CONTENT_WORDS threshold.	Only pages that are BOTH short and pattern-matching are blocked.

ID	Description	Acceptance Criterion
T-MCP-SRCFETCH-014	Access denied page is detected as blocked when the title does not match but the body text contains "Access Denied".	—
T-MCP-SRCFETCH-015	A page with None title and "Redirecting" body text is detected as blocked (covers the case where metadata.title is absent).	—
T-MCP-SRCFETCH-016	The FetchedHtml struct correctly stores all four fields needed by the Phase 3 HTML indexing pipeline.	—
T-MCP-SRCFETCH-017	Email parameter is extracted from params when present.	—
T-MCP-SRCFETCH-018	Email parameter is None when absent from params.	—
<i>Crate: neuroncite-pdf – PDF file discovery and text extraction</i>		
T-PDF-001	Discovery finds all PDFs recursively in nested directories.	Creates a temp directory with PDFs at multiple nesting depths and verifies that all PDF files are found by the discovery function.
T-PDF-002	Discovery ignores non-PDF files.	Creates files with .txt, .docx, and .png extensions alongside a .pdf file and verifies that only the .pdf file appears in the results.
T-PDF-003	Discovery returns sorted paths.	Creates PDF files with names that would be unsorted by filesystem order and verifies that the result is sorted lexicographically.
T-PDF-004	Discovery handles empty directories.	An empty directory produces an empty result list without error.

ID	Description	Acceptance Criterion
T-PDF-005	Symlink cycle detection.	Creates a directory structure with a circular symlink and verifies that discovery terminates without error. On platforms where symlink creation fails (e.g., unprivileged Windows), the test verifies that the visited-path set correctly prevents re-entry.
T-PDF-006	discover_pdfs_flat finds only top-level PDFs and ignores subdirectories.	Creates a structure with PDFs at root and in a nested subdirectory. Only the root-level PDFs are returned.
T-PDF-007	discover_pdfs_flat returns sorted paths, consistent with discover_pdfs behavior.	—
T-PDF-008	discover_pdfs_flat handles empty directories without error.	—
T-PDF-009	discover_pdfs_flat ignores non-PDF files in the top level, consistent with the recursive discovery behavior.	—
T-PDF-010	discover_pdfs (recursive) finds PDFs in subdirectories, while discover_pdfs_flat does not.	Verifies the behavioral difference between the two functions on the same directory structure.
T-PDF-011	discover_documents finds PDFs and HTML files recursively in nested directories.	Verifies that both file types are collected while other file types (e.g., .txt) are ignored.
T-PDF-012	discover_documents finds only PDFs when no HTML files exist, producing the same result as discover_pdfs.	—

ID	Description	Acceptance Criterion
T-PDF-013	A corrupted PDF file (random bytes with .pdf extension) returns a PdfError variant and does not panic.	—
T-PDF-014	A valid PDF with no extractable text returns PageText entries with empty or whitespace-only content.	—
T-PDF-015	QUALITY-004 regression: <code>strip_control_characters</code> removes NUL bytes, form feeds, vertical tabs, and other C0 control characters while preserving tab, newline, and carriage return.	—
T-PDF-016	QUALITY-004 regression: <code>strip_control_characters</code> preserves tab, newline, and carriage return characters since these are legitimate whitespace in extracted PDF text.	—
T-PDF-017	QUALITY-004 regression: <code>strip_control_characters</code> is a no-op on strings that contain no control characters.	—

ID	Description	Acceptance Criterion
T-PDF-018	QUALITY-004 regression: strip_control_characters handles strings composed entirely of control characters, producing an empty string.	—
T-PDF-019	QUALITY-004 regression: strip_control_characters handles an empty string without error.	—
T-PDF-020	QUALITY-004 regression: strip_control_characters correctly handles multi-byte UTF-8 characters mixed with control characters.	Unicode characters outside the C0 range must be preserved.
T-PDF-028a	Single-pass quality check produces identical results to the multi-pass implementation.	Tests a string containing all four character categories (alphabetic, replacement, line break, other) and verifies that each ratio is computed correctly in a single iteration.
T-PDF-029	deps_cache_base returns a path ending with "runtime".	The centralized runtime directory is <Documents>/NeuronCite/runtime/.
T-PDF-030	pdfium_cache_dir returns a path ending with "pdfium" under the runtime directory.	—
T-PDF-031	tesseract_cache_dir returns a path ending with "tesseract" under the runtime directory.	—
T-PDF-032	tessdata_dir returns a "tessdata" subdirectory inside the Tesseract cache directory.	—

ID	Description	Acceptance Criterion
T-PDF-033	<code>cached_pdfium_path</code> returns <code>Some</code> only when the pdfium shared library file exists on disk.	If it returns <code>None</code> , the library has not been downloaded to the cache.
T-PDF-034	<code>cached_pdfium_path</code> is idempotent.	Calling it twice in a row returns the same result (both <code>Some</code> or both <code>None</code>).
T-PDF-035	<code>cached_tesseract_path</code> is idempotent.	Calling it twice in a row returns the same result.
T-PDF-036	<code>ensure_pdfium</code> returns a valid result (either a directory path when pdfium is available/downloadable, or a <code>DepDownload</code> error when the download fails).	The <code>OnceLock</code> serialization ensures this function is safe to call from multiple threads.
T-PDF-037	<code>ensure_tesseract</code> returns a valid result: either a path to the Tesseract binary (when installed), or a <code>DepDownload</code> error (when not installed).	This function does not trigger downloads.
T-PDF-038	Concurrent calls to <code>ensure_pdfium</code> from multiple threads do not cause file lock errors or panics.	The <code>OnceLock</code> serialization ensures only one thread performs the download while others wait.
T-PDF-039	Concurrent calls to <code>ensure_tesseract</code> from multiple threads do not cause panics.	Since <code>ensure_tesseract</code> is check-only (no download), all threads receive the same result.
T-PDF-040	The <code>PDFIUM_TAG</code> constant follows the expected "chromium/<number>" format used by bblanchon/pdfium-binaries releases.	—

ID	Description	Acceptance Criterion
T-PDF-041	The <code>PDFIUM_ARCHIVE</code> constant matches the current platform and ends with ".tgz".	—
T-PDF-042	The <code>PDFIUM_LIB_NAME</code> constant matches the platform-specific shared library extension.	—
T-PDF-043	<code>ensure_tesseract</code> does not trigger downloads.	When Tesseract is not installed, it returns an error instead of attempting to download from the network.
T-PDF-044	The <code>PDFIUM_DOWNLOAD</code> Mutex starts as None and does not permanently cache failures.	After a failed <code>ensure_pdfium</code> call (when pdfium is not cached), the Mutex state remains None, allowing retries. This was changed from <code>OnceLock</code> (which permanently caches errors) to Mutex to support user-initiated retries from the Doctor panel.
T-PDF-045	The <code>TESSERACT_DOWNLOAD</code> Mutex uses the same retry-safe pattern as <code>PDFIUM_DOWNLOAD</code> .	Verifies the Mutex state is never a cached error.
T-PDF-046	Pdfium and Tesseract Mutex guards recover from poisoned state.	If a previous thread panicked while holding the lock, the <code>unwrap_or_else(p p.into_inner())</code> pattern in <code>ensure_pdfium</code> and <code>download_tesseract_with_tessdata</code> must not propagate the poison.
T-PDF-047	Text with a low alphabetic ratio (below 0.3) triggers fallback.	Constructs a string dominated by digits and punctuation so that the alphabetic character ratio falls below the threshold.
T-PDF-048	lopdf per-page extraction produces correct page numbers.	Creates a multi-page PDF and verifies that each page receives its correct 1-indexed page number, rather than all pages being reported as page 1.

ID	Description	Acceptance Criterion
T-PDF-049	Text with a high replacement character ratio (above 0.05) triggers fallback.	Constructs a string where more than 5% of characters are U+FFFD replacement characters.
T-PDF-050	Text with a high line break density (above 0.15) triggers fallback.	Constructs a string where line break characters make up more than 15% of the total character count.
T-PDF-051	Text with a low unique token ratio (below 0.05) triggers fallback.	Constructs a string of many repeated tokens so that the unique/total ratio falls below the threshold.
T-PDF-052b	Well-formed academic text passes all quality thresholds.	Verifies the single-pass implementation correctly classifies representative natural language text as passing.
T-PDF-053	Caesar-cipher-shifted text (character codes shifted by a constant offset) triggers fallback via the function word check.	This type of garbled output passes alphabetic ratio, replacement char, line break, and unique token checks because the shifted characters are still alphabetic and form varied tokens. Only the function word check detects the absence of recognizable English words.
T-PDF-054	Short text fragments (below FUNCTION_WORD_MIN_TOKENS) are exempt from the function word check.	Table captions, equation labels, and headers may legitimately contain no words.
T-PDF-055	Text from real PDF extraction where pdf-extract produces garbled output with leading slashes (observed in production: the /krzhyhu/kdyhqrw... pattern from misinterpreted ToUnicode CMap entries).	This text contains alphabetic characters and varied tokens but zero English function words.

ID	Description	Acceptance Criterion
T-PDF-056	Function word ratio computation correctly identifies function words in mixed-case text.	The comparison is case-insensitive, so "The", "THE", and "the" all count as function words.
T-PDF-057	Extracts text from a well-formed PDF.	Creates a minimal valid PDF containing known text and verifies that <code>extract_with_pdf_extract</code> returns at least one <code>PageText</code> entry annotated with the <code>PdfExtract</code> backend.
T-PDF-058	lopdf fallback to pdf-extract form-feed split.	Verifies that <code>extract_with_formfeed_split</code> filters out empty trailing segments produced by a trailing form-feed character.
T-PDF-059	Empty file returns empty page list without panicking.	—
T-PDF-060	Pdfium extracts multi-column text.	This test requires the pdfium shared library at runtime. It verifies that <code>extract_with_pdfium</code> produces <code>PageText</code> entries with the correct backend annotation. Since the pdfium library may not be available in all CI environments, this test is gated behind <code>#[cfg(feature = "pdfium")]</code> at the module level.
T-PDF-061	OCR fallback triggers for image-only pages.	This test verifies the <code>ocr_page</code> function by passing a valid blank PNG through Tesseract. Tesseract must be installed on the system or in the NeuronCite runtime cache for this test to pass. When Tesseract is missing, the test fails with a descriptive assertion message.
T-PDF-062	<code>download_single_file</code> returns an error for an HTTP 404 response instead of saving the error body as the downloaded file.	Regression test for tessdata downloads that were written as <code>.traineddata</code> files containing an HTTP error page. Uses a one-shot local HTTP server, so no network access is required.
T-PERF-003	Batch OCR processes an empty page list without error.	Verifies that <code>ocr_pdf_pages_batch</code> returns an empty map when given an empty slice of page numbers, without loading pdfium or Tesseract.

ID	Description	Acceptance Criterion
T-PERF-004	Batch hOCR processes an empty page list without error.	Mirrors T-PERF-012 for the hOCR variant. Returns an empty map without loading pdfium or resolving the Tesseract binary.
T-PERF-005	parse_hocr_words extracts word text and bounding box coordinates from well-formed hOCR HTML output.	—
T-PERF-006	parse_hocr_words filters out whitespace-only words that occasionally appear in Tesseract output for blank page regions.	—
T-PERF-007	parse_hocr_words returns an empty vector for HTML input without any ocrx_word spans (e.g., a blank page with no recognized text).	—
T-PERF-008	parse_hocr_words handles multi-digit bounding box coordinates that occur with high-DPI rendering (400 DPI on A4 produces coordinates above 3000 pixels).	—

ID	Description	Acceptance Criterion
T-PERF-009	render_page_to_png uses fast PNG compression (Com- pressionType::Fast) which produces valid, decodable PNG bytes.	Regression test for DEFECT-006: the rendering phase (Phase 1) of ocr_pdf_pages_batch_hocr previously used default PNG compression (level 6, ~2-3s per A4 page), contributing to the observed 251-second annotation hang on a 35-page scanned PDF. Verifies that the fast encoder produces lossless output with correct pixel values – the same data Tesseract reads via its stdin pipe. A synthetic 50x50 RGB image is used so that no pdftum or PDF file is required.
T-PERF-010	parse_hocr_words accepts double-quote attribute syntax produced by Tesseract 5.x builds.	Earlier versions use single quotes; the regex must handle both to prevent empty word lists that cause fallback to page-level bounding boxes (DEFECT-007 root cause).
T-PERF-011	parse_hocr_words handles mixed single/double quote attributes (defensive against non-standard or hand-crafted hOCR).	—
T-SEC-001	validate_tesseract_language accepts a single valid language code.	
T-SEC-002	validate_tesseract_language accepts "+" separated valid codes.	
T-SEC-003	validate_tesseract_language rejects unknown language codes.	
T-SEC-004	validate_tesseract_language rejects path separators.	
T-SEC-005	validate_tesseract_language rejects strings exceeding 100 chars.	
T-SEC-006	validate_tesseract_language rejects empty segments (leading/trailing "+").	

ID	Description	Acceptance Criterion
T-SEC-007	validate_tesseract_language rejects shell metacharacters.	
<i>Crate: neuroncite-pipeline – Background job executor, GPU worker, and indexing pipeline</i>		
T-API-019	rerank_batch returns scores when a reranker is configured.	Spawns the worker with a stub reranker and verifies that rerank_batch returns one score per candidate with the expected values.
T-API-020	embed_batch_search splits large input batches into sub-batches limited by com- pute_batch_size.	Each sub-batch is sent as a separate message to the worker, so the ONNX forward pass processes at most batch_size texts at a time. This prevents the memory exhaustion that occurred when hundreds of refinement sub-chunks were embedded in a single ONNX call. The expected batch_size is derived from compute_batch_size(512) at runtime rather than hardcoded. compute_batch_size uses the host CPU core count in its scaling formula, so a fixed expectation would fail on machines with different core counts.
T-API-021	embed_batch_search with a small input (below batch_size) sends a single message to the worker without splitting.	—
T-API-022	embed_batch_search with empty input returns empty result without sending any messages to the worker.	—
T-API-023	swap_reranker changes reranker_available from false to true.	Spawns the worker without a reranker, verifies reranker_available is false, then hot-swaps a stub reranker and verifies reranker_available is true. This validates the ArcSwap-based runtime reranker activation mechanism.

ID	Description	Acceptance Criterion
T-API-024	rerank_batch succeeds after swap_reranker installs a reranker.	Spawns the worker without a reranker (rerank_batch fails), then hot-swaps a stub reranker and verifies that rerank_batch returns scores.
T-API-025	swap_reranker(None) disables the reranker.	After disabling, reranker_available returns false and rerank_batch returns an error.
T-API-050	WorkerHandle embed_query returns a vector with the correct dimensionality from the stub backend.	—
T-API-051	WorkerHandle priority ordering.	High-priority messages (embed_query) are processed before low-priority messages (embed_batch) when both channels have pending items. This test fills both channels and verifies that the high-priority request completes before the low-priority request.
T-API-052	spawn_worker succeeds when called outside block_on but with an active runtime guard (rt.enter()).	This reproduces the MCP server startup pattern where the tokio runtime is created manually and the server loop runs synchronously on the main thread. Without rt.enter(), tokio::spawn inside spawn_worker panics with "there is no reactor running". This test verifies the guard-based approach works correctly.
T-API-053	rerank_batch returns an error when no reranker is configured.	The worker is spawned without a reranker (None). Calling rerank_batch must return Err, not silently succeed with empty or default scores. This error is what the MCP and API handlers propagate to the caller when the user requests reranking without a configured reranker model.

ID	Description	Acceptance Criterion
T-EXEC-017	Verifies that the cooperative cancellation pattern using <code>Arc<AtomicBool></code> correctly propagates the cancel signal.	This is the regression test for BUG-001 where calling <code>JoinHandle::abort()</code> on a <code>spawn_blocking</code> task did NOT stop the blocking thread, causing pdfium (process-global C library) to remain active while the executor started a new job on a different thread, leading to deadlocks. The fix replaces <code>abort()</code> with a cooperative <code>AtomicBool</code> flag that the blocking pipeline checks before each PDF. This test verifies the flag can be read from a spawned thread.
T-EXEC-018	Verifies that after setting the cancel flag, the executor waits for the blocking task to complete naturally rather than aborting it.	The blocking task returns normally and the <code>JoinHandle</code> resolves with <code>Ok</code> (not a <code>JoinError</code> from <code>abort</code>). This guards against regression to the <code>abort()</code> -based cancellation that caused BUG-001.
T-EXEC-020	The <code>job_notify</code> Notify wakes the executor loop immediately instead of waiting for the full <code>POLL_INTERVAL</code> .	This verifies that <code>notify_one()</code> after job creation triggers a wake-up. Regression test for BUG-002: Before the fix, the executor loop used a fixed <code>sleep(POLL_INTERVAL)</code> without any notification channel. After canceling a job, the executor would sleep for up to <code>POLL_INTERVAL</code> before discovering the next queued job, causing apparent deadlock.
T-EXEC-021	Canceling a queued job transitions its state to <code>Canceled</code> and <code>find_next_queued_job</code> skips it.	Verifies the cancel state machine.
T-EXEC-022	When multiple jobs are queued, the executor picks them up in creation order (FIFO).	This verifies that the executor correctly processes the oldest queued job first.

ID	Description	Acceptance Criterion
T-EXEC-023	The <code>annotate_permit</code> semaphore (initialised with one permit) allows exactly one annotation pipeline to hold a permit at a time.	This prevents concurrent pdfium access across orphaned <code>spawn_blocking</code> threads.
T-EXEC-024	Verifies that the <code>annotate_permit</code> semaphore permit is returned when the <code>spawn_blocking</code> closure returns, even if it returns an error.	This simulates the pattern used in <code>run_annotate_pipeline</code> where the permit is moved into the closure as <code>_permit_guard</code> .
T-EXEC-025	Verifies that the <code>annotate_permit</code> semaphore permit is returned even when the <code>spawn_blocking</code> closure panics.	Rust’s drop semantics guarantee that the <code>OwnedSemaphorePermit</code> destructor runs during stack unwinding.
T-EXEC-026	Verifies that the semaphore serializes two concurrent annotation pipeline attempts.	The second pipeline blocks until the first releases its permit. This prevents the DEFECT-001 scenario where two <code>spawn_blocking</code> threads simultaneously loaded pdfium.
T-EXEC-027	Verifies that <code>CANCEL_GRACE_PERIOD</code> is set to a finite, reasonable duration.	Before the fix, the grace period branch never returned, causing an infinite loop. The fix uses this constant as the timeout after which the executor returns an error.
T-EXEC-028	The cancel check interval must be strictly shorter than the grace period, ensuring the executor polls at least once before the grace period expires.	This prevents the edge case where a single long sleep could skip past the grace period deadline.

ID	Description	Acceptance Criterion
T-EXEC-029	Compile-time verification that the <code>annotate_permit</code> field returned by <code>PipelineContext::annotate_permit()</code> is of type <code>Arc<tokio::sync::Semaphore></code> .	This prevents accidental regression to Mutex or removal of the DEFECT-001 fix.
T-EXEC-030	Simulates the DEFECT-001 scenario where the executor returns early (grace period expired) while the <code>spawn_blocking</code> thread is still running.	The orphaned thread holds the semaphore permit. The next pipeline attempt blocks on acquiring the permit until the orphaned thread finishes.
T-EXEC-DEFECT005-001	<code>ensure_hnsw_for_session</code> returns true immediately when the HNSW index for the session is already present in the in-memory map (fast path).	No database access is performed in this path.
T-EXEC-DEFECT005-002	<code>ensure_hnsw_for_session</code> returns false for a session that has no embeddings in SQLite.	This is the case when indexing produced no chunks (e.g., an empty directory or all PDFs failed extraction).
T-EXEC-DEFECT005-003	<code>ensure_hnsw_for_session</code> builds the HNSW from SQLite and inserts it into the map when embeddings are present but the in-memory index is absent.	This is the repair path for DEFECT-005: the search handler calls this function after Phase 3 silently skipped the HNSW build due to a stale WAL snapshot.
T-EXEC-DEFECT005-004	<code>ensure_hnsw_for_session</code> returns false for a nonexistent session ID.	The repair path must not panic when the session cannot be found in SQLite.

ID	Description	Acceptance Criterion
T-IDX-026	Legacy compute_batch_size produces identical results to com- pute_batch_size_with_caps with default (CPU, 8GB) capabilities.	—
T-IDX-027	CoreML on Apple Silicon with unified memory produces larger batch sizes than the same model on CUDA (discrete VRAM).	—
T-IDX-028	CoreML batch size scales with available RAM.	More unified memory → larger batches (logarithmic scaling).
T-IDX-029	CoreML batch size is clamped to 256 even with enormous RAM.	—
T-IDX-030	CUDA and ROCm batch sizes are unaffected by system memory.	They only depend on the model's sequence length.
T-IDX-031	CPU batch size is clamped to 64.	—
T-IDX-032	All EP configurations return at least 1 for any sequence length.	—
T-IDX-033	CoreML without unified memory (Intel Mac) gets the same treatment as a CPU EP since CoreML on Intel dispatches to Accelerate.	—
T-IDX-034	DirectML batch sizes match CUDA for the same sequence lengths.	Both use discrete VRAM, calibrated identically.
<i>Crate: neuroncite-search – Hybrid search pipeline (vector + keyword)</i>		
T-PERF-001	Batch orphan check identifies active vs.	deleted chunk IDs in a single query round-trip.

ID	Description	Acceptance Criterion
T-PERF-002	Batch orphan check with empty input returns empty set.	—
T-REF-001	parse_divisors parses comma-separated integers correctly.	—
T-REF-002	parse_divisors handles whitespace around values.	—
T-REF-003	parse_divisors filters out values less than 2.	—
T-REF-004	parse_divisors removes duplicates.	—
T-REF-005	parse_divisors handles empty input.	—
T-REF-006	parse_divisors handles single value.	—
T-REF-007	parse_divisors sorts ascending.	—
T-REF-008	parse_divisors ignores non-numeric values.	—
T-REF-009	generate_sub_chunks returns empty for empty results.	—
T-REF-010	generate_sub_chunks returns empty for empty divisors.	—
T-REF-011	generate_sub_chunks produces sub-chunks that are substrings of the parent result's content.	—
T-REF-012	generate_sub_chunks respects parent_index for multiple results.	—
T-REF-013	generate_sub_chunks skips chunks with too few tokens.	A chunk with 3 tokens and divisor 4 has window=0, which is skipped.
T-REF-014	CachedTokenizer::from_json errors on invalid JSON.	

ID	Description	Acceptance Criterion
T-REF-015	generate_sub_chunks handles single-word content gracefully.	—
T-REF-016	cosine_similarity of identical vectors returns 1.0.	—
T-REF-017	cosine_similarity of orthogonal vectors returns 0.0.	—
T-REF-018	cosine_similarity of zero vector returns 0.0.	—
T-REF-019	apply_refinement replaces content when a sub-chunk scores higher than the original.	—
T-REF-020	apply_refinement preserves original content when no sub-chunk scores higher.	—
T-REF-021	apply_refinement handles empty candidates gracefully.	—
T-REF-022	apply_refinement handles multiple results and selects the best sub-chunk independently for each result.	—
T-REF-023	apply_refinement preserves citation metadata after content replacement.	—
T-REF-024	RefinementConfig default values.	—
T-REF-025	generate_sub_chunks with divisor 2 on an 8-token chunk produces sub-chunks with exactly 4 tokens each (window = $8/2 = 4$).	—

ID	Description	Acceptance Criterion
T-REF-026	generate_sub_chunks with multiple divisors produces candidates at different granularities.	—
T-REF-027	CachedTokenizer::from_json succeeds with valid tokenizer JSON and the resulting tokenizer can be used for generate_sub_chunks.	—
T-REF-028	generate_sub_chunks enforces MAX_REFINEMENT_CANDIDATES.	With enough results and fine-grained divisors, the candidate count must not exceed the limit.
T-SCH-001	Vector search returns top-K sorted by descending cosine similarity.	—
T-SCH-002	Vector search filters orphaned labels (HNSW label with no chunk row is skipped) via batch query.	—
T-SCH-003	BM25 search returns ranked IDs.	An FTS5 query for a term present in exactly 3 chunks returns those 3 chunk IDs.
T-SCH-004	BM25 rank positions are 1-indexed.	The highest-ranked result has bm25_rank = 1.
T-SCH-005	RRF fusion combines both lists.	A candidate in both lists scores higher than one in only one list.
T-SCH-006	RRF parameter k effect.	k=10 produces a larger score difference between rank 1 and rank 10 than k=200.
T-SCH-007	BM25 must-match filter.	A vector candidate not in the BM25 list is excluded when must-match is enabled.
T-SCH-008	Dedup by content_hash.	Identical hashes keep only the highest-scoring candidate.

ID	Description	Acceptance Criterion
T-SCH-009	SimHash near-duplicate detection.	Hamming distance ≤ 3 is treated as a near-duplicate. The test constructs two texts that differ by only one short word out of many shared tokens, ensuring the SimHash fingerprints differ by at most 3 bits. A third text shares no meaningful token overlap, producing a Hamming distance well above the threshold.
T-SCH-010	Citation single-page format.	Uses "p. 5" when <code>page_start == page_end == 5</code> . Verifies the batch-loaded citation path.
T-SCH-011	Citation multi-page format.	Uses "pp. 5-7" when <code>page_start=5, page_end=7</code> . Verifies the batch-loaded citation path.
T-SCH-012	Sanitize returns an empty string for input that contains no alphanumeric characters (only operators/punctuation).	—
T-SCH-013	Keyword search with hyphenated terms succeeds instead of failing with "no such column" FTS5 error.	The hyphen in "short-term" was previously interpreted as the FTS5 NOT operator.
T-SCH-014	BM25 keyword search with <code>session_id</code> scoping only returns chunks from the specified session, even though FTS5 is a global virtual table shared across all sessions.	Regression test for BUG-001: Before the fix, <code>keyword_search</code> did not filter by <code>session_id</code> , causing results from unrelated sessions to leak into BM25 results. The RRF merge then included these cross-session results in the final output.
T-SCH-015	BM25 keyword search with <code>file_ids</code> filter combined with <code>session_id</code> scoping.	Ensures both filters work together correctly.
T-SCH-016	<code>extract_query_terms</code> produces cleaned tokens from a query string.	Tokens are stripped of double quotes but preserve their original case and hyphenation.

ID	Description	Acceptance Criterion
T-SCH-017	extract_query_terms strips embedded double quotes from tokens (same sanitization as sanitize_fts5_query).	—
T-SCH-018	extract_query_terms returns an empty Vec for input containing only punctuation and operator characters.	—
T-SCH-019	extract_query_terms preserves hyphenated terms as single tokens, matching the FTS5 sanitization behavior.	—
T-SCH-020	Sanitize wraps each token in double quotes so FTS5 treats hyphens, colons, and other operator characters as literal text.	—
T-SCH-021	Sanitize strips embedded double quotes from tokens to prevent FTS5 syntax errors from unbalanced quote characters.	—
T-SCH-022	Reranker score replaces RRF score.	When reranking is active, the final score of each result equals the reranker score, not the RRF score.
T-SCH-023	relevance_label returns correct labels at exact threshold boundaries.	The four tiers are: - ≥ 0.82 -> "high" - ≥ 0.72 -> "medium" - ≥ 0.60 -> "low" - < 0.60 -> "marginal"
T-SCH-024	relevance_label handles negative vector_score values.	These can occur with cosine similarity for non-normalized vectors and must return "marginal" without panic.
<i>Crate: neuroncite-store – SQLite storage, HNSW index management, and workflow tracking</i>		

ID	Description	Acceptance Criterion
T-AGG-001	all_session_aggregates	— returns correct counts for a session with files, pages, and chunks.
T-AGG-002	all_session_aggregates	— returns zero counts for an empty session.
T-AGG-003	file_chunk_stats_by_session	returns per-file chunk counts.
T-AGG-004	single_file_chunk_stats	— returns zero-count for a file with no chunks.
T-AGG-005	file_page_stats_by_session	returns per-file backend distribution.
T-AGG-006	browse_chunks	— returns paginated results.
T-AGG-007	browse_chunks with page_filter	— narrows results to chunks spanning the specified page.
T-AGG-008	find_files_by_path_pattern	finds files across sessions.
T-AGG-009	session_quality_data	— returns per-file quality rows.
T-AGG-010	page_quality_details	— returns empty and OCR pages.
T-AGG-011	page_quality_details	— returns empty Vec for a session with only clean pages (no empty or OCR pages).
T-ANN-STATUS-001	insert_pending_quotes	— inserts 3 rows with status "pending".

ID	Description	Acceptance Criterion
T-ANN-STATUS-002	update_quote_status transitions a row to "matched" and sets match_method, page, and pdf_filename.	—
T-ANN-STATUS-003	CASCADE delete removes quote status rows when the parent job is deleted.	—
T-ANN-STATUS-004	Pagination with limit and offset.	10 rows inserted, limit=3, offset=3 returns rows 4-6.
T-ANN-STATUS-005	count_quote_statuses_by_status returns correct counts for mixed status values.	
T-ANN-STATUS-006	update_quote_status returns 0 when no matching row exists (wrong job_id or quote combination).	—
T-ANN-STATUS-007	Full round-trip simulating the executor's lifecycle.	Insert pending quotes, update some to "matched" and some to "not_found", verify counts reflect the final state.
T-ANN-STATUS-008	The excerpt_to_meta mapping approach used by the executor correctly handles multiple quotes from the same work.	Two quotes with the same (title, author) but different excerpts are updated independently.
T-ANN-STATUS-009	Status rows survive across separate connection instances (persisted to SQLite, not held in memory).	This validates that the data is available after the executor's blocking thread completes and the MCP handler reads from a different connection.
T-ANN-STATUS-010	Updating all quotes to terminal statuses produces zero "pending" count.	The executor must leave no rows in "pending" state after the pipeline completes.
T-DIFF-001	diff_sessions identifies files that exist only in session A.	—

ID	Description	Acceptance Criterion
T-DIFF-002	diff_sessions identifies files that exist only in session B.	—
T-DIFF-003	files with same path and same hash appear in the identical category.	—
T-DIFF-004	files with same path but different hash appear in the changed category with	—
T-DIFF-005	hash_changed=true. files with same hash but different page_count appear in the changed category.	—
T-DIFF-006	empty session B produces all files in only_in_a.	—
T-EXT-010	read_embedding rejects (offset, length) pairs that overflow u64 when added together.	Without checked_add, the sum would silently wrap to a small number in release builds and the bounds check would pass, leading to an out-of-bounds memory access.
T-EXT-011	ExternalReader::read rejects (offset, length) pairs that overflow u64.	Mirrors T-EXT-012 for the stateful reader path.
T-EXT-012	The BYTES_PER_F32 constant matches the actual size of f32 on the target platform.	This compile-time assertion ensures the constant remains correct if the codebase is ever built on an exotic platform.
T-EXT-013	Byte length calculations using BYTES_PER_F32 produce the same result as std::mem::size_of_val on a concrete f32 slice.	Validates that the refactored constant produces identical byte offsets to the original size_of_val approach.

ID	Description	Acceptance Criterion
T-EXT-014	ExternalWriter byte offset tracking matches BYTES_PER_F32.	Verifies that the writer reports correct cumulative offsets when writing embeddings of different dimensions.
T-PAGE-SEARCH-001	search_page_text finds a literal substring on the correct page and returns the matching page number.	—
T-PAGE-SEARCH-002	case-insensitive search matches uppercase query against lowercase content.	—
T-PAGE-SEARCH-003	case-sensitive search does NOT match when case differs.	—
T-PAGE-SEARCH-004	multiple occurrences on the same page are all reported as separate TextMatch entries.	—
T-PAGE-SEARCH-005	query not found returns empty results (not an error).	—
T-PAGE-SEARCH-006	context is bounded by page content boundaries and does not panic on short pages.	—
T-PAGE-SEARCH-007	case-sensitive search finds a match when the query case matches the content exactly.	Regression test for DEF-007 where swapped arguments caused case_sensitive=true to always return zero.
T-PAGE-SEARCH-008	case-sensitive search finds multiple matches across different pages.	Regression test for DEF-007.
T-ST0-001	Schema migration on empty database creates all tables.	Running the migration function on a freshly created SQLite database creates all expected tables (index_session, indexed_file, page, chunk, chunk_fts, job, idempotency) without error.

ID	Description	Acceptance Criterion
T-ST0-002	Migration idempotency.	Running the migration function twice on the same database does not produce errors or duplicate tables.
T-ST0-003	Session creation returns a positive ID and the session is queryable by that ID with the same configuration values.	—
T-ST0-004	Duplicate session configuration returns a constraint violation error on the second insert.	—
T-ST0-005	Session COALESCE uniqueness for NULL values.	Two sessions that differ only in having NULL vs NULL for <code>chunk_size</code> (page-based strategy) are correctly rejected by the COALESCE-based unique index.
T-ST0-006	File tracking insert and query.	—
T-ST0-007	File change detection mtime/size pre-filter.	—
T-ST0-008	File change detection SHA-256 verification.	—
T-ST0-009	Bulk insert 1000 chunks in a transaction.	—
T-ST0-010	AUTOINCREMENT prevents rowid reuse after delete.	—
T-ST0-011	ON DELETE CASCADE from file deletes chunks and pages.	—
T-ST0-012	FTS5 INSERT trigger sync.	—
T-ST0-013	FTS5 DELETE removal.	—
T-ST0-014	FTS5 <code>is_deleted</code> flag removes from search.	—
T-ST0-015	HNSW serialization roundtrip.	—

ID	Description	Acceptance Criterion
T-ST0-016	<code>bulk_insert_chunks</code> rolls back on failure.	When a constraint violation occurs mid-batch (duplicate <code>chunk_index</code> for the same <code>file_id</code>), the RAII Transaction guard rolls back the entire batch. No partial rows remain in the database.
T-ST0-017	<code>build_hnsw</code> returns an error when a vector's length does not match the declared dimension.	Before this fix, the function used <code>assert_eq!</code> which panics in production. The error variant is <code>StoreError::HnswIndex</code> containing the mismatched chunk ID and both the expected and actual dimensions.
T-ST0-018	<code>build_hnsw</code> succeeds when all vectors match the declared dimension, returning an index that contains all inserted vectors.	—
T-ST0-019	<code>build_hnsw</code> with an empty vector slice returns a valid empty index without error.	—
T-ST0-020	<code>build_hnsw</code> rejects negative chunk IDs that cannot be represented as <code>usize</code> .	Before the fix, negative <code>i64</code> values were silently truncated via <code>as usize</code> , producing incorrect <code>DataId</code> labels in the HNSW graph and corrupting search results.
T-ST0-021	HNSW search returns the original <code>i64</code> chunk IDs that were used during construction.	Verifies the <code>usize->i64</code> round-trip in <code>build_hnsw</code> (<code>i64->usize</code>) and search (<code>usize->i64</code>) is lossless. Uses 20 vectors instead of 3 because <code>parallel_insert_slice</code> needs a sufficiently large dataset to build a well-connected HNSW graph. With fewer than ~10 points the graph connectivity is non-deterministic and the query vector may not appear in search results.
T-ST0-022	Serializing a deserialized index (<code>HnswStorage::Loaded</code>) produces a valid file that deserializes into an index with the same search results.	Verifies that <code>hnsw().file_dump()</code> delegates correctly for both <code>Built</code> and <code>Loaded</code> variants via <code>HnswStorage::hnsw()</code> .

ID	Description	Acceptance Criterion
T-ST0-023	<code>delete_session</code> removes the session and returns count=1.	Calling <code>get_session</code> afterwards returns a NotFound error.
T-ST0-024	The <code>index_session</code> table includes the <code>label</code> column in the v1 schema.	Verifies via <code>PRAGMA table_info</code> .
T-ST0-025	<code>find_sessions_by_directory</code> returns all sessions for a given directory path, including sessions with different models or chunking strategies.	
T-ST0-026	<code>find_sessions_by_directory</code> returns an empty vec when no sessions match the given directory.	
T-ST0-027	<code>delete_sessions_by_directory</code> removes all sessions for a directory and returns the deleted IDs.	ON DELETE CASCADE removes dependent file and chunk records.
T-ST0-028	<code>load_embeddings_for_hnsw</code> returns (id, embedding) pairs for active chunks with non-NULL embeddings, excluding deleted chunks and chunks without embeddings.	
T-ST0-029	The <code>simhash</code> column is stored and retrievable.	Verifies that SimHash values inserted via <code>ChunkInsert</code> are persisted in the database and can be read back through a direct SQL query.
T-ST0-030	The <code>simhash</code> column accepts NULL for backward compatibility.	Chunks inserted without a <code>simhash</code> value store NULL in the database.
T-ST0-031	Jobs not associated with the deleted session remain untouched.	Only jobs whose <code>session_id</code> matches the deleted session should have their <code>session_id</code> set to NULL.

ID	Description	Acceptance Criterion
T-ST0-032	Column repair is idempotent.	Running <code>ensure_columns()</code> twice does not duplicate columns or produce errors.
T-ST0-033	The <code>SELECT</code> queries in <code>job.rs</code> work against a database that was repaired by <code>ensure_columns()</code> .	This is the end-to-end test that reproduces the original "no such column: <code>params_json</code> " bug.
T-ST0-034	The <code>citation_row</code> table exists after migration.	Verifies that the citation verification table is created as part of the v1 schema.
T-ST0-035	<code>insert_file</code> with <code>pdf_page_count</code> stores and retrieves the structural page count correctly.	—
T-ST0-036	Foreign key constraint on <code>citation_row.job_id</code> .	Inserting a <code>citation_row</code> that references a non-existent <code>job_id</code> must fail when foreign keys are enabled.
T-ST0-037	<code>CASCADE</code> delete removes <code>citation_rows</code> when a job is deleted.	Creating a job, inserting <code>citation_rows</code> referencing it, then deleting the job must cascade-delete all associated <code>citation_rows</code> .
T-ST0-038	Migration idempotency with <code>citation_row</code> table.	Running the migration twice does not produce errors or duplicate the table.
T-ST0-039	Column repair adds <code>tags</code> and <code>metadata</code> to an old <code>index_session</code> table that was created without them.	Simulates an old database, then runs <code>ensure_columns()</code> to verify both columns are added.
T-ST0-040	The v1 schema includes the <code>tags</code> and <code>metadata</code> columns in <code>index_session</code> .	Verifies both columns exist via <code>PRAGMA table_info</code> and accept <code>NULL</code> values.
T-ST0-041	The <code>page</code> table has exactly 6 columns: <code>id</code> , <code>file_id</code> , <code>page_number</code> , <code>content</code> , <code>backend</code> , <code>byte_count</code> .	Verifies that the schema definition in <code>create_core_tables</code> produces the expected column set.

ID	Description	Acceptance Criterion
T-ST0-042	The <code>chunk</code> table has the expected set of 16 columns.	Verifies that the schema definition in <code>create_chunk_table_and_indexes</code> produces all required columns for embedding storage, deduplication (<code>content_hash</code> , <code>simhash</code>), soft-delete support (<code>is_deleted</code>), and the <code>created_at</code> timestamp.
T-ST0-043	The <code>job</code> table has the expected set of 11 columns including the <code>params_json</code> column added by the column repair step.	—
T-ST0-044	AUTOINCREMENT on the <code>chunk</code> table prevents ID reuse.	After inserting a chunk with ID N, deleting it, and inserting a new chunk, the new chunk's ID is greater than N.
T-ST0-045	The <code>annotation_quote_status</code> table exists after migration.	—
T-ST0-046	The <code>annotation_quote_status</code> table has exactly 10 columns.	—
T-ST0-047	CASCADE delete on <code>annotation_quote_status</code> .	Deleting a job removes all associated quote status rows.
T-ST0-048	Column repair creates <code>annotation_quote_status</code> on old databases that were created before the table was added.	—
T-ST0-049	The partial index <code>idx_job_state_active</code> exists after migration.	Verifies that the partial index on <code>job(state, kind, created_at)</code> covering active states ('queued', 'running') is created as part of the v1 schema.
T-ST0-050	<code>list_chunks_by_session</code> excludes embedding blobs from the result set.	The embedding column is returned as NULL to avoid transferring large binary data when only metadata is needed.

ID	Description	Acceptance Criterion
T-ST0-051	<code>update_session_tags</code> — and <code>update_session_metadata</code> return 0 rows affected when called with a non-existent session ID.	
T-ST0-052	<code>list_sessions</code> includes tags and metadata fields for each session.	Verifies that both columns are correctly projected in the listing query, for sessions with and without tags/metadata.
T-ST0-053	<code>find_sessions_by_directory</code> returns tags and metadata in the session rows.	
T-ST0-060	<code>get_session</code> returns the correct <code>vector_dimension</code> for sessions created with different model dimensions.	This is the data path exercised by the API search handler's dimension guard, which rejects search requests when the session dimension does not match the currently loaded model's dimension. Two sessions with distinct dimensions must each return their own value.
T-ST0-061	Schema migration creates all four FTS5 triggers.	After running the migration, all four FTS5 synchronization triggers (INSERT, DELETE, soft-delete, un-delete) must exist in <code>sqlite_master</code> . The un-delete trigger re-inserts a chunk's content into FTS5 when its <code>is_deleted</code> flag changes from 1 back to 0.
T-ST0-062	<code>delete_file_by_session</code> removes a file record by its composite key (<code>session_id</code> , <code>file_path</code>) and returns the count of deleted rows.	Verifies that deleting a non-existent path returns 0 and that the correct file is removed when multiple files exist in the session.
T-ST0-063	<code>delete_file_by_session_path</code> enables re-insert of the same file path after deletion, verifying that the UNIQUE constraint is satisfied after cleanup.	This simulates the re-indexing scenario where a stale file record from a previous run blocks a fresh insert.

ID	Description	Acceptance Criterion
T-ST0-064	find_file_by_session_path returns the file record when it exists for the given session and path.	
T-ST0-065	find_file_by_session_path returns None for a non-existent file path within the session.	
T-ST0-066	find_file_by_session_path returns None when the file exists in a different session but not the queried session.	
T-ST0-067	FTS5 un-delete trigger restores chunk to search index.	When a chunk's is_deleted flag changes from 1 back to 0, the un-delete trigger re-inserts its content into the FTS5 index so it appears in keyword search results again. Without the trigger, un-deleted chunks are permanently lost from FTS5.
T-ST0-068	get_chunks_batch returns an empty vector when called with an empty ID slice, without issuing any SQL statement.	—
T-ST0-069	get_chunks_batch retrieves a single chunk correctly when given a one-element ID slice.	—
T-ST0-070	get_chunks_batch retrieves multiple chunks in a single query.	All requested IDs are returned with correct content.
T-ST0-071	get_chunks_batch silently omits IDs that do not exist in the database.	Requesting a mix of valid and non-existent IDs returns only the valid ones.

ID	Description	Acceptance Criterion
T-ST0-072	get_chunks_batch returns an empty vector when all requested IDs are non-existent.	—
T-ST0-073	Atomic job creation with idempotency key — first call creates the job; a second call with the same key returns the existing entry without creating a duplicate job row.	—
T-ST0-074	create_job (without params) stores NULL for params_json.	Verifies that the legacy create_job function produces a job with params_json = None.
T-ST0-075	Atomic job creation with idempotency key — two different keys on the same session create two independent jobs.	—
T-ST0-076	list_jobs returns all jobs including params_json.	This test reproduces the original "no such column: params_json" bug by verifying that list_jobs correctly reads the params_json column.
T-ST0-077	The v1 schema includes the label column in index_session.	Verifies the column exists via PRAGMA table_info and accepts NULL values (no NOT NULL constraint).
T-ST0-078	schema_version() returns the constant SCHEMA_VERSION value.	—
T-ST0-079	The index_session table has exactly the expected set of columns.	Guards against accidentally missing or extra columns.
T-ST0-080	delete_sessions_by_directory returns an empty vec when no sessions match the given directory (no error, just empty).	

ID	Description	Acceptance Criterion
T-ST0-081	The <code>job</code> table has exactly the expected set of columns, including <code>params_json</code> .	This test guards against the bug where job-related endpoints fail with "no such column: params_json" because the column was missing from the schema.
T-ST0-082	BUG-004 regression: <code>delete_session</code> detaches job records instead of cascade-deleting them.	A completed job that referenced a deleted session must remain in the job table with <code>session_id = NULL</code> .
T-ST0-083	The <code>indexed_file</code> table has exactly the expected set of columns, including <code>pdf_page_count</code> .	—
T-ST0-084	BUG-004 regression: <code>delete_sessions_by_directory</code> detaches all job records for all deleted sessions.	Multiple jobs across multiple sessions in the <code>directory</code> must all survive with <code>session_id = NULL</code> .
T-ST0-085	Column repair adds <code>params_json</code> to an old job table that was created without it.	Simulates an old database by creating the job table without <code>params_json</code> , then running <code>ensure_columns()</code> to verify the column is added.
T-ST0-086	Column repair adds <code>pdf_page_count</code> to an old <code>indexed_file</code> table that was created without it.	—
T-ST0-087	The <code>citation_row</code> table has exactly the expected 20 columns.	Guards against accidentally missing or extra columns in the citation verification schema.
T-ST0-088	A freshly created session has a NULL label by default.	After calling <code>update_session_label</code> with a string, <code>get_session</code> returns that label. Calling it again with <code>None</code> clears the label back to NULL.
T-ST0-089	<code>update_session_label</code> returns 0 rows affected when called with a non-existent session ID (no error, just zero updates).	—

ID	Description	Acceptance Criterion
T-ST0-090	<code>list_sessions</code> includes the label field for each session.	Verifies that the label column is correctly projected in the listing query, both for sessions with and without labels.
T-ST0-091	<code>find_session</code> returns the correct ID for a matching config and <code>None</code> for a config that has no matching session.	—
T-ST0-092	A freshly created session has NULL tags and metadata by default.	After calling <code>update_session_tags</code> and <code>update_session_metadata</code> with JSON strings, <code>get_session</code> returns those values. Calling with <code>None</code> clears them back to NULL.
T-ST0-093	Job state transitions.	—
T-ST0-094	Job 24-hour retention cleanup.	—
T-ST0-095	<code>create_job_with_params</code> stores and retrieves <code>params_json</code> .	Verifies that annotation-style jobs with JSON parameters can be created and the parameters are correctly read back via <code>get_job</code> .
T-ST0-096	Idempotency key lookup.	Storing an idempotency entry and querying by key returns the associated <code>job_id</code> and <code>session_id</code> . A non-existent key returns <code>None</code> .
T-STORE-032	SAVEPOINT rollback on partial failure.	Verifies that when a bulk insert fails mid-batch (due to a UNIQUE constraint violation on duplicate <code>page_number</code>), all rows from the batch are rolled back and no partial data remains in the page table. Without the SAVEPOINT, each INSERT would autocommit separately and the first two pages would persist even though the batch as a whole failed.
T-STORE-033	Atomic write leaves no temporary file.	Verifies that after <code>write_manifest</code> completes, only the final <code>models.lock</code> file exists in the directory and the temporary <code>.models.lock.tmp</code> file has been removed by the rename operation. A leftover temp file would indicate the atomic rename failed or was skipped.
T-STORE-WS-001	<code>insert_web_source</code> + <code>get_web_source_by_file_id</code> roundtrip.	Inserting a <code>web_source</code> record and retrieving it by <code>file_id</code> returns all stored fields.

ID	Description	Acceptance Criterion
T-STORE-WS-002	get_web_source_by_url finds a record by URL.	
T-STORE-WS-003	get_web_source_by_url returns None for a URL that does not exist in the database.	
T-STORE-WS-004	list_web_sources returns only records belonging to files within the specified session.	—
T-STORE-WS-005	delete_web_source removes the record and cascade from indexed_file deletion also works.	—
T-STORE-WS-006	insert_web_source with all optional fields as None.	—
T-STORE-WS-007	insert_web_source with raw_html BLOB stores and retrieves binary data correctly.	—
T-STORE-WS-008	Schema migration creates the web_source table with the expected column set.	—
T-STORE-WS-009	Migration is idempotent – running migrate() twice does not produce errors or duplicate the web_source table.	—
T-STORE-WS-010	CASCADE delete removes web_source when the parent indexed_file record is deleted.	—

ID	Description	Acceptance Criterion
T-STORE-WS-011	UNIQUE constraint on file_id prevents duplicate web_source records for the same indexed_file.	—
T-STORE-WS-012	list_web_sources_all returns records across all sessions.	—
<i>Crate: neuroncite-web – Browser-based web frontend server</i>		
T-SSRF-001	Default URL (None) resolves to localhost and passes validation.	—
T-SSRF-002	Explicit localhost URL with the Ollama port passes validation.	—
T-SSRF-003	127.0.0.1 URL with Ollama port passes validation.	—
T-SSRF-004	javascript: scheme is rejected.	—
T-SSRF-005	file:// scheme is rejected.	—
T-SSRF-006	data: scheme is rejected.	—
T-SSRF-007	ftp: scheme is rejected.	—
T-SSRF-008	Remote host is rejected.	The proxy only allows localhost connections to prevent SSRF to arbitrary external or internal services.
T-SSRF-009	https scheme with localhost and Ollama port passes validation.	—
T-SSRF-010	Completely invalid URL is rejected.	—
T-SSRF-011	Empty string URL is rejected (empty is not a valid URL).	—
T-SSRF-012	LAN address (192.168.x.x) is rejected.	The Ollama proxy only allows localhost connections.

ID	Description	Acceptance Criterion
T-SSRF-013	Localhost with a non-Ollama port is rejected to prevent scanning or accessing other local services.	—
T-SSRF-014	Localhost with a non-API path is rejected to prevent proxying to non-Ollama endpoints.	—
T-SSRF-015	Localhost with /api/ path prefix passes validation.	—
T-SSRF-016	IPv6 loopback [::1] with Ollama port passes validation.	—
T-WEB-028	The status endpoint rejects URLs with non-Ollama ports.	After SSRF hardening (M-004), the Ollama proxy only allows port 11434. A request with port 19999 is rejected at validation with HTTP 400.
T-WEB-029	The models endpoint rejects URLs with non-Ollama ports.	After SSRF hardening (M-004), port 19999 is blocked at validation.
T-WEB-030	The running endpoint rejects URLs with non-Ollama ports.	After SSRF hardening (M-004), port 19999 is blocked at validation.
T-WEB-031	The catalog endpoint returns a non-empty list of curated Ollama models with all required fields populated.	—
T-WEB-032	The pull endpoint rejects URLs with non-Ollama ports.	After SSRF hardening (M-004), port 19999 is blocked at validation.
T-WEB-033	The delete endpoint rejects URLs with non-Ollama ports.	After SSRF hardening (M-004), port 19999 is blocked at validation.
T-WEB-034	The pull endpoint returns 400 when the model name is empty.	—

ID	Description	Acceptance Criterion
T-WEB-035	The delete endpoint returns 400 when the model name is empty.	—
T-WEB-036	All catalog entries have non-empty required fields and positive size values.	—

Table 39: Unit test catalog: 1449 tests across all library crates.

22.4 Integration Test Catalog

The following table lists all integration tests that verify cross-crate interactions, end-to-end workflows, and external system integration.

ID	Description	Acceptance Criterion
<i>Full pipeline integration tests</i>		
T-E2E-001	Queries the HNSW index with the embedding vector of the first chunk (seed 1) and verifies that the first chunk is returned as the top result.	Keyword search is still active but has minimal impact because the query text uses a generic term.
T-E2E-002	Queries with a term that appears in exactly one chunk ("bayesian") and verifies that hybrid search returns that chunk with a populated bm25_rank field, confirming that FTS5 keyword search contributed to the fusion ranking.	—

ID	Description	Acceptance Criterion
T-E2E-003	Regression test for the FTS5 query sanitization fix.	Queries containing hyphens (e.g., "short-term") were previously interpreted as the FTS5 NOT operator, causing "no such column: term" errors. This test verifies that the sanitization layer wraps hyphenated tokens in double quotes, allowing FTS5 to treat them as literal terms.
T-E2E-004	Verifies that the citation assembly stage populates the source file path, file display name, page range, and formatted citation string on each returned result.	These fields are resolved from the chunk -> file join in the database.
T-E2E-005	Verifies that the search pipeline returns results sorted in descending order by their final score.	The first result has the highest score and subsequent results have equal or lower scores.
T-E2E-006	Verifies that a query containing embedded quotes ("Box-Cox") is handled correctly by the FTS5 sanitization layer.	The quotes are stripped and the hyphenated token is wrapped in double quotes for FTS5.
T-E2E-007	When <code>bm25_must_match</code> is true, only candidates that appear in both the vector and keyword result sets survive fusion.	A query for "heteroskedasticity" (appearing only in chunk 5) with the embedding of chunk 5 should return chunk 5 (present in both sets) and exclude chunks that only appear in the vector results.
T-E2E-008	Verifies that when a reranker is provided and <code>rerank_enabled</code> is true, the final scores in the search results reflect the reranker's output rather than the RRF fusion scores.	—

ID	Description	Acceptance Criterion
T-E2E-009	Verifies that an empty or whitespace-only query text does not cause errors.	The vector search still runs (using the provided embedding), but keyword search returns no matches. The pipeline completes without error.
T-E2E-010	Verifies that the <code>max_results</code> configuration parameter correctly limits the number of results returned, even when more candidates are available from the HNSW index and FTS5.	—
T-E2E-011	Verifies that the content field of each search result matches the text that was originally inserted into the database.	This confirms that the citation assembly stage correctly retrieves the full chunk text from the chunk table.
T-INT-001	Verifies the complete indexing pipeline from PDF discovery through text chunking to SQLite chunk storage.	Creates two test PDF files in a temporary directory, discovers them via <code>neuroncite_pdf::discover_pdfs</code> , creates a session and file records, applies the word-window chunking strategy, and bulk-inserts the resulting chunks. Asserts that the session, files, and chunks exist in the database with correct relationships.
T-INT-002	Verifies that the indexing pipeline creates entities in the correct hierarchical order: session first, then files referencing the session, then pages referencing files, then chunks referencing both files and the session.	Validates foreign key relationships by checking that each chunk's <code>session_id</code> and <code>file_id</code> match the parent entities.

ID	Description	Acceptance Criterion
T-INT-003	Verifies that the two-stage file change detection correctly identifies files whose mtime and size have not changed since the last indexing run, classifying them as Unchanged .	Simulates a re-indexing scenario by inserting a file record, then calling <code>check_file_changed</code> with identical metadata. The file is reported as unchanged, avoiding unnecessary re-extraction.
T-INT-004	Verifies that the indexing pipeline continues processing remaining files when extraction fails for some PDFs.	Simulates a batch of three files where the second file's extraction "fails" (represented by an empty page). The pipeline stores chunks for the successful files and the overall operation does not abort. Files with zero extractable text still produce a file record but contribute zero chunks to the session.
T-INT-005	Verifies that chunks with identical <code>content_hash</code> values represent duplicates that the deduplication layer can identify.	Inserts multiple chunks where two share the same content hash, then queries the chunk table by <code>content_hash</code> to confirm that the index on <code>content_hash</code> enables lookup of duplicates. The deduplication logic itself is tested at the unit level in <code>neuroncite-search</code> ; this integration test verifies that the database schema supports the hash-based duplicate detection across the store and chunk crates.
T-INT-006	Verifies that the FTS5 full-text search index is automatically populated when chunks are bulk-inserted into the chunk table.	The FTS5 synchronization triggers on the chunk table copy chunk content into the <code>chunk_fts</code> virtual table on INSERT. This test inserts chunks with distinctive terms and then queries the FTS5 index to confirm that the terms are searchable.
T-OCR-001	Confirms that the default pdf-extract backend cannot extract text from the three OCR test PDFs.	Each file either returns an error or triggers a panic (caught by <code>catch_unwind</code>). This test validates the premise that these PDFs require fallback backends.

ID	Description	Acceptance Criterion
T-OCR-002	Verifies that the pdfium backend successfully extracts text from all three OCR test PDFs when called directly.	These PDFs have broken cross-reference tables, malformed CID-range entries, or corrupt Type1 font data that pdf-extract cannot handle, but pdfium's Chrome- derived PDF engine parses correctly. This test requires pdfium.dll / libpdfium.so at runtime. If the library is not available, the test fails with a descriptive assertion message.
T-OCR-003	Verifies that <code>extract_pages(path, None, None)</code> (the automatic fallback chain: pdf-extract -> pdfium -> OCR) produces text for all three OCR test PDFs.	The automatic chain should use pdfium as fallback when pdf-extract fails or produces low-quality text. Requires pdfium.dll / libpdfium.so at runtime.
T-OCR-004	Verifies that the text extracted via the pdfium backend passes the quality heuristics defined in <code>neuroncite_pdf::quality</code> .	For each PDF where pdfium extraction succeeds, at least 50% of pages with non-trivial text content must pass all quality thresholds (alphabetic ratio, replacement char ratio, line break density, unique token ratio). Requires pdfium.dll / libpdfium.so at runtime.
T-OCR-005	Exercises the complete indexing pipeline (<code>extract -> chunk -> store</code>) against the OCR test PDFs using the pdfium fallback.	Verifies that files, pages, and chunks are persisted in SQLite with correct metadata. Requires pdfium.dll / libpdfium.so at runtime.
T-OCR-006	Verifies that after indexing OCR-extracted content, the FTS5 full-text search index returns results for domain-specific terms that are expected to appear in the three OCR test PDFs (statistics, finance, and forecasting papers).	Requires pdfium.dll / libpdfium.so at runtime.

ID	Description	Acceptance Criterion
T-OCR-010	The synthetic xref-corruption PDF has triple-corrupted cross- reference data (xref offsets, startxref pointer, trailer /Size) that causes both pdf-extract and pdfum to fail.	This PDF requires OCR (Tesseract) for text extraction. Without OCR, both backends fail and the fallback chain correctly propagates the error. This test verifies: 1. pdf-extract fails (cross-reference error). 2. pdfum also fails (FormatError from the same corruption). 3. auto-extract returns an error (both backends exhausted, no OCR).
T-OCR-011	The synthetic CID-range PDF has a malformed CMap encoding stream that causes pdf-extract to panic in adobe_cmap_parser (not return an error).	Verifies that pdf-extract panics, pdfum succeeds, and the extracted Helvetica text contains finance terms ("trading", "stock", "returns").
T-OCR-012	The synthetic Type1 PDF has a malformed FontFile stream with an unterminated PostScript string literal that causes pdf-extract to panic in type1_encoding_parser.	Verifies that pdfum succeeds and the extracted Helvetica text contains forecasting terms ("forecast", "accuracy", "error").
T-OCR-013	Verifies that the auto-extract chain (backend=None) produces non-empty pages for all three problematic PDFs and that every page is annotated with a recognized backend (PdfExtract, Pdium, or Ocr).	Mixed backends within a single file are expected because the per-page quality checks route pages individually through the fallback chain.

ID	Description	Acceptance Criterion
T-OCR-020	Verifies that the Tesseract CLI subprocess OCR pipeline works end-to-end.	Creates a minimal PNG image containing a white pixel, invokes <code>neuroncite_pdf::ocr::ocr_page()</code> which writes the image to a temp file and calls <code>tesseract <file> stdout -l eng</code> , and checks the result. Tesseract must be installed (on PATH or in the NeuronCite runtime cache) for this test to pass. If Tesseract is missing, the test fails with a descriptive error message directing the user to install it. Requires the <code>ocr</code> feature to be enabled.
T-OCR-021	Uses one of the two PDFs that pdfium can open (CID-range PDF) and verifies that when the OCR feature is enabled, the auto-extract fallback chain completes without crashing.	Since the CID-range PDF contains embedded text (not scanned images), pdfium extracts text directly and the OCR path is reached only for pages with sparse text. The primary assertion is that extraction succeeds and produces substantial text. Requires pdfium and Tesseract at runtime.
T-REAL-001	Verifies that <code>neuroncite_pdf::discover_pdfs</code> correctly locates PDF files in the synthetic test directory (<code>tests/PDFs/ocr/</code> with 3 subdirectories, each containing one synthetic PDF).	—
T-REAL-002	Verifies that text extraction produces non-empty page content for at least one synthetic PDF.	The auto-fallback chain (<code>pdf-extract -> pdfium -> OCR</code>) handles the malformed fonts in the CID-range and Type1 fixtures.
T-REAL-003	Verifies that the synthetic PDFs with malformed fonts are handled without panics or hangs.	Each PDF is extracted sequentially with per-file timeout protection. Each file is classified by text density.

ID	Description	Acceptance Criterion
T-REAL-004	Exercises the complete indexing pipeline: extract, chunk, and store in SQLite.	Uses the synthetic PDFs via the auto-fallback chain.
T-REAL-005	Verifies that after indexing synthetic PDF content, the FTS5 full-text search index returns results for domain-specific terms present in the synthetic fixtures (statistics, finance, forecasting).	—
T-REAL-007	Verifies that the sentence-based chunking strategy produces valid chunks from synthetic PDF content.	—
T-REAL-008	Verifies that the page-based chunking strategy produces one chunk per non-empty page from synthetic PDF content.	—
T-REAL-009	Verifies that an HNSW index can be built from chunk embeddings produced during the indexing of synthetic PDFs.	Uses stub embeddings (4-dimensional).
<i>CLI integration tests</i>		

ID	Description	Acceptance Criterion
T-CLI-001	Verifies that the version subcommand exits with status code 0 and produces JSON output containing a "version" field on stdout.	The version subcommand prints build metadata including the crate version from Cargo.toml and the list of compiled feature flags.
T-CLI-002	Verifies that the doctor subcommand exits with status code 0 and produces a valid JSON report on stdout.	The doctor subcommand checks for the availability of optional runtime dependencies (CUDA, ONNX Runtime, PDFium, Tesseract) and reports their status. The report must be valid JSON with entries for each dependency.
T-CLI-003	Verifies that the index subcommand exits with status code 1 when given a directory path that does not exist on the filesystem.	The run_index function checks for directory existence before proceeding with the indexing pipeline and returns exit code 1 on failure.
T-CLI-004	Verifies that the search subcommand exits with status code 2 (clap's default for missing required arguments) when invoked without the mandatory <code>-directory</code> , <code>-session-id</code> , and <code>-query</code> arguments.	Clap exits with code 2 and writes a usage error message to stderr when required arguments are not provided.
T-CLI-005	Verifies that the sessions subcommand prints an empty JSON array when pointed at a temporary directory that has no <code>.neuroncite</code> database.	The current implementation returns an empty array regardless of the directory contents (it does not open or create a database). This test documents that behavior: the output is <code>[]</code> , and the exit code is 0.

ID	Description	Acceptance Criterion
T-CLI-006	Verifies that <code>models list</code> exits with status 0 and produces JSON output containing a <code>models</code> array with at least one entry.	Each entry must have <code>model_id</code> , <code>display_name</code> , <code>vector_dimension</code> , <code>cached</code> , and <code>pooling</code> fields. The response also includes <code>total</code> and <code>cached_count</code> summary fields.
T-CLI-007	Verifies that <code>models info</code> with a known model ID exits with status 0 and returns the correct configuration for BGE Small EN v1.5: 384 dimensions, "cls" pooling, 512 max sequence length, and the <code>uses_token_type_ids</code> flag set to true.	—
T-CLI-008	Verifies that <code>models info</code> with an unknown model ID exits with status 1 and writes an error JSON object to stderr with the error code "model_not_found".	—
T-CLI-009	Verifies that <code>models system</code> exits with status 0 and produces JSON output containing <code>gpu</code> (object), <code>backends</code> (array), <code>cache_dir</code> (string), and <code>cache_dir_exists</code> (boolean) fields.	This command initializes the ONNX Runtime shared library (via load-dynamic) and probes GPU execution providers, which can take significant time on first invocation (DLL download) or hang if a GPU provider probe deadlocks. A 90-second timeout prevents the test suite from blocking indefinitely.

ID	Description	Acceptance Criterion
T-CLI-010	Verifies that <code>models verify</code> with an unknown model ID exits with status 1 and writes an error JSON object to <code>stderr</code> .	—
T-CLI-011	Verifies the invariant that <code>cached_count</code> is less than or equal to <code>total</code> in the <code>models list</code> output.	A violation would indicate a counting bug in the list implementation.
T-CLI-012	Verifies that every model in the <code>models list</code> output has a <code>pooling</code> field with one of the three valid values: <code>"cls"</code> , <code>"mean-pooling"</code> , or <code>"last-token"</code> .	These correspond to the <code>PoolingStrategy</code> enum variants and their <code>Display</code> trait implementations.
T-CLI-013	Verifies that <code>models download</code> without the required <code>model_id</code> positional argument exits with status 2 (clap's default for missing required arguments).	—
<i>API server integration tests</i>		
T-SRV-001	Verifies that the server process binds to a port and responds to the <code>/api/v1/health</code> endpoint with a 200 status code and valid JSON.	If the server cannot start (e.g., missing ONNX Runtime or model), the test prints a warning and returns successfully. The test only fails if the server starts but returns incorrect responses.

ID	Description	Acceptance Criterion
T-SRV-002	Verifies that the server handles multiple concurrent requests without panicking or returning server errors.	Sends 10 concurrent GET requests to the health endpoint and verifies that all responses have a 200 status code. This exercises the server's ability to handle parallel request processing.
T-SRV-003	Verifies that the server terminates when a kill signal is sent to the process.	On Windows, the process is terminated via TerminateProcess since SIGINT is not directly supported for child processes. The test verifies that the process exits and does not hang.
T-SRV-004	Verifies that the server handles multiple concurrent GET /api/v1/health requests without returning any error responses.	Spawns 10 tasks, each making 2 sequential requests to the health endpoint for a total of 20 concurrent requests. All responses must have HTTP 200. This exercises the multi-threaded Tokio executor's ability to schedule parallel connections without triggering race conditions on the Axum router or the shared AppState. If the server cannot start (missing ORT library or model), the test prints a diagnostic message and returns without failing.
T-SRV-007	When the server is started with <code>-print-token</code> , requests to /api/v1/health without an Authorization header must receive HTTP 401.	This verifies that the auth middleware enforces bearer token authentication on all API endpoints when a token is configured.
T-SRV-008	When the server is started with <code>-print-token</code> , requests to /api/v1/health with an incorrect bearer token must receive HTTP 401.	This verifies that the auth middleware rejects tokens that do not match the configured server token, even when the Authorization header is present.

ID	Description	Acceptance Criterion
T-SRV-009	When the server is started with <code>-print-token</code> , requests to <code>/api/v1/health</code> with the correct bearer token must receive HTTP 200.	This verifies that the auth middleware allows requests through when the Authorization header contains the server’s configured token.
T-SRV-010	When the server is started with <code>-print-token</code> , all API endpoints including <code>/api/v1/openapi.json</code> require an <code>Authorization: Bearer <token></code> header.	Requests without the header receive 401. Requests with the correct token receive 200. This test verifies that the fix for the OpenAPI-outside-auth finding is effective: <code>/api/v1/openapi.json</code> must now be inside the auth middleware layer and must not be accessible to unauthenticated callers when a token is configured.

Table 40: Integration test catalog: 58 tests covering cross-crate interactions.

22.5 Property-Based and Regression Test Catalog

The following table lists property-based tests (verified via randomized input generation) and regression tests (covering previously identified edge cases and bug fixes).

ID	Description	Acceptance Criterion
<i>Property-based tests</i>		

ID	Description	Acceptance Criterion
T-PROP-001	For any valid pages input, chunk output preserves all text content.	This property verifies that the union of all chunk texts, when their words are collected and concatenated, accounts for every word present in the normalized concatenated document text. The word-window strategy splits on whitespace, so the content of all chunks (accounting for overlap) must contain every word from the original text at least once. The test iterates over 200 pseudo-random inputs with varying page counts and word counts, using both the "word" and "page" chunking strategies. The LCG seed is printed to stderr for failure reproduction.
T-PROP-002	Chunk byte offsets are non-overlapping and cover the entire document.	For the "page" strategy, each chunk's byte range [doc_text_offset_start, doc_text_offset_end) corresponds to exactly one page's content within the concatenated document text. The byte ranges must be non-overlapping (each page occupies a distinct region) and the union of all byte ranges plus the inter-page newline separators must cover the full concatenated text length. For the "word" strategy with zero overlap, consecutive chunk byte ranges must not overlap (though they may have gaps at whitespace boundaries). With non-zero overlap, chunks share bytes in the overlap regions, but the start of each chunk must advance beyond the start of the previous chunk. The test iterates over 200 pseudo-random inputs. The LCG seed is printed to stderr for failure reproduction.
T-PROP-003	<code>l2_normalize</code> produces unit vectors for any non-zero input.	For 100 random non-zero vectors with dimensions ranging from 1 to 1024, the L2 norm of the normalized vector must be within 1e-6 of 1.0. This tolerance accounts for accumulated floating-point rounding in the sum of squares and square root operations.

ID	Description	Acceptance Criterion
T-PROP-004	<code>l2_normalize_batch</code> normalizes each vector independently.	For 50 random batches (each containing 2 to 20 vectors), the function must produce the same result as calling <code>l2_normalize</code> on each vector individually. This test verifies independence by comparing batch-normalized vectors element-by-element against independently normalized copies.
T-PROP-007	SimHash of identical texts always produces Hamming distance 0.	The SimHash algorithm is deterministic: given the same input text, it must produce the same 64-bit fingerprint every time. Therefore, the Hamming distance between the fingerprint of a text and itself must always be 0. This property is tested with 100 random text strings of varying lengths (1 to 200 words).
T-PROP-008	RRF fusion score is always positive and bounded.	For any candidate that appears in at least one result list at a valid 1-indexed rank position, the RRF score must satisfy: $0 < \text{RRF}(d) \leq 2 / (k + 1)$. The upper bound occurs when a candidate appears at rank 1 in both rankers: $\text{RRF_max} = 1/(k+1) + 1/(k+1) = 2/(k+1)$. The lower bound is strictly positive because $1/(k+\text{rank}) > 0$ for all positive k and rank values. This property is tested with 200 random rank combinations across different k values (10, 30, 60, 100, 200).
<i>Regression and edge case tests</i>		
T-REG-001	Empty document (0 pages) returns an error, not a panic.	All chunking strategies require at least one page. Passing an empty slice must produce <code>Err(NeuronCiteError::Chunk(...))</code> without panicking.
T-REG-002	Document with only whitespace pages produces no chunks.	Pages containing only whitespace characters (spaces, tabs, newlines) have no extractable words. The word-window strategy must return an empty chunk list rather than panicking or producing chunks with empty content.

ID	Description	Acceptance Criterion
T-REG-003	Very long single word (10000 chars) handled without panic.	A page containing a single word of 10000 characters must be processed by the word-window strategy without panicking, stack overflow, or producing invalid chunks. The single word forms exactly one chunk.
T-REG-004	Unicode BOM characters handled gracefully during chunking.	The Unicode Byte Order Mark (U+FEFF) appears at the beginning of some PDF-extracted text. The normalization pipeline (whitespace collapsing, hyphenation repair, NFC normalization) does not explicitly strip BOM characters. U+FEFF is not in the set of characters collapsed by <code>collapse_whitespace</code> (which handles only space, tab, and form feed). This test verifies that text containing BOM characters is chunked without panicking, produces valid chunks with correct content hashes, and the content hash is deterministically reproducible.
T-REG-005	NULL bytes in text content handled gracefully.	Some PDF extraction backends may emit NULL bytes (U+0000) in the extracted text. The chunking pipeline must handle them without panicking. NULL bytes are valid UTF-8 characters and should pass through or be stripped by normalization.
T-REG-006	Session with COALESCE(-1) handles NULL directory correctly.	The COALESCE-based unique index on <code>index_session</code> substitutes -1 for NULL values in <code>chunk_size</code> , <code>chunk_overlap</code> , and <code>max_words</code> . This test verifies that two sessions with identical configurations where all nullable fields are NULL (page-based strategy) are correctly rejected by the unique constraint, and that finding a session by config returns the correct ID.
T-REG-007	Chunk with 0-length content hash is computed correctly.	An empty string has a well-defined SHA-256 hash. The content hash function must produce the correct 64-character hex digest for the empty string.

ID	Description	Acceptance Criterion
T-REG-008	HNSW index with a single vector can be searched.	The HNSW index must support building and searching with just one vector. This is the minimum viable index size. The single vector must be its own nearest neighbor with distance 0 (or very close to 0 for cosine distance).
T-REG-009	FTS5 query with special characters (AND, OR, NOT) is escaped.	FTS5 treats AND, OR, NOT, and NEAR as query operators. If user search text contains these words literally, they must either be escaped or the query must be wrapped in quotes to prevent FTS5 syntax errors. This test verifies that inserting and querying chunks containing these reserved words does not cause database errors.
T-REG-010	Database migration is idempotent (running twice is safe).	Calling <code>migrate()</code> on an already-migrated database must succeed without error and must not create duplicate tables, indexes, or triggers.
T-REG-011	Connection pool handles concurrent access from 8 threads.	The r2d2 connection pool is configured with <code>max_size=8</code> . This test verifies that 8 threads can simultaneously acquire connections and perform read operations without deadlock or timeout errors.
T-REG-012	Session label update with NULL clears the label, and update with a string sets the label.	Verifies BUG #5 fix: <code>session_update</code> must correctly distinguish between JSON null (clear label) and a string value (set label).
T-REG-013	Session with <code>max_words</code> stores and retrieves the value correctly.	Verifies DESIGN #5: the <code>max_words</code> field must be readable from <code>SessionRow</code> for display in session listings.
T-REG-014	Schema v2 migration creates the <code>pdf_page_count</code> column on databases that were originally created with the v1 schema.	The column must be nullable (accepting NULL for old files) and accept integer values for new files. Verifies BUG #1 fix.

ID	Description	Acceptance Criterion
T-REG-015	strip_extended_length Verifies correctly handles Windows \\{}\\{}?\\{} prefixed paths, plain Windows paths, and Unix paths.	Verifies DESIGN #3 fix at the utility function level.
T-REG-016	relevance_label returns correct labels for boundary values.	Verifies BUG #6 related logic where relevance labels are computed from vector_score values at the threshold boundaries.
T-REG-017	Session listing includes max_words, chunk_size, chunk_overlap for different chunk strategies.	Verifies DESIGN #5: all chunking parameters must be readable from list_sessions results.
T-REG-018	Finalize-progress SQL sets progress_done = progress_total.	Regression test for NEU-001: Annotation jobs where no input matched a PDF completed with progress_done=0 and progress_total=N, creating an inconsistent terminal state. The fix adds a finalize_progress call before transitioning to the Completed state. This test verifies the SQL logic at the database level: after setting progress_total to a value greater than progress_done, the finalize query must equalize them.
T-REG-019	Finalize-progress is a no-op when progress_done already equals progress_total (normal completion path).	Complementary test to T-REG-021: when a job completes normally with all items processed (progress_done = progress_total), the finalize step must not corrupt the counters.
T-REG-020	Finalize-progress works when progress_done is partially advanced.	Simulates an annotation job where 2 of 5 PDFs were matched and processed, but 3 inputs had no match. The pipeline reports progress_done=2 (matched PDFs processed) while progress_total=5 (total input quotes). After finalize, progress_done must equal progress_total.

ID	Description	Acceptance Criterion
T-REG-021	A <code>source_file</code> field containing HTML special characters in the filename round-trips through JSON serialization without modification.	<code>serde_json</code> 's default serializer preserves angle brackets as literal characters; it does not escape <code><</code> or <code>></code> to <code>\{u003c\}</code> / <code>\{u003e\}</code> unless a custom HTML-safe serializer is configured. XSS prevention for <code>source_file</code> values is the responsibility of the rendering layer: the frontend must assign the value to <code>textContent</code> (or equivalent) rather than inserting it into <code>innerHTML</code> . This test confirms that the JSON value round-trips correctly and that the literal filename content is present in the serialized output.

Table 41: Property-based and regression test catalog: 27 tests covering invariants and edge cases.

22.6 Test Summary

22.7 Continuous Integration Test Matrix

The CI pipeline executes the test suite across a matrix of operating systems and feature flag combinations. Each cell in the matrix represents a separate CI job.

CI pipeline stages:

1. **Format check:** `cargo fmt --all -- --check`. Fails the pipeline if any file deviates from `rustfmt.toml`.
2. **Lint:** `cargo clippy --workspace --all-targets --features <flags> -- -D warnings`. Run once per feature combination.
3. **Unit and integration tests:** `cargo test --workspace --features <flags>`. Run once per OS and feature combination.
4. **Doc tests:** `cargo test --doc --workspace`. Verifies that all code examples in documentation compile and run.
5. **Coverage:** `cargo llvm-cov --workspace --lcov --output-path lcov.info`. Uploaded as a build artifact.
6. **Benchmarks (optional):** `cargo bench --workspace --features <flags>`. Run on the `ort-only` configuration. Results are compared against the stored baseline; regressions beyond 10% produce a CI warning.
7. **Architecture validation:** A suite of six Python validation scripts verifies bidirectional consistency between this document and the codebase (see Section 22.8):
 - (a) `generate_test_chapter.py` – Generates the test catalog $\LaTeX$ from Rust source and verifies the committed file is up to date.
 - (b) `validate_tests.py` – Validates test IDs, counts, sequencing, and coverage

Category	Count	Type
neuroncite-annotate unit tests	259	Unit
neuroncite-api unit tests	29	Unit
neuroncite-chunk unit tests	40	Unit
neuroncite-citation unit tests	103	Unit
neuroncite-core unit tests	62	Unit
neuroncite-embed unit tests	71	Unit
neuroncite-html unit tests	123	Unit
neuroncite-mcp unit tests	431	Unit
neuroncite-pdf unit tests	71	Unit
neuroncite-pipeline unit tests	37	Unit
neuroncite-search unit tests	54	Unit
neuroncite-store unit tests	144	Unit
neuroncite-web unit tests	25	Unit
<i>Unit test subtotal</i>	<i>1449</i>	
Full pipeline integration tests	37	Integration
CLI integration tests	13	Integration
API server integration tests	8	Integration
<i>Integration test subtotal</i>	<i>58</i>	
Property-based tests	6	Property
Regression and edge case tests	21	Regression
<i>Auxiliary test subtotal</i>	<i>27</i>	
Total	1534	

Table 42: Test catalog summary: 1534 tests across all categories.

OS	ort-only	all-features
Windows (x86_64)	clippy, test, bench	clippy, test
Linux (x86_64)	clippy, test, bench	clippy, test

Table 43: CI test matrix. Each cell executes the full sequence: `cargo clippy`, `cargo test`, and optionally `cargo bench`. All jobs use the CPU execution provider.

- between the catalogs and code.
- (c) `validate_consistency.py` – Checks MCP tool catalogs, REST API endpoints, CLI subcommands, crate counts, label integrity, and numeric prose claims.
 - (d) `validate_schemas.py` – Checks database schema tables, columns, indexes, triggers, and FTS5 configuration.
 - (e) `validate_architecture.py` – Checks module structure, feature flags, crate overview, and the dependency graph.
 - (f) `api_parity_check.py` – Verifies REST, MCP, and Python client API parity.
8. **Frontend checks:** `npm ci`, `tsc --noEmit`, `vitest run`, and `vite build` for the SolidJS frontend on Node.js 26. The Rust jobs compile `neuroncite-web` against a placeholder `frontend/dist/`, so this stage is the only one that installs and builds the npm dependency tree.

22.8 Architecture-Code Consistency Validator

A Python script (`tools/validate_architecture.py`) enforces bidirectional consistency between this architecture document and the Rust source tree. The validator runs as the final CI pipeline stage and fails the build if any discrepancy is detected in either direction—code that deviates from the documented structure, or documentation that describes modules, crates, or features not present in the code.

22.8.1 Invocation and Arguments

The script is invoked from the workspace root:

```
python tools/validate_architecture.py --tex docs/architecture.tex --root .
```

Arguments:

- `--tex` – Path to the LaTeX architecture document (required).
- `--root` – Path to the Cargo workspace root directory (required).
- `--strict` – Treat warnings as errors (optional, enabled in CI).
- `--json` – Output results as JSON instead of human-readable text (optional, for machine consumption).

Exit codes: 0 if all checks pass, 1 if any error-level discrepancy is found, 2 if the script cannot parse the LaTeX file or locate the workspace root.

22.8.2 LaTeX Parsing Approach

The script parses the LaTeX source file using regular expressions to extract structured data from three categories of tables:

Crate overview table

The table in Section 2.1 that lists all crates, their types (Library/Binary), and responsibilities. The parser identifies this table by its caption text (“Workspace crates and their responsibilities”) and extracts rows matching the pattern:

```
\texttt{<crate-name>} & <type> & <description>
```

Each row yields a `CrateSpec` record with the crate name and type.

Module structure tables

The per-crate tables in Section 2.3 that list module paths, visibility, and responsibility. The parser identifies each table by its caption text (“Internal module structure of `<crate-name>`”) and extracts rows matching the pattern:

```
src/<path> & <visibility> & <description>
```

Each row yields a `ModuleEntry` record with the file path, visibility (“public” or “private”), and an optional feature flag annotation. Feature flag annotations are detected by scanning for `\midrule` rows containing “compiled under feature `<flag>`” immediately before a group of module rows.

Feature flag table

The table in Section 13 that lists Cargo feature flags. The parser extracts feature names matching `\texttt{<feature>}` in the first column.

22.8.3 Rust Source Tree Scanning

The script scans the workspace by:

1. Parsing the root `Cargo.toml` for `[workspace.members]` to determine the list of crate directories.
2. For each crate directory, recursively collecting all `.rs` files under `src/`. The relative path of each file (e.g., `src/extract/mod.rs`) is recorded as the module path.
3. Parsing each crate’s `src/lib.rs` (or `src/main.rs` for the binary crate) to extract module declarations. The parser identifies lines matching:
 - `pub mod <name>;` – public module
 - `mod <name>;` – private module
 - `#[cfg(feature = "<flag>")]` before a `mod` declaration – feature-gated module
4. Parsing each crate’s `Cargo.toml` for `[dependencies]` and `[features]` sections to verify the dependency graph and feature flag definitions.

22.8.4 Validation Checks

The validator performs seven categories of checks. Each failed check is classified as either an *error* (blocks the CI pipeline) or a *warning* (reported but does not block unless `--strict` is active).

Check	Severity	Condition for Failure
Workspace member mismatch	error	A crate listed in the architecture document’s crate overview table is absent from the <code>[workspace.members]</code> list in the root <code>Cargo.toml</code> , or vice versa.
Missing source file	error	A module path listed in the architecture document’s module structure table for a crate does not correspond to an existing <code>.rs</code> file on disk. This means the documentation describes a module that has not been implemented.

Check	Severity	Condition for Failure
Undocumented source file	error	A <code>.rs</code> file exists in a crate's <code>src/</code> directory but is not listed in the architecture document's module structure table for that crate. This means code was added without updating the documentation. Files matching common exclusion patterns (<code>tests/</code> , <code>benches/</code> , <code>build.rs</code>) are ignored.
Visibility mismatch	error	A module declared as “public” in the architecture document is declared as <code>mod</code> (without <code>pub</code>) in the crate's <code>lib.rs</code> , or a module declared as “private” is declared as <code>pub mod</code> .
Missing feature gate	error	A module listed under a feature flag section in the architecture document does not have a corresponding <code>#[cfg(feature = "...")]</code> attribute on its <code>mod</code> declaration in <code>lib.rs</code> .
Undocumented feature flag	warning	A feature flag defined in a crate's <code>Cargo.toml</code> <code>[features]</code> section is not described in the architecture document's feature flag table.
Dependency graph deviation	warning	A crate-to-crate dependency declared in <code>Cargo.toml</code> is not depicted in the architecture document's dependency graph description (Section 2.2), or the document describes a dependency that does not exist in <code>Cargo.toml</code> .

Table 44: Validation checks performed by the architecture-code consistency validator. Error-level checks block the CI pipeline. Warning-level checks are reported but do not block unless `--strict` is active.

22.8.5 Output Format

In human-readable mode (default), each discrepancy is printed to `stderr` as a single line with the format:

```
[ERROR|WARN] <check-category>: <crate-name> - <detail>
```

Examples:

- `[ERROR] missing_file: neuroncite-store -- src/manifest.rs listed in docs but not found on disk`
- `[ERROR] undocumented_file: neuroncite-pdf -- src/utils.rs exists on disk but not in architecture document`
- `[ERROR] visibility_mismatch: neuroncite-core -- src/types.rs declared public in docs but mod (private) in lib.rs`
- `[WARN] undocumented_feature: neuroncite-embed -- feature "experimental-fp16" not described in architecture document`

A summary line at the end reports the total count of errors and warnings. In JSON mode (`--json`), the output is a JSON object with **errors** (array), **warnings** (array), and **summary** (object with **error_count** and **warning_count** fields).

22.8.6 Bidirectional Enforcement Rationale

The validator enforces consistency in both directions: documentation $\rightarrow$ code and code $\rightarrow$ documentation. This bidirectional check prevents two failure modes that unidirectional checks miss:

- **Documentation drift (code $\rightarrow$ docs):** A developer adds a new module or renames a file without updating the architecture document. The “undocumented source file” check catches this case.
- **Phantom documentation (docs $\rightarrow$ code):** The architecture document describes a planned module that was never implemented, or a module that was removed during refactoring without updating the document. The “missing source file” check catches this case.

By running the validator in CI on every commit, the architecture document remains a reliable and accurate reference for the codebase structure at all times.

A Glossary

ANN (Approximate Nearest Neighbor)

A class of algorithms that find the closest vectors to a query in sub-linear time by trading a small amount of recall accuracy for large speed gains compared to exhaustive linear search.

BM25

Best Matching 25 – a probabilistic ranking function for keyword-based information retrieval. It scores documents by term frequency, inverse document frequency, and document length normalization. SQLite FTS5 implements BM25 natively.

Chunk

A contiguous fragment of text extracted from a PDF document, produced by a chunking strategy. The atomic unit of indexing and retrieval.

Citation

A structured reference to the source of a search result, containing the file name, page range, document-level UTF-8 byte offsets, and a formatted reference string suitable for academic or professional use.

Connection Pool

A set of pre-established database connections that are reused across requests, avoiding the overhead of opening and closing a connection for every operation. Managed by `r2d2` in this application.

Cosine Similarity

A similarity metric between two vectors, computed as the cosine of the angle between them. Ranges from -1 (opposite) to $+1$ (identical).

Cross-Encoder

A transformer model that jointly encodes a (query, document) pair and produces a single relevance score. More accurate than bi-encoder cosine similarity but cannot pre-compute document representations, making it suitable only for reranking a small candidate set.

CSL-JSON

Citation Style Language JSON – a standardized format for representing bibliographic metadata, used by Pandoc, Zotero, and other citation management tools.

Dense Embedding

A fixed-length vector of floating-point numbers that represents the semantic content of a text passage in a continuous vector space.

Feature Flag

A Cargo build-time switch that conditionally compiles a section of code and its dependencies. Used to make embedding backends optional.

FTS5

Full-Text Search version 5 – SQLite’s built-in full-text search engine. Supports BM25

ranking, phrase queries, and prefix queries over text columns.

HNSW

Hierarchical Navigable Small World – a graph-based data structure for approximate nearest neighbor search. It organizes vectors in a multi-layer navigable graph where higher layers contain fewer, more spread-out nodes for fast coarse search, and lower layers contain all nodes for precise refinement.

Immediate-Mode GUI

A GUI paradigm where the interface is fully redrawn every frame based on the current application state, as opposed to retained-mode GUIs that maintain a persistent widget tree.

Index Session

A record in the database that captures a specific combination of directory, model, and chunking configuration. Used to avoid redundant re-indexing.

Model Manifest

A lock file (`models.lock`) that pins the exact revision and checksum of every model used to build an index, ensuring reproducibility.

ONNX

Open Neural Network Exchange – an open format for representing machine learning models. Allows models trained in one framework to run in another.

Reciprocal Rank Fusion (RRF)

A method for combining ranked lists from multiple retrieval systems. Each document receives a score of $1/(k + \text{rank})$ from each system, and the scores are summed. Simple, parameter-light, and does not require score normalization.

Reranking

A second-pass scoring stage that re-orders the top candidates from the initial retrieval using a more accurate (but slower) model, typically a cross-encoder.

SimHash

A locality-sensitive hashing algorithm that produces similar hash values for similar input texts. Pre-computed at index time and stored in the `chunk.simhash` column. Used for near-duplicate detection in search result deduplication by comparing Hamming distances between fingerprints.

Tombstone / Orphan

A vector that remains in the HNSW graph after its corresponding `chunk` row has been physically deleted from the database. Under normal operation, orphans do not accumulate because each indexing run rebuilds the HNSW graph from all active vectors (build-then-swap). Orphans can appear in the persisted graph if the application crashes between chunk deletion and HNSW rebuild completion. During search, orphaned labels are detected when the chunk row lookup returns no result, and the candidate is silently skipped.

Top-K

The K highest-scoring results from a similarity search, where K is a user-configurable

parameter.

Transformer

A neural network architecture based on self-attention mechanisms, widely used for natural language processing tasks including text embedding.

Vector Dimension

The number of floating-point values in an embedding vector. Determined by the model architecture (e.g., 384 for bge-small-en-v1.5).

Headless Mode

An execution mode where the application runs without a graphical user interface. In NeuronCite, headless mode is activated by the `neuroncite serve` subcommand and runs the API server as the sole user-facing interface.

Idempotency Key

A client-chosen unique identifier attached to a request that allows the server to recognize duplicate submissions. The key and the associated response are persisted in the `idempotency` table. If a request with the same key arrives within the 24-hour retention window, the server returns the stored response without performing the operation again.

Job A tracked unit of asynchronous work (e.g., an indexing operation or an HNSW rebuild) that transitions through the states queued, running, completed, failed, or canceled. Each job has a UUID, is persisted in the SQLite `job` table, and can be polled via the API. Jobs are retained for 24 hours after completion.

OpenAPI

A specification standard (formerly Swagger) for describing HTTP APIs in a machine-readable JSON or YAML format. Enables auto-generation of client libraries, documentation, and request validation.

wgpu

A cross-platform GPU API based on the WebGPU standard, supporting Vulkan, Metal, and DirectX 12 backends.